



UNIVERSIDAD DEL TRABAJO DEL URUGUAY
INSTITUTO TECNOLÓGICO SUPERIOR
INSTITUTO DE ADIESTRAMIENTO AERONÁUTICO

PROYECTO FINAL



**Sistema modular de imputación y análisis estadístico
de parámetros de UAV para apoyo al diseño conceptual
de un dron VTOL de ala fija**



Consejo de Educación
Técnico Profesional
Universidad del Trabajo del Uruguay

presenta:

Ezequiel Delpino

para obtener el grado de:

Ingeniero Tecnólogo en Aeronáutica

Tutores del proyecto:

Ing. A. Farré

Ing. F. Herrera



Montevideo - Uruguay

27 de Febrero de 2026

1 AGRADECIMIENTOS

Este trabajo es, en gran parte, una consecuencia de algo mucho más grande que un esfuerzo individual: el conocimiento acumulado por la humanidad a lo largo del tiempo. Cada generación ha construido sobre la anterior, documentando ideas, métodos y descubrimientos para que otras personas puedan retomarlos, cuestionarlos y mejorarlos. Esa continuidad (ese “interés compuesto” del conocimiento) es lo que vuelve posible que hoy exista un punto de partida sólido para crear, investigar y desarrollar.

Dentro de ese recorrido colectivo, merece un reconocimiento especial un hito reciente y decisivo para este proyecto: la inteligencia artificial. Su avance no solo aceleró el aprendizaje y la exploración técnica, sino que también hizo viable, en términos prácticos, el desarrollo del programa comprendido en este trabajo. En ese sentido, este proyecto también es una muestra de cómo las herramientas nacidas del conocimiento compartido pueden potenciar nuevas contribuciones cuando se usan con criterio, responsabilidad y curiosidad.

A nivel más cercano, quiero agradecer especialmente a mi novia, por ser un pilar constante durante todo este proceso: por el apoyo diario, la paciencia, la confianza y por impulsarme a buscar siempre una versión mejor de mí, incluso en momentos de cansancio o dudas.

También quiero agradecer a todas las personas que, a lo largo de los años de carrera, aportaron de alguna manera a sostener este camino: con una explicación a tiempo, una conversación que ordenó ideas, una oportunidad, una corrección, un empujón o simplemente estando presentes. Muchas contribuciones pequeñas terminan construyendo un resultado grande.

Por último, quiero reconocer a mi yo del pasado: el que decidió empezar, el que sostuvo la constancia y el que hizo la inversión de tiempo y energía necesaria para que hoy yo pueda estar cerrando esta etapa. Terminar esta tesis no es solo cumplir un requisito académico, sino también cerrar un ciclo y dejar una base documentada para que otros puedan continuar, mejorar y llevar más lejos el conocimiento aquí compartido.

2 TABLA DE CONTENIDO

1	AGRADECIMIENTOS	1-3
2	TABLA DE CONTENIDO	2-4
3	LISTA DE IMÁGENES	3-6
4	LISTA DE TABLAS.....	4-7
5	GLOSARIO, ABREVIACIONES Y SÍMBOLOS	5-7
5.1	ABREVIACIONES Y SIGLAS DE AERONÁUTICA Y MISIÓN	5-8
5.2	ABREVIACIONES Y MÉTRICAS ESTADÍSTICAS DE VALIDACIÓN	5-8
5.3	MÉTRICAS INTERNAS DEL SISTEMA (SELECCIÓN DE MODELOS E IMPUTACIÓN)	5-9
5.4	SÍMBOLOS Y NOTACIÓN MATEMÁTICA	5-9
5.5	VARIABLES AERONÁUTICAS Y DE PERFORMANCE (CONCEPTOS DE DISEÑO)	5-10
5.6	NOMENCLATURA DE VARIABLES EN EL CÓDIGO (TRAZABILIDAD CON LA IMPLEMENTACIÓN)	5-10
5.7	JERGA INFORMÁTICA Y CIENCIA DE DATOS (CON SINÓNIMOS RECOMENDADOS)	5-11
6	RESUMEN.....	6-12
7	ABSTRACT	7-14
8	INTRODUCCIÓN	8-17
8.1	OBJETIVOS	8-25
8.2	HIPÓTESIS	8-26
8.3	METODOLOGÍA GENERAL.....	8-27
9	MARCO TEÓRICO	9-29
9.1	PROCESO DE DISEÑO DE AERONAVES: VISIÓN GENERAL	9-29
9.2	CORRELACIÓN ENTRE VARIABLES	9-34
10	GÉNESIS DEL SISTEMA.....	10-60
10.1	ADRPY ORIGINAL EN EL CONTEXTO DEL DISEÑO CONCEPTUAL DE KEANE.....	10-60
10.2	INTENCIÓN INICIAL: APLICACIÓN DE ADRPY A UN DRON DE ALA FIJA CON CAPACIDAD VTOL.....	10-62
10.3	LA PRIMERA TABLA DE DRONES: CONSTRUCCIÓN Y CHOQUE CON LA REALIDAD	10-63
10.4	DEL EXCEL CAÓTICO A LA FORMULACIÓN DEL PROBLEMA DE LOS DATOS.....	10-64
10.5	GÉNESIS DEL SISTEMA ADRPY-VTOL Y DE SUS TRES MÓDULOS	10-65
10.6	ROL DEL SISTEMA EN EL PROCESO DE DISEÑO Y EN LA DECISIÓN DE COMPRA	10-67
10.7	RESUMEN Y PUENTE HACIA LA DESCRIPCIÓN TÉCNICA DE LOS MÓDULOS	10-68
11	ARQUITECTURA DEL SISTEMA Y GUÍA DE USO	11-69
11.1	CONCEPTOS BÁSICOS DE PROGRAMACIÓN UTILIZADOS EN ESTE TRABAJO	11-69
11.2	ESTRUCTURA GENERAL DE CARPETAS DE ARCHIVOS	11-70
11.3	ENTORNO DE TRABAJO Y LANZAMIENTO GENERAL DEL SISTEMA	11-72
11.4	MÓDULO 1: SISTEMA PREDICTOR (VISIÓN DESDE EL USUARIO)	11-72
11.5	MÓDULO 2: ANÁLISIS Y VISUALIZACIÓN DE MODELOS	11-76
11.6	MÓDULO 3: ASISTENTE DE DISEÑO PARA NUEVAS AERONAVES.....	11-79
11.7	FLUJO GLOBAL ENTRE LOS TRES MÓDULOS.....	11-81
11.8	SCRIPTS PRINCIPALES Y SU PAPEL EN EL SISTEMA	11-83
11.9	AUTOMATIZACIÓN, MANTENIMIENTO Y LÍMITES PRÁCTICOS	11-84
11.10	RESUMEN Y VÍNCULO CON LOS CAPÍTULO SIGUIENTES	11-84
12	MÓDULO 1: SISTEMA PREDICTOR E IMPUTACIÓN DE LA BASE DE DATOS.....	12-85
12.1	ROL DEL MÓDULO 1 EN EL FLUJO DEL SISTEMA	12-85
12.2	DESCRIPCIÓN DE LA BASE DE DATOS DE DRONES	12-86
12.3	CALIDAD, COMPLETITUD Y LIMITACIONES DE LOS DATOS	12-87

12.4	PREPROCESAMIENTO Y VARIABLES DERIVADAS.....	12-88
12.5	CRITERIOS GENERALES DE IMPUTACIÓN Y FUNCIONES PRINCIPALES	12-89
12.6	IMPUTACIÓN POR SIMILITUD	12-90
12.7	IMPUTACIÓN POR CORRELACIÓN	12-95
12.8	BUCLE DE IMPUTACIÓN Y COMBINACIÓN DE MÉTODOS.....	12-119
12.9	PANEL DE CONFIGURACIÓN DEL MÓDULO 1 Y SENSIBILIDAD DEL SISTEMA.....	12-122
12.10	REGISTRO DE RESULTADOS, SALIDAS EN EXCEL Y JSON	12-125
12.11	LIMITACIONES DEL MÓDULO 1 Y PUENTE HACIA EL MÓDULO 2.....	12-126
13	MÓDULO 2: PANEL INTERACTIVO DE ANÁLISIS DE MODELOS	13-128
13.1	DEL MOTOR PREDICTOR A LA VISUALIZACIÓN	13-128
13.2	DE LOS JSON DEL MÓDULO 1 AL DASHBOARD DEL MÓDULO 2.....	13-128
13.3	ESTRUCTURA GENERAL DEL PANEL: CUATRO ZONAS QUE SE HABLAN ENTRE SÍ	13-129
13.4	USO DEL MÓDULO 2 EN LA PRÁCTICA	13-130
13.5	CASOS DE USO Y LECCIONES OBTENIDAS DEL ANÁLISIS DE MODELOS.....	13-132
13.6	CÓMO EL ANÁLISIS DEL MÓDULO 2 ALIMENTA AL PREDICTOR Y AL ASISTENTE DE DISEÑO	13-135
13.7	CIERRE DEL CAPÍTULO Y PUENTE HACIA EL MÓDULO 3	13-135
14	MÓDULO 3: ASISTENTE DE DISEÑO	14-136
14.1	ROL DEL ASISTENTE DE DISEÑO EN EL FLUJO GLOBAL	14-139
14.2	ENTRADA DE DATOS: DEFINIR LA AERONAVE OBJETIVO	14-139
14.3	REUTILIZACIÓN DE TENDENCIAS Y MODELOS UNIVARIADOS	14-141
14.4	RANKING POR SIMILITUD Y EXPLORACIÓN DE VECINOS (VERSIÓN AMPLIADA CON BASE TEÓRICA)	14-142
14.5	DETECCIÓN DE OUTLIERS Y VERIFICACIONES DE CONSISTENCIA.....	14-143
14.6	MENSAJES DE GUÍA, TOOLTIPS Y NARRATIVA DE APOYO	14-144
15	MÓDULO 3: EJEMPLOS DE USO PARA DEFINIR UNA AERONAVE OBJETIVO	15-146
15.1	OBJETIVO DEL CAPÍTULO.....	15-146
15.2	PASO 1: MIRAR EL “ORDEN DE MAGNITUD” DE LA BASE DE DATOS.....	15-146
15.3	PASO 2: TRADUCIR LA MISIÓN A REQUISITOS NUMÉRICOS INICIALES (CASO A)	15-148
15.4	PASO 3: PRIMERA CORRIDA DEL MÓDULO 3 (CASO A).....	15-148
15.5	PASO 4: USAR LAS TENDENCIAS X–Y PARA ENTENDER EL TRADE-OFF	15-151
15.6	PASO 5: SEGUNDA ITERACIÓN (CASO B) CON REQUISITOS MÁS REALISTAS	15-153
15.7	PASO 6: TERCERA ITERACIÓN (CASO C) AÑADIENDO LA CONFIGURACIÓN DE LA AERONAVE	15-154
15.8	PASO 7: VECTOR FINAL DE PARÁMETROS PARA LA AERONAVE OBJETIVO	15-157
16	DISCUSIÓN, ALCANCES Y CONTINUACIÓN	16-168
16.1	VISIÓN GENERAL: QUÉ APORTA REALMENTE EL SISTEMA.....	16-168
16.2	LECCIONES SOBRE DATOS: CALIDAD, CANTIDAD Y SEGMENTACIÓN	16-168
16.3	LECCIONES SOBRE MODELOS: NO CONFIAR SOLO EN LOS NÚMEROS	16-169
16.4	ANÁLISIS DIMENSIONAL Y DESCUBRIMIENTO DE FÓRMULAS: UN PRIMER PASO	16-170
16.5	POSIBLES EXTENSIONES ESTADÍSTICAS Y DE IA	16-171
16.6	ESCALABILIDAD, CONTINUIDAD Y POTENCIAL INSTITUCIONAL	16-171
16.7	DEL ANÁLISIS ESTADÍSTICO A LA FORMULA FÍSICA	16-172
16.8	MÉTODOS COMPLEMENTARIOS.....	16-173
16.9	LECCIONES DEL DESARROLLO: PROGRAMACIÓN, ESTADÍSTICA E INTELIGENCIA ARTIFICIAL.....	16-176
16.10	SÍNTESIS Y PUENTE A LAS CONCLUSIONES	16-177
17	CONCLUSIONES.....	17-178
17.1	GRADO DE CUMPLIMIENTO DE LOS OBJETIVOS	17-178
17.2	APORTES PRINCIPALES DEL SISTEMA ADRPY-VTOL.....	17-179
17.3	HIPÓTESIS, DESCUBRIMIENTO DE RELACIONES Y ALCANCE REAL DEL SISTEMA	17-179
17.4	LIMITACIONES Y ESTADO ACTUAL DE LA HERRAMIENTA	17-180
17.5	LÍNEAS DE TRABAJO FUTURO	17-181
17.6	CONSIDERACIONES FINALES.....	17-181

18	APÉNDICES.....	18-182
18.1	DESCRIPCIÓN DE VARIABLES DEL VECTOR DE LA AERONAVE OBJETIVO	18-182
18.2	FICHA TÉCNICA “ALTI TRANSITION”	18-185
18.3	PROYECTO FINAL FORMATO REVISTA	18-187
	ABSTRACT	188
3.2	CONSTRUCCIÓN Y NORMALIZACIÓN DE LA BASE DE DATOS DE UAV.....	18-191
19	BIBLIOGRAFÍA & RECURSOS.....	18-199

3 LISTA DE IMÁGENES

IMAGEN 8.1	- ETAPAS CLÁSICAS DEL PROCESO DE DISEÑO DE AERONAVES.....	8-17
IMAGEN 8.2	- EJEMPLO DE DIAGRAMA CLÁSICO DE LA LITERATURA	8-18
IMAGEN 8.3	- DIAGRAMA DE RESTRICCIONES (CONSTRAINT ANALYSIS)	8-25
IMAGEN 9.1	- ETAPAS DEL PROCESO DE DISEÑO AERONÁUTICO Y ALCANCE DE ESTA TESIS	9-30
IMAGEN 9.2	- SUBAJUSTE, AJUSTE ADECUADO Y SOBREAJUSTE.....	9-36
IMAGEN 9.3	- FUNCIÓN DE MAPEO PARA LA CONFIANZA DEL MODELO	9-56
IMAGEN 11.1	- ESTRUCTURA GENERAL DE CARPETAS Y ARCHIVOS.....	11-71
IMAGEN 11.2	- LANZAMIENTO DEL DASHBOARD DEL MÓDULO 1 (PREDICTOR)	11-72
IMAGEN 11.3	- DASHBOARD DEL MÓDULO 1: CONFIGURACIÓN ANTES DE EJECUTAR	11-73
IMAGEN 11.4	- ARCHIVOS DE SALIDA DEL MÓDULO 1	11-74
IMAGEN 11.5	- FLUJO DE PROCESAMIENTO DEL MÓDULO 1	11-76
IMAGEN 11.6	- NOTEBOOK DEL MÓDULO 2: PREPARACIÓN DEL ANÁLISIS DE MODELOS.....	11-76
IMAGEN 11.7	- DASHBOARD DE EJECUCIÓN DE MODULO 2	11-77
IMAGEN 11.8	- DASHBOARD DEL MÓDULO 2: ANÁLISIS INTERACTIVO DE MODELOS.	11-78
IMAGEN 11.9	- ESQUEMA DEL ROL DEL MÓDULO 2 (LECTURA Y CURACIÓN)	11-79
IMAGEN 11.10	- APERTURA DEL DASHBOARD DEL MÓDULO 3 (ASISTENTE DE DISEÑO)	11-80
IMAGEN 11.11	- VISTA GENERAL DEL ASISTENTE DE DISEÑO (MÓDULO 3)	11-81
IMAGEN 11.12	- DIAGRAMA DE FLUJO GLOBAL: DE LA BASE DE DATOS ORIGINAL AL ASISTENTE DE DISEÑO	11-83
IMAGEN 12.1	- EXTRACTO DEL EXCEL DE LA BASE DE AERONAVES	12-86
IMAGEN 12.2	- CURVA DE SIMILITUD EN FUNCIÓN DE LA DIFERENCIA PORCENTUAL.....	12-92
IMAGEN 12.3	- GENERACIÓN DE MODELOS CANDIDATOS A PARTIR DEL SET DE ENTRENAMIENTO	12-99
IMAGEN 12.4	- CONTROL DE CORRELACIÓN ENTRE PREDICTORES (MULTICOLINEALIDAD)	12-100
IMAGEN 12.5	- EVALUACIÓN DEL MODELO: ENTRENAMIENTO VS VALIDACIÓN LOOCV	12-109
IMAGEN 12.6	- CÁLCULO Y CONFIGURACIÓN DEL SCORE DE CONFIANZA DEL MODELO.....	12-114
IMAGEN 12.7	- CONFIGURACIÓN DE CHEQUEO DE EXTRAPOLACIÓN: REGIÓN VÁLIDA DE IMPUTACIÓN	12-118
IMAGEN 12.8	- DASHBOARD COMPLETO DE IMPUTACIÓN Y COMBINACIÓN DE MÉTODOS.....	12-119
IMAGEN 12.9	- PANEL DE CONFIGURACIÓN DEL MÓDULO 1: PESTAÑA ORQUESTACIÓN.....	12-123
IMAGEN 12.10	- PANEL DE CONFIGURACIÓN DEL MÓDULO 1: PESTAÑA SIMILITUD	12-123
IMAGEN 12.11	- PANEL DE CONFIGURACIÓN DEL MÓDULO 1: PESTAÑA CORRELACIÓN	12-124
IMAGEN 12.12	- PANEL DE CONFIGURACIÓN DEL MÓDULO 1: PESTAÑA LOOP/ORQUESTACIÓN	12-124
IMAGEN 12.13	- PANEL DE CONFIGURACIÓN DEL MÓDULO 1: PESTAÑA EXCEL	12-125
IMAGEN 12.14	- PANEL DE CONFIGURACIÓN DEL MÓDULO 1: PESTAÑA HTML.....	12-125
IMAGEN 12.15	- EJEMPLO DE ARCHIVO JSON DE RESULTADOS DE IMPUTACIÓN	12-126
IMAGEN 13.1	- DIAGRAMA SIMPLE MÓDULO 1	13-128
IMAGEN 13.2	- ESTRUCTURA DEL DASHBOARD DEL MÓDULO 2 (CUATRO ZONAS).....	13-130
IMAGEN 13.3	- EJEMPLO 3D: SUPERFICIE CON POCA COBERTURA DE DATOS	13-132
IMAGEN 13.4	- EJEMPLO 2D DE MODELO “ENGAÑOSO” POR EXTRAPOLACIÓN	13-134
IMAGEN 14.1	- INTERFAZ DEL ASISTENTE DE DISEÑO (VISTA DETALLADA EN TRES PARTES)	14-139
IMAGEN 14.2	- ZONAS PRINCIPALES DEL ASISTENTE DE DISEÑO (CAPTURA ANOTADA)	14-141
IMAGEN 14.3	- AYUDA CONTEXTUAL: DEFINICIONES DE CONTROLES DEL ASISTENTE	14-145

IMAGEN 15.1 - CONTROLES POR PARÁMETRO: MODO, RANGO Y PESO EN EL ASISTENTE	15-146
IMAGEN 15.2 - PANEL “RESUMEN GLOBAL/FILTRADO” PARA ALCANCE (KM)	15-147
IMAGEN 15.3 - INGRESO DE REQUERIMIENTOS	15-149
IMAGEN 15.4 - RESULTADO DEL RANKING DEL CASO A (AUTONOMÍA ALTA Y MTOW BAJO)	15-150
IMAGEN 15.5 - CASO A: CONFLICTO AUTONOMÍA VS MASA (VISUALIZACIÓN)	15-152
IMAGEN 15.6 - DISTRIBUCIÓN VELOCIDAD DE CRUCERO VS AUTONOMÍA EN LA BASE	15-152
IMAGEN 15.7 - EJEMPLO DE FILTRADO POR AUTONOMÍA Y MTOW (CANDIDATOS)	15-154
IMAGEN 15.8 - PLATAFORMA VTOL DE REFERENCIA: ALTI TRANSITION	15-156
IMAGEN 15.9 - CATAPULTA NEUMÁTICA PL-80 DE ELI AIRBORNE SOLUTIONS	15-164
IMAGEN 15.10 - DIAGRAMA DE RESTRICCIONES WS-T/W PARA CASO DE ESTUDIO	15-166
IMAGEN 16.1 – UBICACIÓN MACRO DEL PROGRAMA DENTRO DE DIFERENTES METODOLOGÍAS	16-171
IMAGEN 16.2 – ÁRBOL DE DECISIÓN VS RANDOM FOREST EN REGRESIÓN	16-174
IMAGEN 16.3 – ARQUITECTURA GENERAL DE XGBOOST (GRADIENT BOOSTING)	16-174
IMAGEN 16.4 – EJEMPLO DE DIAGRAMA DE ESTADOS Y TRANSICIONES	16-175
IMAGEN 18.1 - FICHA TÉCNICA DE UAV ALTI TRANSITION (1/2)	18-185
IMAGEN 18.2 - FICHA TÉCNICA DE UAV ALTI TRANSITION (2/2)	18-186
IMAGEN 18.3 - DIAGRAMAS DE RESTRICCIONES ADRPY PARA UNA AERONAVE OBJETIVO.	18-189
IMAGEN 18.4 – ARQUITECTURA DEL SISTEMA DE IMPUTACIÓN Y SELECCIÓN INICIAL DE PARÁMETROS PARA DISEÑO PRELIMINAR DE EVTOL	18-189
IMAGEN 18.5 – ENFOQUES DE IMPUTACIÓN Y SU ROL EN EL FLUJO, SIMILITUD VS REGRESIÓN	18-190
IMAGEN 18.6 – DASHBOARD DEL MÓDULO 2: FILTROS DE SELECCIÓN, GRÁFICO PRINCIPAL, RESUMEN DE MODELOS/MÉTRICAS Y PANEL DE DETALLE	18-194
IMAGEN 18.7 – COMENTARIO GENERADO EN CELDAS IMPUTADAS PARA AUDITAR LA LÓGICA Y VERACIDAD DEL VALOR.	18-196

4 LISTA DE TABLAS

TABLA 8.1 - OBJETIVOS DEL TRABAJO	8-26
TABLA 9.1 - TABLA DE CÁLCULO DE COEFICIENTES MÍNIMOS POR TIPO DE FUNCIÓN	9-38
TABLA 10.1 - ENTRADAS Y SALIDAS PRINCIPALES DE ADRPY DENTRO DEL CICLO ITERATIVO DE DISEÑO CONCEPTUAL	10-61
TABLA 12.1 - COMPLETITUD DE LA BASE DE DATOS POR PARÁMETRO (ESTADO INICIAL)	12-88
TABLA 12.2 - EJEMPLO DE DATASET DE ENTRENAMIENTO TRAS EL FILTRADO INICIAL (MÓDULO 1)	12-97
TABLA 15.1 - AERONAVES CON AUTONOMÍA ≥ 20 H EN LA BASE DE DATOS	15-150
TABLA 15.2 - AERONAVES LIGERAS CON AUTONOMÍA MEDIA-ALTA ($\approx 8-16$ H, $MTOW \leq 30$ KG)	15-154
TABLA 15.3 - AERONAVES VTOL/HÍBRIDAS CON AUTONOMÍA MEDIA-ALTA Y MTOW MODERADO	15-155
TABLA 15.4 - VECTOR DE ENTRADA ADRPY (ALTI TRANSITION): REQUERIMIENTOS DE MISIÓN (BRIEF)	15-159
TABLA 15.5 - VECTOR DE ENTRADA ADRPY (ALTI TRANSITION): VARIABLES DE DISEÑO GEOMÉTRICO (DESIGN)	15-161
TABLA 15.6 - VECTOR DE ENTRADA ADRPY (ALTI TRANSITION): COEFICIENTES DE DESPEGUE Y RODADURA (NO APLICABLES EN ESTA ETAPA)	15-162
TABLA 15.7 - VECTOR DE ENTRADA ADRPY (ALTI TRANSITION): COEFICIENTES AERODINÁMICOS EN CONFIGURACIÓN LIMPIA	15-163
TABLA 15.8 - VECTOR DE ENTRADA ADRPY (ALTI TRANSITION): EFICIENCIAS DE HÉLICE Y TIPO DE PROPULSIÓN	15-163
TABLA 16.1 - COMPARACIÓN DE MÉTODOS DE MODELADO PARA IMPUTACIÓN DE DATOS TÉCNICOS	16-176
TABLA 18.1 – TABLA PARCIAL DE COMPLETITUD DE CELDAS.	18-195

5 GLOSARIO, ABREVIACIONES Y SÍMBOLOS

Este capítulo reúne definiciones, abreviaciones, símbolos y términos técnicos utilizados a lo largo del trabajo. Cuando se emplean términos en inglés por ser habituales en bibliografía o herramientas de software, se incluye su equivalente en español y sinónimos recomendados para facilitar la lectura.

5.1 Abreviaciones y siglas de aeronáutica y misión

UAV (Unmanned Aerial Vehicle). Vehículo aéreo no tripulado. Aeronave que opera sin piloto a bordo, ya sea de forma autónoma o teleoperada.

VTOL (Vertical Take-Off and Landing). Despegue y aterrizaje vertical. Describe aeronaves capaces de elevarse y aterrizar sin requerir carrera de pista.

MTOW (Maximum Take-Off Weight). Peso máximo al despegue. Límite superior de peso con el cual la aeronave puede iniciar el despegue; condiciona el diseño estructural, la performance y el dimensionamiento general.

ISA (International Standard Atmosphere). Atmósfera estándar internacional. Modelo de referencia que define condiciones atmosféricas típicas (temperatura, presión y densidad) en función de la altitud para realizar cálculos comparables.

NACA. Nomenclatura de perfiles aerodinámicos desarrollada históricamente por el comité NACA. Se utiliza para identificar geometrías de perfiles alares mediante series numéricas.

5.2 Abreviaciones y métricas estadísticas de validación

MAPE (Mean Absolute Percentage Error). Error porcentual absoluto medio. Cuantifica el error relativo promedio entre valores observados y valores predichos. Su forma estándar se expresa como:

$$MAPE = (1/n) \cdot \sum |(y_i - \hat{y}_i) / y_i| \cdot 100\%$$

donde n es la cantidad de observaciones, y_i el valor real y $\hat{y}_i$ el valor predicho. En la interpretación práctica, un $MAPE$ menor indica mayor precisión relativa. Puede volverse sensible cuando y_i toma valores cercanos a cero.

R^2 (coeficiente de determinación). Mide qué proporción de la variabilidad de la variable objetivo explica el modelo. Una forma estándar de definición es:

$$R^2 = 1 - (SS_{res} / SS_{tot})$$

donde SS_{res} es la suma de cuadrados de los residuos y SS_{tot} la suma total de cuadrados. Un R^2 alto sugiere buen ajuste, pero no implica causalidad y puede resultar poco informativo si la variable objetivo casi no varía.

CV (coeficiente de variación). Indicador de variabilidad relativa, definido como:

$$CV = \sigma / \bar{x}$$

donde σ es la desviación estándar y $\bar{x}$ es la media. Se utiliza para advertir casos donde métricas como R^2 pueden parecer “buenas” aunque la variable objetivo tenga muy poca variabilidad.

LOOCV (Leave-One-Out Cross-Validation). Validación cruzada “dejando uno afuera”. Consiste en entrenar el modelo con todas las muestras menos una y evaluar sobre la muestra excluida, repitiendo el proceso para cada observación. Es útil cuando la disponibilidad de datos es limitada, ya que aprovecha casi toda la información en cada ajuste.

VIF (Variance Inflation Factor). Factor de inflación de la varianza. Indicador de colinealidad entre predictores; valores elevados sugieren que predictores están fuertemente correlacionados y que el ajuste puede volverse inestable o poco interpretable.

Número de condición (condition number). Medida del condicionamiento numérico del problema de ajuste. Valores altos suelen indicar inestabilidad numérica, frecuentemente asociada a colinealidad o escalas muy distintas entre variables.

5.3 Métricas internas del sistema (selección de modelos e imputación)

Corr (coeficiente combinado). Métrica operativa definida en el sistema para comparar modelos usando simultáneamente capacidad explicativa (asociada a R^2) y precisión (asociada a $MAPE$) en una escala común. Se utiliza como criterio integrado de desempeño al evaluar múltiples ecuaciones candidatas.

Confianza (score final entre 0 y 1). Métrica principal de selección utilizada para decidir qué modelo es más conveniente para imputar un valor faltante. Se construye a partir de una calidad base del modelo y se ajusta mediante penalizaciones que contemplan complejidad, tamaño muestral efectivo y verificaciones de calidad del conjunto de datos.

Base (confianza base). Componente inicial que combina términos normalizados de R^2 y $MAPE$ para producir una estimación acotada de la calidad del modelo antes de aplicar penalizaciones.

r2term. Transformación de R^2 a un rango normalizado $[0, 1]$, utilizada para combinarlo con otras métricas dentro de una misma escala.

mapeterm. Transformación de $MAPE$ a un rango normalizado $[0, 1]$, de modo que errores grandes reduzcan fuertemente la evaluación del modelo.

mape_divisor (D). Parámetro de escala que define el rango operativo a partir del cual el $MAPE$ es considerado inaceptable o fuertemente penalizable dentro del término **mapeterm**. Funciona como umbral de sensibilidad del sistema ante el error.

pen_k (penalización por complejidad). Factor entre 0 y 1 que reduce la confianza cuando el modelo posee demasiados parámetros efectivos, en relación con la información disponible.

pen_n (penalización por cantidad de datos). Factor entre 0 y 1 que reduce la confianza cuando el tamaño muestral efectivo es bajo, aumentando el riesgo de falta de robustez.

pen2D (penalización por verificaciones 2D). Factor compuesto entre 0 y 1 asociado a chequeos de calidad del conjunto de entrenamiento (por ejemplo, colinealidad y cobertura del espacio de datos). Su objetivo es penalizar escenarios donde el ajuste puede ser numéricamente inestable o poco representativo.

k (parámetros efectivos). Número efectivo de parámetros del modelo (medida práctica de complejidad). A mayor k , mayor riesgo de sobreajuste si los datos son escasos.

n (muestras efectivas). Número de observaciones válidas utilizadas realmente para entrenar/validar un modelo para una variable objetivo específica.

5.4 Símbolos y notación matemática

y_i . Valor real (observado) de la variable objetivo en la observación i .

$\hat{y}_i$. Valor estimado (predicho) por el modelo para la observación i .

n . Cantidad de observaciones usadas en un cálculo estadístico (por ejemplo, en MAPE).

σ . Desviación estándar de una variable.

$\bar{x}$. Media aritmética de una variable.

w . Peso o coeficiente de ponderación utilizado para combinar componentes o métricas dentro de una ecuación.

Σ . Operador de sumatoria.

5.5 Variables aeronáuticas y de performance (conceptos de diseño)

W/S (carga alar; wing loading). Relación entre el peso de la aeronave y el área alar. Influye en velocidad de pérdida, desempeño de despegue/aterrizaje y comportamiento general. Puede expresarse en N/m^2 o kg/m^2 .

T/W (relación empuje/peso). Relación entre el empuje disponible y el peso. Indica margen de performance para acelerar y trepar. Es adimensional.

AR (aspect ratio; relación de aspecto). Medida de “alargamiento” del ala. Se asocia a eficiencia aerodinámica y, en términos simplificados, relaciona envergadura y superficie alar.

λ (taper ratio; estrechamiento). Relación entre cuerda en la punta y cuerda en la raíz del ala. Afecta distribución de sustentación y consideraciones estructurales.

Techo de servicio (service ceiling). Altitud máxima de operación según un criterio mínimo de tasa de ascenso.

Tasa de ascenso (climb rate). Velocidad vertical de ascenso. Usualmente se expresa en m/s o ft/min .

Carrera de despegue (ground run). Distancia recorrida sobre el suelo hasta alcanzar condiciones de despegue. Usualmente se expresa en metros.

5.6 Nomenclatura de variables en el código (trazabilidad con la implementación)

***wingarea_m2*.** Superficie alar en m^2 .

***aspectratio*.** Relación de aspecto del ala (AR).

***roottaperratio*.** Estrechamiento del ala (λ).

***weight_n*.** Peso expresado en Newtons (cuando el cálculo se expresa en fuerza).

***totalstaticthrust_n*.** Empuje estático máximo total en Newtons.

***loiter_time_h*.** Tiempo de merodeo/patrulla en horas.

***servceil_m*.** Techo de servicio en metros.

***climbrate_fpm*.** Tasa de ascenso en pies por minuto.

groundrun_m. Carrera de despegue en metros.

5.7 Jerga informática y ciencia de datos (con sinónimos recomendados)

Pipeline. Flujo de procesamiento encadenado, compuesto por etapas consecutivas que transforman una entrada en una salida. Sinónimos recomendados: flujo de trabajo, cadena de etapas, proceso secuencial.

Workflow. Secuencia de pasos o procedimiento operativo. Sinónimos recomendados: flujo de trabajo, metodología, secuencia de acciones.

Dataset. Conjunto de datos utilizado para análisis o modelado. Sinónimos recomendados: conjunto de datos, base de datos del estudio, conjunto de registros.

Dataframe. Tabla de datos estructurada típica en herramientas de análisis (por ejemplo, en Python). Sinónimos recomendados: tabla de datos, estructura tabular, matriz tabular.

Script. Archivo de código ejecutable. Sinónimos recomendados: programa, rutina, módulo.

Loop. Bucle de iteración. Sinónimos recomendados: iteración repetitiva, ciclo.

Outlier. Valor atípico. Dato que se aleja marcadamente del comportamiento general y puede distorsionar un ajuste si no se controla o justifica.

Predictor (feature). Variable de entrada usada para estimar otra variable. Sinónimos recomendados: variable explicativa, variable predictora, entrada del modelo.

Target. Variable que se desea estimar. Sinónimos recomendados: variable objetivo, salida del modelo.

Overfitting. Sobreajuste. Situación en la que el modelo se ajusta demasiado a los datos de entrenamiento y pierde capacidad de generalización hacia casos nuevos.

6 RESUMEN

El diseño conceptual de aeronaves se apoya habitualmente en datos de referencia y en diagramas globales que relacionan parámetros como el peso máximo de despegue, la superficie alar o la potencia instalada. En el caso de los vehículos aéreos no tripulados (UAV), la información disponible en catálogos comerciales y fichas técnicas presenta con frecuencia tres problemas combinados: (i) tablas con un porcentaje elevado de celdas vacías, (ii) definiciones no homogéneas entre fabricantes para parámetros clave (por ejemplo, peso en vacío o velocidades características), y (iii) configuraciones modulares cuya documentación técnica es parcial. Estas características dificultan la comparación entre plataformas, la construcción de reglas empíricas de diseño y la alimentación de herramientas de análisis conceptual basadas en datos de mercado.

En paralelo, el diseño conceptual de aeronaves ha incorporado de forma creciente herramientas programables que automatizan parte del ciclo iterativo típico descrito por autores como Keane y Raymer. En ese contexto se enmarca ADRpy, una biblioteca en Python organizada como notebook de Jupyter que implementa modelos aeronáuticos y de peso para estimar prestaciones y generar diagramas clásicos de restricciones, como el plano carga alar (S/W) frente a relación potencia-peso (P/W). ADRpy permite declarar parámetros de misión, masas, geometría y propulsión, ejecutar modelos bien documentados en la literatura y visualizar salidas que guían la iteración de diseño conceptual.

Frente a este contexto, la tesis desarrolla un sistema modular de imputación y análisis estadístico de parámetros de UAV, denominado ADRpy-VTOL. El sistema toma como punto de partida una hoja de cálculo heterogénea de aeronaves de ala fija, con y sin capacidad VTOL, y la transforma en una base de datos organizada, completada y documentada, orientada explícitamente a apoyar el diseño conceptual de un dron de ala fija con capacidad de despegue y aterrizaje vertical. El sistema se implementa en Python y se estructura en tres módulos coordinados: un módulo predictor, un módulo de análisis y auditoría y un módulo asistente de diseño conceptual.

El **Módulo 1 (predictor)** implementa un motor de imputación que combina dos enfoques complementarios. Por un lado, se utilizan esquemas de imputación por similitud entre aeronaves, definidos a partir de parámetros globales seleccionados (como masa máxima, superficie alar o autonomía), para completar celdas vacías en registros con alto grado de proximidad técnica. Por otro, se entrenan modelos de regresión supervisada con distintas formas funcionales (lineal, polinómica de segundo grado, logarítmica, exponencial y de potencia), utilizando hasta dos o tres predictores según la disponibilidad y calidad de los datos. Estos modelos permiten estimar parámetros ausentes a partir de combinaciones de variables observadas, bajo criterios explícitos de coherencia física y estadística.

El **Módulo 2 (análisis y auditoría)** ofrece una interfaz de exploración (implementada como un panel interactivo) que lee los modelos y las imputaciones generadas por el Módulo 1 y permite al usuario: (i) filtrar ecuaciones según métricas de desempeño, (ii) visualizar relaciones entre variables mediante gráficos y tablas, (iii) identificar valores atípicos y rangos típicos de diseño, y (iv) revisar, parámetro por parámetro, las decisiones de imputación realizadas. En este contexto, el panel interactivo se entiende como una herramienta de apoyo al análisis: no ejecuta cálculos adicionales complejos, sino que organiza de manera accesible la información generada por el sistema, de modo que el usuario pueda inspeccionarla críticamente y decidir qué modelos aceptar o descartar.

El **Módulo 3 (asistente de diseño conceptual)** reutiliza el subconjunto de modelos validados para proponer valores de parámetros globales de una aeronave objetivo a partir de unos pocos requerimientos iniciales (por ejemplo, rango de MTOW y tipo de misión). Estas recomendaciones no se presentan como números aislados, sino como un vector de entrada completo, estructurado de forma compatible con el formato requerido por ADRpy para el análisis conceptual. El módulo explicita qué modelos de la base

respaldan cada recomendación y qué nivel de confianza se asocia a cada imputación, de modo que el diseñador pueda trazar el origen de cada valor sugerido.

Metodológicamente, el trabajo combina: (i) la recopilación y unificación de fichas técnicas de UAV comerciales en un rango de masa específico, (ii) la definición de parámetros derivados relevantes para diseño conceptual (como carga alar o relación potencia-peso), (iii) la depuración básica de la base de datos y el tratamiento de valores atípicos mediante medidas robustas, (iv) el entrenamiento sistemático de modelos de regresión con validación cruzada tipo LOOCV, y (v) la evaluación de cada modelo mediante métricas complementarias (MAE, MAPE y coeficiente de determinación R^2). Sobre esta base se define una jerarquía de complejidad de modelos, que establece que, ante desempeños predictivos comparables, se privilegia el uso de ecuaciones de menor complejidad (por ejemplo, modelos lineales frente a modelos polinómicos), con el fin de favorecer la interpretabilidad y reducir el riesgo de sobreajuste.

Para ordenar los modelos generados se introduce un **coeficiente combinado de calidad**, entendido como un indicador sintético que integra, en una sola cifra, tanto la precisión de las predicciones (a través de métricas de error como el MAPE) como la capacidad explicativa (a través de R^2) y la robustez frente a la validación cruzada. Este coeficiente no pretende sustituir el análisis detallado de cada métrica, sino proporcionar un criterio práctico para comparar y priorizar ecuaciones cuando el número de modelos candidatos es elevado. De forma complementaria, cada imputación almacena información sobre el modelo utilizado y su nivel de calidad, lo que permite reconstruir y auditar a posteriori el proceso de estimación de la base de datos.

Los resultados obtenidos muestran que, incluso partiendo de una base de datos con un alto porcentaje de valores faltantes y con diferencias de criterio entre fabricantes, es posible construir (mediante el sistema propuesto) un conjunto de parámetros estimados internamente coherente, acompañado de indicadores explícitos de calidad y trazabilidad. Esta base de datos enriquecida permite identificar rangos de diseño habituales para distintas combinaciones de masa, superficie alar, carga alar y potencia específica, así como detectar configuraciones que se apartan de esos rangos de forma significativa. A partir de los modelos seleccionados se derivan relaciones funcionales entre parámetros globales que pueden interpretarse como reglas empíricas de primer orden para el diseño conceptual de UAV en el rango de masas considerado.

En la versión actual del sistema, el asistente de diseño del Módulo 3 se utiliza, además, para construir un caso de estudio completo: las recomendaciones de parámetros globales se organizan en un vector de entrada compatible con ADRpy y se emplean para generar, con fines ilustrativos, un diagrama clásico de restricciones que relaciona la carga alar (W/S) con la relación empuje-peso (T/W). Este diagrama no se analiza en profundidad desde el punto de vista de prestaciones, sino que se incluye únicamente para materializar el flujo completo propuesto, desde los datos de mercado hasta una salida típica del diseño conceptual.

Desde el punto de vista del estado de desarrollo, el sistema alcanza el nivel de **mínimo producto viable (MVP)**: existe un flujo reproducible desde una planilla inicial desordenada hasta un conjunto de parámetros globales coherentes para una aeronave objetivo, acompañado por herramientas para auditar el camino seguido por los datos y por los modelos. La herramienta no pretende ser un producto industrial acabado, sino una plataforma formativa y extensible que conecta datos de mercado problemáticos con una primera solución numérica defendible para alimentar ADRpy.

Esta visión se materializa en el repositorio público ADRpy-VTOL, que reúne el código, los notebooks y los archivos de configuración necesarios para reproducir los ejemplos de la tesis y experimentar con nuevos conjuntos de datos. El texto de la tesis se complementa con dicho repositorio: el documento explica la lógica del sistema y sus decisiones de diseño, y el código permite ejecutar el flujo completo y explorar variantes. El proyecto deja así una base concreta sobre la que otros trabajos pueden continuar,

ya sea refinando la plataforma ADRpy-VTOL o avanzando en el diseño y eventual construcción de la aeronave VTOL que motivó originalmente el estudio.

En conjunto, la tesis aporta un procedimiento sistemático y reproducible para mejorar la explotación de datos de UAV en el contexto de diseño conceptual. El énfasis se sitúa en la calidad de los datos, en la transparencia de las imputaciones y en la trazabilidad de los modelos empleados, de modo que el sistema actúa como una herramienta de apoyo al criterio del diseñador, sin sustituirlo.

Palabras clave: diseño conceptual, UAV, VTOL, imputación de datos, regresión, MAPE, R^2 , coeficiente combinado de calidad, sistemas de apoyo a la decisión, ADRpy.

7 ABSTRACT

Conceptual aircraft design usually relies on reference data and global diagrams that relate parameters such as maximum take-off weight, wing area or installed power. In the case of unmanned aerial vehicles (UAV), the information available in commercial catalogs and technical datasheets typically presents three combined issues: (i) tables with a high percentage of missing entries, (ii) non-homogeneous definitions of key parameters across manufacturers (for example, empty weight or characteristic speeds), and (iii) modular configurations whose technical documentation is only partial. These features hinder platform-to-platform comparison, the construction of empirical design rules, and the use of market data to feed conceptual design tools.

In parallel, conceptual aircraft design has increasingly incorporated programmable tools that automate parts of the iterative cycle described by authors such as Keane and Raymer. In this context, ADRpy is a Python library, organized as a Jupyter notebook, that implements aerodynamic and weight models to estimate performance and generate classical constraint diagrams, such as wing loading (S/W) versus power-to-weight ratio (P/W). ADRpy allows the user to declare mission, mass, geometry and propulsion parameters, to run models well documented in the literature, and to visualize outputs that guide the conceptual design iteration.

Against this background, the thesis develops a modular system for imputation and statistical analysis of UAV parameters, termed ADRpy-VTOL. The system starts from a heterogeneous spreadsheet of fixed-wing UAV, with and without VTOL capability, and transforms it into an organized, completed and documented database explicitly oriented to supporting the conceptual design of a fixed-wing VTOL drone. The system is implemented in Python and structured into three coordinated modules: a predictor module, an analysis and audit module, and a conceptual design assistant module.

The **Module 1** (predictor) implements an imputation engine that combines two complementary approaches. On the one hand, it applies similarity-based imputation schemes between aircraft, defined from selected global parameters (such as maximum take-off mass, wing area or endurance), to complete missing values in records with a high degree of technical proximity. On the other hand, it trains supervised regression models with different functional forms (linear, second-degree polynomial, logarithmic, exponential and power-law), using up to two or three predictors depending on data availability and quality. These models estimate missing parameters from combinations of observed variables under explicit criteria of physical and statistical consistency.

The **Module 2** (analysis and audit) provides an exploratory interface —implemented as an interactive panel— that reads the models and imputations generated by Module 1 and allows the user to: (i) filter

equations according to performance metrics, (ii) visualize relationships between variables through plots and tables, (iii) identify outliers and typical design ranges, and (iv) review, parameter by parameter, the imputation decisions that were made. In this context, the interactive panel is understood as an analysis-support tool: it does not perform additional complex calculations but rather organizes the information generated by the system in an accessible way, so that the user can inspect it critically and decide which models to accept or discard.

The **Module 3** (conceptual design assistant) reuses the subset of validated models to propose values for the global parameters of a target aircraft from a few initial requirements (for example, MTOW range and mission type). These recommendations are not presented as isolated numbers but as a complete input vector structured to match the format required by ADRpy for conceptual analysis. The module makes explicit which models in the database support each recommendation and what confidence level is associated with each imputation, allowing the designer to trace the origin of each suggested value.

Methodologically, the work combines: (i) the collection and unification of commercial UAV datasheets within a specific mass range; (ii) the definition of derived parameters relevant to conceptual design (such as wing loading or power-to-weight ratio); (iii) basic database cleaning and outlier treatment using robust measures; (iv) systematic training of regression models with leave-one-out cross-validation (LOOCV); and (v) evaluation of each model using complementary metrics (MAE, MAPE and coefficient of determination R^2). On this basis, a hierarchy of model complexity is defined: when predictive performance is comparable, lower-complexity equations (for example, linear rather than polynomial models) are preferred, in order to favour interpretability and reduce the risk of overfitting.

To rank the generated models, a **combined quality coefficient** is introduced, understood as a synthetic indicator that integrates, into a single figure, predictive accuracy (through error metrics such as MAPE), explanatory power (through R^2) and robustness under cross-validation. This coefficient is not intended to replace detailed analysis of each metric but to provide a practical criterion for comparing and prioritizing equations when the number of candidate models is large. In parallel, each imputation stores information about the model used and its quality level, which enables the reconstruction and ex-post audit of the database estimation process.

The results show that, even starting from a database with a high percentage of missing values and with differences in manufacturers' criteria, it is possible —using the proposed system— to construct an internally consistent set of estimated parameters, accompanied by explicit indicators of quality and traceability. This enriched database makes it possible to identify typical design ranges for different combinations of mass, wing area, wing loading and specific power, as well as to detect configurations that deviate significantly from those ranges. From the selected models, functional relationships between global parameters are derived that can be interpreted as first-order empirical rules for the conceptual design of UAV within the considered mass range.

In the current version of the system, the design assistant in Module 3 is also used to build a complete case study: the recommended global parameters are organized into an input vector compatible with ADRpy and used, for illustrative purposes, to generate a classical constraint diagram relating wing loading (W/S) and thrust-to-weight ratio (T/W). This diagram is not analyzed in depth from a performance standpoint; it is included solely to materialize the full proposed flow, from market data to a typical conceptual design output.

From a development-status viewpoint, the system reaches the level of a **minimum viable product (MVP)**: there is a reproducible flow from an initial unstructured spreadsheet to a coherent set of global parameters for a target aircraft, accompanied by tools to audit the path followed by both data and models. The tool is not intended as a finished industrial product but as a formative and extensible platform that links problematic market data with a first defensible numerical solution to feed ADRpy.

This vision is materialized in the public ADRpy-VTOL repository, which gathers the code, notebooks and configuration files required to reproduce the thesis examples and to experiment with new datasets. The thesis document complements this repository: the text explains the system logic and design decisions, while the code allows the user to execute the complete workflow and explore variants. The project thus provides a concrete foundation on which further work can build, either by refining the ADRpy-VTOL platform or by advancing the design and eventual construction of the VTOL aircraft that originally motivated the study.

Taken together, the thesis provides a systematic and reproducible procedure for improving the exploitation of UAV data in the context of conceptual design. Emphasis is placed on data quality, transparency of imputations and traceability of the models employed, so that the system acts as a decision-support tool for the designer rather than a replacement for engineering judgement.

Keywords: conceptual design; UAV; VTOL; data imputation; regression; MAPE; R^2 ; combined quality coefficient; decision-support systems; ADRpy.

8 INTRODUCCIÓN

El diseño de una aeronave comienza con la definición precisa de la misión y de los requisitos que debe satisfacer. La naturaleza de esa misión —transporte de carga, vigilancia, fumigación agrícola, reconocimiento, formación o apoyo a tareas civiles y militares— condiciona desde el inicio las decisiones principales de diseño. En función de estos objetivos se fijan parámetros globales como el peso máximo al despegue, la superficie alar, la carga alar, la potencia o el empuje instalados, la autonomía y la velocidad de crucero. Estos parámetros, a su vez, influyen en la configuración general de la aeronave, en la elección de materiales y en el tipo de sistemas de propulsión y control necesarios para cumplir la misión.

La etapa de diseño conceptual tiene como propósito seleccionar de manera coherente esos parámetros globales, sabiendo que se encuentran fuertemente acoplados entre sí. Un aumento de la potencia instalada puede mejorar determinadas prestaciones, pero incrementa el peso de la planta motriz y, en consecuencia, el peso máximo al despegue. Un incremento del peso máximo eleva, por regla general, la velocidad de pérdida, lo que puede requerir un aumento de la superficie alar para mantener márgenes aceptables en despegue y aterrizaje. Ese aumento de área alar, a su vez, repercute en el peso estructural. El diseño conceptual puede entenderse, en este sentido, como un proceso de ajuste fino entre parámetros interdependientes, en el que cada modificación desencadena una serie de compensaciones.

De forma más amplia, la literatura clásica describe el proceso completo de diseño como una secuencia de etapas que comienza con la definición de requisitos, continúa con el diseño conceptual y preliminar, y culmina en el diseño de detalle y la fabricación. La **Imagen 8.1** muestra de manera esquemática esta organización general, adaptada de Raymer.

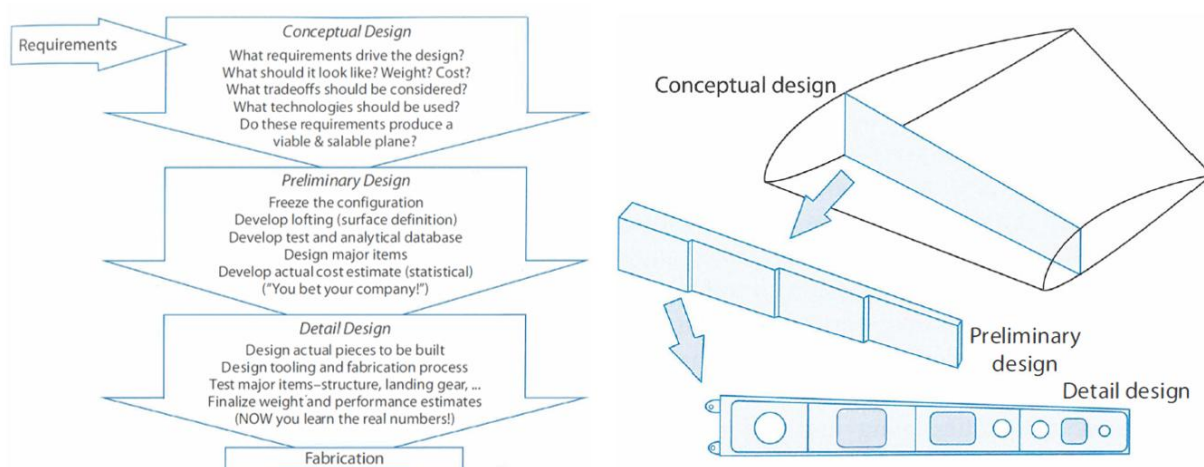


Imagen 8.1 Etapas clásicas del proceso de diseño de aeronaves.

Esquema adaptado de Raymer que muestra la secuencia desde los requisitos iniciales hasta el diseño conceptual, preliminar y de detalle, y su vínculo con la fabricación. [1]

En la práctica, este proceso se apoya tanto en modelos teóricos como en la experiencia acumulada de la industria. Una de las herramientas habituales consiste en utilizar aeronaves existentes como referencia. La literatura clásica de diseño reúne, por ejemplo, diagramas en los que se representa la fracción de peso vacío en función del peso máximo al despegue para distintas familias de aeronaves, o gráficos donde se comparan

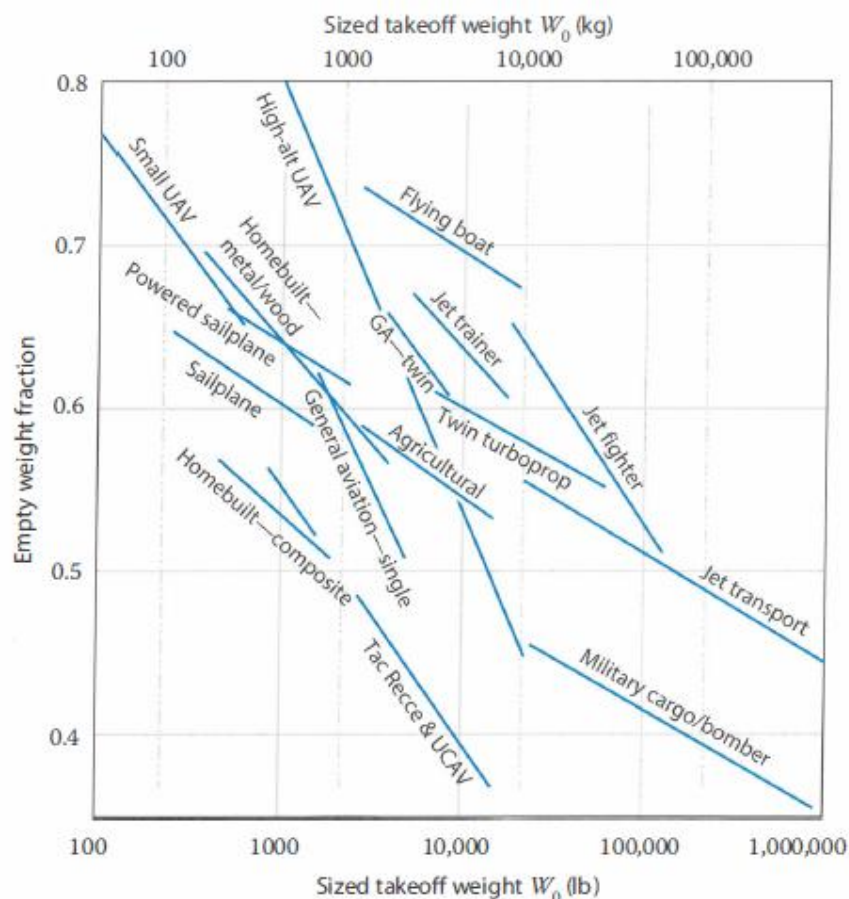


Imagen 8.2 - Ejemplo de diagrama clásico de la literatura

En el eje horizontal, el peso al despegue en escala logarítmica; en el eje vertical, la fracción de peso vacío; varias rectas representan categorías como jets de entrenamiento, aviones agrícolas, UAV pequeños, etc. [1][Empty-weight fraction trends, [1, p. 30] Fig. 3. 1.]

cargas alares y relaciones empuje/peso de configuraciones típicas. Estos recursos permiten identificar rangos de diseño “habituales” y detectar configuraciones que se apartan de manera significativa de lo esperado para una determinada categoría o rango de masas.

El desarrollo de software especializado ha añadido, en las últimas décadas, una capa adicional a este proceso. En lugar de trabajar exclusivamente con tablas y gráficos estáticos, es posible integrar modelos de peso y prestaciones en herramientas programables que permiten evaluar rápidamente diferentes configuraciones. ADRpy es un ejemplo de este tipo de herramienta: se trata de una biblioteca en Python orientada al análisis conceptual, que implementa modelos aeronáuticos y de peso para estimar prestaciones y construir diagramas de restricciones clásicos. Para operar, ADRpy requiere que el usuario proporcione un conjunto de parámetros globales que describan la aeronave que se desea analizar.

La calidad de las salidas que produce ADRpy depende directamente de la calidad de ese vector de entrada. Si los parámetros iniciales no son coherentes entre sí, o si se encuentran muy alejados de la práctica habitual en el rango de masas considerado, los resultados numéricos pueden carecer de utilidad para el diseño. Por ello, disponer de una base de datos organizada de aeronaves de referencia y de métodos sistemáticos para extraer de ella rangos plausibles de parámetros globales resulta especialmente relevante para la etapa conceptual.

En el caso de los vehículos aéreos no tripulados (UAV) de ala fija, este problema adquiere características particulares. El mercado de UAV ha crecido de forma rápida y con una gran diversidad de fabricantes, tamaños y configuraciones. Existen plataformas ligeras, de pequeño tamaño y bajo costo, destinadas a aplicaciones recreativas o a servicios sencillos de filmación; y existen, al mismo tiempo, sistemas más complejos, de mayor envergadura y capacidad, orientados a misiones industriales, de seguridad o de investigación. Esta diversidad se refleja en la forma en que los fabricantes presentan la información técnica de sus productos.

En términos generales, la información disponible para UAV de ala fija procede de fuentes como:

- fichas técnicas y catálogos oficiales publicados por los fabricantes;
- descripciones de productos en páginas web institucionales;
- material técnico distribuido como documentación comercial o de soporte;
- y, en algunos casos, resúmenes de características presentes en repositorios o portales de divulgación.

A partir de estas fuentes es posible reunir tablas con parámetros globales de diferentes plataformas, pero la experiencia muestra que estas tablas presentan una **serie de dificultades** que limitan su uso directo en el diseño conceptual.

En primer lugar, muchos fabricantes publican solo un subconjunto de parámetros. Es frecuente encontrar fichas que informan el peso máximo al despegue y la velocidad de crucero, pero omiten la superficie alar; otras proporcionan la carga útil y la autonomía, pero no detallan el peso en vacío ni la potencia instalada; en algunos casos se declara únicamente el peso total del sistema de despegue y aterrizaje vertical, sin separar componentes. Al registrar estos datos en una hoja de cálculo compartida, el resultado es una tabla con un número considerable de celdas vacías distribuidas de manera irregular entre los distintos parámetros.

En segundo lugar, las definiciones empleadas no son homogéneas. Conceptos aparentemente sencillos, como “peso en vacío” o “autonomía”, pueden estar sujetos a interpretaciones diferentes según el fabricante. En ciertos casos, el peso en vacío incluye todos los sistemas de a bordo salvo combustible y carga útil; en otros, se refiere a la plataforma básica sin determinados equipos auxiliares o sensores. De forma similar, la autonomía puede estar asociada a condiciones de vuelo, configuraciones de carga y perfiles de misión que no siempre se especifican con claridad. Esta falta de homogeneidad introduce una fuente de incertidumbre adicional al comparar valores entre aeronaves.

En tercer lugar, una parte importante del mercado de UAV se organiza en torno a plataformas modulares. El fabricante ofrece una configuración básica y un conjunto de opciones de carga útil, baterías, sensores o equipos de comunicaciones que el usuario puede combinar según sus necesidades. Las prestaciones declaradas suelen corresponder a una configuración de referencia, pero la descripción detallada de esa configuración no siempre está disponible. Esto complica la comparación con otros productos en los que sí se explicita cada componente y sus efectos sobre el peso y las prestaciones.

Cuando se integran en una única base de datos decenas de aeronaves con estas características, se obtiene un panorama útil para valorar órdenes de magnitud, pero difícil de explotar de manera directa. Es posible verificar, por ejemplo, que los pesos máximos se agrupan en ciertos rangos y que determinadas misiones se asocian a escalas de tamaño y autonomía reconocibles. Sin embargo, la presencia de muchos datos ausentes, la heterogeneidad de definiciones y la existencia de configuraciones parcialmente documentadas hacen que cualquier intento de extraer reglas cuantitativas de diseño requiera un tratamiento previo cuidadoso.

La presente tesis parte precisamente de esta situación. El punto de partida es una hoja de cálculo heterogénea de UAV de ala fija, en un rango de masa específico, que reúne información procedente de fichas técnicas, catálogos y documentación pública de fabricantes, complementada de forma puntual con otras fuentes abiertas cuando fue necesario aclarar definiciones generales o completar datos básicos. La tabla incluye, según disponibilidad, datos de masa, dimensiones principales, prestaciones y características generales de configuración y misión.

En este estado inicial, la base de datos resulta valiosa como inventario, pero limitada como herramienta cuantitativa. Para que pueda utilizarse como apoyo al diseño conceptual y para alimentar con garantías un análisis programable basado en ADRpy, es necesario abordar dos problemas principales:

1. **La presencia de valores faltantes** en un número significativo de celdas, que impide aplicar métodos estadísticos estándar sin un tratamiento previo.

2. **La falta de uniformidad en las definiciones y criterios de declaración**, que obliga a introducir supuestos claros y cautelas a la hora de interpretar y combinar los datos.

A partir de esta necesidad se formula la idea central del trabajo: diseñar un sistema que permita organizar, completar y analizar esta base de datos de forma sistemática, con métodos transparentes y con un registro ordenado de las decisiones adoptadas en cada etapa. Este sistema, que se desarrollará en los capítulos siguientes, constituye el vínculo entre la información disponible en el mercado y las herramientas de análisis conceptual que requieren vectores de entrada coherentes para describir una aeronave de referencia.

La necesidad de tratar de manera más rigurosa la base de datos descrita en la parte anterior conduce al planteo de un sistema específico de organización, imputación y análisis. En lugar de considerar la hoja de cálculo únicamente como un listado de referencia, se propone utilizarla como punto de partida de un proceso estructurado que permita:

- completar una fracción significativa de los valores faltantes;
- evaluar, con métricas cuantitativas, la calidad de las estimaciones realizadas;
- identificar rangos típicos de diseño para distintos parámetros globales;
- y generar, a partir de esa información, vectores de entrada coherentes para herramientas de análisis conceptual como ADRpy.

Este enfoque se materializa en el desarrollo de un sistema modular denominado **ADRpy-VTOL**. El sistema se concibe como un puente entre dos dominios que, en la práctica, suelen abordarse por separado: por un lado, el dominio de los datos de mercado, con sus limitaciones y heterogeneidades; por otro, el dominio de los modelos de cálculo programables, que requieren entradas organizadas y consistentes. ADRpy-VTOL se sitúa entre ambos, con la función de transformar una base de datos incompleta en un conjunto de parámetros globales estimados, acompañados de información sobre su origen y su calidad.

Desde el punto de vista académico, el proyecto general se orienta al diseño de un dron de ala fija con capacidad de despegue y aterrizaje vertical (VTOL) en un rango de masas concreto. Con el fin de abordar este propósito de manera ordenada, el programa se estructura en dos trabajos de fin de carrera complementarios. La **presente tesis** se centra en la construcción y explotación de una base de datos de aeronaves de referencia, y en el diseño e implementación del sistema ADRpy-VTOL. Un **trabajo paralelo** profundiza en el uso de ADRpy y en el análisis conceptual de configuraciones VTOL, tomando como punto de partida los vectores de entrada que pueden generarse con el sistema aquí descrito. De este modo, el

recorrido completo abarca, de forma escalonada, desde la organización de la información disponible en el mercado hasta la aplicación de modelos de prestaciones sobre un caso de estudio representativo.

El sistema ADRpy-VTOL se organiza en tres componentes principales que actúan de forma coordinada:

1. Módulo predictor.

Este módulo se ocupa de preparar la base de datos para su explotación y de generar estimaciones de parámetros faltantes. En una primera etapa, ordena y transforma los datos originales, unifica unidades y selecciona subconjuntos utilizables según el parámetro que se desee estudiar. A continuación, aplica dos enfoques complementarios:

- **Imputación por similitud entre aeronaves**, en la que se buscan plataformas con características globales próximas (por ejemplo, masa máxima, superficie alar, autonomía) para estimar valores faltantes en registros incompletos.

- **Modelos de regresión supervisada**, donde se ajustan funciones lineales y no lineales sencillas (polinómicas de bajo grado, logarítmicas, exponenciales, de potencia) que relacionan un parámetro objetivo con uno, dos o tres predictores, según la disponibilidad y calidad de la información.

Cada modelo se entrena con un conjunto de datos válido, se evalúa mediante métricas como MAE, MAPE y R^2 , y se almacena junto con indicadores que resumen su comportamiento.

2. Módulo de análisis y auditoría.

El segundo componente del sistema toma como entrada los modelos y las imputaciones generadas por el módulo predictor y los organiza en una estructura de consulta. Se desarrolla un panel interactivo que permite:

- filtrar ecuaciones según métricas de desempeño y nivel de complejidad;
- visualizar relaciones entre variables mediante gráficos y tablas;
- identificar valores atípicos y rangos típicos de diseño para diferentes combinaciones de parámetros;
- y revisar, parámetro por parámetro, qué modelos se han utilizado para imputar cada dato y con qué nivel de calidad.

El papel de este módulo es ofrecer una vista ordenada y flexible del “estado de los modelos”, de manera que el usuario pueda inspeccionar la información con criterios técnicos, confirmar decisiones de imputación, detectar posibles inconsistencias y, en caso necesario, descartar determinadas ecuaciones.

3. Módulo asistente de diseño conceptual.

Una vez identificados los modelos que presentan un comportamiento satisfactorio, el tercer componente del sistema los reutiliza para proponer parámetros globales de una aeronave objetivo. A partir de unos pocos requisitos iniciales (por ejemplo, rango de peso máximo al despegue, tipo de misión o familia de referencia), el asistente:

- consulta los modelos disponibles;
- genera estimaciones para parámetros que no se hayan fijado de antemano;
- y construye un vector de entrada completo para ADRpy, en el que cada valor se asocia a la ecuación o procedimiento de imputación del que procede.

El asistente no devuelve un único diseño “óptimo”, sino un conjunto de valores coherentes con el conocimiento estadístico derivado de la base de datos, acompañado de información de trazabilidad que permite al diseñador comprender y, si es necesario, ajustar manualmente cada componente del vector.

La implementación de estos módulos se realiza en Python, con una estructura basada en notebooks y scripts que pueden ejecutarse y modificarse de forma incremental. Esta elección busca dos objetivos simultáneos. Por un lado, facilita la integración con ADRpy, que está escrito en el mismo lenguaje y orientado al trabajo en entornos similares. Por otro, favorece el carácter formativo del proyecto: el código se organiza de manera que pueda ser leído, verificado y extendido por otros estudiantes o por equipos de trabajo que deseen adaptar el sistema a nuevas bases de datos o a rangos de masa distintos.

El diseño de ADRpy-VTOL tiene un enfoque deliberadamente práctico. No se parte de una formulación abstracta y luego se busca un conjunto de datos para validarla, sino del problema concreto de trabajar con una base de datos real, con sus vacíos e inconsistencias, y de la necesidad de contar con una herramienta que permita mejorarla hasta donde sea razonable. El sistema no elimina las limitaciones de las fuentes, pero sí introduce una forma organizada de tratarlas: cada imputación queda asociada a un modelo y a una medida de calidad; cada modelo se evalúa de forma explícita; y el resultado final se presenta de manera que se pueda seguir el rastro de las decisiones tomadas.

Desde un punto de vista más general, el uso sistemático de modelos de regresión en este trabajo no se concibe únicamente como una herramienta de ajuste numérico. La literatura reciente sobre descubrimiento de leyes físicas a partir de datos ha mostrado que ciertos métodos de aprendizaje automático pueden proponer estructuras funcionales que luego se interpretan en términos físicos. Entre otros, cabe mencionar trabajos que utilizan redes neuronales o modelos simbólicos para extraer ecuaciones compactas a partir de registros experimentales o de simulación, y que plantean una integración explícita entre datos y formulaciones analíticas [2]; [3]; [4].

En la presente tesis no se implementan algoritmos de aprendizaje profundo de este tipo, ni se busca formular nuevas leyes físicas a partir de la base de datos disponible. No obstante, la idea de fondo está presente: si se dispone de datos razonablemente consistentes, los modelos pueden emplearse no solo para predecir y completar información faltante, sino también para sugerir relaciones funcionales que, más adelante, pueden ser analizadas e interpretadas desde el punto de vista aeronáutico.

Este enfoque enlaza de manera natural con métodos clásicos como el análisis dimensional y el teorema π de Buckingham [5]; [6], que permiten construir grupos adimensionales y proponer relaciones entre variables a partir de consideraciones puramente físicas. Aunque en esta tesis no se desarrolla en profundidad una combinación sistemática de análisis dimensional y búsqueda automatizada de modelos, se reconoce este camino como una línea clara de trabajo futuro: integrar la organización de datos y las técnicas de regresión utilizadas aquí con herramientas más avanzadas de descubrimiento de modelos podría enriquecer significativamente el sistema y ampliar su capacidad para apoyar el diseño conceptual.

En este punto resulta útil resumir la posición que ocupa esta tesis dentro del proceso de diseño de una aeronave pequeña. El trabajo no pretende abordar todas las etapas, ni sustituir análisis aerodinámicos o estructurales de alto nivel de detalle. Su contribución se concentra en una zona específica del flujo: la que conecta los datos de aeronaves existentes con la elección inicial de parámetros globales para una nueva configuración. En términos simples, ADRpy-VTOL se ocupa de mejorar la información de entrada, de forma que las herramientas de análisis conceptual dispongan de vectores más coherentes y trazables sobre los que trabajar.

Sobre este marco general se apoyan los supuestos de partida y las limitaciones reconocidas del enfoque, así como el objetivo general y los objetivos específicos de la tesis. En la parte siguiente de la introducción se describen dichas certezas e incertidumbres iniciales, con el fin de delimitar con claridad

qué se asume como válido, qué se considera un resultado esperable y qué aspectos deben manejarse con especial cautela al interpretar las conclusiones del trabajo.

El desarrollo de ADRpy-VTOL se apoya en una serie de **supuestos de partida** que conviene dejar claros desde el inicio, ya que orientan las decisiones metodológicas y la interpretación de los resultados.

En primer lugar, se asume que los UAV incluidos en la base de datos representan productos reales que han superado, al menos, un umbral mínimo de viabilidad técnica y operativa. Aunque las fichas técnicas puedan contener imprecisiones locales o diferencias de criterio entre fabricantes, los parámetros globales publicados (masas, dimensiones, prestaciones) se consideran suficientemente fiables como para buscar en ellos tendencias de primer orden dentro del rango de masas seleccionado. Dicho de otro modo, no se entiende la base de datos como un registro exhaustivo ni perfecto del mercado, pero sí como una muestra razonable de diseños que han demostrado ser utilizables en la práctica.

En segundo lugar, se considera que los parámetros globales empleados —peso máximo al despegue, peso en vacío cuando se dispone de él, superficie alar, envergadura, cargas alares, potencias específicas, autonomías y velocidades características— constituyen una descripción adecuada para la fase de diseño conceptual. Estos parámetros no agotan el espacio de diseño, pero permiten caracterizar la “escala” de la aeronave y su misión de forma suficiente como para alimentar modelos de peso y prestaciones como los que implementa ADRpy. El sistema se centra deliberadamente en este conjunto de variables, aun cuando existan otros aspectos relevantes (detalles de control, sistemas embarcados, integraciones con carga útil específica) que se abordan en etapas posteriores del desarrollo de un producto aeronáutico.

En tercer lugar, se adopta como criterio metodológico que la organización modular y la trazabilidad de las operaciones aportan un valor significativo. Cada imputación que realiza el sistema queda asociada al modelo o al procedimiento que la generó; cada modelo de regresión se almacena junto con sus métricas de error y con información sobre el conjunto de datos utilizado para ajustarlo; las decisiones de filtrado y las reglas de selección de modelos se documentan en estructuras de datos que pueden consultarse desde el panel de análisis. Este enfoque busca que el sistema no sea una “caja negra” que simplemente entrega números, sino una herramienta cuyos pasos intermedios puedan revisarse y, si es necesario, modificarse.

Por último, se entiende que ADRpy-VTOL debe actuar como apoyo a la decisión y no como sustituto del criterio del diseñador. El sistema aporta estimaciones, rangos plausibles y visualizaciones que ayudan a comprender cómo se comportan las aeronaves del conjunto de referencia, pero la elección final de parámetros para un proyecto concreto, así como la aceptación o rechazo de determinadas recomendaciones, sigue siendo responsabilidad del ingeniero o del equipo de diseño. En consecuencia, las salidas del sistema se conciben como propuestas preliminares coherentes con la base de datos y con los modelos empleados, no como soluciones definitivas.

Junto con estos supuestos de base, es necesario reconocer un conjunto de **incertidumbres y limitaciones** que condicionan el alcance de los resultados.

Una **primera** limitación está asociada a la calidad y coherencia de las fuentes de datos. Las fichas técnicas y descripciones de productos proceden de fabricantes distintos, elaboradas en momentos diferentes y con criterios de presentación que no siempre se explican con detalle. Aunque en la tesis se realiza una depuración básica de la base de datos (corrección de valores evidentemente erróneos, unificación de unidades, detección de duplicados), no puede garantizarse la ausencia completa de errores de carga ni de inconsistencias en las definiciones empleadas por cada fabricante.

Una **segunda** limitación se relaciona con el tamaño y la representatividad de la muestra. El conjunto de UAV considerados está acotado por el rango de masa elegido y por la disponibilidad pública de información. Las relaciones funcionales que se derivan de la base de datos describen el comportamiento de este subconjunto concreto del mercado y no deben extrapolarse de manera automática a otros rangos

de masa, a configuraciones muy distintas (por ejemplo, plataformas de gran altitud y larga duración) o a aplicaciones que no estén adecuadamente representadas en la muestra.

En **tercer** lugar, algunos parámetros presentan un número reducido de observaciones válidas. En esos casos, la construcción de modelos de regresión está sujeta a una mayor incertidumbre estadística y a un riesgo superior de sobreajuste. La tesis señala explícitamente estos casos y propone interpretar los resultados asociados a esos parámetros como indicaciones de orden de magnitud o como puntos de partida para análisis más detallados, y no como reglas de diseño firmes.

Otra fuente de incertidumbre proviene de la definición de similitud entre aeronaves utilizada en los procedimientos de imputación. Para estimar un valor faltante mediante comparación con plataformas “parecidas” es necesario escoger qué variables definen esa semejanza (por ejemplo, peso máximo al despegue, superficie alar, autonomía, tipo de misión, tipo de propulsión) y cómo se combinan en una medida numérica de distancia. Distintas elecciones razonables de estas variables y pesos pueden conducir a resultados ligeramente diferentes, en particular en regiones del espacio de diseño donde los datos son escasos. El sistema adopta una definición concreta de similitud, adecuada al contexto de la tesis, pero reconoce que no se trata de una definición única ni universal.

Finalmente, el alcance funcional de ADRpy-VTOL se limita de manera deliberada a la etapa de diseño conceptual. El sistema no aborda directamente análisis aerodinámicos de alta fidelidad, estudios estructurales detallados ni aspectos de control, fabricación o certificación. La generación de un vector de entrada completo para ADRpy y la construcción de un diagrama clásico de restricciones (W/S frente a T/W) (**Imagen 8.3**) se utilizan como ejemplo de integración del sistema en un flujo de diseño más amplio, pero no sustituyen las etapas posteriores de verificación y refinamiento que serían necesarias en un proceso de desarrollo industrial.

En conjunto, estos supuestos y limitaciones definen el marco en el que deben interpretarse los resultados de la tesis. El trabajo no pretende ofrecer un modelo exhaustivo del mercado de UAV ni un entorno de diseño completamente automatizado, sino un procedimiento sistemático para mejorar la explotación de una base de datos real y para apoyar, con información organizada y trazable, la elección de parámetros globales en el diseño conceptual de un dron de ala fija con capacidad VTOL.

Sobre esta base se formulan el objetivo general y los objetivos específicos del trabajo, que estructuran el desarrollo de los capítulos siguientes y precisan, en términos operativos, qué se espera lograr con el sistema ADRpy-VTOL y cómo se evalúa su desempeño dentro del marco conceptual aquí presentado.

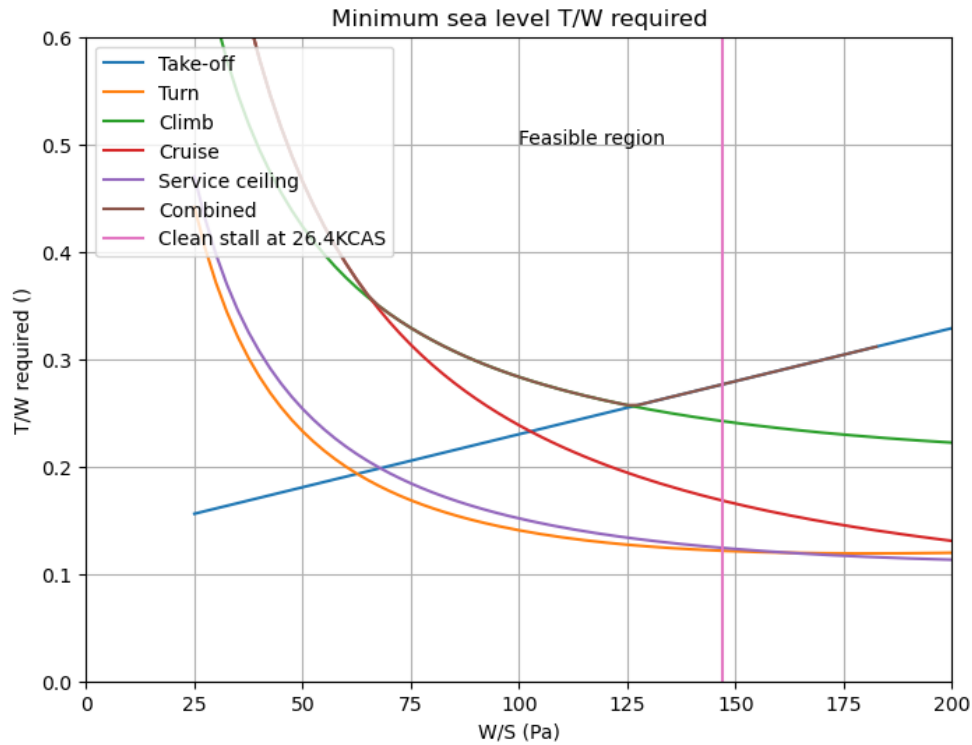


Imagen 8.3- Diagrama de restricciones (constraint analysis)

Mínimo T/W requerido en función de W/S para distintas condiciones de desempeño. Sobester muestra este tipo de diagramas para pequeños UAV, donde la zona en blanco indicada con la frase “Feasible región”, representa el conjunto de pares (W/S , T/W) que satisfacen simultáneamente todas las condiciones. [7]

8.1 Objetivos

En este contexto, el propósito de la tesis puede resumirse en un objetivo general y varios objetivos específicos.

8.1.1 Objetivo general

Desarrollar y documentar un sistema modular que permita completar, analizar y explotar una base de datos de aeronaves existentes, de modo que se convierta en una herramienta de apoyo al diseño conceptual de un dron de ala fija con capacidad VTOL.

8.1.2 Objetivos específicos

En términos más concretos, se plantean los siguientes objetivos específicos:

1. Diseñar un **módulo predictor** que utilice modelos de regresión y criterios físicos simples para imputar valores faltantes en la base de datos, manteniendo un registro explícito de la calidad y la confianza de cada imputación.
2. Construir un **módulo de análisis y visualización de modelos** que permita explorar las ecuaciones generadas, sus métricas de error y su coherencia técnica, mediante gráficos y paneles interactivos.
3. Implementar un **asistente de diseño** que, apoyado en la base de datos completada y en los modelos validados, sugiera rangos plausibles para los parámetros de nuevas aeronaves, incluyendo configuraciones VTOL.

4. Documentar el funcionamiento interno de cada módulo de manera pedagógica, de forma que otros estudiantes puedan comprender, verificar y extender el sistema sin necesidad de ser especialistas en programación o estadística.

5. Dejar claramente identificados los puntos de conexión del sistema con etapas posteriores del diseño (análisis de prestaciones, aerodinámica, estructuras) y con herramientas como ADRpy, explicitando para un caso de estudio la construcción de un vector de entrada completo y trazable, de modo que el proyecto pueda integrarse en un programa de desarrollo más amplio.

Para mayor claridad, estos objetivos se resumen en la siguiente tabla:

<i>Tipo de objetivo</i>	<i>Descripción</i>
<i>General</i>	Convertir una base de datos incompleta de aeronaves en una herramienta utilizable para apoyar el diseño conceptual de un dron VTOL de ala fija.
<i>Específico</i> 1	Imputar valores faltantes mediante un módulo predictor basado en regresión y reglas físicas simples.
<i>Específico</i> 2	Analizar y visualizar los modelos generados para evaluar su calidad estadística y su coherencia técnica.
<i>Específico</i> 3	Asistir al usuario en la selección de parámetros de diseño para nuevas aeronaves, utilizando la base de datos completada como referencia.
<i>Específico</i> 4	Documentar el sistema con enfoque pedagógico, facilitando su uso y extensión por futuros estudiantes.
<i>Específico</i> 5	Enlazar el sistema con etapas posteriores del diseño y con otras herramientas de cálculo, como ADRpy, mediante la definición y documentación de un vector de entrada completo y trazable para un caso de estudio representativo.

Tabla 8.1- Objetivos del trabajo

Resumen del objetivo general y de los objetivos específicos del proyecto ADRpy-VTOL, organizado por tipo de objetivo para facilitar su lectura y seguimiento en la Introducción.

8.2 Hipótesis

8.2.1 Hipótesis general

El desarrollo de un sistema modular que combine imputación por similitud, modelos de regresión supervisados y herramientas de auditoría estadística permite transformar una base de datos incompleta de UAV en un conjunto de parámetros globales coherentes, trazables y útiles para la toma de decisiones en diseño conceptual, y organizarlos en un vector de entrada completo, compatible con la estructura de ADRpy, para su uso en etapas posteriores de análisis.

8.2.2 Hipótesis específicas

A partir de esta hipótesis general se plantean tres hipótesis específicas, alineadas con los tres grandes objetivos del sistema.

Hipótesis I – Imputación de datos

Mediante la combinación de imputación por similitud y modelos de regresión ajustados y validados estadísticamente, es posible completar una fracción significativa de las celdas vacías de la base de datos de UAV manteniendo una coherencia interna aceptable para su uso en diseño conceptual.

Hipótesis II – Descubrimiento de relaciones entre parámetros

El análisis sistemático de los modelos generados y de sus representaciones gráficas, apoyado por el panel interactivo, permite identificar o redescubrir relaciones funcionales entre parámetros globales de las aeronaves (por ejemplo, entre peso máximo de despegue, superficie alar, carga alar o potencia específica) sin imponer de antemano una forma particular para dichas relaciones.

Hipótesis III – Automatización del flujo de diseño conceptual

La automatización del flujo de trabajo —desde la depuración e imputación de la base de datos, pasando por la selección y auditoría de modelos, hasta la generación de recomendaciones de parámetros globales— permite reducir el tiempo y el esfuerzo cognitivo necesarios en la etapa de diseño conceptual, manteniendo al mismo tiempo la trazabilidad de las decisiones. Al organizar el resultado en un vector de entrada completo y documentado, compatible con ADRpy, el sistema facilita la integración con herramientas de análisis conceptual sin sustituir el juicio del ingeniero.

8.3 Metodología general

La metodología adoptada en esta tesis se articula alrededor del desarrollo y la integración de tres módulos de software. A un nivel global, el proceso puede describirse en las siguientes etapas:

Etapas 1. Relevamiento y estructuración inicial de datos.

Se recopila información de UAV comerciales en el rango de interés a partir de fichas técnicas y otras fuentes públicas. Los datos se integran en una única tabla, unificando unidades y nombres de parámetros. En esta fase se prioriza conservar la información disponible, aun cuando existan numerosas celdas vacías.

Etapas 2. Definición de parámetros derivados y depuración básica.

A partir de los datos originales se calculan parámetros derivados relevantes para diseño conceptual (por ejemplo, carga alar y relación potencia–peso). Se identifican registros evidentemente inconsistentes y se establecen criterios iniciales para tratar valores atípicos o dudosos.

Etapas 3. Desarrollo del Módulo 1: sistema predictor.

Se implementan rutinas para preparar subconjuntos de datos utilizables, normalizar variables y aplicar imputación por similitud entre aeronaves. Sobre estos datos se entrenan modelos de regresión (lineales y polinómicos) con uno, dos o tres predictores, utilizando esquemas de validación como LOOCV y métricas de desempeño (MAPE, MAE, R^2). Los resultados se almacenan en tablas y archivos estructurados (por ejemplo, JSON) que incluyen información suficiente para su análisis posterior.

Etapas 4. Desarrollo del Módulo 2: panel de análisis y visualización.

Se diseña un panel interactivo que lee los modelos y las imputaciones generadas por el Módulo 1. El panel permite filtrar ecuaciones según distintos criterios, visualizar tendencias y dispersiones, identificar outliers y revisar, parámetro por parámetro, el comportamiento de los modelos y de los datos imputados.

Etapas 5. Desarrollo del Módulo 3: asistente de diseño conceptual.

Se reutilizan los modelos considerados aceptables para construir un asistente que, a partir de unos pocos requerimientos iniciales de una aeronave objetivo, sugiera valores coherentes para otros

parámetros globales. Este módulo incorpora verificaciones básicas de consistencia y mecanismos para evidenciar al usuario en qué modelos se está apoyando cada recomendación y, organiza esas recomendaciones en un vector de entrada completo y trazable, compatible con la estructura de ADRpy para análisis conceptual.

Etapas 6. Evaluación integrada y reflexión final.

Se analiza el funcionamiento del flujo completo, desde la tabla original hasta las salidas del asistente de diseño. La evaluación se realiza tanto en términos cuantitativos (calidad de las imputaciones y de los modelos) como cualitativos (grado de organización logrado, facilidad de uso y potencial para apoyar el diseño conceptual). A partir de esta evaluación se discuten los alcances y las posibles extensiones futuras del sistema. Para un caso de estudio se recorre explícitamente este flujo hasta la construcción del vector ADRpy y la generación de una salida ilustrativa típica del diseño conceptual (un diagrama W/S–T/W), sin pretender aún un análisis detallado de prestaciones.

El trabajo se ubica con claridad en la etapa inicial del diseño. Por un lado, aborda el problema de la información disponible: cómo construir y limpiar una base de datos útil de UAV de ala fija. Por otro, desarrolla herramientas que permiten explotar esa base de datos, pero sin pretender reemplazar análisis aerodinámicos o estructurales detallados.

El sistema no incluye todavía:

- diseño de perfiles alares ni optimización geométrica avanzada;
- simulaciones CFD o análisis por elementos finitos;
- construcción ni ensayo de prototipos físicos.

Además, la base de datos actual, con unas decenas de aeronaves y fuerte heterogeneidad en las definiciones, impone limitaciones claras. Las tendencias y modelos que se obtienen deben interpretarse como indicadores y no como leyes definitivas del mercado de UAV. En particular, la falta de verificaciones externas sistemáticas hace que muchas de las conclusiones deban leerse con cautela y siempre bajo la supervisión del criterio del ingeniero.

Aun así, el sistema tiene un valor formativo importante. Permite trabajar con datos reales, enfrentarse a sus imperfecciones y ver cómo conceptos de regresión, métricas de error, análisis dimensional y visualización interactiva se integran en un proyecto de ingeniería concreto. Desde esta perspectiva, la tesis funciona también como un puente entre la formación aeronáutica tradicional y herramientas más recientes de ciencia de datos e inteligencia artificial.

9 MARCO TEÓRICO

El sistema ADRpy-VTOL se apoya en técnicas estadísticas y geométricas para transformar una base de datos aeronáutica (incompleta y heterogénea) en información útil para el diseño conceptual. En particular, el flujo incluye limpieza, control de valores atípicos, imputación de celdas faltantes y evaluación de modelos, incorporando umbrales configurables (“knobs”) que permiten ajustar el nivel de exigencia del análisis.

En este marco teórico se presentan los conceptos necesarios para interpretar esos controles: qué problema resuelven, por qué se eligen, qué ventajas y limitaciones tienen, y cómo se relacionan entre sí. El cálculo detallado se ejecuta de forma automática a través de funciones en Python apoyadas en implementaciones científicas ampliamente utilizadas en ingeniería y ciencia de datos. Por ello, se prioriza una explicación conceptual orientada a la toma de decisiones y a la lectura crítica de resultados, acompañada por referencias numéricas sobre la sensibilidad de los umbrales y por ejemplos aplicados en los capítulos metodológicos.

9.1 Proceso de diseño de aeronaves: visión general

En este apartado se pretende ubicar el trabajo dentro del proceso completo de diseño de una aeronave. No diseñamos un avión “desde cero” en todas sus fases, sino que intervenimos en una parte concreta del camino: el diseño conceptual, en el contexto de aeronaves pequeñas no tripuladas. Para describir ese camino se toman dos referencias principales: el enfoque clásico de Raymer [1] para aeronaves tripuladas y el enfoque de Keane [8] para aeronaves pequeñas de ala fija y flujos de trabajo asistidos por software.

En términos simples, diseñar una aeronave es acompañar una idea desde el nivel de misión (“qué quiero que haga el avión”) hasta el nivel de piezas concretas que se pueden fabricar, probar y operar. A lo largo de ese camino, el diseño pasa por varias fases con niveles de detalle crecientes.

9.1.1 Fases clásicas del diseño de aeronaves

De forma sintética, podemos describir el proceso de diseño de una aeronave como una secuencia de fases:

1. Definición de misión y requisitos

Se describe qué debe hacer la aeronave, en qué entorno y con qué restricciones (reglamentarias, operativas, económicas). Esa descripción se traduce en números: carga útil, alcance, altitud, velocidad, tipo de despegue/aterrizaje, condiciones meteorológicas, etc.

2. Diseño conceptual

A partir de los requisitos, se exploran distintas configuraciones posibles. Se estiman pesos, superficies, potencias y prestaciones con modelos relativamente simples. Se generan alternativas, se comparan y se selecciona una configuración candidata.

3. Diseño preliminar

Se congela la configuración general y se la analiza con más detalle. Se refina el dimensionamiento, se mejora la calidad de los modelos aerodinámicos y estructurales, se realiza el *lofting* (definición geométrica precisa) y se coordinan los trabajos de distintos equipos.

4. Diseño de detalle

Se diseñan piezas concretas: espesores, materiales, uniones, sistemas, cableado. Se cierran tolerancias, procesos de fabricación y documentación técnica.

5. Fabricación, prototipos y ensayos

Se construyen prototipos, se realizan ensayos estructurales y aerodinámicos, pruebas en tierra y vuelos de prueba. Se ajustan modelos y se corrigen defectos.

6. Operación, mantenimiento y mejora

La aeronave entra en servicio. Se ajustan procedimientos, se recopilan datos de operación, se realizan modificaciones y mejoras. En algunos casos se extiende la vida útil, en otros se acelera el retiro.

7. Retirada y reemplazo

Finalmente, la aeronave se retira del servicio y es sustituida por nuevos diseños, que vuelven a recorrer el mismo ciclo.

En la práctica, estas fases no son compartimentos estancos. Hay bucles de realimentación: resultados de ensayos pueden obligar a volver a diseño preliminar, cambios de requisitos pueden hacer replantear el diseño conceptual. Tanto Raymer como Keane insisten en que el diseño de aeronaves es inherentemente iterativo y que las decisiones iniciales influyen en todo lo que viene después [1] [8].

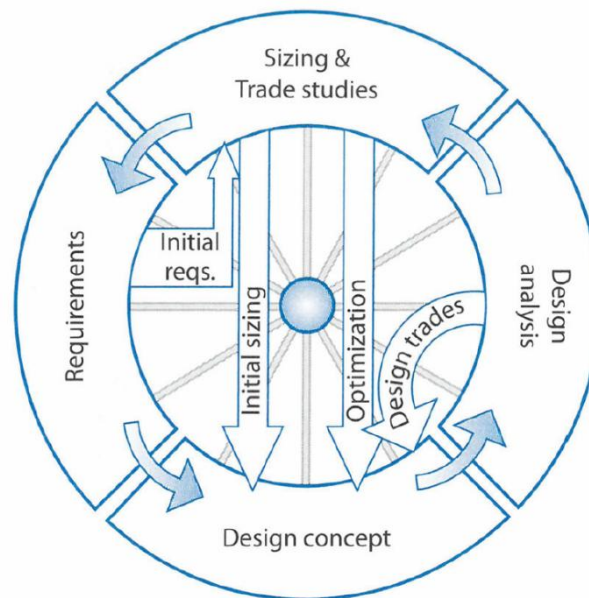


Imagen 9.1 - Etapas del proceso de diseño aeronáutico y alcance de esta tesis

Esquema de las fases del proceso de diseño (conceptual–preliminar–detallado, ensayos, operación) presentado por Raymer y la ubicación del alcance de este trabajo dentro de ese ciclo. [1]

9.1.2 Definición de misión y requisitos

La misión es la forma técnica de expresar el rol del avión. Por ejemplo, transportar una cierta carga útil a una determinada distancia, realizar vigilancia a baja velocidad durante un número de horas, operar desde pistas cortas o desde un buque, etc. A partir de esa descripción cualitativa, se construyen requisitos de diseño cuantitativos:

- masa de carga útil,
- alcance o autonomía,
- altitud y velocidad de crucero,

- perfiles de vuelo típicos,
- condiciones ambientales y regulatorias,
- restricciones de coste y mantenimiento.

Raymer resalta que una misión mal formulada contamina todas las fases posteriores: si subestimamos la carga útil o sobreestimamos el alcance, podemos terminar con un diseño que “es bonito en papel” pero no cumple las necesidades reales. Por eso, los manuales de diseño dedican un esfuerzo considerable a ayudar al diseñador a explicitar correctamente la misión antes de entrar en números [1].

En este caso, el contexto es el de un dron de ala fija con capacidad VTOL, pensado para ciertas misiones de vigilancia y transporte ligero. No desarrollamos esta misión en detalle aquí, pero es el telón de fondo sobre el que se interpretan los datos de mercado y las tendencias que analizamos en la tesis.

9.1.3 Diseño conceptual: descripción general

Una vez definidos los requisitos, entramos en el diseño conceptual. Esta es la fase donde el diseño todavía es “blando”: probamos configuraciones, cambiamos parámetros, comparamos alternativas. El objetivo no es cerrar el avión definitivo, sino responder a una pregunta clave:

¿existe alguna configuración que pueda cumplir la misión de forma técnicamente viable y razonablemente eficiente?

Raymer describe el diseño conceptual como la etapa de exploración y filtrado. Se generan varias alternativas (por ejemplo, configuraciones de ala alta o baja, distintas distribuciones de motores, diferentes geometrías de cola), se estiman pesos, prestaciones y costes con modelos simplificados y se descartan las opciones que no cumplen requisitos o que son claramente menos atractivas que otras [1].

Keane complementa esta visión para aeronaves pequeñas no tripuladas. En su enfoque, el diseño conceptual puede describirse con un conjunto relativamente reducido de parámetros: masa máxima al despegue, carga útil, geometría básica del ala y del fuselaje, perfil de misión, etc. A partir de esos parámetros, se aplican relaciones empíricas y modelos simplificados para estimar pesos parciales, arrastres y requerimientos de potencia, generando diagramas globales como la carga alar frente a la relación empuje-peso [8].

En términos de herramientas, el diseño conceptual se apoya mucho en:

Reglas empíricas basadas en estadística de flotas existentes.

Métodos analíticos simplificados, como los que recopila Raymer para peso y aerodinámica.

Herramientas computacionales ligeras, como hojas de cálculo estructuradas o notebooks tipo ADRpy, que encapsulan esas relaciones y permiten recalcular rápidamente cuando se cambian las entradas.

Es precisamente en esta fase, y en este tipo de herramientas, donde se ubica nuestro trabajo.

9.1.4 Diseño conceptual en detalle y rol de nuestro sistema

Dentro del diseño conceptual, podemos distinguir varias **sub-etapas** que nos interesan especialmente porque ayudan a ubicar el papel de ADRpy y de nuestro sistema ADRpy-VTOL.

9.1.4.1 De la misión a una primera familia de configuraciones

El primer paso es traducir la misión en una **familia de configuraciones plausibles**. Por ejemplo, para un UAV ligero de vigilancia podemos considerar:

- ala fija con cola convencional,
- ala fija con capacidad VTOL basada en rotores adicionales,
- configuraciones más exóticas (canard, alas en tándem, etc.).

En esta etapa se combinan experiencia, bibliografía y, a menudo, intuición de ingeniería. Los manuales sugieren aplicar checklists de configuración para asegurar que no se pasa por alto alguna alternativa evidente [8] [1].

9.1.4.2 Estudios de compromiso (trade studies) y selección de configuración

Una vez identificadas varias opciones, se realizan estudios de compromiso (trade studies): se analiza cómo cambia el rendimiento cuando se modifican parámetros como el alargamiento de ala, la relación de aspecto, la distribución de masas, el número y la ubicación de los motores, entre otros. El objetivo es visualizar compromisos típicos del tipo “más carga útil” versus “más peso” o “mayor velocidad” versus “más consumo”, para entender qué se gana y qué se pierde con cada decisión de configuración.

Raymer y Keane proponen distintas formas de organizar estos estudios, pero la idea central es la misma: usar modelos simplificados para explorar un espacio de diseño amplio y, a partir de esa exploración, quedarse con una configuración candidata que parezca prometedora. En el contexto de este proyecto, este mismo principio se utiliza como guía: facilitar el proceso de exploración temprana con la intención de generar más candidatos prometedores consumiendo menos tiempo y recursos, manteniendo un marco de comparación ordenado entre alternativas. [8]

De todas formas, en el mundo de los UAV esta aproximación puede resultar en parte simplificadora, ya que tiende a apoyarse en configuraciones “tipo” y puede limitar la exploración de arquitecturas realmente nuevas. Además, especialmente en el rango aproximado de 1 a 100 kg, los costos de inversión para probar nuevos conceptos no suelen ser tan elevados como en aeronaves convencionales, lo que habilita iteraciones físicas más rápidas. Sin embargo, una fortaleza importante del programa es que no solo permite estudiar datos empíricos del mercado, sino también realizar variaciones controladas sobre una configuración base: por ejemplo, mantener el resto del diseño constante y variar únicamente la posición de un motor, generar un modelo 3D, registrar resultados (p. ej., CFD o ensayos), construir una tabla de casos y analizar cuantitativamente cómo se correlaciona el desempeño con ese cambio de posición (coordenadas), obteniendo tendencias útiles para guiar la selección de configuración.

9.1.4.3 Estimaciones iniciales de masa, aerodinámica y potencia

Con una configuración candidata definida, el siguiente paso es realizar estimaciones iniciales de, masa máxima al despegue (MTOW) y pesos parciales, coeficientes aerodinámicos en condiciones clave, requerimientos de potencia o empuje en distintas fases de vuelo.

Aquí es donde entran en juego las recopilaciones de fórmulas y gráficos de Raymer, que permiten estimar pesos de estructura, sistemas y combustible a partir de configuraciones y tamaños aproximados, y herramientas como ADRpy, que implementan esas relaciones en forma de código reutilizable [1] [8].

En el caso de ADRpy, el diseñador define parámetros clave en un notebook de Jupyter (masas, geometría básica, perfil de misión), y el código se encarga de encadenar las fórmulas y generar salidas como, estimaciones de pesos, requerimientos de potencia, diagramas de carga alar frente a relación empuje-peso.

9.1.4.4 Uso de datos históricos y del mercado

Un elemento importante, que aparece tanto en Raymer como en Keane, es el uso de datos históricos y de otras aeronaves como referencia. En lugar de confiar solo en modelos analíticos, se contrasta el diseño con lo que ya vuela: familias de aviones existentes, prototipos o, en el caso de UAV, plataformas comerciales.

El trabajo se inserta precisamente aquí. En lugar de utilizar únicamente bases de datos “limpias” de aeronaves clásicas, tomamos como referencia un conjunto de drones del mercado actual, cuyas fichas técnicas son heterogéneas e incompletas. Nuestro sistema, reúne esos datos dispersos en una tabla, imputa valores faltantes con métodos de similitud y correlación, analiza la consistencia de los modelos estadísticos resultantes y ofrece una base de datos mejorada desde la cual se pueden extraer relaciones entre masa, carga útil, autonomía, velocidad, etc.

El objetivo es que el diseñador no elija los valores de entrada para ADRpy “a ciegas”, sino apoyándose en tendencias observadas en el mercado, filtradas y verificadas con criterios estadísticos y físicos. En ese sentido, nuestro sistema es una capa previa al uso de herramientas como ADRpy: prepara datos y tendencias que alimentan el dimensionamiento conceptual y facilitan las primeras iteraciones del diseño.

9.1.4.5 Diagramas de restricciones y zona plausible de diseño

Finalmente, con las estimaciones de masa, aerodinámica y potencia, se construyen diagramas de restricciones: típicamente, gráficos de carga alar (S/W) frente a relación empuje-peso (P/W), donde se representan requisitos de despegue, ascenso, crucero, etc. La intersección de esas restricciones define una zona plausible de diseño, donde la aeronave puede operar cumpliendo todas las condiciones impuestas [8] [1].

En la filosofía Keane–ADRpy, estos diagramas se generan de forma automatizada a partir de las entradas del notebook. En esta tesis, no modificamos directamente la lógica de estos diagramas, pero sí trabajamos en lo que viene antes: la definición de esos parámetros de entrada a partir de datos reales de mercado.

9.1.5 Diseño preliminar

Una vez que el diseño conceptual ha identificado una configuración viable y una zona plausible en el espacio S/W – P/W , se pasa al diseño preliminar. Aquí ya no se cambian grandes rasgos del avión cada día: se congela la configuración global y se empieza a refinarla.

En esta fase, se mejora la calidad de los modelos aerodinámicos y estructurales, se realiza el *lofting* y se define con precisión la geometría externa, se evalúan cargas estructurales con mayor detalle, se coordinan los trabajos de diferentes especialidades (aerodinámica, estructuras, sistemas, costes).

El diseño preliminar puede involucrar herramientas más pesadas, como CFD y análisis por elementos finitos, y consume mucho más tiempo de ingeniería que el diseño conceptual.

Nuestra tesis no entra en este nivel de detalle, pero quiere alimentarlo mejor: al proponer parámetros de entrada coherentes y defendibles para un dron VTOL, facilitamos que el trabajo de diseño conceptual se haga sobre una base de datos más sólida.

9.1.6 Diseño de detalle, fabricación y ensayos

En el diseño de detalle, la aeronave deja de ser un conjunto de volúmenes y líneas principales para convertirse en un conjunto de piezas concretas, se eligen materiales específicos, se definen espesores,

rigidizadores, uniones, fijaciones, se diseñan sistemas (eléctrico, combustible, control, aviónica) a nivel de componentes y cableados, se generan planos de fabricación y modelos 3D detallados.

A continuación, en la etapa de fabricación y ensayos, se construyen prototipos, se realizan ensayos estructurales (por ejemplo, pruebas de carga en alas), ensayos en tierra (rodaje, sistemas) y vuelos de prueba. Los resultados pueden retroalimentar el diseño preliminar o de detalle, corrigiendo errores o afinando márgenes [1].

Estas fases están fuera del alcance directo de nuestro trabajo. Sin embargo, es importante recordarlas para entender que el diseño conceptual que apoyamos con nuestro sistema es solo el primer peldaño de una escalera mucho más larga.

9.1.7 Operación, mantenimiento y retirada

Tras la certificación y entrega, la aeronave entra en fase de operación, donde aparecen nuevas fuentes de información, datos de consumo real, tasas de fallo de sistemas, costes de mantenimiento, restricciones operativas no previstas.

Esta información suele conducir a modificaciones (Service Bulletins, mejoras de versiones, cambios en manuales) y, a la larga, a decisiones de retirada, reemplazo o desarrollo de nuevas generaciones. Muchos diseños futuros se apoyan en estadísticas de flotas anteriores, cerrando así el ciclo de uso de datos históricos en nuevas fases conceptuales [8].

9.1.8 Ubicación de nuestro trabajo dentro del proceso global

Con este panorama, podemos ubicar con precisión nuestra tesis:

Trabajamos dentro de la fase de diseño conceptual, más específicamente en la sub-etapa de uso de datos históricos/mercado para fijar parámetros de diseño iniciales.

Nuestro sistema se apoya en la filosofía de flujos automatizados que proponen Keane y herramientas como ADRpy, pero se coloca antes: se encarga de limpiar, completar y explotar una base de datos de drones existentes para que los valores de entrada a esos flujos no sean arbitrarios, sino coherentes con el mercado.

La fase 2, posterior a ésta tesis, se sitúa en el paso siguiente: adaptar ADRpy para un dron VTOL concreto y explotar la zona plausible de diseño; es decir, entra más directamente en el uso de herramientas de diseño conceptual/preliminar sobre un caso específico.

En resumen, nuestra contribución no es diseñar todo el avión ni sustituir a herramientas como ADRpy, sino mejorar el tramo del proceso en el que se conectan datos reales, métodos estadísticos y decisiones de diseño conceptual. Ese es el lugar exacto que ocupa el sistema ADRpy-VTOL dentro del proceso de diseño descrito por Raymer y Keane.

9.2 Correlación entre variables

Esta técnica se basa en la premisa fundamental de la estadística: si dos variables están fuertemente correlacionadas, es posible estimar una a partir de la otra. En nuestro caso, buscamos explotar esta propiedad para completar automáticamente valores faltantes en parámetros críticos del diseño, como la

autonomía, el peso vacío o el consumo, utilizando como predictores otros parámetros conocidos (por ejemplo, MTOW, capacidad de combustible o área alar).

9.2.1 ¿Qué es una correlación?

En estadística, correlación describe el grado en que dos variables están relacionadas. Cuando decimos que dos parámetros están correlacionados, nos referimos a que existe un patrón de comportamiento conjunto: cuando uno varía, el otro tiende a variar de manera predecible. Esta relación puede adoptar diversas formas:

Lineal: una variable aumenta o disminuye proporcionalmente a la otra (por ejemplo, $y = mx + b$).

Polinómica: la relación sigue una curva, como una parábola de segundo grado ($y = ax^2 + bx + c$).

Exponencial o logarítmica: relaciones donde el ritmo de cambio se acelera o desacelera progresivamente.

No determinística: relaciones con estructura aparente pero no predecible de forma funcional simple.

En este estudio comenzamos aplicando dos:

1. **Lineal:** por su simplicidad, interpretabilidad y aplicación directa en modelos de regresión básicos.

2. **Polinómica de segundo grado:** para capturar relaciones más complejas sin comprometer la eficiencia computacional o la robustez del modelo.

Pero una vez que nos dimos cuenta que debíamos restringir la polinómica de manera considerable, decidimos sumar otros 3, para brindarle un punto medio de representación entre la rigidez de la lineal y la flexibilidad de la polinómica:

3. **Exponencial.**

4. **Potencia.**

5. **Logarítmica.**

Esta elección se basa en un criterio de relación costo-beneficio, ya que estos cinco enfoques cubren un amplio espectro de relaciones observables en ingeniería sin requerir herramientas estadísticas avanzadas ni grandes volúmenes de datos.

La incorporación de modelos polinómicos de segundo grado sin duda agrega complejidad al desarrollo computacional y al diseño del algoritmo, pero aporta beneficios significativos desde el punto de vista predictivo. No solo amplía nuestra capacidad para modelar relaciones no lineales presentes en los datos, sino que también contribuye a una comprensión más profunda de la estructura de las variables.

9.2.2 Modelos de regresión y su comportamiento frente a los datos

En ingeniería y ciencia de datos, el uso de modelos matemáticos para representar fenómenos reales es una práctica común. Cuando disponemos de un conjunto de observaciones (por ejemplo, características técnicas de aeronaves y sus respectivas prestaciones), es habitual buscar una relación funcional que permita estimar o predecir una variable objetivo a partir de una o más variables conocidas. Esta técnica se conoce como regresión.

El proceso de regresión consiste en encontrar una ecuación matemática que se aproxime lo mejor posible a los datos observados. Sin embargo, lograr esa aproximación no es trivial. Si el modelo es demasiado simple, no logrará captar la tendencia real de los datos. Si es demasiado complejo, se ajustará tan fuertemente a los datos conocidos que perderá la capacidad de predecir correctamente nuevos casos. Esto da lugar a tres escenarios típicos: **underfitting**, **ajuste adecuado/Good Fit** y **overfitting**.



Imagen 9.2 - Subajuste, ajuste adecuado y sobreajuste

Comparación visual de los tres escenarios de ajuste de un modelo sobre datos (underfitting, ajuste adecuado y overfitting) para introducir el problema sesgo/varianza en imputación. [9]

Underfitting (subajuste)

El primer caso se denomina underfitting o subajuste. Ocurre cuando el modelo elegido es tan simple que no logra capturar la forma real de los datos. En la **Imagen 9.2**, se observa una línea recta que intenta representar una clara tendencia curva. Como resultado, muchos puntos quedan lejos de la línea. El modelo tiene errores grandes y sistemáticos.

Esto suele pasar cuando se usa una regresión lineal (una recta) para un conjunto de datos que claramente sigue una relación no lineal. El modelo falla tanto en los datos conocidos como en los nuevos.

Good Fit (ajuste adecuado)

En el segundo gráfico se presenta un modelo con ajuste adecuado. La curva representa fielmente la tendencia general de los datos sin intentar pasar exactamente por todos los puntos. Algunos puntos quedan levemente por encima o por debajo, lo cual es normal, ya que los datos reales suelen tener cierta variabilidad o “ruido”.

Este tipo de modelo logra un buen balance: es lo suficientemente flexible como para capturar la forma real de la relación, pero no se deja llevar por fluctuaciones menores. Es un modelo robusto y generalizable, útil para estimar tanto los datos conocidos como nuevos casos similares.

Overfitting (sobreajuste)

El tercer gráfico muestra el caso de overfitting o sobreajuste. Aquí, el modelo es tan complejo que intenta pasar exactamente por cada punto observado. El resultado es una curva muy irregular, con múltiples subidas y bajadas que no responden a una tendencia lógica.

Esto ocurre cuando el modelo tiene demasiados grados de libertad (por ejemplo, muchos términos polinómicos) y se entrena con pocos datos. El modelo “memoriza” los datos de entrenamiento y se ajusta

incluso al ruido o a los errores de medición. Aunque en los datos originales parece tener un error casi nulo, su desempeño con nuevos datos suele ser muy pobre.

Una vez comprendido por qué un modelo demasiado complejo puede ajustarse en exceso a los datos (overfitting), surge una pregunta clave:

¿Cómo saber si tengo suficientes datos para ajustar un modelo de forma segura?

La respuesta está en la estructura matemática de la regresión.

9.2.3 El sistema de ecuaciones que representa una regresión

Cuando ajustamos un modelo de regresión, lo que hacemos es resolver un sistema de ecuaciones lineales. Si usamos por ejemplo una regresión polinómica, cada observación (es decir, cada fila de datos) aporta una ecuación. Los términos del modelo (como $x_1, x_2, x_1^2, x_1x_2, x_2^2$, etc.) son las incógnitas del sistema, es decir, los valores que se quiere estimar (los coeficientes).

Ese sistema puede escribirse así:

$$X \cdot \beta = y$$

Donde:

- X es la **matriz de diseño**, que contiene los valores de los predictores y sus combinaciones polinómicas.
- β es el **vector de coeficientes** que el modelo debe calcular.
- y es el vector de resultados conocidos (por ejemplo, payload).

El sistema tiene solución única solamente si hay más ecuaciones (filas) que incógnitas (columnas). Si hay menos filas que columnas, el sistema tiene infinitas soluciones posibles: no puede determinar con certeza los coeficientes del modelo, y cualquier resultado que obtengamos será numéricamente inestable. En otras palabras si se quiere calcular 5 incógnitas, pero solo se tiene 3 datos disponibles, estamos pidiendo que el modelo “invente” cómo llenar los huecos. Y lo hará, pero no será una solución confiable.

9.2.4 Cantidad de coeficientes en función del grado y número de predictores

A medida que agregamos más predictores, o aumentamos el grado del polinomio, el modelo genera más y más términos. Cada uno de esos términos tiene su propio coeficiente. Por ejemplo, un modelo polinómico de segundo grado con dos variables (x_1 y x_2) incluye los siguientes:

$$\text{Términos: } 1, x_1, x_2, x_1^2, x_1x_2, x_2^2 \Rightarrow 6 \text{ coeficientes}$$

La fórmula general que te permite calcular cuántos términos va a tener un modelo de regresión polinómica es:

$$p = \binom{d + g}{g}$$

Donde:

- d = número de predictores (variables independientes)
- g = grado del polinomio
- p = número total de parámetros (coeficientes)

Esta es una fórmula de combinatoria: te dice cuántas formas hay de combinar los predictores y sus potencias hasta un cierto grado.

Ejemplos para nuestros casos:

<i>Tipo de modelo</i>	<i>d</i>	<i>g</i>	$p = \binom{d + g}{g}$
<i>Lineal 1 predictor</i>	1	1	2
<i>Lineal 2 predictores</i>	2	1	3
<i>Polinómico 1 predictor (grado 2)</i>	1	2	3
<i>Polinómico 2 predictores (grado 2)</i>	2	2	6

Tabla 9.1- Tabla de cálculo de coeficientes mínimos por tipo de función

La tabla indica cuántos coeficientes debe estimar el modelo según el grado y la cantidad de predictores, y se utiliza para definir el mínimo de datos necesarios para ajustar cada combinación sin que el problema quede subdeterminado.

Para que el modelo tenga una solución matemática válida y no caiga en sobreajuste, se necesita:

$$n > p$$

Es decir: el número de datos válidos debe ser mayor que la cantidad de coeficientes (β_i) que el modelo intenta ajustar.

Si se cumple esta condición, el sistema está **sobredeterminado** y puede encontrar una única solución óptima. Si no se cumple, el sistema está **subdeterminado**, y cualquier resultado será, en el mejor de los casos, una “memoria” de los datos, no un modelo que explique una tendencia real.

Los conceptos de sobredeterminación, overfitting y requisitos mínimos de datos no son solo consideraciones teóricas, sino que tienen un impacto directo en la arquitectura del sistema de imputación desarrollado en este trabajo. Recordemos que el sistema tiene como objetivo predecir con la mayor precisión posible valores faltantes en una tabla de datos aeronáuticos, utilizando diferentes combinaciones de variables técnicas disponibles.

Cada vez que el sistema detecta una celda vacía, activa un proceso de búsqueda del modelo más adecuado para imputarla. Este proceso consiste en probar múltiples combinaciones de predictores (entre uno y dos, según disponibilidad), junto con cinco formas funcionales: modelos lineales y modelos polinómicos de segundo grado, potencia, exponencial y logarítmica. Para cada una de estas combinaciones, el sistema entrena un modelo específico y evalúa su capacidad predictiva.

Con esto en mente es que se diseñó el sistema para que antes de calcular cualquier coeficiente o evaluar la precisión de un modelo, el sistema realizara una validación de datos válidos suficientes para poder entrenar ese modelo.

El mecanismo es el siguiente:

1. Se define el tipo de modelo que se quiere evaluar (lineal o polinómico).
2. Se selecciona un subconjunto de predictores (por ejemplo, MTOW y Potencia).
3. Se limpia el dataframe inicial para el caso correspondiente (eso incluye eliminar todas las filas que no tengan reciprocidad de datos en los parámetros o el parámetro seleccionado en conjunto con el parámetro objetivo)
4. Se calcula cuántos parámetros requiere ese modelo, según la fórmula combinatoria.
5. Se contabiliza la cantidad de filas disponibles de datos observados para la tabla solo con filas validas.
6. Se verifica que la cantidad de datos válidos n sea estrictamente mayor que la cantidad de parámetros p (siendo éste parámetro un valor configurable en el panel).

Solo si esa condición se cumple, el modelo se entrena. De lo contrario, la combinación se descarta automáticamente y el sistema pasa a la siguiente.

Este paso actúa como un filtro lógico que garantiza que ningún modelo se entrene con menos información de la que necesita. En términos prácticos, evita que el sistema memorice puntos aislados o genere ecuaciones sin sentido físico por falta de datos.

Gracias a este filtro, el sistema puede adaptarse dinámicamente a la calidad de los datos disponibles. Por ejemplo: si para un parámetro determinado existen solo tres aeronaves con valores válidos, no se podrá calcular ningún modelo polinómico de segundo grado con dos predictores (ya que requiere al menos 7 datos); si hay cuatro datos válidos, sí se podrá ajustar un modelo lineal con dos predictores, pero no uno polinómico; a su vez se agregó condicionales como puntos o valores únicos para cubrirme en el espectro y no caer en falsas representaciones; si hay siete o más datos válidos, todas las combinaciones serán consideradas y comparadas, y se elegirá la que produzca mejores métricas (MAPE, R^2 , correlación y confianza).

De esta forma, el sistema se vuelve flexible y robusto, ajustando la complejidad del modelo al nivel de información disponible, sin intervención manual.

Cabe destacar que este enfoque no es exclusivo para los modelos polinómicos. También se aplica a las regresiones lineales: aunque sean más simples, requieren al menos tantos datos como parámetros más uno. El criterio es exactamente el mismo.

9.2.5 Evaluación de modelos mediante validación cruzada

Hasta aquí hemos discutido cómo se seleccionan y ajustan los modelos de imputación para predecir datos faltantes a partir de combinaciones de variables técnicas. Sin embargo, que un modelo se ajuste

bien a los datos disponibles no garantiza necesariamente que sea fiable. Por ejemplo, un modelo puede ajustarse casi perfectamente a los puntos de entrenamiento y, aun así, fallar al generalizar su comportamiento a nuevos casos. Por eso, evaluar la capacidad real de predicción de cada modelo se vuelve fundamental, sobre todo si los datos disponibles son escasos o están influenciados por casos extremos.

A continuación, explicaremos por qué se implementa un proceso de validación interna dentro del sistema, cómo se realiza técnicamente y qué información aporta para la selección del mejor modelo de imputación.

9.2.5.1 Diferencia entre ajuste y capacidad predictiva

Cuando se entrena un modelo, se optimizan los coeficientes de una ecuación para que el error respecto a los datos conocidos sea mínimo. Este error suele medirse con métricas como el MAPE (Error Porcentual Absoluto Medio) o el coeficiente de determinación R^2 .

Estas métricas, cuando se calculan sobre los mismos datos usados para entrenar el modelo, reflejan el *ajuste*. Sin embargo, el verdadero interés del sistema no está en qué tan bien el modelo reproduce datos que ya conoce, sino en qué tan bien puede estimar valores que no ha visto aún. A esta propiedad se le llama capacidad de generalización o calidad predictiva, y se evalúa mediante un proceso conocido como validación cruzada.

En el contexto del sistema de imputación, los conjuntos de datos disponibles para cada combinación de predictores suelen ser pequeños: a veces se dispone de 4, 5 o 7 aeronaves válidas para entrenar un modelo. En esos casos, dividir el conjunto en entrenamiento y prueba (como en un 70/30 %) no es práctico: se perdería demasiada información útil.

Por eso, se utiliza una técnica llamada LOOCV (Leave-One-Out Cross-Validation), o validación cruzada dejando uno fuera. Esta técnica permite aprovechar al máximo cada dato disponible sin comprometer la independencia de la evaluación.

¿Cómo funciona LOOCV?

El principio es simple:

1. Si hay n datos válidos, el sistema entrena el modelo n veces.
2. En cada iteración, se deja fuera un solo dato (en nuestro caso eso significa quitar una aeronave).
3. El modelo se entrena con los $n - 1$ restantes y luego se usa para predecir el dato que fue excluido.
4. Se calcula el error de predicción para esa única instancia.
5. Al final, se obtienen n errores individuales, uno por cada predicción "ciega".
6. A partir de esos errores se calcula el MAPE validado, que representa un indicador objetivo de qué tan bien se comporta el modelo ante datos nuevos.

Este procedimiento, aunque más costoso computacionalmente que un simple entrenamiento, es especialmente valioso cuando se trabaja con muestras pequeñas y cuando se desea evitar la sobreconfianza que pueden producir los errores entrenados.

Uno de los aspectos más informativos del sistema es la comparación directa entre:

- El **MAPE de entrenamiento**: qué tan bien el modelo ajusta los datos conocidos.
- El **MAPE de validación (LOOCV)**: qué tan bien predice valores que no ha visto durante el entrenamiento.

Esta diferencia revela si el modelo está sobreajustando (overfitting) los datos o si realmente ha aprendido una relación general válida. Por ejemplo:

- Si el MAPE entrenado es bajo (2 %) y el validado también es bajo (3 %), el modelo es confiable.
- Si el MAPE entrenado es bajo (1 %) pero el validado es alto (20 %), el modelo es frágil y probablemente esté memorizando los datos en lugar de aprender una estructura útil.

Con esta comparación, el sistema puede no solo identificar el modelo con menor error entrenado, sino también penalizar aquellos cuya validación muestre poca robustez.

9.2.6 El rol de los valores extremos (*outliers*) en modelos físicos y estadísticos

9.2.6.1 *Tratamiento de valores extremos (outliers)*

Cuando se trabaja con conjuntos de datos técnicos o experimentales es habitual encontrarse con valores que se alejan considerablemente de la mayoría. A estos casos se los conoce como *outliers* o valores atípicos. En muchos análisis estadísticos tradicionales los outliers son motivo de preocupación, porque pueden distorsionar promedios, aumentar errores y alterar resultados. Sin embargo, la relevancia o el riesgo que implican depende del tipo de modelo que se construye y del objetivo del análisis.

9.2.6.2 *¿Qué es un outlier?*

En términos simples, un outlier es un valor que se aleja de forma marcada del resto de los datos. Esa desviación puede verse con respecto a la media, a una tendencia general o a un rango esperado.

Por ejemplo, si en una tabla de aeronaves casi todas tienen una potencia entre 5 y 30 hp y aparece una con 300 hp, ese valor es claramente atípico dentro de ese conjunto. Lo mismo ocurriría con una autonomía de 40 h en una base donde la mayoría oscila entre 1 y 6 h.

Existen distintos métodos para detectar outliers univariantes, como el criterio de Tukey (basado en cuartiles e IQR) o los análisis de tipo z-score. Pero en este trabajo lo central no es solo *cómo* se detecta un outlier, sino *qué se hace con él* y cómo se integra esa decisión al flujo de imputación y selección de modelos.

9.2.6.3 *Modelos físicos deterministas y modelos estadísticos empíricos*

En modelos físicos deterministas (aerodinámica, estructuras, cargas, etc.) se parte de leyes naturales bien establecidas. En ese contexto, un outlier suele indicar un error de medición, una condición experimental anómala o un fallo en la captura de datos. Se tiende a corregir o descartar esos valores, porque pueden sesgar la estimación de constantes o hacer que el modelo aparente violar leyes físicas. La interpretación se centra en la veracidad del fenómeno: un solo punto erróneo puede arruinar una recta físicamente significativa.

En modelos estadísticos empíricos, como el sistema de imputación utilizado aquí, el objetivo no es derivar una ley universal sino representar el comportamiento observado en un conjunto de diseños reales. En este caso, un outlier no es necesariamente un error. Puede ser un diseño poco frecuente pero técnicamente válido (por ejemplo, un UAV experimental con geometría no convencional). Eliminarlo sin analizarlo puede implicar perder parte de la diversidad real del mercado. Incluirllo, con el tratamiento adecuado, permite construir un espacio estadístico más amplio y realista.

Dicho de otra forma, si una aeronave real presenta parámetros por fuera de la tendencia general pero la ficha técnica es consistente, ese punto es evidencia de que esa combinación de parámetros es posible, aunque sea minoritaria.

9.2.6.4 Papel de los outliers en el sistema de imputación

El sistema desarrollado en esta tesis no pretende crear una aeronave en tiempo real, sino proponer estimaciones razonables para parámetros faltantes y para vectores de diseño conceptual, a partir de tendencias observadas en una base de datos de UAV. Las predicciones funcionan como valores iniciales plausibles, que luego deberán ser revisados y afinados en modelos aerodinámicos, estructurales y de performance más detallados.

Desde esa perspectiva, los outliers cumplen varios roles:

- Ayudan a delimitar el rango realista de valores posibles (si los datos son veraces).
- Impiden que el sistema se restrinja artificialmente a un “promedio estrecho” y pierda soluciones no convencionales.
- Aportan diversidad a las combinaciones de predictores, permitiendo que el sistema sugiera diseños cercanos a aeronaves poco comunes pero reales.

Al mismo tiempo, un outlier puede desviar un ajuste si se lo trata como un dato más. Por eso el sistema combina tres ideas:

1. Diagnóstico y visualización de outliers por columna (para que el usuario vea qué hay en la base).
2. Ponderación de puntos alejados al entrenar las regresiones (para que influyan menos si son sospechosos).
3. Validación cruzada (LOOCV) para confirmar si un modelo se sostiene o no en presencia de esos valores.

Si un modelo se ve afectado negativamente por uno o varios outliers, la degradación se manifiesta en las métricas de validación (por ejemplo, MAPE validado) y ese modelo se descarta en la etapa de selección, aunque los datos extremos sigan estando presentes en la base.

9.2.6.5 Medidas robustas y z-score: cómo se cuantifica “qué tan raro” es un punto

Para que los umbrales de outliers sean configurables de manera coherente es necesario trabajar con una medida de “rareza” que no dependa de las unidades de la variable. En lugar de preguntar “¿cuántos kg por encima de la mediana estoy?” o “¿cuántos m/s por encima?”, el sistema utiliza un z-score robusto que responde a una pregunta más general:

¿Cuántas dispersiones típicas me estoy alejando del valor central de la columna?

El procedimiento es:

1. Se toma como centro la **mediana** de la columna, porque es menos sensible a valores extremos que la media.
2. Se calcula la **MAD** (Median Absolute Deviation), que es la mediana de $|x - \text{mediana}|$.
3. Se escala la MAD por una constante 1,4826, lo que permite que esta medida sea comparable a una desviación estándar en el caso de distribuciones aproximadamente normales. Este factor proviene de la teoría de estadística robusta y no se elige de forma arbitraria.
4. El z-score robusto de un valor x queda, de manera aproximada, como:

$$z = \frac{|x - \text{mediana}|}{1,4826 \cdot MAD}$$

El numerador y el denominador están en las mismas unidades, por lo que z es adimensional. Así, z puede interpretarse como “cuántas MAD ajustadas” se aleja un dato del centro. Valores de z cercanos a 0 indican datos muy típicos; valores crecientes indican alejamiento progresivo del comportamiento central.

Ejemplo conceptual: si en una columna de MTOW la mediana está alrededor de 20 kg y la dispersión típica robusta equivale a unos 3 kg, un valor cercano a 29 kg estará aproximadamente a 3 unidades típicas del centro ($z \approx 3$), y uno cercano a 38 kg estará alrededor de 6 unidades típicas ($z \approx 6$), independientemente de que la variable esté expresada en kg, N o en otra unidad consistente.

9.2.6.6 Umbral suave, umbral duro y ponderación de outliers

Una vez que cada valor tiene asignado un z-score, el sistema compara ese z contra dos umbrales.

Un **umbral suave**, a partir del cual un dato se considera “sospechoso” y su influencia en el ajuste comienza a reducirse;

Un **umbral duro**, a partir del cual el dato se considera extremadamente alejado y puede ser candidato a exclusión si la configuración lo permite.

Estos umbrales no tienen un máximo teórico, porque z puede crecer sin límite cuando un valor está muy lejos del resto. Lo que importa es que, en la práctica, valores razonables para un conjunto de datos aeronáuticos heterogéneo suelen estar en rangos como:

umbral suave: aproximadamente entre 2,5 y 3,5;

umbral duro: aproximadamente entre 5 y 7.

Por debajo de estos rangos, el sistema se vuelve muy estricto y empieza a tratar como problemáticos datos que son simplemente menos frecuentes. Muy por encima de estos rangos, se vuelve demasiado permisivo y deja de reaccionar ante errores claros de unidad o de transcripción.

Además de marcar puntos extremos, el sistema utiliza un esquema de ponderación. Cada fila recibe un peso w que disminuye cuando el z-score crece. Conceptualmente, un punto con z grande no “desaparece”, pero su peso en la regresión es menor. La forma general del peso es decreciente en z y se acota entre un valor mínimo $w_{\min}$ y 1. De esta manera, el ajuste deja de tratar todos los puntos por igual y pasa a ser más robusto frente a valores muy alejados.

En términos prácticos, esto significa que un outlier suave (cercano al umbral) todavía influye, pero menos que un punto típico; un outlier muy fuerte, si no fue eliminado, aporta solo una fracción de lo que aportaría un dato central. El efecto se ve reflejado en:

- los coeficientes de los modelos ajustados,
- las métricas de error (por ejemplo MAE y MAPE),
- y el ranking final de modelos candidatos que el sistema retiene.

Así, el tratamiento no es “todo o nada”: existe un gradiente de influencia que depende de cuán lejos se encuentre el dato del cuerpo principal del conjunto.

9.2.6.7 Criterio IQR y su integración con el z-score

El criterio basado en el **IQR** (Interquartile Range) ofrece otra manera robusta de identificar valores atípicos, especialmente útil cuando la distribución es asimétrica o tiene colas largas. Se basa en los cuartiles:

- $Q1$: valor por debajo del cual se encuentra el 25 % de los datos,
- $Q3$: valor por debajo del cual se encuentra el 75 %,
- $IQR = Q3 - Q1$: ancho del 50 % central de la distribución.

Con la regla habitual, se consideran atípicos los valores que están por debajo de

$$Q1 - 1,5 \cdot IQR$$

o por encima de

$$Q3 + 1,5 \cdot IQR$$

En el contexto de esta tesis, el criterio IQR se utiliza como complemento del z-score por varias razones:

- permite detectar valores raros aun cuando el z-score no sea muy grande, en columnas con fuerte asimetría;
- es independiente de suposición de normalidad;
- facilita la visualización mediante boxplots y rangos típicos para cada parámetro aeronáutico.

En el módulo de exploración de datos, el IQR se usa para generar resúmenes y gráficos que ayudan a identificar posibles errores de unidad o registros claramente inconsistentes. Durante el entrenamiento de modelos, el criterio IQR puede emplearse como condición adicional para marcar outliers suaves, de modo que ciertos puntos queden sujetos a ponderación, aunque su z-score no fuese extremo.

La decisión de eliminar filas se toma en función de los umbrales definidos para outliers duros y de la necesidad de conservar un tamaño mínimo de muestra, de modo que el criterio IQR no actúa como “borrador automático” por sí solo, sino como una capa extra de evidencia para el usuario y para el sistema.

9.2.6.8 Secuencia de aplicación en el flujo estadístico

El tratamiento de valores extremos en ADRpy-VTOL se integra al flujo de trabajo de la siguiente manera:

1. En la fase de análisis exploratorio, se calculan para cada columna estadísticas básicas (mínimo, máximo, mediana, percentiles) y criterios de outliers basados en z-score robusto e IQR. Esto permite al usuario revisar histogramas, boxplots y tablas, y detectar errores groseros o valores sospechosos antes de entrenar modelos.

2. En la fase de entrenamiento de modelos de imputación por correlación, cada combinación de predictores se ajusta utilizando pesos derivados del z-score: los puntos muy alejados tienen menor peso en la regresión, y los que superan el umbral duro pueden ser excluidos si la configuración así lo establece.

3. La validez de cada modelo se evalúa después mediante validación cruzada (por ejemplo, LOOCV). Si la presencia de outliers —aunque hayan sido ponderados— genera un comportamiento inestable, esto se refleja en métricas de error elevadas y el modelo se descarta en la etapa de selección comparativa.

De este modo, el tratamiento de outliers no se limita a “limpiar la tabla”, sino que acompaña todo el proceso: ayuda a interpretar la base de datos, a estabilizar los ajustes estadísticos y a filtrar modelos que, aun siendo matemáticamente aceptables, se apoyan excesivamente en puntos poco representativos del comportamiento global de la flota.

9.2.7 Correlaciones multivariantes

Hasta ahora mencionamos correlaciones entre pares de variables, pero en la práctica, los fenómenos complejos (como el comportamiento de una aeronave) rara vez dependen de un solo parámetro. Por eso, además del estudio de correlaciones simples, se incorporó el análisis de correlaciones multivariantes, en donde se combinan dos o más predictores para explicar una misma variable objetivo.

Esta estrategia permite detectar efectos combinados entre variables que individualmente no muestran una fuerte correlación, pero que en conjunto sí lo hacen. Por ejemplo, el peso máximo al despegue (MTOW) podría no estar fuertemente correlacionado ni con la envergadura ni con la potencia por separado, pero sí con una combinación de ambas.

En este trabajo, se analizaron combinaciones lineales y polinómicas de hasta dos variables predictoras. Esto implica generar modelos de regresión del tipo:

- Lineal múltiple:

$$y = \beta_0 + \beta_1 x_1 + \beta_2 x_2 + \beta_3 x_3$$

- Polinómico de segundo grado (con interacción):

$$y = \beta_0 + \beta_1 x_1 + \beta_2 x_2 + \beta_3 x_1^2 + \beta_4 x_1 x_2 + \beta_5 x_2^2$$

Estas combinaciones no solo enriquecen el análisis, sino que también permiten construir modelos más precisos y robustos, especialmente cuando se busca imputar valores faltantes en entornos donde las dependencias entre variables no son triviales.

9.2.8 ¿Qué es la multicolinealidad y por qué importa?

En un sentido estadístico, la multicolinealidad aparece cuando dos o más predictores de un modelo de regresión están fuertemente correlacionados entre sí. En esa situación el modelo tiene dificultades

para asignar de forma clara el “peso” correspondiente a cada variable, porque varias de ellas contienen información muy similar.

Un ejemplo simple ayuda a visualizarlo: si el MTOW y el área alar estuvieran casi perfectamente correlacionados y ambos predijeran muy bien la carga útil, la regresión podría repartir el efecto entre los dos predictores de muchas maneras distintas (por ejemplo, $\beta_1 = 0,10$ y $\beta_2 = 1,0$, o $\beta_1 = 0,15$ y $\beta_2 = 0,5$) sin cambiar demasiado la capacidad predictiva global. Esto genera coeficientes inestables, de magnitudes erráticas o incluso con signos poco intuitivos.

Al inicio de este trabajo podría parecer que la multicolinealidad no es un problema grave, porque el objetivo principal es obtener buenas imputaciones y no interpretar físicamente los coeficientes. Sin embargo, en la práctica se observó que:

- combinaciones muy colineales favorecen extrapolaciones fuertes en algunas gráficas 3D,
- pequeños cambios en los datos pueden producir variaciones grandes en los coeficientes,
- la estabilidad de las imputaciones fuera del rango central se resiente.

Por eso, aunque la tesis no busque “derivar fórmulas de ingeniería” a partir de los coeficientes, sí necesita modelos estables y razonables, y eso obliga a controlar la multicolinealidad.

9.2.8.1 Herramientas teóricas para detectar multicolinealidad

Para analizar la redundancia entre predictores se usan varias herramientas complementarias:

Correlación lineal entre predictores (coeficiente de Pearson $|r|$): detecta pares de variables que prácticamente se mueven juntas.

Factor de Inflación de la Varianza (VIF): mide cuánto aumenta la varianza de un coeficiente por la colinealidad con los otros predictores.

Número de condición de la matriz de diseño: evalúa la estabilidad numérica global del sistema de ecuaciones que define la regresión.

En un modelo puramente lineal, una receta clásica sería:

1. Evaluar la correlación de cada predictor con la variable objetivo.
2. Calcular VIF para detectar colinealidad entre predictores.
3. Eliminar predictores redundantes.
4. Ajustar el modelo con los predictores restantes.
5. Validar el modelo con R^2 , MAE y algún esquema de validación cruzada.

En esta tesis, el planteo es similar, pero con dos matices importantes: se trabaja también con predictores polinómicos (por ejemplo x, x^2), donde cierta colinealidad es estructural y esperada; el foco está en la capacidad de imputar bien, no en interpretar cada coeficiente.

En nuestro caso al detectar colinealidad el modelo se descarta completamente, no se eliminan predictores redundantes, ya que en verdad hacemos una combinación de todas las combinaciones posibles de variables.

Por eso no se busca “eliminar toda colinealidad”, sino fijar umbrales razonables que eviten modelos numéricamente frágiles o contruidos con información casi duplicada.

9.2.8.2 Correlación lineal: coeficiente de Pearson entre predictores

Para evaluar si dos variables se relacionan de manera aproximadamente lineal se utiliza el **coeficiente de Pearson**, que toma valores entre -1 y 1:

$r = 1$: correlación positiva perfecta;

$r = -1$: correlación negativa perfecta;

$r = 0$: ausencia de correlación lineal clara.

Una forma habitual de escribir la fórmula es:

$$r = \frac{\sum (x_i - \bar{x})(y_i - \bar{y})}{\sqrt{\sum (x_i - \bar{x})^2 \sum (y_i - \bar{y})^2}}$$

donde x_i y y_i son los valores de las dos variables que se comparan, y $\bar{x}$, $\bar{y}$ son sus medias.

En este trabajo, el coeficiente de Pearson se utiliza en dos planos:

1. **Para construir modelos útiles de imputación:** se exige una correlación mínima entre predictor y variable objetivo (por ejemplo $|r| > 0,6$) para considerar que una relación lineal simple tiene fuerza suficiente como para ser candidata a modelo.

2. **Para vigilar la multicolinealidad entre predictores:** se evita que dos predictores muy correlacionados (por ejemplo $|r| \approx 0,95$ o más) aparezcan juntos en un mismo modelo, salvo casos muy justificados, porque aportan información casi duplicada y pueden generar inestabilidad.

En términos de interfaz, estos criterios se traducen en límites configurables sobre $|r|$, un **umbral mínimo** para considerar útil la relación predictor–objetivo y un **umbral máximo** aceptable para la correlación entre predictores dentro de un mismo modelo.

9.2.8.3 VIF y número de condición: multicolinealidad en conjunto

Cuando se utilizan dos o más predictores en una misma regresión, la correlación par a par ya no alcanza para describir la colinealidad global. Ahí entra el **Factor de Inflación de la Varianza (VIF)**.

Para un predictor x_j , el VIF se define como:

$$\text{VIF}_j = \frac{1}{1 - R_j^2}$$

donde R_j^2 es el coeficiente de determinación al ajustar una regresión en la que la variable explicada es x_j y las variables explicativas son los demás predictores del modelo. Si x_j puede explicarse casi perfectamente a partir de los otros, R_j^2 se acerca a 1 y el VIF crece mucho.

Escalas habituales de interpretación:

- VIF ≈ 1 : sin colinealidad relevante.
- VIF > 5 : colinealidad moderada, conviene revisar.
- VIF > 10 : colinealidad alta, generalmente se recomienda eliminar o reemplazar alguno de los predictores.

En esta tesis se introducen límites sobre el VIF como criterio de calidad: los modelos que superan el umbral máximo definido para VIF se penalizan o descartan, ya que son más sensibles al ruido y a las variaciones de la base de datos.

El **número de condición** de la matriz de diseño es otra forma de medir inestabilidad. Se calcula a partir de la relación entre la mayor y la menor escala de variación (por ejemplo, mediante valores singulares o autovalores). Si el número de condición es grande, el sistema es mal condicionado: pequeñas perturbaciones en los datos pueden producir grandes cambios en los coeficientes.

Orientativamente:

- valores por debajo de 30 suelen considerarse seguros,
- entre 30 y 100 se entra en una zona de alerta,
- por encima de 100 se habla de problemas serios de condicionamiento.

En el flujo de selección de modelos de esta tesis, tanto el VIF como el número de condición se utilizan para penalizar o directamente descartar combinaciones de predictores que, aunque puedan mostrar buen ajuste aparente, se apoyan en estructuras numéricamente frágiles.

9.2.8.4 Correlación polinómica, R^2 y variabilidad de la variable objetivo

Cuando se incorporan términos polinómicos (por ejemplo x , x^2 , combinaciones de dos predictores, etc.), la colinealidad se vuelve, en parte, estructural: es normal que x y x^2 estén muy correlacionados. En este contexto, seguir mirando solo $|r|$ entre columnas deja de ser suficiente.

En lugar de un único coeficiente de correlación “generalizado”, se recurre a métricas que describen el rendimiento del **modelo completo**:

- el **coeficiente de determinación** R^2 , que mide qué proporción de la variabilidad de la variable objetivo es explicada por el modelo,
- y el **MAPE** (Mean Absolute Percentage Error), que mide el error relativo promedio en porcentaje.

El R^2 se define como:

$$R^2 = 1 - \frac{\sum (y_i - \hat{y}_i)^2}{\sum (y_i - \bar{y})^2}$$

donde y_i son los valores reales, $\hat{y}_i$ los valores predichos por el modelo y $\bar{y}$ la media de los valores reales.

Su interpretación estándar es:

$R^2 = 1$: el modelo reproduce exactamente todos los datos.

$R^2 = 0$: el modelo no mejora sobre usar la media.

$R^2 < 0$: el modelo es peor que predecir siempre la media.

Sin embargo, cuando la variabilidad de la variable objetivo es casi nula (por ejemplo, cuando todos los valores son muy parecidos), incluso un modelo trivial que predice siempre la media puede alcanzar un R^2 muy alto. Para evitar interpretar estos casos como “modelos excelentes”, se utiliza el **coeficiente de variación**:

$$CV = \frac{\sigma}{\bar{x}}$$

donde σ es la desviación estándar y $\bar{x}$ la media de la variable objetivo. De forma simplificada:

- si $CV < 1\%$, la variabilidad es despreciable y no tiene sentido interpretar un R^2 alto,
- si $1\% \leq CV < 5\%$, el modelo se revisa con más cuidado,
- si $CV \geq 5\%$, se considera que hay variación suficiente como para que R^2 sea informativo.

El **MAPE** se define como:

$$MAPE = \frac{1}{n} \sum \left| \frac{y_i - \hat{y}_i}{y_i} \right| \times 100$$

y describe la precisión práctica del modelo en términos porcentuales. Un MAPE bajo indica que, en promedio, las predicciones están cerca de los valores reales en términos relativos, lo cual es especialmente útil cuando las magnitudes varían mucho entre aeronaves.

9.2.8.5 Coeficiente combinado “Corr” como medida operativa¹

Para disponer de una escala común que combine capacidad explicativa y error porcentual se define un **coeficiente combinado de correlación “Corr”**:

$$Corr = \frac{1}{2} R^2 + \frac{1}{2} \left(1 - \frac{MAPE}{15} \right)$$

La idea es:

- el primer término premia modelos con buen R^2 ,
- el segundo término premia modelos con MAPE bajo; cuando el MAPE se acerca o supera el 15 %, este término se reduce fuertemente e incluso puede volverse negativo.

En términos operativos:

- si $MAPE = 15\%$, el segundo término se hace 0;
- si $MAPE > 15\%$, el segundo término puede volverse negativo y arrastrar a Corr hacia abajo.

Como criterio práctico, se considera aceptable una combinación de predictores cuando $Corr \geq 0,6$. En la práctica, esto ocurre cuando se cumple simultáneamente algo similar a:

¹ En la implementación actual, *Corr* se calcula y se conserva principalmente como indicador resumido para fines de reporte y análisis (por ejemplo, visualización en módulos de auditoría). La selección de modelos se basa en la métrica Confianza, que parte de un componente base (combinación acotada de R^2 y MAPE) y aplica penalizaciones adicionales (por tamaño muestral, cobertura y tamaño global de la base). En validación LOOCV se calcula *Corr_LOOCV* y, a partir de él, se obtiene *Confianza_LOOCV* = *Corr_LOOCV* · *penalización_por_k(n_LOOCV)*; posteriormente se combina con la confianza de entrenamiento mediante un peso *w*, (desarrollo en capítulos posteriores).

- R^2 moderado o alto (por ejemplo $R^2 \gtrsim 0,7$),
- $MAPE$ bajo o moderado (por ejemplo $MAPE \lesssim 7,5\%$).

Este coeficiente se utiliza como una especie de “correlación unificada” para comparar tanto modelos lineales como polinómicos, en lugar de trabajar solo con el $|r|$ de Pearson.

9.2.8.6 Criterios globales de selección y validación

Finalmente, la tesis adopta un conjunto de criterios coherentes para decidir qué modelos de imputación conservar:

- **multicolinealidad controlada** mediante correlaciones entre predictores, VIF y número de condición,
- **variabilidad mínima** de la variable objetivo (coeficiente de variación), para que R^2 tenga sentido,
- **umbral mínimo de calidad estadística** mediante R^2 y $MAPE$,
- **coeficiente combinado Corr** como métrica única de comparación,
- **y validación cruzada tipo leave-one-out** (incluyendo variantes en las que se dejan fuera columnas simulando datos faltantes) para evaluar la estabilidad de cada modelo frente a la realidad de una base incompleta.

La selección final no se basa en la independencia perfecta entre predictores, algo poco realista en un contexto aeronáutico, sino en la combinación de, buena capacidad para explicar la variación relevante, errores porcentuales moderados, estabilidad numérica y robustez frente a datos faltantes y outliers.

9.2.9 Comprobaciones geométricas 2D y diversidad de datos

Cuando se usan modelos con **dos predictores** a la vez, cada aeronave de la base se representa como un punto en el plano (x_1, x_2) . No basta con tener muchos puntos: interesa **cómo están distribuidos** en ese plano.

Si los puntos ocupan una superficie razonable, el modelo tiene información sobre distintas combinaciones de predictores.

Si los puntos están todos apilados en una franja muy estrecha, o casi alineados, el modelo “cree” que conoce 2D, pero en realidad solo ha visto una situación muy particular.

Para cuantificar esta idea de “buena distribución” se usan tres herramientas geométricas:

1. **Convex hull** → qué tanto del rectángulo que encierra los datos está realmente ocupado.
2. **PCA 2D y PC2_ratio** → cuánta variación hay en la “segunda dirección independiente”.
3. **Elipse de covarianza** → qué área típica ocupa la nube de puntos.

Estas tres medidas se convierten luego en **umbrales configurables** (por ejemplo, *hull_ratio_min*, *pc2_ratio_min*, *ellipse_ratio_min*) que el sistema utiliza para aceptar, penalizar o descartar modelos con dos predictores.

9.2.9.1 Convex hull: contorno mínimo de los datos

Dado un conjunto de puntos en el plano (x_1, x_2) , el convex hull (envolvente convexa) es el polígono convexo más pequeño que contiene a todos los puntos.

Una forma intuitiva de imaginarlo es, colocar todos los puntos sobre una mesa, rodearlos con una bandita elástica estirada, soltarla, la forma que adopta la goma es el convex hull.

En el sistema se comparan dos áreas:

- el área del rectángulo mínimo alineado con los ejes que contiene a todos los puntos (bounding box),
- el área del convex hull.

Con esas dos áreas se define:

$$hull_ratio = \frac{\text{área del hull}}{\text{área del rectángulo}}$$

Interpretación:

- Si los puntos ocupan casi toda la “caja” (por ejemplo, repartidos por todo el rango de MTOW y autonomía), el *hull_ratio* será cercano a 1.
- Si los puntos están concentrados en una franja estrecha o en un rincón, el *hull_ratio* será pequeño: el hull ocupa muy poca parte del rectángulo que podría ocupar.

En esta tesis se utiliza un umbral mínimo *hull_ratio_min*. Valores típicos de configuración están alrededor de, *hull_ratio_min* $\approx 0,15$.

Esto significa que, si el área del hull es menos del 15 % del área del rectángulo que encierra todos los datos, se considera que la distribución es “pobre” para un modelo 2D: hay riesgo de extrapolaciones fuertes cuando se combinan valores de predictores fuera de la región principal. En esos casos el modelo se penaliza o no se utiliza.

9.2.9.2 PCA 2D y PC2_ratio: qué tan “realmente 2D” es la nube

El análisis de componentes principales (PCA, por *Principal Component Analysis*) es una herramienta que permite describir la dispersión de los datos buscando **direcciones independientes** en las que los puntos varían más o menos.

En 2D, aplicado a los predictores (x_1, x_2) , la idea es:

1. Se centra la nube de puntos, restando la media de cada predictor.
2. Se busca la dirección (un eje en el plano) donde los puntos se dispersan más. Esa dirección se llama **primer componente principal (PC1)**.
3. Se busca una segunda dirección **perpendicular** a la anterior, donde los puntos sigan variando, aunque menos. Esa dirección es el **segundo componente principal (PC2)**.

- A lo largo de **PC1** se concentra la mayor parte de la variación.
- A lo largo de **PC2** está la variación que queda “en otra dirección independiente”.

Si todos los puntos estuvieran exactamente alineados en una recta, ocurriría esto:

- PC1 capturaría prácticamente toda la variación,
- PC2 casi no tendría variación: todos los puntos estarían sobre una línea, sin “espesor” alrededor.

Para cuantificar cuánta información hay en cada dirección se calcula la **varianza** a lo largo de PC1 y PC2:

- Var_1 : varianza en PC1,
- Var_2 : varianza en PC2.

La **varianza total** en el plano es la suma:

$$Var_{total} = Var_1 + Var_2$$

Con esto se define:

$$PC2_ratio = \frac{Var_2}{Var_{total}}$$

Interpretación:

Si $PC2_ratio \approx 0$, significa que la varianza en PC2 es casi nula: la nube de puntos es prácticamente una línea. El problema en realidad es 1D, aunque se estén usando dos predictores.

Si $PC2_ratio$ es significativo (por ejemplo 0,1 o 0,2), quiere decir que una parte importante de la variación ocurre en la segunda dirección independiente: la nube ocupa una región verdaderamente 2D.

Valores típicos:

$PC2_ratio \approx 0,5 \rightarrow$ nube prácticamente “redonda”: los datos se dispersan de forma parecida en ambas direcciones.

$PC2_ratio \approx 0,1 \rightarrow$ la mayor parte de la variación está en PC1, pero PC2 sigue aportando una cantidad apreciable de información (un 10 % de la varianza).

$PC2_ratio \ll 0,05 \rightarrow$ la variación en PC2 es muy pequeña; los datos están casi en una recta.

En el sistema, el parámetro de configuración $pc2_ratio_min$ fija un valor mínimo aceptable para este cociente. Un valor típico es, **$pc2_ratio_min \approx 0,03$** .

Esto significa que se exige que al menos un 3 % de la variación total se encuentre en la segunda dirección independiente. Si $PC2_ratio$ cae por debajo de ese valor, se considera que:

el segundo predictor aporta muy poca información nueva,

el modelo de dos predictores se comporta casi como un modelo de uno solo,

y la regresión puede volverse numéricamente frágil o muy sensible al ruido.

En esos casos, el sistema penaliza o descarta esa combinación de predictores y privilegia modelos donde los dos ejes aportan variación real.

9.2.9.3 **Elipse de covarianza: tamaño y forma del “núcleo” de datos**

Otra forma de describir cómo se dispersan los puntos (x_1, x_2) es a través de la **matriz de covarianza**. Esta matriz resume, cuánto varía cada predictor por separado (varianzas), y cómo se mueven juntos (covarianza).

A partir de esa matriz se puede construir una **elipse de dispersión** que representa el “núcleo” de la nube de datos: la zona donde se concentran la mayoría de los puntos.

Conceptualmente:

1. Se calcula la matriz de covarianza de (x_1, x_2) .
2. Se obtienen sus autovalores λ_1 y λ_2 , que son números no negativos relacionados con la varianza a lo largo de dos ejes ortogonales (estos ejes coinciden con los componentes principales PC1 y PC2).
3. Se construye una elipse con semiejes proporcionales a $\sqrt{\lambda_1}$ y $\sqrt{\lambda_2}$.

Si λ_1 y λ_2 son parecidos, la elipse se parece a un círculo: los datos se dispersan de forma similar en ambas direcciones.

Si uno de los autovalores es mucho mayor que el otro, la elipse es muy alargada: los datos se extienden principalmente en una dirección, con poco “espesor” transversal.

En la implementación utilizada, a partir de esa elipse teórica se calcula un área *ellipse_area* (para un nivel estándar) y se normaliza respecto al área del rectángulo mínimo que contiene a todos los puntos:

$$ellipse_ratio = \frac{ellipse_area}{bbox_area}$$

Interpretación:

Si *ellipse_ratio* es relativamente alto (por ejemplo, mayores a 0,1), la elipse central es grande respecto al rectángulo: los puntos se reparten en una región amplia y la nube ocupa bien el espacio disponible.

Si *ellipse_ratio* es muy bajo, la elipse es pequeña comparada con el rectángulo: aunque el rango total de valores sea grande, los datos típicos se concentran en una zona pequeña, y hay muchas combinaciones teóricas de (x_1, x_2) para las cuales no hay datos.

El parámetro *ellipse_ratio_min* fija el valor mínimo aceptable. Un valor de referencia es, ***ellipse_ratio_min* $\approx$ 0,10**.

Esto impone, en combinación con *hull_ratio* y *PC2_ratio*, que los datos de entrenamiento de un modelo 2D:

- no estén todos comprimidos en una franja muy estrecha,
- no se limiten a una región mínima dentro del rectángulo,
- y aporten información suficiente para explorar el espacio de diseño sin extrapolar de forma extrema.

9.2.9.4 Cobertura de pares únicos y rango efectivo de entrenamiento

Además de la forma de la nube de puntos, el sistema controla cuánta información distinta hay realmente en los datos usados para entrenar un modelo con dos predictores.

En muchos catálogos técnicos, varias aeronaves comparten exactamente los mismos valores en un par de variables (por ejemplo, mismo MTOW y misma autonomía) porque pertenecen a una misma familia con ligeras variantes. Si se cuentan esas filas tal cual, el modelo “cree” que tiene muchos datos, pero en realidad está viendo siempre la misma combinación.

Para capturar esto se calcula:

$$\text{unique_pair_ratio} = \frac{\text{número de pares } (x_1, x_2) \text{ distintos}}{\text{número total de filas válidas } n}$$

Interpretación:

Si muchas filas comparten los mismos (x_1, x_2) , el numerador crece poco y el *unique_pair_ratio* se aproxima a 0.

Si casi todas las filas tienen combinaciones distintas de predictores, el cociente se acerca a 1.

En la configuración por defecto del sistema se exige un mínimo de, **unique_pair_ratio_min $\approx$ 0,60**.

Es decir, al menos un 60 % de las filas deberían aportar pares realmente distintos para considerar que el modelo está suficientemente alimentado con variaciones de diseño. Si el ratio cae por debajo de ese umbral, el modelo se considera poco diverso, aumenta el riesgo de sobreajuste a unas pocas configuraciones y esa combinación de predictores se descarta o se penaliza antes de calcular la confianza final.

Además de la diversidad de combinaciones, se controla el tamaño efectivo de la muestra en relación con la complejidad del modelo. Un modelo de dos predictores lineales ajusta aproximadamente 3 parámetros (intercepto + 2 pendientes). Un modelo polinómico cuadrático de dos predictores ajusta del orden de 6 parámetros (intercepto, términos lineales, cuadráticos y cruzados).

Por eso el sistema calcula:

$$n_{\text{por_parámetro}} = \frac{n}{p}$$

donde:

n es el número de filas válidas usadas para entrenar,

p es el número aproximado de parámetros libres del modelo (3 para lineal, 6 para polinómico de 2 predictores).

En la configuración utilizada se aplican los siguientes umbrales, que debido a la baja cantidad de datos los mismos no son estadísticamente aceptables para garantizar un modelo confiable:

para modelos lineales de 2 predictores se exige aproximadamente $n_{\text{por_parámetro}} \gtrsim 8 \rightarrow$ unas 10 filas válidas o más,

para modelos polinómicos de 2 predictores se exige $n_{\text{por_parámetro}} \gtrsim 10 \rightarrow$ unas 12 filas válidas o más.

Si no se alcanza ese mínimo, el modelo se descarta automáticamente, aunque el ajuste local parezca bueno. Con esto se evita entrenar modelos “sofisticados” con muy pocos datos, que tenderían a memorizar ruido en lugar de capturar tendencias generales de la familia de aeronaves.

9.2.9.5 Integración de los checks geométricos 2D en el flujo de modelos

Todos los indicadores anteriores (colinealidad, PCA 2D, coverage del hull, elipse, pares únicos y tamaño muestral efectivo) se integran en una lógica de selección de modelos de dos predictores.

En términos conceptuales, el flujo para un modelo 2D es el siguiente:

1. **Se forma la nube de puntos** (x_1, x_2, y) a partir de la base depurada (tras limpieza, imputación preliminar y tratamiento de outliers).

2. **Se calculan métricas de estabilidad y forma:**

- correlación lineal y *VIF* (para detectar multicolinealidad fuerte),
- número de condición y *PC2_ratio* (para detectar problemas numéricos y nubes casi 1D),
- *unique_pair_ratio*, *hull_ratio* y *ellipse_ratio* (para medir diversidad y cobertura espacial),
- *n_per_param* (para asegurar suficiente cantidad de datos por parámetro).

3. **Se aplican umbrales mínimos/máximos** predefinidos:

- si la correlación entre predictores es excesiva, si el número de condición es muy alto o si *PC2_ratio* cae por debajo de un cierto mínimo, el modelo se considera numéricamente inestable;
- si *unique_pair_ratio*, *hull_ratio* o *ellipse_ratio* son muy bajos, se detecta que la nube ocupa mal el espacio disponible;
- si *n_per_param* es insuficiente, se evita entrenar un modelo demasiado complejo con pocos datos.

4. Si una o varias de estas condiciones no se cumplen, el modelo se **descarta** antes de entrar siquiera al ranking. Solo los modelos que superan este filtro pasan a la etapa en la que se comparan mediante R^2 , MAPE y confianza.

De esta manera, los checks 2D no son un adorno extra, sino un filtro previo que garantiza que, los modelos que alimentan el sistema de imputación trabajan con datos razonablemente variados, el espacio de diseño está cubierto de manera suficiente, y las extrapolaciones que el sistema realiza sobre la base de estos modelos tienen respaldo en una nube de datos que realmente ocupa una región 2D, y no una línea delgada en el plano.

Estos mismos indicadores reaparecen, con distinto peso, dentro del cálculo global de **confianza del modelo**, que se detalla a continuación.

9.2.10 Confianza del modelo de imputación

En esta tesis se utiliza un **score interno de confianza** para ordenar los modelos de imputación y decidir cuáles se consideran “aceptables” o “recomendables”. Este score no es una probabilidad en sentido estricto, sino una medida adimensional entre 0 y 1 que combina:

- calidad del ajuste (R^2 y MAPE),
- cantidad de datos utilizados,
- calidad geométrica de la nube (checks 2D),
- y la validación por LOOCV.

La idea es que un modelo con **confianza alta** haya superado varias pruebas simultáneas: ajusta bien los datos, se basa en una cantidad suficiente de ejemplos, no depende de una geometría degenerada y se comporta razonablemente bien cuando se lo valida “dejando datos fuera”.

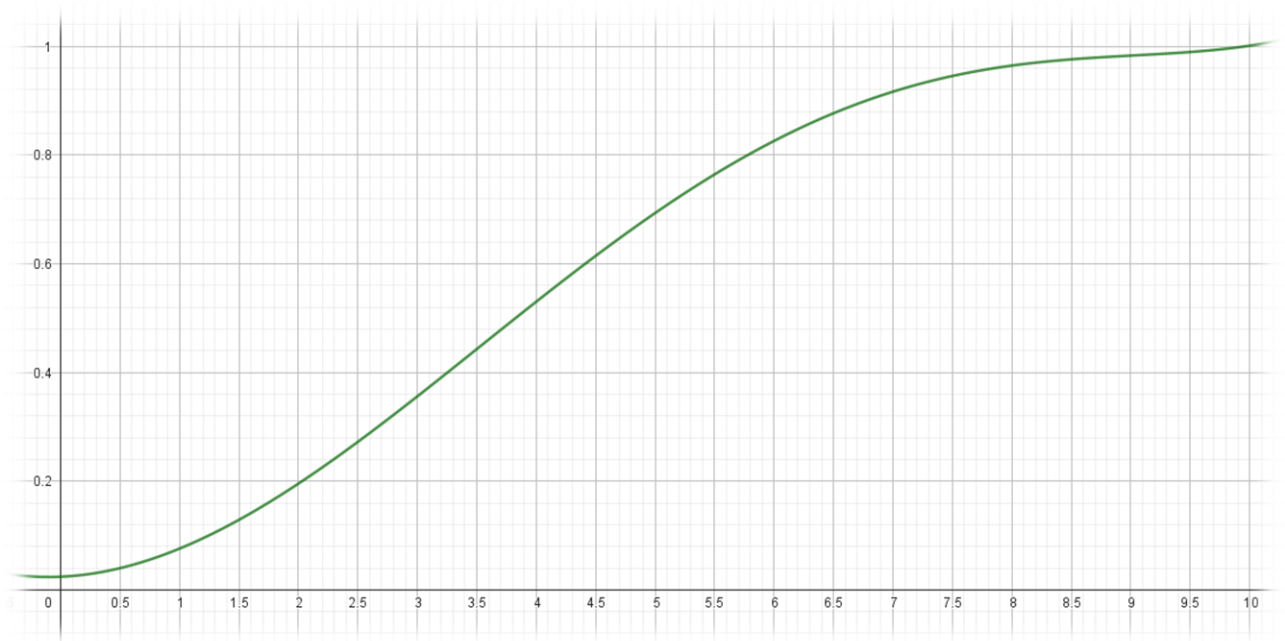


Imagen 9.3 Función de mapeo para la confianza del modelo

Curva usada para transformar una medida de desempeño del modelo (p. ej., combinación de R^2 y MAPE) en un score de confianza normalizado (0–1) dentro del sistema.

9.2.10.1 Componente base: combinación de R^2 y MAPE

El primer bloque de la confianza parte de dos métricas ya usadas en el resto de la tesis:

- el **coeficiente de determinación R^2** , que mide qué fracción de la variabilidad de la variable objetivo explica el modelo,
- el **MAPE**, que mide el error porcentual medio de las predicciones.

Ambas se normalizan a un rango 0–1:

- $r2_term$ se toma como R^2 truncado a $[0, 1]$.
- $mape_term$ se construye como:

$$mape_term = 1 - \frac{MAPE}{mape_divisor}$$

y luego también se transforma a $[0, 1]$.

El parámetro **mape_divisor** fija la escala a partir de la cual los errores se consideran inaceptables. Un valor típico es:

- **mape_divisor $\approx 15\%$**

Con esto:

si $MAPE \approx 0\%$, entonces $mape_term \approx 1$,

si $MAPE \approx 15\%$, entonces $mape_term \approx 0$,

si MAPE es mayor que 15% , el término se satura en 0 para no penalizar “hacia abajo” sin límite.

La combinación base se calcula como:

$$^2\text{base} = w_{R^2} \cdot r^2_{\text{term}} + w_{MAPE} \cdot \text{mape_term}$$

donde w_{R^2} y w_{MAPE} son pesos configurables que, por defecto, suelen elegirse cercanos a 0,5–0,5 para dar un peso similar a la capacidad explicativa y a la precisión porcentual. El resultado se limita después al rango 0–1.

En términos cualitativos:

un modelo con R^2 alto y MAPE bajo se aproxima a base ≈ 1 ,

un modelo con R^2 bajo o MAPE muy alto tendrá base cercana a 0,

combinaciones intermedias producen valores de base intermedios (por ejemplo, R^2 moderado pero MAPE muy bueno, o al revés).

9.2.10.2 Penalización por tamaño muestral k

El siguiente bloque introduce una penalización que depende del número de muestras efectivas usadas para entrenar el modelo, denotado aquí por k . No es lo mismo obtener un R^2 alto a partir de 7 u 8 puntos que a partir de 50.

Para capturar esta idea se define una función polinómica suave $p_k(k)$, ajustada de forma que:

para valores de k muy pequeños (por ejemplo, $k \approx 3$ –4 puntos), la penalización sea moderada (valores del orden de 0,15–0,20);

a medida que k crece, el factor se aproxima a 1;

para k de unas 20 muestras o más, el factor es prácticamente 1 (no se penaliza adicionalmente al modelo por falta de datos).

Este factor se aplica multiplicando la base:

$$\text{base_k} = \text{base} \times p_k(k)$$

De este modo, dos modelos con el mismo R^2 y MAPE pueden tener confianzas distintas si uno se entrenó con pocas muestras y el otro con un conjunto mucho más amplio.

9.2.10.3 Penalización por tamaño global de la base N

Además del tamaño muestral concreto del modelo, la configuración permite introducir una penalización que depende del tamaño global de la base relevante, representado aquí por N . Conceptualmente es el número de aeronaves para las cuales el parámetro objetivo es conocido dentro del subconjunto considerado.

Se utiliza otra función polinómica $p_N(N)$ que típicamente, toma valores bajos cuando N es muy pequeño (poca evidencia global), crece hacia 1 cuando la base de datos es más amplia.

El resultado de esta etapa es:

² base es la combinación normalizada (acotada a [0,1]) usada internamente para Confianza; Corr se mantiene como indicador reportado.

$$\text{base_kN} = \text{base} \times p_k(k) \times p_N(N)$$

Esta estructura refuerza una idea central del trabajo: **no alcanza con que un modelo ajuste bien; también debe estar basado en una cantidad razonable de evidencia.**

9.2.10.4 Penalizaciones métricas 2D (colinealidad y cobertura)

El bloque siguiente introduce penalizaciones adicionales asociadas a las métricas 2D vistas antes:

correlación absoluta entre predictores $|r|$,

VIF (Factor de Inflación de la Varianza),

número de condición,

PC2_ratio,

unique_pair_ratio, *hull_ratio*, *ellipse_ratio*.

Para cada una de estas métricas el sistema:

1. Extrae el valor correspondiente del resumen estadístico del modelo.
2. Lo normaliza o transforma mediante un pequeño polinomio definido en la configuración.
3. Obtiene un factor entre 0 y 1 que se multiplica al resto.

Por ejemplo, de manera cualitativa:

si $|r|$ está por debajo de un cierto límite, el factor asociado a “*pearson_abs*” vale ≈ 1 ; si se acerca demasiado a 1, el factor descende;

si *PC2_ratio* o *hull_ratio* caen por debajo de los mínimos recomendados, su factor asociado se hace significativamente menor que 1, reduciendo la confianza;

si la cobertura es razonable y las métricas están en zona “saludable”, estos factores quedan cerca de 1 y apenas afectan al valor base.

El resultado de multiplicar todos estos factores es un peso geométrico que refuerza modelos contruidos a partir de nubes bien distribuidas y penaliza aquellos que dependen de geometrías degeneradas (datos casi alineados, poca cobertura, pocos pares distintos, etc.).

9.2.10.5 Aporte de la validación LOOCV³

Finalmente, la confianza se ajusta en función de la calidad de la validación externa:

- se calcula una validación tipo LOOCV (Leave-One-Out) apropiada para imputación,
- de ella se obtienen R^2_{LOOCV} y $\text{MAPE}_{\text{LOOCV}}$,
- en base a estos valores, el modelo se clasifica en tres categorías:
 - **robusto**,
 - **no robusto**,

³ El sistema calcula $\text{Corr}_{\text{LOOCV}}$ y deriva $\text{Confianza}_{\text{LOOCV}}$ (en función de $\text{Corr}_{\text{LOOCV}}$ y una penalización por tamaño n_{LOOCV}); luego mezcla $\text{Confianza}(\text{entrenamiento})$ y $\text{Confianza}_{\text{LOOCV}}$ con un peso w .

- **rechazado.**

A cada categoría se le asigna un factor de ajuste, por ejemplo:

robusto $\rightarrow$ factor $\approx 1,0$

no robusto $\rightarrow$ factor $\approx 0,85$

rechazado $\rightarrow$ factor $\approx 0,6$

La confianza final se obtiene entonces combinando, la confianza construida con datos de entrenamiento y checks geométricos, con un pequeño ajuste multiplicativo según la clase de LOOCV y un peso global que controla cuánto influye esta corrección.

En términos intuitivos, si un modelo parece bueno por métricas internas pero la validación LOOCV sugiere problemas, la confianza final baja; si un modelo es consistente en entrenamiento y en LOOCV, la confianza se acerca a 1; si LOOCV lo clasifica como “rechazado”, la confianza queda limitada, aunque R^2 y MAPE pudieran ser aparentemente buenos.

Con esta estructura, el **score de confianza** resume en un solo número muchas capas de información:

ajuste (R^2 , MAPE),

tamaño y calidad de la muestra,

estabilidad numérica y buena geometría 2D,

y comportamiento bajo validación cruzada.

Ese número es el que finalmente se usa en los módulos posteriores para ordenar modelos, filtrar aquellos que no alcanzan un nivel mínimo aceptable y, en última instancia, para decidir qué ecuaciones son las candidatas a usarse en la imputación de parámetros faltantes.

10 GÉNESIS DEL SISTEMA

Antes de entrar en los detalles de implementación de nuestro sistema, en este capítulo se quiere contar cómo llegamos hasta aquí. Partimos del ADRpy original del libro de Keane, con la idea bastante directa de usarlo para diseñar un dron de ala fija con capacidad VTOL. En el camino descubrimos que el obstáculo principal no era el código de Keane, sino los datos que teníamos disponibles y la forma en que intentábamos usarlos.

Este capítulo funciona como puente entre el MARCO TEÓRICO y la descripción técnica de los módulos que haremos en los capítulos siguientes. Aquí no volvemos a desarrollar la teoría de regresión ni las métricas; nos concentramos en la historia del proyecto, en el problema de los datos y en cómo eso nos llevó a concebir el sistema ADRpy-VTOL en tres módulos.

10.1 ADRpy original en el contexto del diseño conceptual de Keane

Al comienzo de este proyecto, no se pretendía inventar un método de diseño conceptual desde cero. Queríamos apoyarnos en herramientas que ya estuvieran consolidadas en la literatura, y en particular en el enfoque de diseño conceptual que presentan Keane y otros autores clásicos del área [8] [1].

En términos simples, el proceso de diseño que propone Keane es iterativo:

1. Partimos de unos requisitos de misión (peso máximo al despegue, tipo de misión, autonomía, velocidad deseada, tipo de despegue/aterrizaje, etc.).
2. Proponemos valores iniciales para parámetros de diseño (área alar, relaciones de aspecto, prestaciones objetivo, potencias instaladas, entre otros).
3. Usamos modelos aerodinámicos y de peso para calcular si ese conjunto de parámetros es razonable: estimamos pesos, arrastres, consumos, márgenes de potencia.
4. Con esos resultados generamos diagramas globales, como el clásico plano carga alar (S/W) vs relación potencia-peso (P/W), que nos muestran la “zona plausible” donde el diseño cumple simultáneamente varios requisitos.
5. A partir de esos gráficos hacemos correcciones, ajustamos parámetros de entrada y volvemos a iterar.
6. En pocas palabras: el diseño conceptual no es un cálculo único, sino un ciclo de prueba y ajuste. Justamente por eso, la repetición de procesos es parte central del enfoque de Keane y de Raymer. Cada vez que cambiamos un valor de entrada, debemos repetir toda la cadena de cálculos hasta ver de nuevo dónde cae el diseño en el diagrama $S/W-P/W$.
7. El ADRpy surge en ese contexto como una herramienta para automatizar una parte importante de ese ciclo. En lugar de repetir los cálculos “a mano” o con hojas de cálculo dispersas, Keane propone agruparlos en un entorno programable (Python) donde:
8. definimos las entradas del diseño (parámetros de geometría, masas, misión, configuración del avión),
9. ejecutamos una serie de rutinas aerodinámicas y de peso basadas en modelos teóricos y empíricos,

10. y obtenemos como salida estimaciones preliminares y gráficos clave, incluidos los diagramas S/W y P/W que delimitan regiones de diseño aceptables.

En nuestro caso concreto, el sistema ADRpy no es una simple hoja de entrada aislada, sino un notebook de Jupyter: un entorno donde podemos declarar variables en celdas de código, ejecutar funciones que encapsulan fórmulas aeronáuticas clásicas y visualizar rápidamente los resultados en forma de gráficos [8].

De manera muy resumida, las **entradas típicas** del ADRpy abarcan: parámetros de misión (perfil de vuelo, altitud de crucero, velocidad de diseño), parámetros de masa (MTOW, fracciones de peso para estructura, combustible, carga útil), parámetros geométricos (superficie alar, alargamiento, configuraciones de cola, etc.), características de propulsión (tipo de motor, potencia o empuje instalados).

Y las **salidas** incluyen, entre otras: estimaciones de pesos parciales y totales, coeficientes aerodinámicos en puntos de operación relevantes, requerimientos de potencia y empuje, y los diagramas integrados donde se combinan carga alar y potencia-peso para identificar la región del plano S/W–P/W en la que el diseño puede funcionar de manera aceptable.

Marcamos estas entradas y salidas de forma resumida en la siguiente tabla conceptual:

<i>Bloque</i>	<i>Entradas típicas (ejemplos)</i>	<i>Salidas típicas / para qué se usan</i>
<i>Misión</i>	Perfil de vuelo, altitud de crucero, velocidad de diseño	Parámetros de operación para evaluar consumo, prestaciones y restricciones
<i>Masas</i>	MTOW, fracciones de peso (estructura, combustible, carga útil)	Estimaciones de pesos parciales y total; coherencia del diseño
<i>Geometría</i>	Superficie alar, alargamiento, configuración de cola	Coeficientes aerodinámicos en puntos relevantes; sensibilidad geométrica
<i>Propulsión</i>	Tipo de motor, potencia o empuje instalados	Requerimientos de potencia/empuje y márgenes; verificación de viabilidad
<i>Integración (constraint analysis)</i>	Conjunto anterior (diseño + misión)	Diagramas globales S/W vs P/W y “zona plausible”; guía para iterar valores

Tabla 10.1- Entradas y salidas principales de ADRpy dentro del ciclo iterativo de diseño conceptual.

Síntesis conceptual de las entradas típicas (misión, masas, geometría y propulsión), las rutinas/modelos ejecutados y las salidas principales (estimaciones preliminares y gráficos de restricciones) que ADRpy entrega para iterar un diseño.

¿Por qué elegimos trabajar con ADRpy?

Elegimos trabajar con el ADRpy por dos motivos que siguen presentes en la versión actual del proyecto:

1. Queríamos un flujo lo más automatizado posible.

Si el proceso de diseño es necesariamente iterativo, entonces tiene mucho sentido que la repetición sea barata: cambiar unos pocos parámetros de entrada y volver a lanzar el cálculo debería ser nuestro trabajo principal, y no repetir a mano todas las operaciones intermedias. El sistema ADRpy ya materializa esa filosofía, una vez armada la estructura de código, cada nueva iteración requiere solo cambiar entradas y volver a ejecutar.

2. Queríamos una base técnicamente sólida y didáctica.

El ADRpy está construido a partir de modelos bien documentados en la literatura de diseño conceptual y está organizado de forma relativamente clara en el notebook. Eso nos permitía estudiarlo, entender las fórmulas y al mismo tiempo usarlo como plataforma para nuestro propio proyecto, en lugar de empezar desde una hoja en blanco.

Por eso, nuestro punto de partida fue simple, usar el ADRpy como “motor” de dimensionamiento, adaptarlo al caso de un dron de ala fija con capacidad VTOL y automatizar alrededor de él todo lo que hiciera falta para que las iteraciones de diseño fueran ágiles.

10.2 Intención inicial: aplicación de ADRpy a un dron de ala fija con capacidad VTOL

Con esta herramienta en la mano, el objetivo inicial era muy concreto, diseñar un dron de ala fija con capacidad VTOL. Queríamos aprovechar el ADRpy como punto de partida para dimensionar pesos, superficies, potencias y otros parámetros clave de la aeronave, pero adaptándolo a una configuración que el código original no contempla de forma directa.

Desde el principio, imaginamos el proceso de diseño como un recorrido por fases encadenadas:

- Una **primera fase**, centrada en entender el mercado existente, construir una base de datos de drones, extraer regularidades estadísticas y, a partir de ahí, proponer valores iniciales razonables para los parámetros de nuestro diseño VTOL.
- Una **segunda fase**, orientada a modificar el ADRpy y sus modelos físicos para incorporar explícitamente las particularidades de un sistema VTOL (nuevos modos de operación, diferentes repartos de peso, efectos en el rendimiento, etc.), y calcular la región plausible de diseño en el diagrama S/W – P/W para esa aeronave objetivo.

En ese esquema, nuestra tesis se concentra en la fase 1: construir y procesar una base de datos que nos permita definir esa “primera solución razonable” para un dron VTOL, apoyándonos en métodos de imputación, regresión y análisis de modelos. La fase 2 se ocupa en: adaptar el ADRpy para incorporar la configuración VTOL y avanzar en el diseño preliminar propiamente dicho, trabajando directamente con las ecuaciones y modelos físicos que se ven afectados por ese cambio de configuración.

Ambas tesis son independientes, pero temáticamente contiguas: compartimos el mismo objetivo de fondo (diseñar un dron de ala fija con capacidad VTOL) y usamos el ADRpy como hilo conductor, pero cada trabajo aborda un tramo distinto del camino.

Cuando intentamos aplicar el ADRpy “tal cual” a nuestro caso VTOL, el primer obstáculo no fue de programación, sino de datos de entrada. Antes de poder lanzar el notebook y obtener el diagrama S/W–P/W, necesitábamos responder preguntas muy concretas:

¿Qué rango de MTOW es razonable para el tipo de misión que se busca?

¿Qué fracciones de peso podemos esperar para estructura, carga útil y sistemas VTOL?

¿Qué superficies alares, autonomías y velocidades de crucero son coherentes con lo que ya existe en el mercado?

Si esas suposiciones iniciales son completamente arbitrarias, el diagrama S/W–P/W que obtenemos puede ser “matemáticamente correcto” pero irrelevante desde el punto de vista de diseño. La solución que propone Keane funciona bien cuando tenemos una intuición previa sobre el tipo de aeronave y los órdenes de magnitud razonables. En nuestro caso, queríamos ir un paso más allá, usar el mercado como guía para fijar esos órdenes de magnitud de forma más informada.

Ahí aparece la idea de construir una tabla de drones comerciales como puente entre el mercado y el ADRpy, a partir de esa tabla, extraer tendencias (por ejemplo, cómo se relacionan peso máximo, carga útil y autonomía) y usar esas tendencias para proponer valores de entrada coherentes para nuestro dron VTOL objetivo.

Como veremos en las secciones siguientes, el intento de construir y explotar esa tabla nos llevó directamente al problema central de este capítulo: el problema de los datos y la necesidad de diseñar un sistema completo (ADRpy-VTOL) para poder abordarlo.

10.3 La primera tabla de drones: construcción y choque con la realidad

El primer paso para “aterrizar” el ADRpy en el mundo real fue construir una tabla con drones comerciales. La idea era sencilla: si lográbamos recopilar suficientes datos de plataformas existentes, podríamos ver cómo se relacionan sus parámetros principales y usar esas relaciones como guía para nuestro diseño VTOL.

Empezamos con una tabla relativamente modesta, con unos 15 parámetros por aeronave: peso máximo al despegue, peso vacío, carga útil, autonomía, velocidad de crucero, envergadura, superficie alar estimada, tipo de planta motriz, configuración general (ala fija, multirrotor, VTOL de ala fija), entre otros. A medida que avanzábamos, fuimos ampliando esa lista hasta acercarnos a los 30 parámetros, en un intento deliberado de incluir más información y ver si aparecían correlaciones útiles entre variables que a primera vista no estaban tan conectadas.

Las fuentes eran principalmente fichas técnicas y sitios web de fabricantes. No necesitábamos que la tabla fuera exhaustiva respecto al mercado mundial de drones; nos bastaba con tener un conjunto representativo de plataformas en un rango de MTOW compatible con nuestro caso de estudio. Sin embargo, cuando empezamos a llenar la tabla, el entusiasmo inicial se mezcló rápidamente con la frustración.

Lo que en un principio imaginábamos como una planilla ordenada resultó ser un mosaico de datos heterogéneos. Nos encontramos con problemas como:

Muchos campos vacíos: algunos fabricantes informaban solo el MTOW y la autonomía, otros solo la carga útil máxima, otros ni siquiera especificaban si la autonomía se refería a vuelo con carga o sin carga.

Definiciones inconsistentes: el “peso vacío” no siempre significaba lo mismo; en ciertos casos incluía batería y sensores, en otros parecía ser una estimación sin equipamiento completo. Lo mismo ocurría con “autonomía” o “endurance”, que podía referirse a tiempos de vuelo en condiciones poco comparables.

Dificultad para comparar plataformas: algunas aeronaves incluían de fábrica sensores, sistemas redundantes o equipamiento específico que otras ofrecían como opción. Esto hacía que dos drones con el mismo MTOW pudieran estar en puntos muy distintos del espacio de diseño real.

Cuando representamos algunos de esos datos en gráficos simples, esperábamos ver tendencias claras: por ejemplo, que la autonomía creciera de forma razonable con el MTOW o que la relación entre carga útil y peso máximo siguiera patrones similares a los que describen autores como Raymer. En la práctica, lo que veíamos eran nubes de puntos muy dispersas, con valores aparentemente contradictorios o difícilmente explicables sin conocer en detalle el contexto de cada plataforma.

Esta tabla no pretende ser un catálogo del mercado, sino un espejo de nuestro problema: una base de datos pequeña, llena de huecos y con definiciones poco homogéneas. A partir de allí, la idea de “usar el mercado como guía” dejó de ser un paso menor y se convirtió en un problema de investigación por derecho propio.

10.4 Del Excel caótico a la formulación del problema de los datos

Trabajar con esa primera tabla nos obligó a ponerle nombre a algo que, al principio, solo percibíamos como una incomodidad. No era simplemente que “faltaban datos”: lo que teníamos delante era un problema estructural de datos que afectaba a toda la lógica del proyecto.

En esencia, nos encontramos con dos dificultades entrelazadas:

1. Pocos datos, con muchos huecos y ambigüedades.

Incluso ampliando la tabla a unos 30 parámetros, el número total de aeronaves seguía siendo relativamente reducido y muchos campos permanecían vacíos. Además, la falta de estándares claros en las fichas técnicas hacía que ciertos parámetros fueran difíciles de interpretar, aunque estuvieran informados.

2. Herramientas insuficientes para explorar relaciones en un dataset ruidoso.

Con la planilla en la mano, podíamos hacer gráficos simples en Excel o en un notebook, pero pronto vimos que “mirar gráficos” ya no alcanzaba. Ante tanta dispersión y tantas inconsistencias, nuestra capacidad como humanos para detectar patrones útiles a ojo se volvía muy limitada.

Aquí es donde MARCO TEÓRICO entra en juego, pero de una forma distinta a la prevista inicialmente. En el marco teórico hablamos de regresión, MAPE, R^2 , underfitting, overfitting y otros conceptos estadísticos, suponiendo que teníamos una base de datos razonablemente limpia. Al enfrentarnos con la tabla real, descubrimos que antes de ajustar cualquier modelo debíamos resolver un problema previo: lograr una base de datos mínimamente coherente y disponer de herramientas para inspeccionarla en profundidad.

De esta confrontación surgió nuestro problema de los datos, que podríamos resumir así:

Necesitamos una base de datos que represente de forma realista el mercado de drones en el rango que nos interesa.

Esa base de datos, por construcción, va a tener huecos, outliers y definiciones poco homogéneas.

Si no diseñamos herramientas específicas para **imputar**, **analizar** y **visualizar** esos datos, la información que podemos extraer será pobre, y nuestras decisiones de diseño serán poco defendibles.

En otras palabras, el problema no era solo “llenar celdas vacías”, sino entender qué tipo de relaciones podía ofrecernos el mercado, qué tan confiables podían ser y cómo podíamos trasladarlas, de forma transparente, al proceso de diseño conceptual.

Este diagnóstico nos llevó a una conclusión importante: no bastaba con usar el ADRpy como estaba. Teníamos que construir alrededor de él un sistema capaz de tratar con datos reales, con todas sus imperfecciones, y hacerlo de un modo que fuese trazable, configurable y defendible desde el punto de vista de ingeniería.

10.5 Génesis del sistema ADRpy-VTOL y de sus tres módulos

Una vez formulado el problema, el proyecto empezó a cambiar de naturaleza. Lo que había comenzado como un intento de “usar el ADRpy para un VTOL” se convirtió, poco a poco, en el diseño de un sistema completo que conecta datos de mercado, estadística y herramientas de diseño preliminar.

En términos conceptuales, la solución que fuimos construyendo tiene tres pilares, que luego cristalizaron en los tres módulos del sistema ADRpy-VTOL:

Módulo 1 – Predictor e imputación: rellenar huecos de la base de datos de forma trazable, combinando métodos de similitud y correlación.

Módulo 2 – Análisis y visualización de modelos: inspeccionar con detalle los modelos generados, detectar comportamientos engañosos y quedarnos solo con aquellos que sean estadísticamente razonables y físicamente defendibles.

Módulo 3 – Asistente de diseño: usar la base imputada y las tendencias identificadas para ayudar a tomar decisiones de diseño, proponiendo valores razonables para una aeronave objetivo y comparándola con lo que ya existe en el mercado.

La historia del desarrollo refleja bien cómo creció la complejidad del proyecto. Empezamos construyendo algo muy básico, con la idea de tener un script que nos ayudara a completar algunas celdas y a ver unas pocas correlaciones sencillas. El código se fue desarrollando casi por completo en Visual Studio Code, apoyándonos en herramientas como Copilot y aprendiendo, en paralelo, tanto Python como las bibliotecas necesarias.

Lo que en un principio imaginábamos como un módulo relativamente acotado terminó convirtiéndose en un sistema mucho más amplio. El primer módulo nos llevó varios meses de trabajo continuo, con múltiples iteraciones, reescrituras y decisiones de arquitectura. Mirando hacia atrás, sentimos que hoy podríamos reimplementar una versión equivalente en mucho menos tiempo, porque el principal esfuerzo no fue solo programar, sino descubrir qué problemas había que resolver y en qué orden.

En las primeras versiones, muchos parámetros se pedían directamente al usuario mediante ventanas o cuadros de diálogo (por ejemplo, con interfaces sencillas tipo Tkinter) o por consola. Esa estrategia funcionaba para pruebas rápidas, pero se volvió problemática cuando el sistema empezó a crecer: era fácil que el pipeline quedara bloqueado esperando una entrada manual en medio de una ejecución,

especialmente si lanzábamos todo desde un botón en un notebook. Esa experiencia nos llevó a repensar la arquitectura: separar claramente la lógica de cálculo del módulo predictor de las interfaces de usuario, y concentrar la configuración en paneles o secciones específicas que se establecen antes de lanzar el procesamiento.

Esta separación entre “motor” e “interfaz” fue una decisión clave. Nos permitió rediseñar las interfaces sin romper el núcleo del sistema y facilitó la incorporación posterior de paneles de configuración más claros y notebooks con pestañas dedicadas a similitud, correlación y pipeline de imputación.

10.5.1 El nacimiento del Módulo 2

El **Módulo 2** no estaba en el plan original. Surgió como respuesta a un problema muy concreto que encontramos al empezar a ajustar modelos con nuestro módulo predictor. En algunos casos, obteníamos modelos con métricas excelente según los criterios estadísticos (por ejemplo, errores medios bajos o coeficientes de determinación altos), pero que producían predicciones absurdas, valores negativos para magnitudes físicas que no pueden ser negativas, o valores de la orden de magnitud totalmente alejados de lo que es razonable.

Este fenómeno se hacía especialmente evidente en los modelos con dos predictores, donde ya no teníamos una línea de regresión, sino una superficie en un espacio bidimensional. La complejidad del comportamiento crecía, y con ella la posibilidad de que el modelo se ajustara muy bien a los puntos disponibles, pero extrapolara mal en zonas con pocos datos.

Para entender qué estaba ocurriendo y poder defender las decisiones del sistema, necesitábamos herramientas que fueran más allá de un simple índice numérico. De ahí nació la idea del Módulo 2: un conjunto de funciones y visualizaciones que nos permitieran, ver la superficie del modelo en relación con los datos de entrenamiento, analizar la cobertura de los datos respecto a la superficie, y aplicar medidas adicionales, como indicadores basados en colinealidad (VIF) o componentes principales, para detectar cuando la combinación de predictores llevaba a modelos poco fiables.

En este contexto, el tratamiento de outliers pasó a ser algo más que un detalle técnico: era una parte esencial de la robustez del sistema. Tuvimos que decidir cómo identificar puntos atípicos, cómo ponderarlos o filtrarlos y cómo reflejar esas decisiones en las métricas de confianza explicadas en el MARCO TEÓRICO. La discusión detallada de estos métodos se desarrollará más adelante; aquí nos interesa remarcar que el Módulo 2 nació precisamente para enfrentar este tipo de problemas, donde los modelos “mienten” si solo miramos un número.

10.5.2 El papel de cada módulo en el diseño de una aeronave

Aunque los tres módulos se apoyan mutuamente, no juegan el mismo papel en el proceso de diseño. Desde la perspectiva de quien quiere diseñar una aeronave concreta, el **Módulo 3** es el más directamente útil: es el que toma el dataset (enriquecido por el Módulo 1 y filtrado por el Módulo 2) y lo convierte en recomendaciones concretas para una aeronave objetivo, ayudando a ordenar qué valores de masa, carga útil, autonomía o potencia tienen sentido usar como punto de partida.

El **Módulo 1** es, sin embargo, el más complejo desde el punto de vista estadístico y de flujo de datos. Es donde se combinan métodos de similitud, regresiones de distinta estructura, manejo de outliers y validaciones encadenadas para imputar valores faltantes de forma trazable. Diseñar este módulo implicó partir casi desde cero, enfrentando problemas que no habíamos anticipado y revisando la arquitectura varias veces a medida que aparecían nuevos casos límite.

El **Módulo 2**, por su parte, se convirtió en el puente entre la estadística y la ingeniería: permite ver, evaluar y depurar los modelos que salen del módulo predictor, de manera que las decisiones de diseño no se apoyen en relaciones espurias o físicamente inaceptable.

Mirando el conjunto, podríamos decir que:

el **Módulo 1** construye y limpia la base de conocimiento;

el **Módulo 2** la audita y la hace interpretable;

el **Módulo 3** la traduce en decisiones de diseño.

Si hubiéramos conocido desde el principio la magnitud de este trabajo, probablemente habríamos imaginado un proyecto más acotado. Pero precisamente en ese crecimiento inesperado está buena parte del valor formativo de esta tesis: nos obligó a pensar el diseño conceptual no solo como un conjunto de fórmulas, sino como un sistema de datos, modelos y herramientas que debe convivir con la realidad desordenada del mercado.

10.6 Rol del sistema en el proceso de diseño y en la decisión de compra

El sistema ADRpy-VTOL cumple dos funciones complementarias. Por un lado, actúa como asistente de diseño: a partir de la base de datos imputada y depurada, nos ayuda a elegir valores iniciales razonables para una aeronave objetivo (pesos, autonomía, carga útil, potencias, etc.), coherentes con lo que observamos en el mercado. Por otro lado, funciona como un mapa de mercado: al situar esa aeronave objetivo en el mismo espacio de variables que los drones comerciales, podemos ver si estamos apuntando a una zona ya muy poblada —donde quizá tenga más sentido comprar o adaptar una plataforma existente— o a una región poco explorada, que justifica invertir en un diseño nuevo.

Ambas funciones se apoyan en el mismo flujo de información. De forma condensada, el sistema trabaja así:

1. Partimos de un Excel inicial, con datos incompletos y heterogéneos de drones del mercado.
2. El **Módulo 1 (predictor e imputación)** procesa esa base: filtra, rellena huecos y genera una versión enriquecida del dataset, donde los valores imputados tienen trazabilidad y se ha tenido en cuenta la presencia de outliers.
3. El **Módulo 2 (análisis y visualización de modelos)** nos permite revisar manualmente los modelos generados si es que detectamos algún inconveniente o simplemente se busca inspeccionar por curiosidad.
4. El **Módulo 3 (asistente de diseño)** usa esa base depurada para dos tareas: comparar una aeronave objetivo con el mercado y proponer conjuntos de parámetros coherentes que luego pueden emplearse como entradas en el ADRpy adaptado para VTOL o como referencia para decidir entre diseñar o comprar.

En este contexto, el manejo de outliers es parte del diseño del sistema y no un detalle marginal. Sabemos que los datos reales de mercado contienen valores extremos, algunos legítimos y otros probablemente poco fiables. Por eso incorporamos criterios explícitos para detectarlos, regular su influencia y reflejar esas decisiones en las métricas de confianza de los modelos. No buscamos “limpiar” los datos hasta que parezcan ideales, sino trabajar con su complejidad de forma controlada, de modo que las relaciones que finalmente utilizamos en el diseño sigan teniendo sentido físico.

10.7 Resumen y puente hacia la descripción técnica de los módulos

En este capítulo mostramos cómo pasamos del ADRpy original de Keane al sistema ADRpy-VTOL. Al intentar usar el ADRpy para un dron de ala fija con capacidad VTOL, nos encontramos con una base de datos de mercado pequeña, incompleta y heterogénea. Ese choque nos obligó a formular el problema de los datos y a diseñar una solución en tres módulos: un predictor que imputa y mejora la tabla, un analizador que permite revisar y filtrar los modelos entrenados y un asistente de diseño que traduce esa información en recomendaciones concretas y comparaciones con el mercado.

Los siguientes capítulos entrarán en la “cocina” de este sistema. Primero describiremos con más detalle cómo funciona el Módulo 1, qué tipos de modelos utiliza y cómo organiza el flujo de imputación. Luego abordaremos el Módulo 2, centrado en el análisis y la visualización de modelos, y finalmente el Módulo 3, donde se materializa la interacción con la aeronave objetivo y el vínculo con el ADRpy adaptado para VTOL. De este modo, lo que aquí presentamos como historia y motivación se convierte, en los capítulos técnicos, en un conjunto concreto de algoritmos, estructuras de datos y decisiones de implementación.

11 ARQUITECTURA DEL SISTEMA Y GUÍA DE USO

En los capítulos anteriores hablamos de motivaciones, de diseño conceptual de aeronaves y del papel que puede jugar un enfoque estadístico. A partir de aquí comienza la parte más aplicada de la tesis. Antes de describir la lógica interna del módulo predictor, del análisis de modelos o del asistente de diseño, nos parece importante abrir la “tapa” del sistema y mostrar qué hay adentro.

En este capítulo presentamos la **arquitectura general del proyecto ADRpy-VTOL** y explicamos **cómo se usa** en la práctica: qué carpetas contiene, qué significan los distintos scripts, qué rol tienen los notebooks y qué ve el usuario cuando lanza los dashboards de cada módulo. El objetivo es que cualquier ingeniero o estudiante, aunque no tenga experiencia programando, pueda orientarse dentro del código y entender qué ocurre cuando se ejecuta el sistema.

Nuestro sistema no es una aplicación web empaquetada ni un programa de escritorio que se instala con un clic. Es un **proyecto de código en Python** organizado en carpetas, scripts y cuadernos de trabajo (notebooks).

Desde el punto de vista del usuario final, todo podría reducirse a “abrir tres notebooks y trabajar con sus dashboards”. Sin embargo, detrás de esos paneles hay una serie de decisiones de diseño que conviene documentar, cómo se separan las responsabilidades entre los distintos módulos, qué archivos intervienen cuando se imputan datos o se analizan modelos, dónde se guardan los resultados y cómo se podrían reutilizar o extender.

Explicar esta arquitectura tiene dos ventajas claras:

1. **Transparencia:** el sistema deja de ser una caja negra; se puede inspeccionar, auditar y criticar.
2. **Continuidad:** futuros estudiantes podrán retomar el proyecto sin empezar desde cero, entendiendo qué hace cada pieza y dónde encaja en el flujo general.

Por eso dedicamos un capítulo completo a describir la estructura antes de entrar en el detalle de algoritmos.

11.1 Conceptos básicos de programación utilizados en este trabajo

En esta sección introducimos algunos conceptos mínimos de programación. No pretendemos dar un curso de Python, sino crear un lenguaje común con el lector para poder hablar del sistema sin tecnicismos innecesarios.

En términos sencillos:

Un **script** es un archivo de texto con extensión .py que contiene instrucciones que Python ejecuta de arriba hacia abajo. Cada script suele encargarse de una parte del problema (por ejemplo, “cargar datos”, “imputar celdas”, “generar gráficos”).

Un **módulo** es un conjunto de scripts relacionados que trabajan sobre un mismo tema. En nuestro caso distinguimos tres grandes módulos:

- el **módulo predictor**, que limpia los datos e imputa valores faltantes;
- el **módulo de análisis de modelos**, que permite inspeccionar y comparar las ecuaciones generadas;
- el **módulo asistente de diseño**, que utiliza la base imputada para sugerir parámetros para nuevas aeronaves.

Un **paquete** es una carpeta que agrupa módulos y contiene un archivo especial `__init__.py`. Eso permite que otras partes del sistema puedan “importar” su contenido con una instrucción de Python.

Una **función** es un bloque de código que realiza una tarea concreta y que podemos reutilizar las veces que haga falta. Por ejemplo, una función puede leer un archivo Excel y devolver un conjunto de datos listo para procesar.

Un archivo de tipo **main** es el punto de entrada a un módulo: un script que decide en qué orden se llaman las funciones y que coordina el trabajo de los demás scripts. En nuestro proyecto existen varios “main”, asociados a distintos usos.

Un **notebook** (.ipynb) es un cuaderno interactivo donde podemos mezclar texto y código. El código se ejecuta por celdas y los resultados aparecen debajo. En nuestro proyecto se usaron los notebooks como puerta de entrada para el usuario.

Un **dashboard** es una interfaz gráfica que aparece al ejecutar ciertas celdas del notebook. En él se muestran controles (botones, casillas, menús desplegables) con los que se puede trabajar sin escribir código.

Con estas nociones en mente, podemos hablar del proyecto con frases como “el notebook del módulo 1 llama al dashboard, que a su vez dispara el script de imputación” sin que quede en el aire qué significa cada palabra.

11.2 Estructura general de carpetas de archivos

El proyecto ADRpy-VTOL se organiza en una estructura de carpetas que refleja las distintas funciones del sistema. De forma simplificada, la parte que utilizamos en esta tesis se puede describir así:

Una carpeta principal del proyecto, que contiene:

- una carpeta asociada al módulo predictor y de análisis (en las capturas la vemos como una carpeta de análisis, con subcarpetas para scripts (“Módulos”), datos de entrada y resultados);
- una carpeta para el módulo asistente de diseño, con sus propios scripts;
- una carpeta notebooks con los tres cuadernos de trabajo principales;
- una carpeta scripts que contiene el código original de ADRpy, usado como base para modelos atmosféricos, análisis de restricciones y otras herramientas de diseño.

En la tesis resumimos esta estructura mediante la **Imagen 11.1 - Estructura general de carpetas y archivos** **Imagen 11.1**, la misma no pretende listar absolutamente todos los archivos del repositorio, sino destacar aquellos que son relevantes para el flujo de trabajo descrito en los capítulos siguientes.

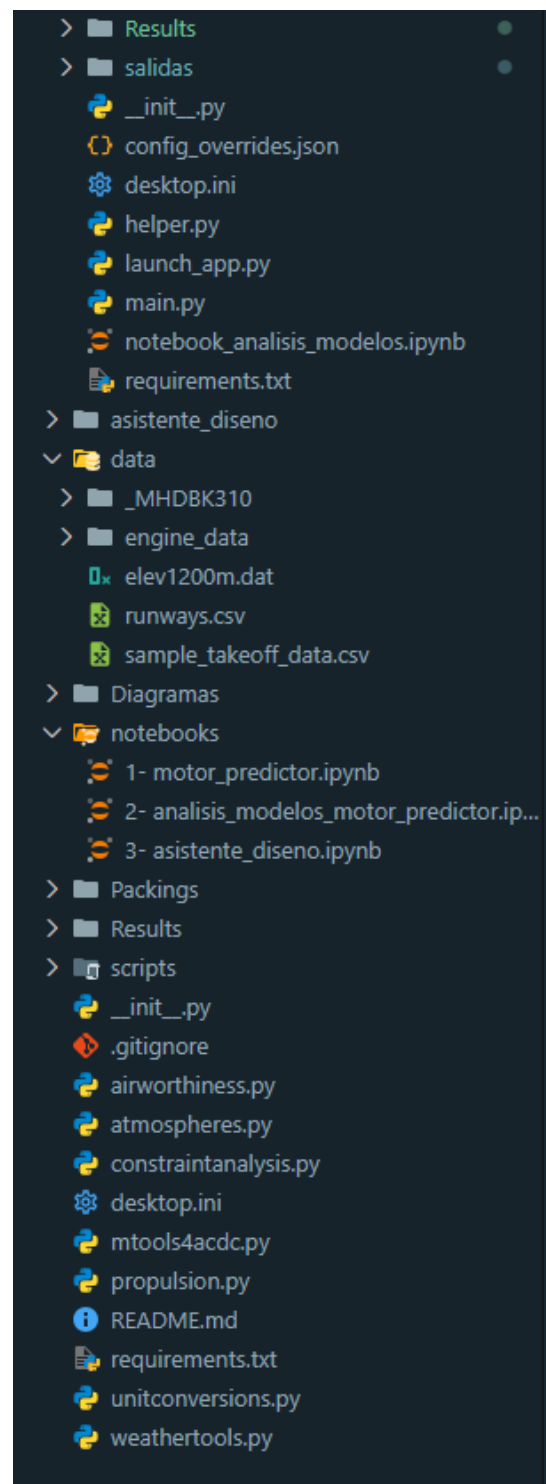
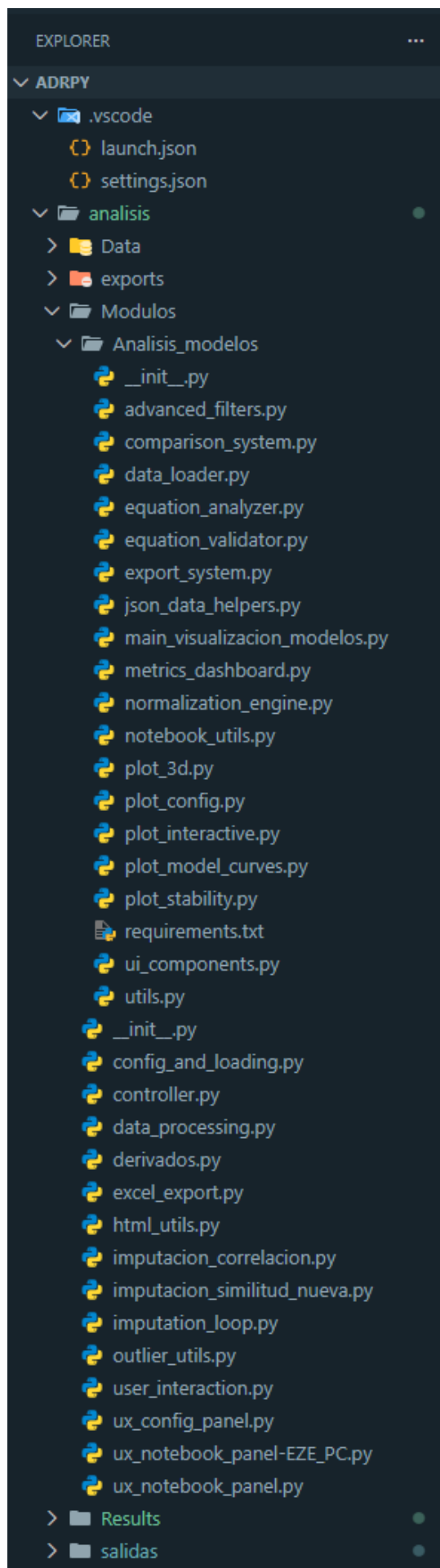


Imagen 11.1- Estructura general de carpetas y archivos

Esta imagen muestra un árbol de directorios reducido, donde se distinguen claramente; el bloque de scripts del módulo predictor, el bloque de scripts del módulo de análisis de modelos, el bloque de scripts del asistente de diseño, la carpeta de notebooks, y la carpeta de código base de ADRpy.

11.3 Entorno de trabajo y lanzamiento general del sistema

El sistema está pensado para ejecutarse en un entorno de Python que soporte notebooks, como puede ser un editor de código con soporte para Jupyter (VS Code en nuestro caso) o un entorno Jupyter clásico.

A nivel práctico, el usuario necesita:

1. tener Python instalado y configurar un entorno donde pueda ejecutar cuadernos (.ipynb);
2. descargar el proyecto ADRpy-VTOL y abrir la carpeta raíz en ese entorno;
3. instalar las dependencias indicadas en los archivos de requisitos del proyecto.

No reproducimos aquí todas las instrucciones técnicas, ya que pueden variar con el tiempo y se describen en el material de soporte del repositorio. Lo importante para la tesis es entender que, la interfaz principal no es una ventana de aplicación independiente, sino los tres notebooks que acompañan al proyecto, al ejecutar ciertas celdas en esos notebooks, se abren dashboards que permiten usar el sistema sin escribir código adicional.

En las secciones siguientes describimos cómo se organiza este uso para cada módulo.

11.4 Módulo 1: sistema predictor (visión desde el usuario)

El primer módulo con el que trabaja el usuario es el sistema predictor, responsable de transformar una base de datos incompleta en una tabla imputada que pueda usarse para análisis y diseño.

El flujo de uso, visto desde la persona que opera el sistema, puede resumirse así:

1. Se abre el notebook del primer módulo (1- motor_predictor.ipynb) en el entorno de trabajo.

Se ejecutan las celdas iniciales, que: preparan el entorno, incorporan la ruta a los scripts del módulo predictor e importan la función que construye el dashboard.



Imagen 11.2 - Lanzamiento del dashboard del Módulo 1 (predictor)

Captura del notebook donde se invoca la función que abre el dashboard del módulo predictor para configurar parámetros de imputación y ejecutar el proceso.

Una celda específica invoca esa función y abre el dashboard del módulo predictor. Este panel central ofrece varias pestañas para configurar el comportamiento de la imputación. Entre ellas se incluyen secciones dedicadas a: criterios de orquestación y control general del proceso, parámetros de similitud entre aeronaves, parámetros de correlación y validación de modelos, opciones relacionadas con el tratamiento del Excel y la exportación de resultados.

Importante: Por defecto no se pedirán confirmaciones manuales; podés ajustar el comportamiento en la sección Confirmaciones.

Orquestación Similitud **Correlación** Loop/Orquestación Excel HTML

▼ Checks 2D

☒ Activar checks 2D

☒	r máx	0.9
☒	VIF máx	10
☒	PC2 ratio mín	0.03
☒	Rango mínimo	2
☒	Condición máx	100000
☒	Cobertura pares únicos mín	0.6
☒	Cobertura hull mín	0.15
☒	Cobertura elipse mín	0.1
☒	n/param (linear-2)	8
☒	n/param (poly-2)	10
☒	Modo agresivo r mín	0.95

► Diversidad mínima

► Extrapolación

► Modelos

► Confianza

► Selección/LOOCV

► Outliers

Guardar overrides **Guardar y ejecutar**

Imagen 11.3 - Dashboard del Módulo 1: configuración antes de ejecutar

Vista del panel donde el usuario ajusta opciones (similitud/correlación, Top-K, penalizaciones, segmentación, etc.) y luego inicia la corrida de imputación.

Una vez que el usuario ajusta los parámetros según sus preferencias de estudio, dispone de un botón que guarda la configuración y lanza el proceso de imputación. En ese momento se encadena la ejecución de varios scripts del módulo predictor, que se encargan, entre otras tareas, de: cargar y normalizar los datos de entrada, detectar y tratar valores atípicos, aplicar la imputación por similitud, aplicar la imputación por correlación, combinar resultados y registrar índices de confianza.

2. Como salida principal, el módulo genera:

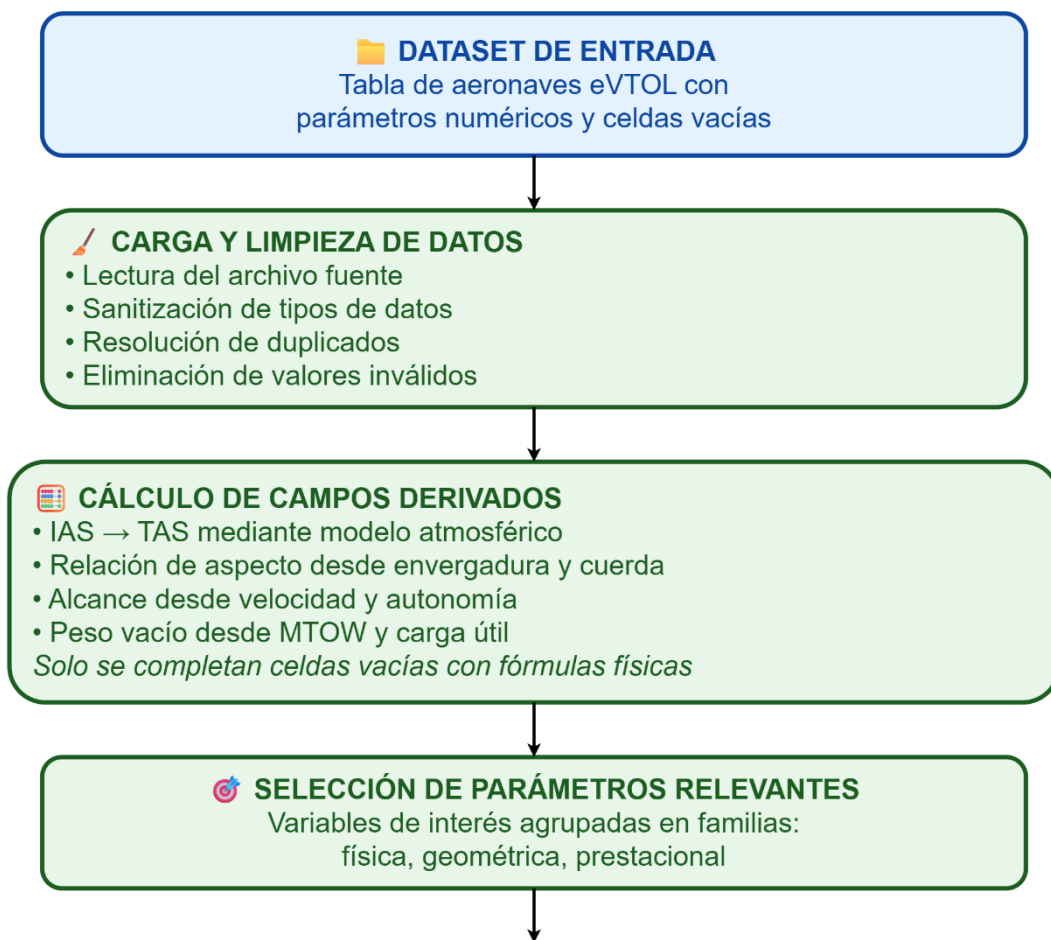
un archivo de datos imputados (en formato de hoja de cálculo), que contiene la misma estructura de la tabla original, pero con muchas de las celdas vacías completadas;

uno o varios archivos auxiliares (por ejemplo, en formato de texto estructurado “.json”) que describen los modelos de correlación creados y la información necesaria para analizarlos posteriormente.

	A	B	C	D	E	F	G	H	I	J	K	L	M	N	O	P	Q	R	S	T	U	V	W	X	Y	Z	AA
	Modelo	Distancia de carrera requerida para despegue	Altitud a la que se realiza el crucero	Velocidad a la que se realiza el crucero	Techo de servicio máximo	Velocidad de pérdida (m/s CAS)	Área del ala	Relación de aspecto del ala	Longitud del fuselaje	Profundidad del fuselaje	Ancho del fuselaje	Peso máximo al despegue (MTOW)	Alcance de la aeronave (km)	Autonomía de la aeronave (h)	Velocidad máxima (m/s IAS)	Velocidad de pérdida (m/s CAS)	Tasa de ascenso	Radio de giro	Envergadura	Cuerda	Payload	duración en VTOL	Velocidad crucero (m/s IAS)	Battery Weight	Peso frame	Peso vacío fuel y battery	Peso Vaso (MTOW payload)
1	Stalker XE	5,000.0	16,63	12,000.0	NaN	0,87	15,301	2,1	NaN	0,211	13,6	370,0	8,0	20,0	NaN	NaN	NaN	5,657	0,239	2,495	2,0	15,433	NaN	NaN	NaN	10,886	
2	Stalker VXE30 Fuel Cell	5,000.0	17,34	12,000.0	NaN	1,158	15,326	2,591	NaN	0,2	19,95	433,0	8,0	25,034	NaN	NaN	NaN	4,877	0,318	2,495	NaN	16,093	NaN	NaN	NaN	17,46	
3	Stalker VXE30 Battery	5,000.0	17,34	12,000.0	NaN	1,158	15,326	2,591	NaN	0,2	21,8	222,0	4,0	25,034	NaN	NaN	NaN	4,877	0,318	2,495	NaN	16,093	NaN	NaN	NaN	19,28	
4	Edge Autonomy Penguin C Mk 2.5 VTOL	5,000.0	16,59	9,700.0	NaN	NaN	13,421	NaN	NaN	0,283	41,0	665,28	12,0	33,5	NaN	NaN	NaN	4,12	NaN	NaN	NaN	15,4	NaN	NaN	NaN	NaN	
5	Aerosonde Mk. 4.7 Fixed Wing	5,000.0	26,93	14,700.0	NaN	1,55	12,5	3,0	NaN	0,277	42,2	1,782,0	19,8	33,439	NaN	NaN	NaN	4,4	0,352	14,5	NaN	25,0	NaN	NaN	NaN	27,7	
6	Aerosonde Mk. 4.7 VTOL	5,000.0	26,93	9,700.0	NaN	1,55	12,5	3,0	NaN	0,277	53,5	1,080,0	12,0	33,439	NaN	NaN	NaN	4,4	0,352	11,3	NaN	25,0	NaN	NaN	NaN	42,2	
7	Aerosonde Mk. 4.8 Fixed wing	5,000.0	26,93	18,200.0	NaN	1,55	12,5	3,0	NaN	0,277	54,4	1,782,0	19,8	33,439	NaN	NaN	NaN	4,4	0,352	17,7	NaN	25,0	NaN	NaN	NaN	36,7	
8	Aerosonde Mk. 4.8 VTOL FTUAS	5,000.0	26,93	15,000.0	NaN	1,55	12,5	3,0	NaN	0,277	93,0	NaN	14,0	40,837	NaN	NaN	NaN	4,4	0,352	22,7	NaN	25,0	NaN	NaN	NaN	70,3	
9	AAI Aerosonde	5,500.0	37,64	15,000.0	NaN	0,57	14,754	1,7	NaN	0,217	13,1	3,270,0	26,0	30,846	NaN	NaN	NaN	2,9	0,197	NaN	NaN	34,94	NaN	10,0	NaN	NaN	
10	Fulmar X	5,000.0	29,95	9,842	NaN	NaN	14,994	1,2	NaN	0,24	20,0	800,0	8,0	41,7	NaN	NaN	NaN	3,0	NaN	NaN	NaN	NaN	NaN	NaN	NaN	43,0	
11	Orbiter 4	5,000.0	NaN	NaN	NaN	NaN	12,944	1,2	NaN	0,265	55,0	NaN	24,0	36,0	NaN	NaN	NaN	5,2	NaN	NaN	12,0	NaN	NaN	NaN	NaN	26,5	
12	Orbiter 3	5,000.0	26,429	NaN	NaN	NaN	13,964	1,2	NaN	0,204	32,0	514,569	6,0	36,0	NaN	NaN	NaN	4,4	NaN	NaN	5,5	NaN	NaN	NaN	NaN	43,0	
13	Menfis	5,000.0	17,99	NaN	NaN	1,192	15,002	1,48	NaN	0,212	6,5	25,0	2,0	25,6	NaN	NaN	NaN	2,1	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	
14	ScanEagle	5,000.0	30,16	19,500.0	NaN	1,238	14,377	1,71	NaN	0,238	26,5	1,814,4	18,0	41,2	NaN	NaN	NaN	3,1	0,316	NaN	NaN	NaN	NaN	NaN	NaN	NaN	
15	Integrator	5,000.0	30,49	19,500.0	NaN	1,145	NaN	2,5	NaN	NaN	74,8	2,445,12	24,0	46,3	NaN	NaN	NaN	4,8	0,301	NaN	NaN	NaN	NaN	NaN	NaN	NaN	
16	Integrator VTOL	5,000.0	NaN	NaN	NaN	2,039	12,533	NaN	NaN	NaN	75,0	NaN	16,0	37,351	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	
17	Integrator Extended Range	5,000.0	7,87	19,500.0	NaN	1,145	14,674	2,5	NaN	0,222	74,8	500,0	19,0	46,3	NaN	NaN	NaN	4,8	0,301	NaN	NaN	NaN	NaN	NaN	NaN	NaN	
18	ScanEagle 3	5,000.0	25,32	20,0	NaN	1,065	12,91	2,4	NaN	0,265	36,3	1,522,8	18,0	41,2	NaN	NaN	NaN	4,0	0,289	NaN	NaN	NaN	NaN	NaN	NaN	NaN	
19	RQ Nan 21A BlackJack	5,000.0	33,29	20,0	NaN	1,145	12,885	2,5	NaN	0,271	61,0	1,779,84	16,0	46,3	NaN	NaN	NaN	4,8	0,301	NaN	NaN	NaN	NaN	NaN	NaN	NaN	
20	DeltaQuad Evo	5,000.0	17,82	15,0	NaN	0,84	14,934	0,75	NaN	NaN	10,0	270,0	4,53	32,064	NaN	NaN	NaN	2,69	0,253	NaN	NaN	NaN	NaN	NaN	NaN	NaN	
21	DeltaQuad Pro RMAP	5,000.0	17,24	13,123	NaN	NaN	NaN	0,9	NaN	NaN	6,2	100,0	1,83	NaN	NaN	NaN	NaN	2,35	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	
22	DeltaQuad Pro RCARGO	5,000.0	17,24	13,123	NaN	NaN	NaN	0,9	NaN	NaN	6,2	100,0	1,83	NaN	NaN	NaN	NaN	2,35	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	
23	V21	5,000.0	19,39	15,000.0	14,0	0,8	NaN	0,93	NaN	0,287	10,0	194,4	3,0	33,0	14,0	NaN	100,0	2,15	0,247	NaN	NaN	NaN	NaN	NaN	NaN	NaN	
24	V25	5,000.0	21,55	16,000.0	15,5	0,52	NaN	0,93	NaN	0,287	12,5	288,0	4,0	33,0	15,5	NaN	120,0	2,45	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	
25	V32	5,000.0	21,55	16,000.0	17,0	0,945	14,192	1,0	NaN	0,287	23,5	324,0	4,5	33,0	17,0	NaN	150,0	3,2	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	
26	V35	5,000.0	26,93	16,000.0	NaN	1,566	13,186	1,88	NaN	0,287	32,0	252,0	2,8	33,0	NaN	NaN	NaN	3,5	0,223	NaN	NaN	NaN	NaN	NaN	NaN	NaN	
27	V39	5,000.0	26,93	16,000.0	NaN	0,945	14,192	NaN	NaN	0,287	24,0	405,0	4,5	33,0	NaN	NaN	NaN	3,9	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	
28	Volitation VT370	5,000.0	26,93	17,000.0	NaN	2,039	12,333	2,02	NaN	0,287	40,0	1,350,0	15,0	33,0	NaN	NaN	NaN	6,5	0,24	NaN	NaN	NaN	NaN	NaN	NaN	NaN	
29	Skyeye 2600	5,000.0	35,55	NaN	10,0	0,88	14,516	2,05	NaN	0,195	15,0	237,6	2,0	31,171	10,0	NaN	NaN	2,6	0,259	NaN	NaN	NaN	NaN	NaN	NaN	NaN	
30	Skyeye 2930 VTOL	5,000.0	25,85	NaN	18,0	1,0	13,938	2,03	NaN	0,208	28,0	259,2	3,0	30,0	18,0	NaN	NaN	2,93	0,277	NaN	NaN	NaN	NaN	NaN	NaN	NaN	
31	Skyeye 3600	50,0	5,000.0	25,898	NaN	12,5	1,33	13,186	2,488	NaN	0,243	28,0	515,96	4,5	32,663	12,5	NaN	3,6	0,327	NaN	NaN	NaN	NaN	NaN	NaN	NaN	
32	Skyeye 3600 VTOL	5,000.0	32,32	NaN	24,0	1,32	13,186	2,42	NaN	0,242	40,0	300,0	6,0	33,0	24,0	NaN	NaN	3,6	0,325	NaN	NaN	NaN	NaN	NaN	NaN	NaN	
33	Skyeye 5000	60,0	5,000.0	35,55	NaN	15,0	2,615	12,18	3,5	NaN	0,375	90,0	950,4	8,0	42,0	15,0	NaN	5,0	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	
34	Skyeye 5000 VTOL	5,000.0	30,16	NaN	NaN	2,615	NaN	3,5	NaN	0,375	100,0	800,0	8,0	42,0	NaN	NaN	NaN	5,0	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	
35	Skyeye 5000 VTOL octo	5,000.0	37,7	NaN	24,0	2,615	12,598	3,5	NaN	0,375	100,0	14,955	38,0	24,0	NaN	NaN	NaN	5,0	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	
36	Volitation VT510	5,000.0	32,32	17,000.0	25,0	2,629	NaN	2,905	NaN	0,375	100,0	540,0	5,0	50,0	25,0	NaN	NaN	5,1	0,346	NaN	NaN	NaN	NaN	NaN	NaN	NaN	
37	Ascend	5,000.0	21,55	10,000.0	13,0	NaN	NaN	1,562	NaN	0,26	9,5	432,0	6,0	30,0	13,0	NaN	NaN	2,0	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	
38	Transition	5,000.0	21,55	13,000.0	13,0	0,929	NaN	2,3	NaN	0,26	18,0	864,0	12,0	30,0	13,0	NaN	NaN	3,0	0,273	NaN	NaN	NaN	NaN	NaN	NaN	NaN	
39	Reach	5,000.0	26,93	16,000.0	13,0	1,113	13,704	4,712	NaN	0,306	91,0	1,800,0	20,0	35,0	13,0	NaN	NaN	6,0	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	
40	HYFLY H510	5,000.0	29,84	NaN	NaN	NaN	13,186	NaN	NaN	NaN	75,0	NaN	NaN	NaN	NaN	NaN	NaN	3,0	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	
41	HYFLY H405	5,000.0	26,93	NaN	NaN	NaN	14,192	1,5	NaN	NaN	25,0	NaN	NaN	NaN	NaN	NaN	NaN	1,5	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	
42	JOUAV CW-007	5,000.0	18,31	19,500.0	14,372	NaN	NaN	NaN	NaN	NaN	7,0	56,3	0,92	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	
43	JOUAV CW-15	5,000.0	18,25	20,000.0	NaN	NaN	14,934	2,06	NaN	NaN	17,0	182,95	3,0	28,579	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	
44	JOUAV CW-25E	5,000.0	20,9	13,000.0	NaN	NaN	13,928	2,18	NaN	NaN	31,58	244,44	3,5	NaN	NaN	NaN	NaN	4,35	0,259	NaN	NaN	NaN	NaN	NaN	NaN	NaN	
45	JOUAV CW-25H	5,000.0	23,92	NaN	NaN	1,095	14,516	2,1	NaN	NaN	31,0	435,0	5,5	30,577	NaN	NaN	NaN	4,4	0,249	NaN	NaN	NaN	NaN	NaN	NaN	NaN	

Imagen 11.4 - Archivos de salida del Módulo 1

Ejemplo de los artefactos generados por el predictor: DataFrame imputado (Excel) y reportes/archivos auxiliares (p. ej., JSON de resultados y HTML/plots).



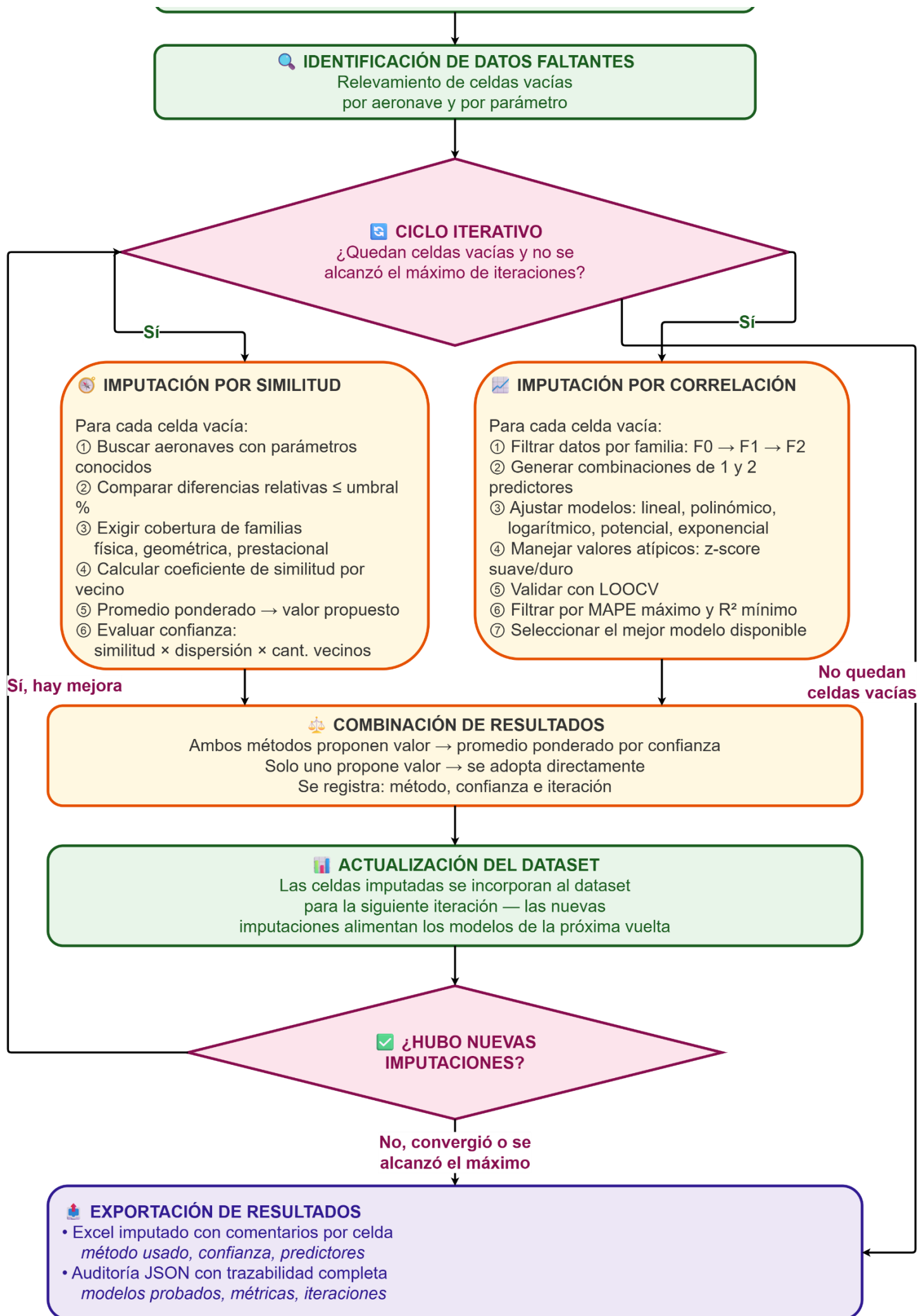


Imagen 11.5 - Flujo de procesamiento del Módulo 1

Diagrama de pipeline iterativo que combina imputación por similitud y por regresión para completar parámetros faltantes en el dataset de aeronaves eVTOL, con exportación de auditoría y trazabilidad por celda.

11.5 Módulo 2: análisis y visualización de modelos

Una vez ejecutado el sistema predictor, el usuario dispone de una base de datos completada y de un conjunto de modelos de correlación almacenados. El **módulo 2** permite inspeccionar esos modelos con calma, comprobar si tienen sentido y compararlos entre sí.

El uso típico es el siguiente:

1. Se abre el notebook del segundo módulo (2- *analisis_modelos_motor_predictor.ipynb*).

```
# Analisis de modelos - Motor Predictor (notebooks)
print("Preparando entorno...")

import sys
import os
import time

# Asegurar rutas correctas
BASE_DIR = r"C:/Users/delpi/OneDrive/Tesis/ADRPY-VTOL/ADRPY"
ANALISIS_DIR = os.path.join(BASE_DIR, 'analisis')
if ANALISIS_DIR not in sys.path:
    sys.path.insert(0, ANALISIS_DIR)
if os.getcwd() not in sys.path:
    sys.path.append(os.getcwd())

# Importar utilidades (sin reload para rapidez)
print("Importando utilidades...")
start = time.time()
try:
    from Modulos.Analisis_modelos import notebook_utils
    from Modulos.Analisis_modelos.notebook_utils import ControlPanel
    print(f"Importado en {time.time()-start:.2f}s")
except Exception as e:
    print(f"Error importando utilidades: {e}")
    raise SystemExit
```

Imagen 11.6 - Notebook del Módulo 2: preparación del análisis de modelos

Celdas iniciales del módulo de análisis que localizan los archivos producidos por el Módulo 1, cargan la base imputada y preparan estructuras para el dashboard.

Las celdas iniciales: localizan los archivos generados por el módulo predictor, cargan la información necesaria sobre ecuaciones y métricas e importan funciones especializadas en análisis y visualización.

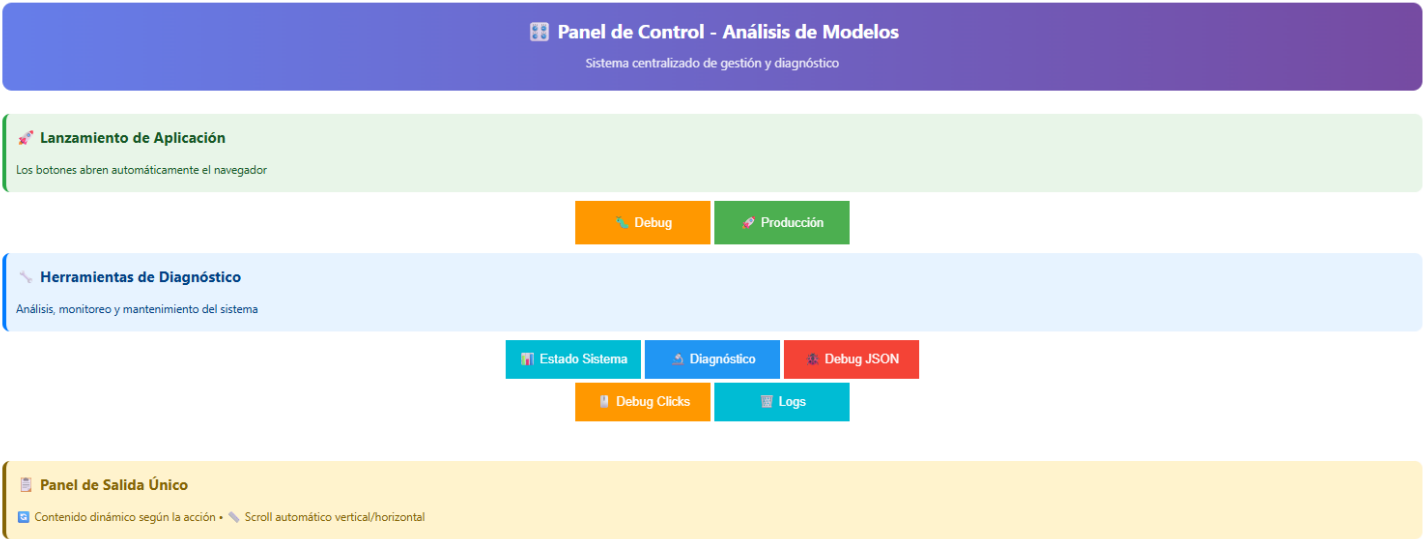
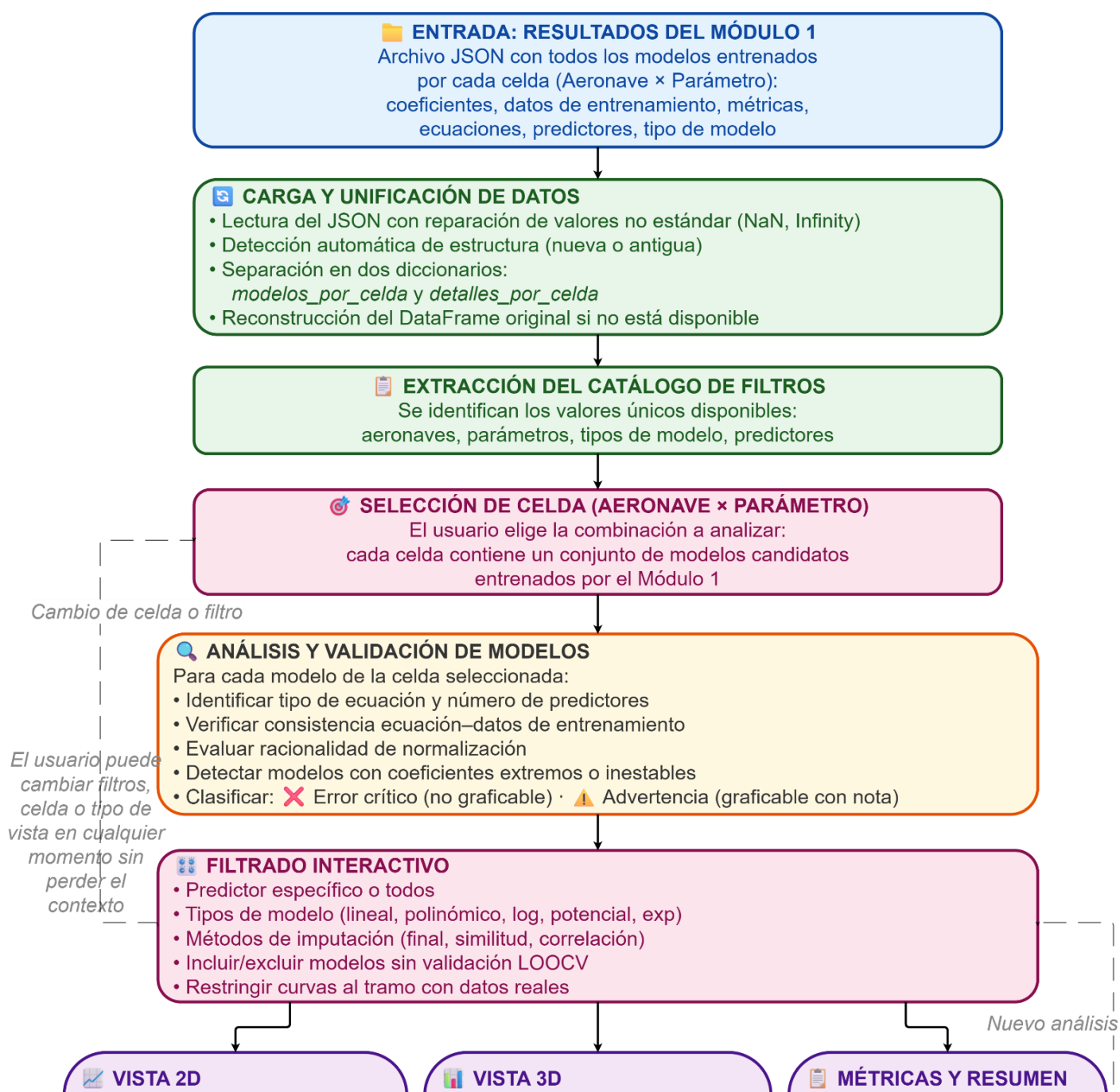


Imagen 11.8 - Dashboard del Módulo 2: análisis interactivo de modelos.

Captura del panel que permite seleccionar parámetro y predictores, comparar modelos, visualizar curvas/superficies y consultar métricas (incluida LOOCV).

El módulo 2 no modifica la base de datos imputada; se limita a leer lo que ya existe y a ofrecer herramientas para interpretar y depurar el conjunto de modelos.

En capítulos posteriores se detallará cómo se calculan las métricas, cómo se representan las curvas y qué criterios se usan para aceptar o descartar modelos. Aquí nos enfocamos en mostrar la posición del módulo 2 dentro del flujo global.



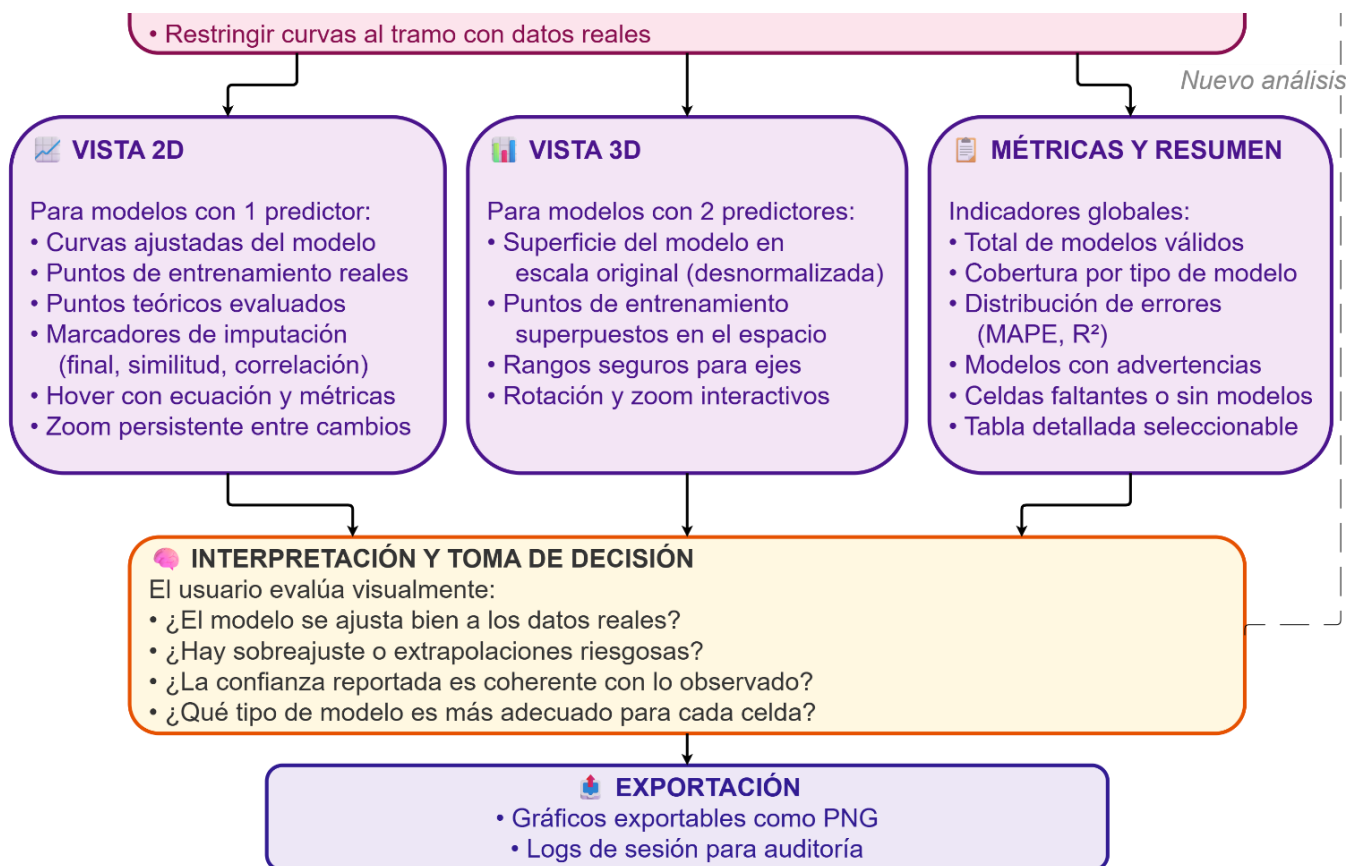


Imagen 11.9 - Esquema del rol del Módulo 2 (lectura y curación)

Diagrama/concepto: el Módulo 2 trabaja en modo “solo lectura” sobre resultados del predictor para interpretar, auditar y depurar modelos, sin modificar el Excel imputado.

11.6 Módulo 3: asistente de diseño para nuevas aeronaves

El tercer módulo cierra el ciclo, utilizando la base de datos imputada y las tendencias derivadas para guiar al usuario en la definición de parámetros para nuevas aeronaves. En la práctica, el asistente toma los requerimientos iniciales “en lenguaje de usuario” (tipo de misión, masa máxima al despegue, tipo de despegue y aterrizaje, autonomía, velocidad de crucero, etc.) y los convierte en un conjunto de parámetros globales coherentes para una aeronave objetivo. Estos valores de partida serán los que, en capítulos posteriores, se tomen como base para construir el vector de entrada que alimentará los cálculos en ADRpy.

El esquema general de uso es el siguiente:

1. Se abre el notebook del tercer módulo (3- asistente_diseno.ipynb).

Las celdas iniciales: cargan la base de datos imputada y otros archivos necesarios, importan funciones que organizan la información por tipos de aeronaves, tendencias y rangos típicos, preparan la interfaz gráfica.



Imagen 11.10 - Apertura del dashboard del Módulo 3 (asistente de diseño)

Captura del notebook donde se lanza el asistente; el usuario define objetivos y explora valores coherentes con la base imputada y las tendencias derivadas.

2. Una celda abre el dashboard del asistente de diseño.

En él, el usuario puede:

- definir una aeronave objetivo (por ejemplo, tipo de misión, masa máxima al despegue, tipo de despegue y aterrizaje);
- explorar qué valores de otros parámetros (como superficie alar o potencia instalada) parecen coherentes con los que se observan en la base de datos;
- detectar combinaciones que se alejan de las tendencias conocidas y que podrían requerir una justificación especial o un diseño más arriesgado.
- obtener un conjunto de parámetros globales de partida, explícitamente documentados, que servirán como insumo para la construcción del vector de entrada de la aeronave objetivo en ADRpy.

El módulo asistente no escribe nada en la base de datos original. Funciona como una herramienta de consulta y simulación ligera: ayuda a proponer valores razonables, pero no modifica los registros almacenados.

Resumen global/filtrado: Alcance de la aeronave (km)

DataFrame original	DataFrame filtrado
Resumen (global)	Resumen (filtrado)
Cobertura: 64/83 (NaN 22.89%)	Cobertura: 64/83 (NaN 22.89%)
Min: 25.00 • Q1: 210.60 •	Min: 25.00 • Q1: 210.60 •
Mediana: 365.00 • Media:	Mediana: 365.00 • Media:
669.27 • Q3: 982.80 • Max:	669.27 • Q3: 982.80 • Max:
3270.00	3270.00
Rango IQR: [-947.70, 2141.10]	Rango IQR: [-947.70, 2141.10]
Sugerido (Top-K): 434.00	
Objetivo: 500.00	

Ayuda de controles

Ranking ... Tendenc... Sugerenc... O... Panel ...

- Ranking y similitud
- Tendencias X-Y
- Sugerencias (Top-K)
- Outliers

Imagen 11.11 - Vista general del asistente de diseño (Módulo 3)

Interfaz del asistente mostrando controles globales, lista de parámetros y panel de detalle, como base del flujo “definir–explorar–ajustar” una aeronave objetivo.

11.7 Flujo global entre los tres módulos

Aunque cada módulo se puede usar por separado, el sistema adquiere todo su sentido cuando se ve como un flujo continuo de datos y decisiones. De forma resumida:

1. Partimos de un Excel original con datos incompletos de aeronaves.
2. El **Módulo 1 (sistema predictor)** limpia la tabla, imputa valores faltantes y genera:
 - un nuevo archivo con datos completados,
 - un archivo con la descripción de los modelos de correlación utilizados.
3. El **Módulo 2 (análisis de modelos)** toma esos resultados y permite:
 - inspeccionar ecuaciones y métricas,
 - descartar modelos poco fiables,
 - entender mejor las relaciones que el sistema ha identificado.
4. El **Módulo 3 (asistente de diseño)** utiliza la base imputada y las tendencias derivadas para:
 - apoyar la selección de parámetros para nuevas aeronaves,
 - advertir sobre combinaciones inusuales o alejadas de la práctica observada.
 - producir un conjunto de valores de partida coherentes para la aeronave objetivo, que luego se traducen en el vector de entrada utilizado en el caso de estudio con ADRpy.

Este encadenamiento se representa en:

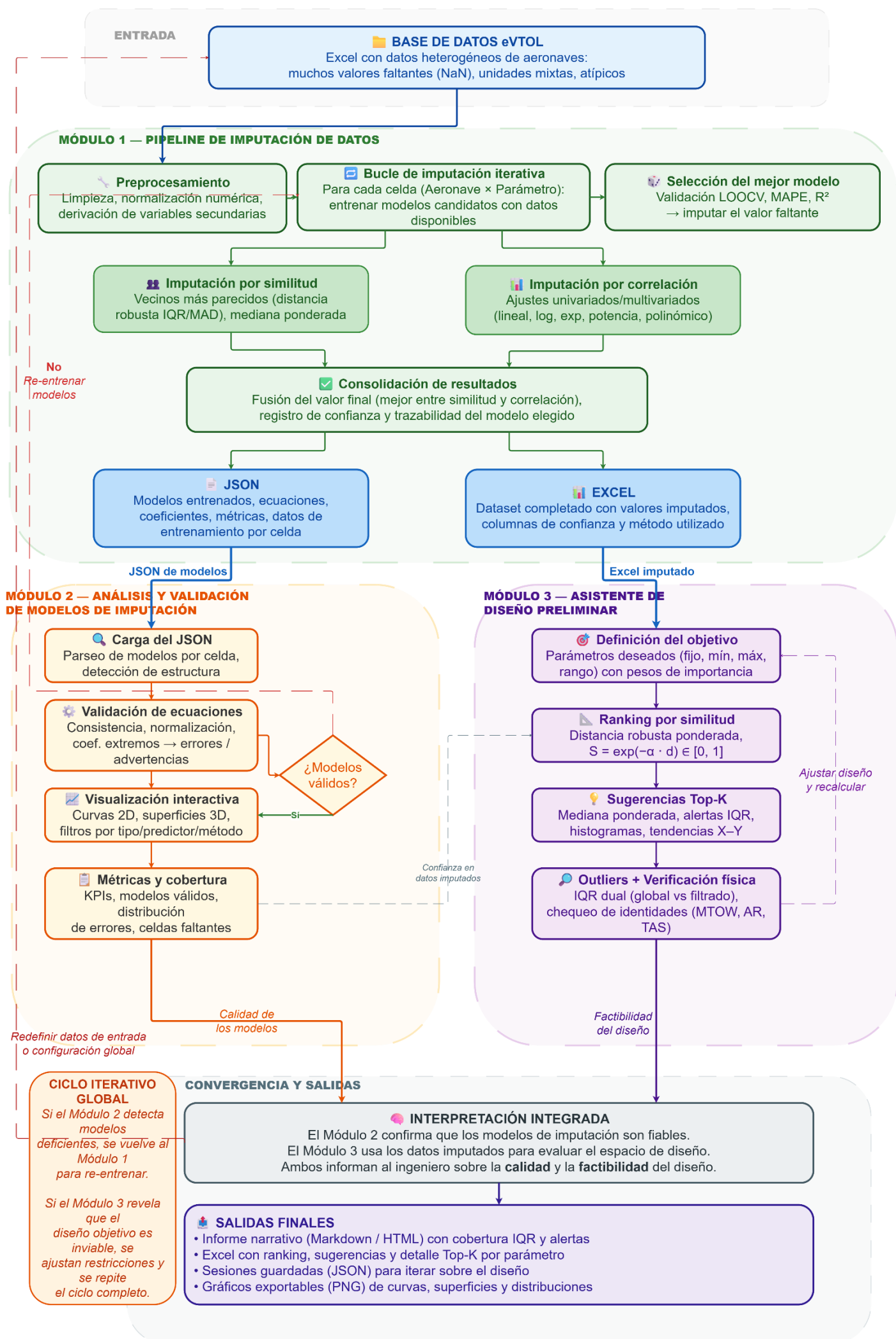


Imagen 11.12 - Diagrama de flujo global: de la base de datos original al asistente de diseño

En el diagrama se ven las tres etapas como bloques conectados por archivos intermedios (datos imputados y descripción de modelos), destacando que el Módulo 2 y el Módulo 3 se alimentan de lo que produce el Módulo 1.

De esta manera, el lector puede situar cada capítulo técnico posterior (dedicado a similitud, correlación o selección de modelos) dentro de un contexto claro.

11.8 Scripts principales y su papel en el sistema

Para completar la visión de alto nivel, resulta útil disponer de una referencia rápida sobre qué hace cada archivo importante. No listamos aquí todos los scripts del proyecto, sino aquellos que son centrales en el flujo descrito:

Scripts del módulo predictor

- `config_and_loading.py`: se encarga de leer los archivos de entrada y aplicar configuraciones generales.
- `data_processing.py`: realiza transformaciones previas sobre los datos, como conversiones de unidad y creación de variables derivadas.
- `derivados.py`: agrupa cálculos de parámetros que no vienen explícitos en la base original y se obtienen a partir de otros.
- `outlier_utils.py` y archivos relacionados de outliers: implementan la detección y el tratamiento de valores atípicos.
- `imputacion_similitud_nueva.py` y scripts de similitud: contienen la lógica de imputación basada en aeronaves vecinas.
- `imputacion_correlacion.py`: implementa la imputación por modelos de regresión.
- `imputation_loop.py`: coordina el bucle general que recorre las celdas, decide qué método aplicar y registra los resultados.

Scripts del módulo de análisis de modelos

- `equation_analyzer.py`: carga y organiza la información sobre las ecuaciones generadas.
- `comparison_system.py`: ayuda a comparar modelos y métricas asociadas.
- `plot_3d.py` y otros scripts de gráficos: generan visualizaciones bidimensionales y tridimensionales.
- `normalization_engine.py`: gestiona la normalización de variables cuando se representan y comparan modelos.

Scripts del módulo asistente de diseño

- `datos.py`: carga y organiza la base imputada para su uso en el asistente.
- `tipos.py`: clasifica las aeronaves por tipo de misión y otras categorías relevantes.
- `tendencias.py`: calcula tendencias y rangos típicos que servirán como referencia para nuevas aeronaves.

- `similitud.py` y `verificacion.py`: aplican criterios de similitud y de verificación de coherencia a las propuestas del usuario.

11.9 Automatización, mantenimiento y límites prácticos

El proyecto ADRpy-VTOL surge de una necesidad muy concreta: automatizar un flujo de trabajo que sería inabordable a mano si se quisiera repetir para distintas bases de datos o distintas configuraciones de estudio.

Al programar el sistema, podemos volver a ejecutar todo el pipeline con nuevas aeronaves o con diferentes umbrales de similitud y criterios de modelo, sin reescribir fórmulas ni macros, reducimos errores de cálculo repetitivos, ya que las mismas funciones se aplican de forma consistente a todos los casos, dejamos la puerta abierta para que otros estudiantes incorporen mejoras, ya sea añadiendo nuevos parámetros, extendiendo los dashboards o implementando métodos de imputación más avanzados.

Al mismo tiempo, la automatización trae consigo algunas responsabilidades, el código debe mantenerse y adaptarse si cambian las versiones de las bibliotecas utilizadas, los cambios en la lógica de imputación o en la interfaz deben probarse sobre conjuntos de datos pequeños antes de aplicarse a toda la base, es recomendable documentar las decisiones importantes (por ejemplo, umbrales por defecto, criterios de decisión entre métodos) para evitar que el sistema se convierta en una caja negra incluso para quienes lo desarrollaron.

En esta tesis damos los primeros pasos en esa dirección: dejamos descrita la arquitectura general, el papel de los módulos y los archivos principales, y mostramos cómo se encadenan las piezas desde el Excel original hasta el asistente de diseño.

11.10 Resumen y vínculo con los capítulos siguientes

En este capítulo presentamos la estructura arquitectónica del sistema ADRpy-VTOL y una guía de uso de sus tres módulos, vimos qué entendemos por script, módulo, paquete, notebook y dashboard en el contexto de este proyecto, describimos las carpetas y archivos principales de manera superficial mostramos cómo el usuario interactúa con el sistema a través de tres notebooks y cómo, a partir de ellos, se lanzan dashboards que ejecutan los procesos de imputación, análisis y asistencia de diseño, resumimos el flujo global que lleva de una base incompleta a una herramienta interactiva para definir parámetros de nuevas aeronaves.

A partir de esta base, el lector ya puede orientarse dentro del código y entender qué ocurre, al menos a nivel de bloques, cuando se ejecuta el sistema.

En el capítulo siguiente nos centraremos en el primer módulo, el sistema predictor. Allí describiremos con más detalle el conjunto de datos de aeronaves, las dificultades específicas que presenta y la lógica con la que el módulo limpia la tabla e imputa los valores faltantes, apoyándonos en el contenido presentado aquí.

12 MÓDULO 1: Sistema predictor e imputación de la base de datos

En los capítulos anteriores fuimos construyendo el contexto de este trabajo: partimos de ADRpy y del proceso clásico de diseño conceptual, vimos el problema real de los datos de drones y describimos cómo organizamos el software en tres módulos. En este capítulo entramos, por fin, en el “primer motor” operativo del sistema: el Módulo 1, responsable de transformar una tabla de aeronaves llena de huecos en una base imputada, trazable y utilizable por los módulos siguientes.

El objetivo aquí no es solo describir algoritmos. Se pretende que el lector pueda seguir el flujo completo: desde abrir el Excel de entrada hasta obtener un Excel enriquecido y un archivo JSON listo para que el Módulo 2 visualice modelos y el Módulo 3 apoye el diseño de una nueva aeronave. Por eso combinamos una mirada de usuario (qué se ve, qué decisiones hay que tomar) con una mirada interna (qué hace realmente el código en cada etapa).

12.1 Rol del Módulo 1 en el flujo del sistema

En nuestro sistema, el Módulo 1 ocupa el primer eslabón operativo después del diagnóstico del problema de datos explicado en capítulos anteriores. Recibe como entrada una base de datos de drones (**Imagen 12.1**) que está incompleta, heterogénea y a veces inconsistente, y la transforma en dos productos principales:

- un archivo Excel con la misma estructura básica, pero con muchas de las celdas faltantes imputadas y claramente identificadas, y
- un archivo JSON que resume, celda por celda, qué se imputó, con qué método y con qué información de apoyo, que luego aprovecharán los dashboards de los Módulos 2 y 3.

	A	B	C	D	E	F	G	H	I	J	K	L	M	N	O	P	Q	R	S	T	U	V	W	X	Y	Z
	Modelo	Distancia de carrera requerida para despegue	Altitud a la que se realiza el crucero (m/s IAS)	Velocidad a la que se realiza el crucero (m/s IAS)	Techo de servicio máximo	Velocidad de pérdida (m/s CAS)	Área del ala	Relación de aspect ratio	Longitud del fuselaje	Profundidad del fuselaje	Ancho del fuselaje	Peso máximo al despegue (kg)	Alcance de la aeronave (km)	Autonomía de la aeronave (h)	Velocidad máxima (m/s IAS)	Velocidad de pérdida (m/s CAS)	Tasa de ascenso	Radio de giro	Inversora	Cuerda	Páyan	duración en VTOL	Velocidad crucero (m/s IAS)	Battery Weight	Peso frame	Peso vacío fuel+ battery
1					12.000,0		0,87	15,301	2,1		0,211	13,6	370,0	8,0	20,0				3,657	0,239	2,495	2,0	15,433			
2	Stalker XE				12.000,0		1,158	15,326	2,591		0,2	19,95	433,0	8,0	25,034				4,877	0,318	2,495		16,093			
3	Stalker VXE30 Fuel Cell				12.000,0		1,158	15,326	2,591		0,2	21,8	222,0	4,0	25,034				4,877	0,318	2,495		16,093			
4	Stalker VXE30 Battery											41,0		12,0	33,5				4,12				15,4			
5	Edge Autonomy Penguin C Mk 2.5 VTOL				14.700,0		1,55	12,5	3,0		0,277	42,2	19,8	33,439					4,4	0,352	14,5		25,0			
6	Aerosonde Mk. 4.7 Fixed Wing				9.700,0		1,55	12,5	3,0		0,277	53,5	12,0	33,439					4,4	0,352	11,3		25,0			
7	Aerosonde Mk. 4.7 VTOL				18.200,0		1,55	12,5	3,0		0,277	54,4	19,8	33,439					4,4	0,352	17,7		25,0			
8	Aerosonde Mk. 4.8 Fixed wing				15.000,0			12,5				93,0		14,0							22,7					
9	Aerosonde Mk. 4.8 VTOL FTUAS	5.500,0			15.000,0		0,57	14,754	1,7		13,1	3.270,0	26,0	30,846					2,9	0,197					10,0	
10	AAI Aerosonde				9.842				1,2		20,0	800,0	8,0	41,7					3,0				27,8			
11	Fulmar X								1,2		55,0		24,0	36,0					5,2		12,0					
12	Orbiter 4								1,2		32,0		6,0	36,0					4,4		5,5					
13	Orbiter 3								1,48		6,5	25,0	2,0	25,6					2,1				16,7			
14	Mantis				19.500,0		1,71				26,5		18,0	41,2					3,1		5,0		28,0			
15	ScanEagle				19.500,0				2,5		74,8		24,0	46,3					4,8		18,0		28,3			
16	Integrator										75,0		16,0									18,0				
17	Integrator VTOL	5.000,0											19,0	41,2					4,8		18,0					
18	Integrator Extended Range				19.500,0				2,5		74,8	500,0	19,0	46,3					4,8		18,0					
19	ScanEagle 3				20,0				2,4		36,3		18,0	41,2					4,0		8,6		23,5			
20	RQ Nan 21A BlackJack				20,0				2,5		61,0		16,0	46,3					4,8		17,7		30,9			
21	DeltaQuad Evo				13,0		0,84		0,75		10,0	270,0	4,53						2,69		3,0		16,54		4,8	4,8
22	DeltaQuad Pro #MAP				13.123				0,9		6,2	100,0	1,83						2,35		1,2		16,0			
23	DeltaQuad Pro #CARGO				13.123				0,9		6,2	100,0	1,83						2,35		1,2		16,0			
24	V21				15.000,0	14,0	0,8		0,93		10,0		3,0	33,0	14,0			100,0	2,15		1,5		18,0		2,65	
25	V25				16.000,0	15,5	0,52		0,93		12,5		4,0	33,0	15,5			120,0	2,45		2,2		20,0		8,45	
26	V32				16.000,0	17,0			1,0		23,5		4,5	33,0	17,0			150,0	3,2		5,0		20,0		6,45	
27	V35				16.000,0				1,88		32,0		2,8	33,0					3,5		10,0		25,0			
28	V39				16.000,0						24,0		4,5	33,0					3,9		5,0		25,0			
29	Volitation VT370				17.000,0				2,02		40,0		15,0	33,0					6,5		18,0		25,0			
30	Skyeye 2600					10,0	0,88		2,05		15,0		2,0		10,0				2,6		4,0		33,0		6,5	
31	Skyeye 2930 VTOL					18,0	1,0		2,03		28,0		3,0	30,0	18,0				2,93		6,0		24,0		7,1	
32	Skyeye 3600	50,0				12,5	1,33		2,488		28,0		4,5		12,5				3,6		10,0				11,5	
33	Skyeye 3600 VTOL					24,0	1,32		2,42		40,0	300,0	6,0	33,0	24,0				3,6		10,0		30,0		11,0	
34	Skyeye 5000	60,0				15,0	2,615		3,5		0,375	90,0	8,0	42,0	15,0				5,0		20,0		33,0		32,0	
35	Skyeye 5000 VTOL						2,615		3,5		0,375	100,0	800,0	8,0	42,0				5,0		25,0		28,0			
36	Skyeye 5000 VTOL octo								3,5		0,375	100,0			38,0	24,0			5,0		15,0		35,0		35,0	
37	Volitation VT510				17.000,0	25,0			2,905		100,0		5,0	50,0	25,0				5,1		25,0		30,0			
38	Ascend				10.000,0				1,552		9,5		6,0	30,0	13,0				2,0		0,6	0,05	20,0		3,0	6,0
39	Transition				13.000,0				2,3		18,0		12,0	30,0	13,0				3,0		1,5	0,05	20,0		5,8	11,8
40	Reach				16.000,0				4,712		91,0		20,0	35,0	13,0				6,0		7,0	0,05	25,0		31,0	54,0
41	HYFLY H510										75,0								3,0		10,0		27,7			
42	HYFLY H405								1,5		25,0								1,5		5,0		25,0			
43	JOUAV CW-007				19.500,0						7,0		0,92								1,0		17,0			
44	JOUAV CW-15				20.000,0				2,06		17,0		3,0								3,0		16,94			
45	JOUAV CW-25E				13.000,0				2,18		31,58		3,5						4,35		6,0		19,4			
46	JOUAV CW-25H								2,1		31,0	435,0	5,5						4,4		4,0		22,2			

Imagen 12.1 - Extracto del Excel de la base de aeronaves

Ejemplo de la estructura de la base de datos (filas por aeronave y columnas por parámetro), incluyendo celdas faltantes que motivan la imputación.

Desde el punto de vista del flujo de diseño conceptual, el Módulo 1 cumple un papel similar al de las bases empíricas que utilizan autores como Raymer para obtener relaciones preliminares de diseño: prepara un “terreno estadístico” razonable para que luego tenga sentido hablar de tendencias, correlaciones y comparaciones con una aeronave objetivo [1].

En resumen, este módulo responde a una pregunta muy concreta:

“si esta es la base de datos de drones que tenemos, con todos sus agujeros, ¿cómo podemos completarla de forma razonable, documentada y configurable, en lugar de improvisar valores a mano en Excel?”.

12.2 Descripción de la base de datos de drones

La materia prima del Módulo 1 es un archivo Excel que reúne del orden de ochenta aeronaves de distintos fabricantes y configuraciones. No buscamos abarcar todos los UAV del mercado, sino tener una muestra suficientemente diversa como para capturar tendencias de diseño en drones de ala fija de tamaño pequeño y mediano, con distintas misiones y niveles de complejidad.

Las columnas del archivo representan parámetros que, desde el punto de vista del diseño preliminar, consideramos relevantes. A grandes rasgos, podemos agruparlos en cuatro familias:

- **Parámetros de masa y carga útil**, como masa máxima al despegue (MTOW), peso vacío, carga útil y, cuando corresponde, peso de baterías o combustible.
- **Parámetros geométricos**, como envergadura, superficie alar, cuerda media aerodinámica y dimensiones generales del fuselaje.
- **Parámetros de propulsión y energía**, como potencia instalada, número de motores, tensión y capacidad de baterías o potencia del motor de combustión.
- **Parámetros de misión y prestaciones**, como velocidad de crucero, autonomía, alcance, techo de servicio y razón de ascenso.

En muchos casos, los fabricantes no entregan todos estos datos de forma directa. Parte del trabajo previo a este módulo consistió en recopilar información dispersa en fichas técnicas, páginas web oficiales y documentación complementaria de cada plataforma. La experiencia fue la típica de cualquier intento de construir una base de datos en ingeniería: para algunas aeronaves hay fichas muy detalladas y para otras apenas sabemos el MTOW y la autonomía anunciada.

En términos prácticos, el Módulo 1 trata este Excel como un “contrato” de entrada: asume que las columnas básicas estarán presentes y que el usuario podrá añadir nuevas variables en el futuro, siempre que respete la estructura general. Toda la lógica de lectura y preprocesamiento se diseñó para ser lo más tolerante posible a pequeñas variaciones, sin perder el control de qué significa cada columna.

12.3 Calidad, completitud y limitaciones de los datos

Como ya adelantamos en la INTRODUCCIÓN, la base de datos que usamos no es un conjunto “perfecto” de datos experimentales, sino una colección de información pública que arrastra inevitables limitaciones. Aquí nos interesa mirar esas limitaciones con la lupa del Módulo 1, es decir, desde el punto de vista de alguien que tiene que programar qué hacer con cada celda vacía.

En términos de completitud, la situación típica es la siguiente:

- Hay parámetros casi siempre presentes, como MTOW, misión general de la aeronave o tipo de planta motriz.
- Hay otros con presencia intermedia, como autonomía, velocidad de crucero o superficie alar, que están disponibles para un porcentaje significativo de aeronaves, pero no para todas.
- Finalmente, hay variables muy valiosas, pero escasamente reportadas, como el peso vacío detallado, la capacidad de baterías o la razón de ascenso, que aparecen solo en una fracción de la muestra.

A esto se suma la ambigüedad de las definiciones. Distintos fabricantes usan criterios diferentes para “peso vacío”, combinan o no la carga útil con sensores y a veces redondean prestaciones de forma optimista. Desde el punto de vista matemático, esto introduce ruido y posibles sesgos en cualquier regresión; desde el punto de vista de la implementación, nos obliga a ser prudentes con la imputación, evitando dar una falsa sensación de precisión allí donde la propia base es incierta.

En lugar de ocultar estos problemas, el Módulo 1 los hace explícitos. Cuando documentamos el porcentaje de completitud por parámetro clave (por ejemplo, mediante una tabla tipo [Tabla 12.1]), no lo hacemos para exhibir una “estadística bonita”, sino para justificar las decisiones posteriores: qué parámetros se imputan con más confianza, cuáles se dejan sin completar y en qué casos un modelo estadístico, por muy sofisticado que parezca, no debería utilizarse.

PARÁMETRO	Registros totales	Valores numéricos completos	Valores faltantes/no numéricos	% completado
Battery weight	83	1	82	1,20%
Potencia específica (p/w)	83	1	82	1,20%
Maximum crosswind (rachas)	83	2	81	2,41%
Tasa de ascenso	83	2	81	2,41%
Distancia de carrera requerida para	83	2	81	2,41%
Radio de giro	83	3	80	3,61%
Altitud a la que se realiza el crucero	83	3	80	3,61%
Tiempo de emergencia en vuelo	83	4	79	4,82%
Wing loading	83	4	79	4,82%
Duración en vtol	83	4	79	4,82%
Potencia watts	83	4	79	4,82%
Consumo	83	5	78	6,02%
Potencia hp	83	6	77	7,23%
Cuerda	83	7	76	8,43%
Capacidad combustible	83	7	76	8,43%

Relación de aspecto del ala	83	8	75	9,64%
Ancho del fuselaje	83	10	73	12,05%
Peso vacío - fuel y battery	83	10	73	12,05%
Velocidad de pérdida limpia (m/s cas)	83	14	69	16,87%
Velocidad de pérdida (m/s cas)	83	14	69	16,87%
Precio	83	15	68	18,07%
Peso frame	83	15	68	18,07%
Maximum crosswind (sostenido)	83	17	66	20,48%
Área del ala	83	17	66	20,48%
Peso vacío (mtow - payload)	83	20	63	24,10%
Alcance de la aeronave (km)	83	21	62	25,30%
Velocidad máxima (m/s ias)	83	48	35	57,83%
Rango de comunicación	83	52	31	62,65%
Techo de servicio máximo	83	53	30	63,86%
Velocidad crucero (m/s ias)	83	65	18	78,31%
Longitud del fuselaje	83	70	13	84,34%
Payload	83	72	11	86,75%
Envergadura	83	75	8	90,36%
Autonomía de la aeronave (h)	83	79	4	95,18%
Misión	83	83	0	100,00%
Cantidad de motores propulsión	83	83	0	100,00%
Cantidad de motores propulsión vertical	83	83	0	100,00%
Peso máximo al despegue (mtow)	83	83	0	100,00%
Propulsión horizontal	83	83	0	100,00%
Despegue	83	83	0	100,00%
Propulsión vertical	83	83	0	100,00%
Clase	83	83	0	100,00%

Tabla 12.1 - Completitud de la base de datos por parámetro (estado inicial)

Para cada variable de la base de UAV se reportan registros totales, valores numéricos disponibles, faltantes/no numéricos y porcentaje de completitud, como insumo para priorizar qué parámetros conviene imputar y con qué nivel de cautela.

12.4 Preprocesamiento y variables derivadas

Antes de hablar de imputación, necesitamos explicar qué ocurre cuando el usuario pulsa por primera vez el botón de “cargar datos” en el Módulo 1. Aunque por fuera parezca un paso trivial, por dentro se desencadena una secuencia de operaciones que preparan el terreno para todo lo que viene después.

En la versión actual del sistema, el flujo básico de preprocesamiento puede resumirse así:

Primero, cargamos el archivo Excel de entrada y lo convertimos en un DataFrame de pandas. En este momento se aplican algunas comprobaciones mínimas de integridad: se valida que ciertas columnas clave estén presentes, se homogenizan nombres y se identifican columnas que no son de datos (por ejemplo,

notas o columnas auxiliares que el usuario pueda haber añadido y que el sistema debe ignorar o tratar con cuidado).

Después, se realizan algunas transformaciones de unidades y cálculos de variables derivadas. En lugar de limitarse a las magnitudes “tal cual” vienen del fabricante, el Módulo 1 construye ratios que son más útiles desde el punto de vista del diseño conceptual, como carga alar (MTOW/superficie alar), densidad de potencia (potencia instalada/MTOW) o relaciones entre peso vacío y MTOW. Estas variables derivadas no solo son útiles para las correlaciones posteriores, sino que también ayudan a detectar inconsistencias en los datos originales.

Finalmente, los datos se organizan en estructuras internas que el resto del módulo puede utilizar de forma coherente: DataFrames filtrados por tipo de aeronave, diccionarios con metadatos de cada columna, listas de parámetros candidatos para ser predictores, etc. Todo esto ocurre antes de intentar rellenar una sola celda, porque la calidad de la imputación depende directamente de cómo hayamos entendido y estructurado la base.

Este mismo paso de preprocesamiento marca el punto en el que empezamos a construir los artefactos de salida: además del Excel enriquecido, el Módulo 1 irá acumulando información que luego se volcará en el archivo JSON para el Módulo 2. Desde aquí ya se decide qué campos de contexto acompañarán a cada imputación: identificadores de aeronave, nombres de parámetros, unidades y, cuando corresponde, información sobre normalización de variables.

12.5 Criterios generales de imputación y funciones principales

Una vez que tenemos la base preprocesada, el corazón del Módulo 1 es la decisión que tomamos cada vez que encontramos una celda vacía: ¿qué hacemos con ese hueco? La respuesta general del sistema combina dos enfoques complementarios, que ya adelantamos en el MARCO TEÓRICO: imputación por similitud e imputación por correlación.

A nivel conceptual, definimos tres principios de diseño para cualquier imputación:

1. **Coherencia física:** el valor propuesto debe tener sentido con la familia de aeronaves a la que pertenece el dron objetivo. No basta con que la ecuación ajuste bien los datos; también debe respetar el orden de magnitud y las restricciones básicas del problema (por ejemplo, no producir pesos negativos o autonomías imposibles).

2. **Trazabilidad:** el sistema debe registrar no solo el valor imputado, sino también cómo se obtuvo: qué vecinos se usaron en la similitud, qué modelo de correlación se empleó, qué métricas de calidad obtuvo y en qué rango de datos se entrenó. Esta información es la que luego explota el Módulo 2.

3. **Configurabilidad:** en lugar de fijar “a fuego” umbrales y criterios en el código, decidimos exponerlos en el dashboard del Módulo 1. Parámetros como el porcentaje máximo de diferencia permitido entre aeronaves similares (por defecto 20 %) o el número mínimo de datos para entrenar un modelo de correlación se pueden ajustar desde la interfaz, sin tocar el código.

Desde el punto de vista de la implementación, estos principios se concretan en tres familias de funciones principales:

- Funciones de **similitud**, que reciben una aeronave objetivo y un parámetro faltante, buscan vecinos razonablemente parecidos y devuelven un valor imputado por promedio o combinación ponderada.

- Funciones de **correlación**, que construyen modelos entre el parámetro objetivo y uno o varios predictores, evalúan su calidad con métricas como MAPE y R^2 (según lo discutido en el MARCO TEÓRICO) y, si pasan los filtros definidos, los emplean para imputar.

- Funciones de **orquestación**, que recorren la base celda por celda, deciden en qué orden aplicar similitud y correlación, combinan resultados cuando ambos métodos están disponibles y registran todo en las estructuras que luego se exportan a Excel y JSON.

En lo que resta del capítulo vamos a detallar estos dos bloques de imputación, empezando por la similitud, que fue el primer enfoque que implementamos y el más intuitivo para alguien que viene del mundo del diseño de aeronaves y no necesariamente de la estadística.

12.6 Imputación por similitud

En el Módulo 1, la imputación por similitud es nuestro primer intento para rellenar un dato faltante usando únicamente aeronaves reales que se parezcan a la aeronave objetivo. La idea es intuitiva: si encontramos varios drones “parecidos” en masa, geometría y prestaciones, podemos estimar el valor que falta a partir de un promedio ponderado de lo que hacen esos vecinos.

En esta sección explicamos el proceso paso a paso, utilizando las mismas reglas que implementa el script `imputacion_similitud_nueva.py`, pero en lenguaje accesible.

12.6.1 Punto de partida: una celda vacía en la base

Partimos de una tabla donde cada fila es una aeronave y cada columna es un parámetro (MTOW, envergadura, autonomía, potencia, etc.). El sistema toma:

- una **aeronave objetivo** (fila X),
- un **parámetro faltante** que se quiere imputar (por ejemplo, potencia instalada).

El resto del módulo ya se encargó de cargar el Excel, limpiar columnas y calcular algunas variables derivadas. Aquí nos centramos solo en cómo rellenar esa celda vacía.

12.6.2 Comparar la aeronave objetivo con posibles vecinos

Para decidir si otra aeronave puede ser “vecina” de la nuestra, comparamos los valores que ambas tienen en común (MTOW, superficie alar, autonomía, etc.). Para cada uno de esos parámetros calculamos una diferencia porcentual:

$$\Delta\% = \frac{|valo_candidato - valor_objetivo|}{|valor_objetivo|} \times 100$$

Por ejemplo, si nuestra aeronave X tiene una MTOW de 20 kg y otra aeronave Y tiene 22 kg:

$$\Delta\% = \frac{|22 - 20|}{|20|} \times 100 = 10\%$$

Hacemos esto para todos los parámetros donde las dos aeronaves tienen datos válidos.

Además, cada parámetro se clasifica en una de tres familias:

- familia de **masas** (por ejemplo, MTOW, peso vacío, carga útil),
- familia **geométrica** (envergadura, superficie alar, etc.),
- familia de **prestaciones/misión** (velocidad de crucero, autonomía, alcance...).

La lógica es que dos aeronaves solo son realmente comparables si se parecen en más de una “dimensión” del diseño, no solo en un número suelto.

12.6.3 Filtros de parecido físico: umbral del 20 % y familias mínimas

Para considerar que una aeronave candidata es similar:

1. Exigimos que, en los parámetros que comparamos, la diferencia porcentual no supere un cierto umbral.

En la configuración por defecto usamos *umbral de diferencia* = 20 %.

Si para algún parámetro clave la diferencia supera ese 20 %, descartamos a esa aeronave como vecina.

2. Contamos en cuántas familias (masas, geometría, prestaciones) encontramos al menos un parámetro dentro del umbral.

Con esto aplicamos dos reglas:

- **Regla principal:** aceptamos la aeronave candidata como vecina si está dentro del 20 % en al menos 3 familias distintas.

- **Excepción:** si cumple al menos 2 familias pero aporta muchos parámetros (por ejemplo, 6 o más parámetros dentro de umbral), también la aceptamos.

El objetivo es evitar “falsos vecinos” que se parezcan solo en una dimensión muy específica y, en cambio, quedarnos con plataformas que tienen un parecido razonable en masa, forma y prestaciones.

12.6.4 Convertir diferencias en un coeficiente de similitud

Una vez que una aeronave candidata supera estos filtros, calculamos qué tan parecida es, no solo si entra o no entra. Para eso, transformamos cada diferencia porcentual en un número entre 0 y 1 al que llamamos “coeficiente de similitud”:

- 1 significa “coincidencia perfecta”,
- 0 significa “sin similitud útil”.

En la configuración por defecto usamos una forma polinómica suave que hace lo siguiente:

- si la diferencia es 0 % → similitud = 1,
- si la diferencia está alrededor de 10 % → similitud ≈ 0,7,
- si la diferencia llega al 20 % → similitud ≈ 0,

- por encima del 20 % ya consideramos que no hay similitud (se recorta a 0).

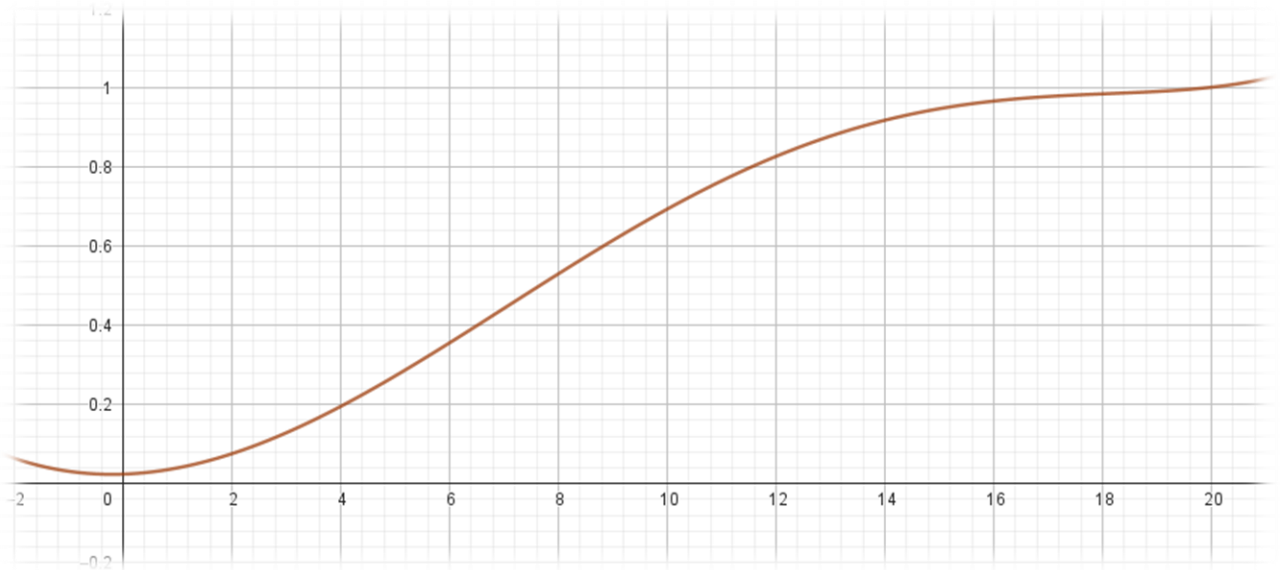


Imagen 12.2- Curva de similitud en función de la diferencia porcentual

Función que convierte la diferencia relativa entre dos valores en un coeficiente de similitud (≈ 1 para diferencias pequeñas y decreciente hasta el umbral definido, p. ej. 20%).

No hace falta que el lector recuerde la fórmula exacta; lo importante es la idea, a mayor diferencia porcentual, menor similitud, y a partir del 20 % la similitud se considera nula.

Para cada aeronave candidata calculamos la similitud en todos los parámetros que hemos comparado y luego sacamos un promedio:

$$\text{similitud_global} = \text{promedio de las similitudes en todos los parámetros comparados}$$

Ese valor, entre 0 y 1, resume qué tan parecida es la candidata a la aeronave objetivo.

12.6.5 Elegir cuántos vecinos usar

Con todos los vecinos aceptados calculamos su similitud global y los ordenamos de mayor a menor. Luego aplicamos dos decisiones:

- usamos como máximo un cierto número de vecinos “top” (por ejemplo, los 10 más parecidos),
- exigimos al menos un número mínimo de vecinos para imputar (por defecto, $k_{\text{mín}} = 3$).

Si, después de todos los filtros, solo encontramos uno o dos vecinos válidos, el módulo de similitud no imputa y deja el problema para el módulo de correlación. Esta es una decisión deliberada: preferimos evitar imputaciones basadas en evidencia muy pobre.

12.6.6 Cómo se calcula el valor imputado (promedio ponderado)

Una vez elegido el conjunto de vecinos (por ejemplo, 3 a 10 aeronaves) recogemos:

- su valor conocido del parámetro que pretendemos imputar (por ejemplo, potencia instalada),
- su similitud global con la aeronave objetivo.

Primero convertimos las similitudes en pesos. La lógica es simple, cuanto más parecida es una aeronave, más peso tiene en el promedio.

Luego normalizamos esos pesos para que entre todos sumen 1.

Finalmente, el valor imputado se obtiene como un promedio ponderado:

$$valor_imputado = peso_1 \times valor_1 + peso_2 \times valor_2 + \dots + peso_k \times valor_k$$

Donde:

- $valor_1, valor_2, \dots, valor_k$ son los valores conocidos del parámetro objetivo en cada vecino,
- $peso_1, peso_2, \dots, peso_k$ son los pesos normalizados basados en la similitud (y, opcionalmente, en la detección de outliers, como explicamos ahora).

12.6.7 Tratamiento suave de valores atípicos (outliers)

No todos los vecinos “parecidos” se comportan igual. Puede haber alguno con un valor del parámetro objetivo muy diferente al resto (un outlier). Para evitar que uno o dos valores extremos dominen el promedio:

1. El sistema calcula una medida de dispersión entre los vecinos (compara cuánto se aleja cada uno del grupo).
2. Si un vecino es demasiado extremo, se le asigna un peso menor o, en situaciones muy severas (según la configuración), directamente se lo excluye del cálculo.

En términos sencillos, la fórmula del promedio ponderado sigue siendo la misma, pero los pesos de los vecinos “raros” se reducen para que no arruinen la imputación.

12.6.8 Cómo calculamos la confianza de la imputación

Cada vez que imputamos un valor por similitud, el sistema calcula una confianza entre 0 y 1. Esta confianza combina tres ideas:

1. Lo parecidos que son los vecinos

Si las similitudes globales son altas (todos los vecinos se parecen mucho a la aeronave objetivo), la confianza sube.

2. Cuántos vecinos tenemos

Con más vecinos consistentes, la estimación es más robusta. Con pocos vecinos, la confianza baja.

3. Cuánto se parecen entre sí los valores de los vecinos

Si los vecinos tienen valores muy parecidos del parámetro objetivo, la confianza es alta. Si entre ellos están muy dispersos (unos muy altos y otros muy bajos), la confianza baja.

Sin entrar en todos los detalles internos, la idea general que usamos es:

$$\text{confianza_final} = (\text{qué tan similares son los vecinos}) \times (\text{qué tan buenos son los datos de esos vecinos})$$

y esta confianza final se guarda junto con el valor imputado.

12.6.9 Ejemplo ilustrativo (no corresponde a los datos reales de la tesis)

Para fijar ideas, imaginemos un caso sintético:

aeronave objetivo x ,

param. faltante: potencia;

tras aplicar los filtros, el sistema encuentra 3 vecinos válidos: A, B y C;

A: 1800 W

B: 2000 W

C: 1900 W

sus similitudes globales con X son:

A: 0,9

B: 0,8

C: 0,7⁴

Primero convertimos similitudes en pesos proporcionales:

$$\text{suma_sim} = 0,9 + 0,8 + 0,7 = 2,4$$

$$\text{peso_A} = 0,9 / 2,4 \approx 0,375$$

$$\text{peso_B} = 0,8 / 2,4 \approx 0,333$$

$$\text{peso_C} = 0,7 / 2,4 \approx 0,292$$

Luego calculamos el promedio ponderado:

$$\text{potencia_imputada} \approx 0,375 \times 1800 + 0,333 \times 2000 + 0,292 \times 1900$$

$$\text{potencia_imputada} \approx 675 + 666 + 555 \approx 1896 \text{ W}$$

⁴ (suponiendo que un umbral del 30% es aceptable)

En la práctica, el sistema redondearía de acuerdo a los criterios elegidos, pero lo importante es ver la lógica:

- los vecinos más parecidos pesan más,
- el valor final está en el rango de los vecinos,
- y la confianza será razonablemente alta porque:
 - hay 3 vecinos,
 - son bastante similares,
 - y sus potencias no están muy alejadas entre sí.

12.6.10 Resumen

Con esto, el lector tiene una visión clara del proceso de similitud:

1. comparar la aeronave objetivo con posibles vecinos,
2. filtrar por familias y umbral de diferencia,
3. convertir diferencias en un índice de similitud entre 0 y 1,
4. elegir un conjunto de vecinos suficientes,
5. hacer un promedio ponderado,
6. ajustar pesos si hay outliers,
7. y calcular una confianza que refleja lo robusta que es la imputación.

12.7 Imputación por correlación

En esta sección describimos con calma cómo funciona la imputación por correlación en el Módulo 1. La idea es seguir el recorrido de una única celda vacía y ver qué hace el sistema en cada paso:

- cómo selecciona los datos de entrenamiento,
- cómo decide qué modelos probar,
- qué filtros geométricos aplica antes de ajustar nada.

Para que no quede todo en abstracto, acompañamos el flujo con un ejemplo ilustrativo basado en un modelo polinómico de segundo grado con dos predictores (el caso más exigente en validaciones). Este ejemplo no corresponde a los datos reales de la tesis, pero respeta el tipo de cálculos y órdenes de magnitud que maneja el sistema.

12.7.1 Paso 0 - Punto de partida: una celda vacía y sus predictores

Partimos de una tabla donde cada fila es una aeronave y cada columna, un parámetro (MTOW, potencia instalada, peso vacío, autonomía, etc.). El Módulo 1 detecta una celda vacía, por ejemplo:

- aeronave objetivo: fila A^* ,⁵
- parámetro a imputar: peso vacío.

Independientemente del resultado el sistema está configurado de manera nativa para pasar a correlación se encuentre o no un valor de similitud.

En este punto sabemos:

- qué columna se pretende imputar (objetivo y),
- qué otras columnas tienen valores conocidos en esa fila (candidatas a predictores x),
- y que vale la pena intentar encontrar una relación estadística entre y y alguno de esos predictores.

12.7.2 Paso 1 - Construcción del conjunto de entrenamiento para esa celda

El primer bloque de lógica consiste en construir un conjunto de datos de entrenamiento específico para esa celda. No entrenamos un solo modelo global para “peso vacío”; entrenamos, si hace falta, muchos modelos locales, uno por cada combinación de predictores que tenga sentido y cumpla los requisitos mínimos.

En términos conceptuales, nuestro sistema hace lo siguiente:

1. Conservar solo la aeronave objetivo y las filas con objetivo conocido.

Nos quedamos con:

- la fila A^* (donde el peso vacío está vacío),
- todas las filas donde el peso vacío sí está informado.

2. Eliminar predictores que estén vacíos en la aeronave objetivo.

Si para A^* falta el valor de autonomía, esa columna no se puede usar como predictor en esta imputación concreta y se descarta.

3. Aplicar filtros de “familia” (F0, F1, F2, o sin filtro).

Intentamos trabajar primero con aeronaves lo más parecidas posible a A^* . Para eso usamos atributos categóricos como:

- misión,
- tipo de despegue,
- presencia o no de propulsión vertical y horizontal.

De forma progresiva:

- F0: misma misión, mismo tipo de despegue, misma configuración de propulsión.
- F1: misma misión y mismo tipo de despegue, con cierta tolerancia en la propulsión.
- F2: misma misión y mismo tipo de despegue, ignorando detalles más finos.
- Sin filtro: si nada de lo anterior da suficientes datos, flexibilizamos y mezclamos más.

⁵ Convención: el superíndice $**$ indica el punto objetivo (aeronave a imputar). Por ejemplo, x_1^* es el valor del predictor x_1 en la aeronave A^* , y $\hat{y}^*$ es la predicción para esa aeronave.

En cada nivel se exige un número mínimo de aeronaves con el objetivo conocido (por ejemplo, al menos 5). Si no se alcanza, pasamos a la familia siguiente.

4. Evitar extrapolación extrema en predictores.

Para cada predictor numérico (MTOW, potencia, envergadura, etc.) comparamos el valor de A^* ⁶ con el rango observado en las aeronaves de entrenamiento:

- Si el valor de A^* queda claramente **fuera del rango** (más allá de una tolerancia configurable), ese predictor se descarta para esta imputación.
- La idea es evitar modelos que terminen extrapolando muy lejos de los datos usados para entrenar.

Si después de este filtrado no queda una cantidad suficiente de aeronaves con el objetivo conocido, o si se descartan todas las columnas útiles, el flujo de correlación se detiene para esa celda: el sistema **prefiere no imputar** antes que hacerlo con un modelo mal apoyado.

Ejemplo ilustrativo.

Supongamos que se busca imputar el peso vacío de la aeronave A^* , y que tras aplicar estas reglas obtenemos:

- 8 aeronaves de entrenamiento ($A1...A8$) con peso vacío conocido,
- 2 predictores numéricos válidos para A^* :
 - x_1 = MTOW (kg),
 - x_2 = potencia instalada (kW).

Los valores de entrenamiento quedan así:

<i>Aeronave</i>	<i>MTOW</i> x_1	<i>Potencia</i> x_2	<i>Peso vacío</i> y
A1	10	19	30
A2	12	26	35
A3	14	21	39
A4	16	29	46
A5	18	24	51
A6	20	33	57
A7	22	27	63
A8	24	35	66

Tabla 12.2 - Ejemplo de dataset de entrenamiento tras el filtrado inicial (Módulo 1)

⁶ Convención: el superíndice $**$ indica el punto objetivo (aeronave a imputar). Por ejemplo, x_1^* es el valor del predictor x_1 en la aeronave A^* , y $\hat{y}^*$ es la predicción para esa aeronave.

Ejemplo ilustrativo del conjunto de entrenamiento que queda disponible para ajustar una regresión: incluye la variable objetivo (y) y los predictores (x) únicamente en las filas con datos válidos, antes de entrenar el modelo.

La aeronave A* ⁷(objetivo) tiene:

- $x_1^* = 19\text{kg}$ de MTOW,
- $x_2^* = 28\text{kW}$ de potencia,
- y^* (peso vacío) desconocido.

Los rangos de entrenamiento son:

- MTOW: de 10 a 24 kg,
- potencia: de 19 a 35 kW.

Como 19 está entre 10 y 24, y 28 entre 19 y 35, concluimos que A* queda dentro del rango de entrenamiento de ambos predictores: no hay extrapolación extrema.

12.7.3 Paso 2 - Generación de candidatos: combinaciones de predictores y familias de modelos

Modelos

Dimensionalidad

Permitir modelos 1D

☒

Permitir modelos 2D

☒

Grado polinómico

Tipos de modelos habilitados

Tipos activos

☒ lineal
 ☒ poly2
 ☒ log
 ☒ pot
 ☒ exp

Calidad y validación

MAPE máximo (%)

Usar LOOCV

☒

LOOCV ponderado

☐

Ponderaciones de selección

Configuración pesos

⁷ Convención: el superíndice '*' indica el punto objetivo (aeronave a imputar). Por ejemplo, x_1^* es el valor del predictor x_1 en la aeronave A*, y $\hat{y}^*$ es la predicción para esa aeronave.

Imagen 12.3- Generación de modelos candidatos a partir del set de entrenamiento

Dashboard de configuración para entrenar múltiples modelos candidatos para competir en la imputación.

Con el conjunto de entrenamiento listo, el sistema genera los modelos candidatos que va a evaluar combinando:

- el número de predictores (1 o 2),
- la familia de modelo (lineal, polinómica, potencia, etc.).

De forma esquemática:

Con **1 predictor** x :

○ Lineal:

$$y = a_0 + a_1 \cdot x$$

○ Polinómico de segundo grado:

$$y = a_0 + a_1 \cdot x + a_2 \cdot x^2$$

○ Logarítmico:

$$y = a_0 + a_1 \cdot \ln(x)$$

○ De potencia:

$$y = a_0 \cdot x^b$$

○ Exponencial:

$$y = a_0 \cdot e^{(a_1 \cdot x)}$$

Con **2 predictores** x_1, x_2 :

○ Lineal (plano):

$$y = a_0 + a_1 \cdot x_1 + a_2 \cdot x_2$$

○ Polinómico de segundo grado (superficie curva):

$$y = a_0$$

$$\blacksquare a_1 \cdot x_1 + a_2 \cdot x_2$$

$$\blacksquare a_3 \cdot x_1^2 + a_4 \cdot x_2^2$$

$$\blacksquare a_5 \cdot x_1 \cdot x_2$$

Cada tipo de modelo tiene un número de parámetros distinto y, por tanto, requisitos mínimos de muestras. A modo de ejemplo:

- un modelo lineal 1D (2 parámetros: a_0, a_1) necesita menos datos que un polinomio 2D de segundo grado (6 parámetros),
- para un polinómico 2D solemos exigir más muestras y mayor diversidad (umbral configurable en el dashboard).

En nuestro ejemplo, como solo disponemos de dos predictores (x_1, x_2), las combinaciones son:

- modelos con un solo predictor: usar solo x_1 o solo x_2 ,
- modelos con dos predictores: usar x_1 y x_2 a la vez (lineal 2D o polinómico 2D).

En esta sección nos centraremos en el caso más exigente: el modelo polinómico de segundo grado con dos predictores. Los pasos que describimos ahora (chequeos geométricos) se aplican también a los modelos lineales 2D; lo único que cambia es el número de parámetros.

12.7.4 Paso 3 - Chequeos geométricos y de diversidad para modelos 2D

Antes de ajustar un modelo con dos predictores, el sistema realiza una serie de chequeos preliminares en el plano (x_1, x_2) . El objetivo es evitar entrenar superficies sobre nubes de puntos degeneradas (casi alineadas, muy amontonadas, etc.).

Importante: Por defecto no se pedirán confirmaciones manuales; podés ajustar el comportamiento en la sección Confirmaciones.

Orquestación	Similitud	Correlación	Loop/Orquestación	Excel	HTML
Checks 2D					
Validación de modelos 2D					
☒ Activar checks 2D					
Correlación y multicolinealidad					
☒	r máx	0.8	i		
☒	VIF máx	10	i		
☒	PC2 ratio mín	0.03	i		
☒	Rango mínimo	2	i		
☒	Condición máx	100000	i		
Cobertura del espacio					
☒	Cobertura pares únicos mín	0.6	i		
☒	Cobertura hull mín	0.15	i		
☒	Cobertura elipse mín	0.1	i		
Mínimo de muestras por parámetro					
☒	n/param (linear-2)	6	→ n≥24	i	
☒	n/param (poly-2)	8	→ n≥60	i	
Modo agresivo					
☒	Modo agresivo r mín	0.94	i		
Diversidad mínima					

Imagen 12.4- Control de correlación entre predictores (multicolinealidad)

Ilustración del chequeo de correlaciones/VIF para descartar combinaciones de predictores redundantes o inestables antes del ajuste.

12.7.4.1 Correlación entre predictores

Primero calculamos la correlación de Pearson entre x_1 y x_2 . La fórmula general es:

$$r = \frac{\sum_{i=1}^n (x_{1i} - \bar{x}_1)(x_{2i} - \bar{x}_2)}{\sqrt{\sum_{i=1}^n (x_{1i} - \bar{x}_1)^2} \sqrt{\sum_{i=1}^n (x_{2i} - \bar{x}_2)^2}}.$$

En nuestro ejemplo:

- Número de muestras: $n = 8$.
- Media de x_1 :

$$\bar{x}_1 = \frac{10 + 12 + 14 + 16 + 18 + 20 + 22 + 24}{8} = \frac{136}{8} = 17.$$

- Media de x_2 :

$$\bar{x}_2 = \frac{19 + 26 + 21 + 29 + 24 + 33 + 27 + 35}{8} = \frac{214}{8} = 26,75.$$

Calculamos algunas diferencias (mostramos dos como ejemplo):

- Para A1:

$$\begin{aligned} x_{11} - \bar{x}_1 &= 10 - 17 = -7, \\ x_{21} - \bar{x}_2 &= 19 - 26,75 = -7,75. \end{aligned}$$

- Para A2:

$$\begin{aligned} x_{12} - \bar{x}_1 &= 12 - 17 = -5, \\ x_{22} - \bar{x}_2 &= 26 - 26,75 = -0,75. \end{aligned}$$

El producto de cada par de diferencias se suma:

$$\sum (x_{1i} - \bar{x}_1)(x_{2i} - \bar{x}_2) = (-7)(-7,75) + (-5)(-0,75) + \dots$$

Si hacemos esta suma completa para las 8 aeronaves y dividimos por $n - 1 = 7$, obtenemos la covarianza cruzada:

$$\sigma_{12} = \frac{1}{7} \sum (x_{1i} - \bar{x}_1)(x_{2i} - \bar{x}_2) \approx 21,14.$$

De forma análoga:

$$\begin{aligned} \sigma_{11} &= \frac{1}{7} \sum (x_{1i} - \bar{x}_1)^2 = \frac{1}{7} (49 + 25 + 9 + 1 + 1 + 9 + 25 + 49) = \frac{168}{7} = 24, \\ \sigma_{22} &= \frac{1}{7} \sum (x_{2i} - \bar{x}_2)^2 \approx 30,5. \end{aligned}$$

Así, las desviaciones estándar son:

$$s_1 = \sqrt{\sigma_{11}} = \sqrt{24} \approx 4,90, s_2 = \sqrt{\sigma_{22}} \approx \sqrt{30,5} \approx 5,52.$$

Y la correlación resulta:

$$r = \frac{\sigma_{12}}{s_1 s_2} \approx \frac{21,14}{4,90 \cdot 5,52} \approx \frac{21,14}{27,08} \approx 0,78.$$

En nuestro sistema, si $|r|$ es demasiado alto (por ejemplo, $\geq 0,90$), consideramos que los dos predictores casi contienen la misma información y **descartamos el modelo 2D**.

En este ejemplo el, $|r| \approx 0,78 < 0,90$, entonces el par de predictores pasa el filtro de correlación.

12.7.4.2 VIF (Factor de Inflación de Varianza)

Con el valor de r calculado, obtenemos el VIF, que mide cuánto se inflan las varianzas de los coeficientes por colinealidad:

$$\text{VIF} = \frac{1}{1 - r^2}$$

Sustituyendo:

$$\begin{aligned} r^2 &\approx (0,78)^2 \approx 0,61, \\ 1 - r^2 &\approx 1 - 0,61 = 0,39, \\ \text{VIF} &\approx \frac{1}{0,39} \approx 2,57 \end{aligned}$$

En la configuración típica, usamos un umbral de $\text{VIF} \leq 10$.

En nuestro ejemplo:

- $\text{VIF} \approx 2,57$ está muy por debajo del límite; el modelo 2D no se descarta por este motivo.

12.7.4.3 Segundo componente principal (PC2) y proporción de varianza

Para entender si los puntos (x_1, x_2) ocupan realmente una región bidimensional o si están casi alineados, usamos la descomposición en componentes principales. La idea es:

- calcular la matriz de covarianza:

$$\Sigma = \begin{bmatrix} \sigma_{11} & \sigma_{12} \\ \sigma_{12} & \sigma_{22} \end{bmatrix} = \begin{bmatrix} 24 & 21,14 \\ 21,14 & 30,5 \end{bmatrix},$$

- obtener sus autovalores λ_1, λ_2 (varianzas a lo largo de los ejes principales),
- ver qué fracción de la varianza total corresponde a la segunda componente (PC2).

La suma de autovalores es igual a la traza de Σ :

$$\lambda_1 + \lambda_2 = \sigma_{11} + \sigma_{22} = 24 + 30,5 = 54,5.$$

Usando la fórmula estándar para autovalores de una matriz 2×2 :

$$\lambda_{1,2} = \frac{\sigma_{11} + \sigma_{22}}{2} \pm \sqrt{\left(\frac{\sigma_{11} - \sigma_{22}}{2}\right)^2 + (\sigma_{12})^2}.$$

Calculamos primero:

$$\frac{\sigma_{11} + \sigma_{22}}{2} = \frac{24 + 30,5}{2} = \frac{54,5}{2} = 27,25.$$

El determinante:

$$\det(\Sigma) = \sigma_{11}\sigma_{22} - \sigma_{12}^2 = 24 \cdot 30,5 - (21,14)^2.$$

Calculamos:

$$24 \cdot 30,5 = 732,$$

$$(21,14)^2 \approx 447,0,$$

por lo que:

$$\det(\Sigma) \approx 732 - 447 = 285.$$

Luego:

$$\left(\frac{\sigma_{11} + \sigma_{22}}{2}\right)^2 - \det(\Sigma) = (27,25)^2 - 285 \approx 743,56 - 285 \approx 458,56.$$

La raíz cuadrada:

$$\sqrt{458,56} \approx 21,41.$$

De aquí:

$$\lambda_1 \approx 27,25 + 21,41 = 48,66,$$

$$\lambda_2 \approx 27,25 - 21,41 = 5,84.$$

(Numéricamente, si usamos todos los decimales de σ_{12} , obtenemos $\lambda_1 \approx 48,64$ y $\lambda_2 \approx 5,86$; las pequeñas diferencias son por redondeo.)

La varianza total es:

$$\lambda_1 + \lambda_2 \approx 48,64 + 5,86 \approx 54,5.$$

La proporción explicada por la segunda componente (PC2) es:

$$\text{ratio PC2} = \frac{\lambda_2}{\lambda_1 + \lambda_2} \approx \frac{5,86}{54,5} \approx 0,11 \text{ (11 \%)}.$$

En nuestro sistema solemos exigir que la PC2 explique al menos un cierto porcentaje (por ejemplo, 3–5 %) para considerar que la nube es realmente bidimensional.

En este ejemplo el PC2 $\approx$ 11 % de la varianza total por lo que hay variación apreciable en la segunda dirección; el modelo 2D pasa este filtro.

12.7.4.4 Relación entre número de muestras y parámetros del modelo

Antes de ajustar el polinomio 2D completo, miramos también la relación:

$$\frac{n}{p}$$

donde:

- n = número de muestras de entrenamiento,
- p = número de parámetros del modelo.

En nuestro caso, el polinomio 2D de segundo grado tiene 6 parámetros ($a_0, a_1, a_2, a_3, a_4, a_5$), y el número de aeronaves de entrenamiento $n = 8$.

Por tanto:

$$\frac{n}{p} = \frac{8}{6} \approx 1,33.$$

En una configuración conservadora, lo ideal sería tener un cociente bastante mayor que 1 (por ejemplo, 2, 3 o más). En el software real, ese umbral se controla mediante parámetros de configuración, si exigimos, por ejemplo, $n/p \geq 2$, un modelo con 8/6 no se permitiría y se descartaría automáticamente, si trabajamos con una configuración más permisiva (por falta de datos), se puede aceptar este modelo, pero los chequeos posteriores (LOOCV, outliers) se vuelven más importantes.

En el ejemplo didáctico que mostramos aquí seguimos adelante con el polinomio 2D para poder ilustrar el resto del flujo. En la implementación real, este caso quedaría o no habilitado según los umbrales elegidos en el dashboard.

12.7.4.5 Cobertura espacial: área del hull y elipse de dispersión

Finalmente, evaluamos cómo se distribuyen los puntos (x_1, x_2) en el plano, más allá de la correlación y el VIF. Usamos dos ideas:

- el **bounding box** (rectángulo mínimo alineado con los ejes),
- el **casco convexo** (“envolvente de goma elástica” que rodea los puntos),
- y una **elipse de dispersión** basada en las varianzas y covarianzas.

a) Bounding box

El bounding box tiene lados:

en x_1 : de 10 a 24 $\rightarrow$ longitud = $24 - 10 = 14$,

en x_2 : de 19 a 35 $\rightarrow$ longitud = $35 - 19 = 16$.

El área del rectángulo es:

$$\text{Área bbox} = 14 \cdot 16 = 224.$$

b) Casco convexo y su área

Tomamos los puntos extremos de la nube (los vértices del casco convexo). En este ejemplo, el casco convexo incluye seis puntos, por ejemplo en el orden:

$$(10,19), (14,21), (22,27), (24,35), (20,33), (12,26).$$

Calculamos el área con la fórmula del polígono (regla del “cordón de zapatos”):

$$\text{Área hull} = \frac{1}{2} \left| \sum_{i=1}^n x_i y_{i+1} - x_{i+1} y_i \right|,$$

cerrando el polígono con $(x_{n+1}, y_{n+1}) = (x_1, y_1)$.

Sustituimos:

Producto hacia adelante:

$$10 \cdot 21 = 210$$

$$14 \cdot 27 = 378$$

$$22 \cdot 35 = 770$$

$$24 \cdot 33 = 792$$

$$20 \cdot 26 = 520$$

$$12 \cdot 19 = 228$$

Suma:

$$210 + 378 + 770 + 792 + 520 + 228 = 2898.$$

Producto hacia atrás:

$$14 \cdot 19 = 266$$

$$22 \cdot 21 = 462$$

$$24 \cdot 27 = 648$$

$$20 \cdot 35 = 700$$

$$12 \cdot 33 = 396$$

$$10 \cdot 26 = 260$$

Suma:

$$266 + 462 + 648 + 700 + 396 + 260 = 2732.$$

La diferencia es:

$$2898 - 2732 = 166.$$

El área del casco convexo es:

$$\text{Área hull} = \frac{1}{2} \cdot 166 = 83.$$

La razón hull/bbox es:

$$\text{ratio hull} = \frac{\text{Área hull}}{\text{Área bbox}} = \frac{83}{224} \approx 0,37.$$

En nuestro sistema usamos un umbral del orden de 0,15. Si la razón fuera mucho menor, significaría que los puntos ocupan una región muy pequeña del rectángulo (por ejemplo, casi una diagonal), y el modelo 2D podría extrapolar dentro de su propio dominio.

En este ejemplo, 0,37 está muy por encima del mínimo, así que el modelo pasa este chequeo.

c) Elipse de dispersión y su área

Para complementar el análisis, consideramos una elipse que representa la dispersión de los puntos en el plano, basada en la matriz de covarianza Σ .

Si usamos los autovalores λ_1 y λ_2 calculados antes, los semiejes principales de la elipse de 1 desviación estándar son:

$$a = \sqrt{\lambda_1}, b = \sqrt{\lambda_2}.$$

En nuestro ejemplo:

$$a = \sqrt{48,64} \approx 6,98, b = \sqrt{5,86} \approx 2,42.$$

El área de la elipse es:

$$\text{Área elipse} = \pi a b \approx \pi \cdot 6,98 \cdot 2,42.$$

Calculamos:

$$6,98 \cdot 2,42 \approx 16,9, \\ \text{Área elipse} \approx \pi \cdot 16,9 \approx 53,0.$$

La razón elipse/bbox es:

$$\text{ratio elipse} = \frac{\text{Área elipse}}{\text{Área bbox}} \approx \frac{53,0}{224} \approx 0,24.$$

También aquí usamos un umbral (por ejemplo, 0,10). Si la elipse fuera muy pequeña comparada con el rectángulo, significaría que la nube “real” está muy concentrada en el centro y que, aunque el bounding box sea grande, la diversidad efectiva es baja.

En este ejemplo, $\text{ratio elipse} \approx 0,24 > 0,10$, la dispersión en ambas direcciones es razonable.

12.7.4.6 Resumen

En resumen, hasta el final del **Paso 3** nuestro ejemplo queda así, el par de predictores ($x_1 = \text{MTOW}, x_2 = \text{potencia}$)

- tiene una correlación moderada ($r \approx 0,78$),
- un VIF razonable ($\approx 2,57$),
- una PC2 que explica ≈ 11 % de la varianza,
- una nube que ocupa ≈ 37 % del bounding box (hull ratio),
- y una elipse de dispersión que ocupa ≈ 24 % del bounding box (ellipse ratio);

según los umbrales configurados, el modelo 2D **pasa todos los chequeos geométricos**, la relación $n/p = 8/6$ es relativamente baja para un polinomio 2D general, por lo que en el código real este modelo podría ser aceptado o descartado según la configuración; en nuestro ejemplo didáctico lo mantenemos para poder seguir mostrando el flujo.

12.7.5 Paso 4 - Ajuste del modelo polinómico de dos predictores

En este punto el sistema ya:

- eligió un objetivo (por ejemplo, peso vacío),
- seleccionó dos predictores (por ejemplo MTOW y envergadura),
- filtró las aeronaves válidas,
- pasó los chequeos geométricos (Pearson, VIF, PC2, cobertura, etc.).

Ahora intentamos ajustar un modelo polinómico de segundo grado con dos predictores.

12.7.5.1 Forma general del modelo

Para dos predictores x_1 y x_2 , el modelo polinómico de segundo grado que usamos tiene la forma:

$$y = a_0 + a_1 x_1 + a_2 x_2 + a_3 x_1^2 + a_4 x_1 x_2 + a_5 x_2^2$$

donde:

- y es la variable objetivo (por ejemplo, peso vacío),
- x_1 podría ser MTOW,
- x_2 podría ser envergadura,
- $a_0, \dots, a_5$ son los coeficientes que el sistema ajusta con mínimos cuadrados.

En el código, el ajuste se hace sobre una versión reescalada de las variables (para mejorar la estabilidad numérica), pero al final reconstruimos una ecuación equivalente en unidades originales, que es la que presentamos al usuario.

12.7.5.2 Cómo se obtienen los coeficientes (visión práctica)

A nivel práctico, el script construye una tabla con los términos:

$$\{1; x_1; x_2; x_1^2; x_1 x_2; x_2^2\},$$

para todas las aeronaves de entrenamiento.

Con esa tabla, resuelve el problema clásico de mínimos cuadrados, encontrar los coeficientes $a_0, \dots, a_5$ que minimizan la suma de los cuadrados de los residuos $(y_i - \hat{y}_i)^2$.

El detalle algebraico de los mínimos cuadrados ya se desarrolla en el MARCO TEÓRICO; aquí nos concentramos en cómo usamos el resultado dentro del flujo de imputación.

12.7.5.3 Ejemplo numérico de evaluación del modelo

Supongamos que, para nuestro subconjunto de entrenamiento, el ajuste polinómico entrega (en escala original):

$$a_0 = 10$$

$$a_1 = 0,8$$

$$a_2 = 1,2$$

$$a_3 = 0,02$$

$$a_4 = -0,01$$

$$a_5 = 0,03$$

y que una de las aeronaves de entrenamiento tiene:

$$x_1 = 10(\text{por ejemplo, MTOW} = 10 \text{ kg})$$

$$x_2 = 5(\text{por ejemplo, envergadura} = 5 \text{ metros})$$

$$\text{valor real } y = 28(\text{por ejemplo, peso vacío} = 28 \text{ kg})$$

Entonces la predicción del modelo para esa aeronave se calcula sustituyendo en la ecuación:

1. Escribimos la fórmula:

$$\hat{y} = a_0 + a_1x_1 + a_2x_2 + a_3x_1^2 + a_4x_1x_2 + a_5x_2^2$$

2. Sustituimos valores:

$$\hat{y} = 10 + 0,8 \cdot 10 + 1,2 \cdot 5 + 0,02 \cdot 10^2 - 0,01 \cdot 10 \cdot 5 + 0,03 \cdot 5^2$$

3. Calculamos término a término:

$$0,8 \cdot 10 = 8$$

$$1,2 \cdot 5 = 6$$

$$0,02 \cdot 10^2 = 0,02 \cdot 100 = 2$$

$$-0,01 \cdot 10 \cdot 5 = -0,01 \cdot 50 = -0,5$$

$$0,03 \cdot 5^2 = 0,03 \cdot 25 = 0,75$$

4. Sumamos todo:

$$\hat{y} = 10 + 8 + 6 + 2 - 0,5 + 0,75 = 26,25$$

Este cálculo es exactamente el que hace internamente el script cuando, más adelante, necesita predecir un valor para una aeronave (sea de entrenamiento o para imputación).

12.7.6 Paso 5 - Métricas de ajuste y validación LOOCV

$$\text{MAPE}_{\text{train}} = \frac{1}{n} \sum_{i=1}^n \text{APE}_i$$

Ejemplo numérico (un término del MAPE)

Volvamos a la aeronave del ejemplo anterior:

$$y = 28$$

$$\hat{y} = 26,25$$

Calculamos el APE:⁹

1. Diferencia absoluta:

$$|y - \hat{y}| = |28 - 26,25| = 1,75$$

2. Dividimos por el valor real y pasamos a porcentaje:

$$\frac{1,75}{28} \approx 0,0625$$

$$\text{APE} \approx 0,0625 \times 100 = 6,25 \%$$

El script repite este cálculo para todas las aeronaves de entrenamiento y luego promedia. Supongamos que, al promediar las 8 aeronaves de nuestro ejemplo, obtenemos:

- $\text{MAPE}_{\text{train}} \approx 5,8 \%$

Cálculo de R^2 en entrenamiento

El coeficiente de determinación R^2 mide qué fracción de la variabilidad de y explica el modelo. Usamos la fórmula clásica:

- Suma de cuadrados de residuos (errores):

$$\text{SSE} = \sum_{i=1}^n (y_i - \hat{y}_i)^2 \quad 10$$

- Suma total de cuadrados respecto a la media de y :

$$\text{SST} = \sum_{i=1}^n (y_i - \bar{y})^2 \quad 11$$

- Coeficiente de determinación:

⁹ APE = *Absolute Percentage Error* (error porcentual absoluto por muestra). Se define como el "error porcentual absoluto" de cada aeronave y luego se promedia para obtener MAPE

¹⁰ SSE = *Sum of Squared Errors* (suma de cuadrados de los residuos/errores) $\text{SSE} = \sum (y_i - \hat{y}_i)^2$

¹¹ SST = *Total Sum of Squares* (suma total de cuadrados respecto a la media de y) $\text{SST} = \sum (y_i - \bar{y})^2$

$$R^2 = 1 - \frac{SSE}{SST}$$

Para no extender demasiado la aritmética, mostramos un solo término de SSE, para la aeronave del ejemplo:

$$(y - \hat{y})^2 = (28 - 26,25)^2 = 1,75^2 = 3,0625$$

El sistema suma este tipo de términos para todas las aeronaves y calcula R^2 completo. Supongamos que en este ejemplo:

$$R_{\text{train}}^2 \approx 0,92$$

12.7.6.2 Métrica combinada “Corr” en entrenamiento

En el Módulo 1 no elegimos modelos solo por una métrica. Combinamos el MAPE y el R^2 de entrenamiento en una métrica escalar que llamamos “Corr”, con la forma:

$$\text{Corr}_{\text{train}} = w_{R^2} \cdot R_{\text{train}}^2 + w_{\text{MAPE}} \cdot (1 - \min(\frac{\text{MAPE}_{\text{train}}}{D}, 1))$$

donde:

- w_{R^2} y w_{MAPE} son pesos configurables (por defecto 0,5 y 0,5),
- D es un “divisor de MAPE” (por defecto 15), que define cuándo un MAPE se considera “razonable”.

Ejemplo numérico

Con los valores ilustrativos:

$$R_{\text{train}}^2 = 0,92$$

$$\text{MAPE}_{\text{train}} = 5,8 \%$$

$$w_{R^2} = 0,5$$

$$w_{\text{MAPE}} = 0,5$$

$$D = 15$$

Calculamos primero el término de MAPE:

1. Cociente:

$$\frac{\text{MAPE}_{\text{train}}}{D} = \frac{5,8}{15} \approx 0,3867$$

2. Parte entre paréntesis:

$$1 - 0,3867 \approx 0,6133$$

Luego combinamos:

$$\text{Corr}_{\text{train}} = 0,5 \cdot 0,92 + 0,5 \cdot 0,6133$$

$$\text{Corr}_{\text{train}} \approx 0,46 + 0,3067 = 0,7667$$

Este valor resume, en una sola escala, que el modelo tiene buen R^2 y un MAPE razonablemente bajo.

12.7.6.3 Validación Leave-One-Out (LOOCV)

Para verificar la robustez, el script repite el ajuste en un esquema LOOCV:

1. Tenemos n aeronaves de entrenamiento.

2. En cada iteración, dejamos una aeronave fuera, ajustamos el modelo con las otras $n - 1$ y predecimos el valor de la que quedó fuera.

3. Guardamos el error porcentual de esa predicción.

4. Al final, tenemos n predicciones “fuera de muestra” y calculamos:

$\text{MAPE}_{\text{LOOCV}}$: promedio de los n errores porcentuales.

R^2_{LOOCV} : R^2 entre los valores reales y las predicciones LOOCV.

El código repite exactamente el mismo pipeline ¹² que en el entrenamiento (escalado, polinomios, pesos de outliers, etc.), pero siempre ajustando sin la aeronave que se va a predecir.

Ejemplo numérico esquemático

Imaginemos que, al terminar el LOOCV con nuestras 8 aeronaves, obtenemos:

$$\text{MAPE}_{\text{LOOCV}} = 7,4 \%$$

$$R^2_{\text{LOOCV}} = 0,88$$

La forma de combinar estos valores en una métrica “ Corr_LOOCV ” es análoga a la de entrenamiento:

$$\text{Corr}_{\text{LOOCV}} = w_{R^2} \cdot R^2_{\text{LOOCV}} + w_{\text{MAPE}} \cdot (1 - \min(\frac{\text{MAPE}_{\text{LOOCV}}}{D}, 1))$$

Sustituimos:

1. Término MAPE:

$$\frac{7,4}{15} \approx 0,4933$$

$$1 - 0,4933 \approx 0,5067$$

2. Combinación:

$$\text{Corr}_{\text{LOOCV}} = 0,5 \cdot 0,88 + 0,5 \cdot 0,5067$$

$$\text{Corr}_{\text{LOOCV}} \approx 0,44 + 0,2533 = 0,6933$$

¹² Pipeline = Flujo

12.7.6.4 Pesos de outliers en LOOCV (visión simplificada)

El LOOCV también puede usar pesos para reducir la influencia de puntos atípicos.

La idea es que se calcule una medida de “distancia” de cada punto a la tendencia general (por ejemplo, un z-score robusto), a partir de ese valor, se asigna un peso:

$$w_i = \frac{1}{1 + \alpha \cdot z_i}$$

donde α es un parámetro configurable (por ejemplo, 0,5) y z_i es el z-score robusto.

- Los puntos muy alejados de la tendencia (z grande) reciben pesos pequeños (se los escucha menos en el ajuste).
- En casos extremos, el código puede directamente excluir (“remover duro”) ciertos puntos si se supera un umbral de z-score muy alto.

Ejemplo rápido: si para un punto tenemos $z = 4$ y $\alpha = 0,5$:

$$w = \frac{1}{1 + 0,5 \cdot 4} = \frac{1}{1 + 2} = \frac{1}{3} \approx 0,333$$

Es decir, ese punto no se elimina del ajuste: su error se sigue considerando, pero su influencia sobre el ajuste (y por tanto sobre los coeficientes del modelo) es aproximadamente tres veces menor que la de un punto con peso 1.

12.7.6.5 Clasificación LOOCV: robusto, no robusto, rechazado

Con $\text{MAPE}_{\text{LOOCV}}$ y R^2_{LOOCV} el sistema clasifica el modelo en tres categorías:

- **robusto**: errores bajos y R^2 alto,
- **no robusto**: intermedio,
- **rechazado**: LOOCV indica que el modelo no se sostiene.

Los umbrales por defecto (configurables) son, por ejemplo:

- robusto si:
 - $\text{MAPE}_{\text{LOOCV}} \leq 7,5 \%$
 - $R^2_{\text{LOOCV}} \geq 0,6$
- no robusto si:
 - $\text{MAPE}_{\text{LOOCV}} \leq 12,5 \%$
 - $R^2_{\text{LOOCV}} \geq 0,45$
- en cualquier otro caso: rechazado.

Con los valores de nuestro ejemplo:

- Con estos umbrales, el modelo quedaría clasificado como **robusto**.

Modelos

Conianza

Ponderaciones básicas

Peso R²

0.5

Peso MAPE

0.5

MAPE divisor

15

Penalización por k (polinomio)

Penalización por k (polinomio)

a5

0.00002281

a4

-0.00024

a3

-0.0036

a2

0.046

a1

0.0095

a0

0.024

Penalización por N (polinomio)

Penalización por N (polinomio)

b3

0

b2

0.0025

b1

0.02

b0

0.1

Penalizaciones por métricas 2D

Aporte LOOCV a la conianza

Peso LOOCV

0

Factor ROBUSTO

1

Factor NO ROBUSTO

0.85

Factor RECHAZADO

0.4

Selección/LOOCV

Dashboard de configuración del cálculo de confianza combinando métricas (MAPE, R^2 , correlación) y penalizaciones para obtener un indicador único por modelo.

12.7.7.1 Confianza base a partir de MAPE y R^2

12-114

1. Se normaliza R^2 :

$$r2_{\text{term}} = \max(0, \min(1, R^2_{\text{train}}))$$

2. Se transforma el MAPE en un término entre 0 y 1:

$$mape_{\text{term}} = 1 - \min\left(\frac{\text{MAPE}_{\text{train}}}{D}, 1\right)$$

3. Se combinan:

$$\text{base} = w_{R^2} \cdot r2_{\text{term}} + w_{\text{MAPE}} \cdot mape_{\text{term}}$$

Con nuestros números:

$$r2_{\text{term}} = 0,92$$

$$mape_{\text{term}} \approx 0,6133(\text{como vimos antes})$$

$$w_{R^2} = w_{\text{MAPE}} = 0,5$$

Luego:

$$\text{base} = 0,5 \cdot 0,92 + 0,5 \cdot 0,6133 \approx 0,7667$$

12.7.7.2 Penalización por complejidad y tamaño de muestra

Después, la confianza se ajusta con dos factores, una penalización por número de parámetros efectivos del modelo (k), una penalización por número efectivo de muestras (n).

A nivel conceptual:

- $\text{pen}_k = f_k(k)$,
- $\text{pen}_n = f_n(n)$,

donde f_k y f_n son curvas configurables que devuelven valores entre 0 y 1. La idea es que modelos muy complejos con pocos datos reciban penalizaciones más fuertes, mientras que modelos con complejidad razonable y buena cantidad de datos se penalicen poco.

En los modelos 2D, además se incorpora un factor adicional que resume distintos chequeos sobre la “calidad” del conjunto de entrenamiento en dos dimensiones (por ejemplo, si los predictores están muy relacionados entre sí o si el espacio de datos está poco cubierto). Este factor, al igual que los anteriores,

toma valores entre 0 y 1: valores cercanos a 1 indican que no se detectan problemas relevantes; valores bajos reducen la confianza final.¹³

De este modo, la confianza intermedia se calcula como:

$$\text{conf}_{\text{intermedia}} = \text{base} \times \text{pen}_k \times \text{pen}_n \times \text{pen}_{2D}$$

donde pen_{2D} corresponde justamente a ese “producto de penalizaciones 2D”.

donde “penalización_métricas_2D” es el producto de factores asociados a Pearson, VIF, PC2, cobertura, etc. Cada métrica se transforma en un factor entre 0 y 1 con una expresión simple del tipo:

$$\text{factor} = \max(0, \min(1, p_0 + p_1x + p_2x^2))$$

Por ejemplo, si el PC2_ratio del modelo es suficientemente alto, el factor correspondiente será cercano a 1; si es muy bajo, se acercará a 0 y bajará la confianza.

Para mantener el ejemplo manejable, supongamos que:

$$\text{pen}_k = 0,95$$

$$\text{pen}_n = 0,95$$

$$\text{pen}_{2D} = 0,95$$

Entonces:

1. Producto de penalizaciones:

$$0,95 \times 0,95 \times 0,95 \approx 0,857$$

2. Confianza intermedia:

$$\text{conf}_{\text{intermedia}} = 0,7667 \times 0,857 \approx 0,657$$

12.7.7.3 Aporte del LOOCV a la confianza final

El LOOCV no solo da una etiqueta (robusto / no robusto / rechazado), sino que también ajusta la confianza final según esa etiqueta.

Usamos una mezcla convexa:

$$\text{Confianza}_{\text{final}} = \text{conf}_{\text{intermedia}} \times ((1 - w_{\text{LOOCV}}) + w_{\text{LOOCV}} \cdot f_{\text{clase}})$$

donde:

- w_{LOOCV} es el peso asignado al LOOCV (por defecto 0,2),
- f_{clase} es un factor según la clase:

robusto $\rightarrow 1,0$

no robusto $\rightarrow 0,85$

¹³ En la implementación, pen_{2D} se obtiene multiplicando varios factores parciales (cada uno entre 0 y 1), asociados a distintos indicadores calculados en 2D. La intención es que, si alguno de esos indicadores muestra un problema importante, el factor total disminuya y la confianza baje en consecuencia.

rechazado $\rightarrow 0,6$ (valores ilustrativos compatibles con el código).

En nuestro ejemplo la clase LOOCV es “robusto”, por lo que,

- $f_{\text{clase}} = 1,0$,
- $w_{\text{LOOCV}} = 0,2$.

Entonces:

1. Parte entre paréntesis:

$$(1 - 0,2) + 0,2 \cdot 1,0 = 0,8 + 0,2 = 1,0$$

2. Confianza final:

$$\text{Confianza}_{\text{final}} = 0,657 \times 1,0 = 0,657$$

Si el modelo hubiera sido “no robusto”:

$$(1 - 0,2) + 0,2 \cdot 0,85 = 0,8 + 0,17 = 0,97^{14}$$

Si hubiera sido “rechazado”, la confianza se reduciría más.

En la práctica, el sistema descarta modelos cuya confianza quede por debajo de ciertos umbrales configurables y solo conserva los que superan un nivel mínimo.

12.7.7.4 Uso del modelo para imputar una celda

Cuando un modelo pasa todos los filtros y su confianza final es aceptable, el Módulo 1 lo usa para imputar celdas faltantes.

Para una aeronave objetivo con valores de predictores:

$$x_1^* = 16 \quad ^{15}$$

$$x_2^* = 7$$

la predicción del modelo polinómico se calcula igual que en el ejemplo de entrenamiento, sustituyendo en la ecuación:

1. Fórmula:

$$\hat{y}^* = a_0 + a_1 x_1^* + a_2 x_2^* + a_3 (x_1^*)^2 + a_4 x_1^* x_2^* + a_5 (x_2^*)^2$$

2. Sustitución (con los mismos coeficientes ilustrativos):

$$\hat{y}^* = 10 + 0,8 \cdot 16 + 1,2 \cdot 7 + 0,02 \cdot 16^2 - 0,01 \cdot 16 \cdot 7 + 0,03 \cdot 7^2$$

3. Cálculo término a término:

$$0,8 \cdot 16 = 12,8$$

$$1,2 \cdot 7 = 8,4$$

$$0,02 \cdot 16^2 = 0,02 \cdot 256 = 5,12$$

¹⁴ la confianza final bajaría ligeramente.

¹⁵ Convención: el superíndice “*” indica el punto objetivo (aeronave a imputar). Por ejemplo, x_1^* es el valor del predictor x_1 en la aeronave A^* , y $\hat{y}^*$ es la predicción para esa aeronave.

$$-0,01 \cdot 16 \cdot 7 = -0,01 \cdot 112 = -1,12$$

$$0,03 \cdot 7^2 = 0,03 \cdot 49 = 1,47$$

4. Suma:

$$\hat{y}^* = 10 + 12,8 + 8,4 + 5,12 - 1,12 + 1,47$$

$$\hat{y}^* = 36,67(\text{aprox.})$$

Este valor se escribe en la celda faltante de la tabla de resultados.

12.7.7.5 Comprobación de extrapolación antes de aceptar la imputación

▼ Extrapolación

Extrapolación 1D

Modo predictores

eliminar

▼

Tolerancia (%)

0

Configuración 2D

Modo 2D

marginal

▼

Hull padding

0

Imagen 12.7 – Configuración de chequeo de extrapolación: región válida de imputación

dashboard de control geométrico que evita imputar cuando el punto objetivo cae fuera de la región cubierta por los datos (extrapolación riesgosa).

Antes de aceptar la imputación, el script verifica si el punto (x_1^*, x_2^*) está “dentro” de la región donde hay datos:

En el caso 2D, puede usar un enfoque marginal (comparar con los mínimos y máximos de cada predictor) o un enfoque geométrico (convex hull de los puntos de entrenamiento).

Si el punto está claramente fuera del rango, se marca la imputación con una advertencia de extrapolación.

Junto con el valor imputado, el Módulo 1 guarda:

- la ecuación del modelo,
- la confianza final numérica,
- el tipo de modelo (por ejemplo, “poly-2 predictores”),
- y las advertencias.

Toda esta información se vuelca tanto en el Excel de salida como en los archivos JSON que servirán de entrada al Módulo 2 para la visualización y el análisis detallado de modelos.

12.8 Bucle de imputación y combinación de métodos

Importante: Por defecto no se pedirán confirmaciones manuales; podés ajustar el comportamiento en la sección Confirmaciones.

Orquestación	Similitud	Correlación	Loop/Orquestación	Excel	HTML
--------------	-----------	-------------	-------------------	-------	------

▼ Orden de ejecución

Primero ▼ Segundo: correlacion

▼ Combinación de métodos

comb. ▼ conf_min

▼ Criterios de parada

min_nuevas_por_iter sin_mejora_consecutivas

▼ Export/HTML

☒ Export JSON dir fname

☒ Mostrar HTML df_base

Guardar overrides

Guardar y ejecutar

Imagen 12.8 - Dashboard completo de imputación y combinación de métodos

Panel del pipeline que integra imputación por similitud, por correlación y su combinación (promedio ponderado, mejor confianza, prioridad correlación, prioridad similitud), incluyendo outliers y controles de parada y exportación.

En los apartados anteriores explicamos la lógica interna de la imputación por similitud y por correlación como si fueran “cajas negras” separadas. En esta sección describimos cómo el sistema las coordina en el bucle de imputación (implementado en `imputation_loop.py`), que es quien toma, finalmente, la decisión de qué valor entra en cada celda vacía de la base de datos.

En términos simples, el bucle hace tres cosas:

1. Llama a la similitud y a la correlación para que propongan valores.
2. Fusiona esas propuestas, celda por celda, según reglas configurables.
3. Actualiza la tabla, repite el proceso varias veces y, al final, genera los archivos de salida (Excel imputado y JSON para el Módulo 2).

12.8.1 Iteraciones y llamadas a similitud y correlación

El bucle trabaja sobre una copia de la base original que se va completando de a poco. Antes de empezar, calcula cuántas celdas están vacías y carga los parámetros de parada: cuántas iteraciones máximas permitimos y cuántas iteraciones “sin mejora” toleramos.

En cada iteración ocurre lo siguiente:

- **Similitud** recibe la tabla filtrada y la tabla de trabajo. Devuelve:
 - una tabla donde algunas celdas vacías fueron rellenas por similitud;
 - una lista de imputaciones, donde cada registro indica aeronave, parámetro, valor propuesto y nivel de confianza.

• **Correlación** recibe la tabla de trabajo. Internamente entrena y evalúa muchos modelos candidatos (lineales, polinómicos, transformados y con uno o dos predictores, según la configuración), aplica todos los filtros de calidad y se queda con un modelo “ganador” por celda cuando encuentra algo razonable. Como salida, entrega:

- una tabla con las imputaciones por correlación;
- una lista de imputaciones (aeronave, parámetro, valor, confianza, advertencias);
- y un resumen de todos los modelos entrenados, que se guardará en el JSON para el Módulo 2.

El bucle no vuelve a elegir entre modelos internos de correlación: confía en el “mejor modelo” que ya le entrega esa etapa. Su tarea es decidir cómo combinar esa propuesta con la de similitud.

12.8.2 Reunir candidatos por celda

Con las dos listas de imputaciones, el bucle arma, para cada celda vacía identificada por (aeronave, parámetro), la lista de **candidatos** disponibles:

- si **ningún método** propuso un valor numérico, esa celda queda sin cambios en esa iteración;
- si **solo uno** de los métodos (similitud o correlación) propuso un valor, el candidato disponible es ése;
- si **ambos** propusieron, tenemos dos candidatos para la misma celda, cada uno con su propio valor y confianza.

A partir de ahí, el problema ya no es estadístico sino de fusión de propuestas.

12.8.3 Estrategias de combinación y ejemplo ilustrativo

El sistema admite varias políticas de combinación, configurables desde el dashboard:

- elegir siempre el método con mayor confianza (por encima de un mínimo);
- priorizar correlación (si hay modelo aceptable, usarlo directamente; si no, recurrir a similitud);
- priorizar similitud (la versión simétrica);
- o hacer un promedio ponderado por confianza, que es la configuración que usamos por defecto.

La idea del promedio ponderado es sencilla: si tenemos:

- valor de similitud V_s con confianza C_s ,
- valor de correlación V_c con confianza C_c ,

el valor final se coloca “más cerca” del método en el que más confiamos.

Ejemplo ilustrativo (no corresponde a datos reales de la tesis):

- Se busca imputar la potencia instalada de la aeronave A17.
- Similitud propone $V_s = 1300W$ con confianza $C_s = 0,60$.
- Correlación propone $V_c = 1500W$ con confianza $C_c = 0,80$.

El valor final se calcula como una media ponderada:

- multiplicamos cada valor por su confianza:

$$1300 \times 0,60 = 780;$$

$$1500 \times 0,80 = 1200.$$

- sumamos ambos:

$$780 + 1200 = 1980.$$

- dividimos por la suma de las confianzas:

$$C_s + C_c = 0,60 + 0,80 = 1,40.$$

Valor final:

$$V_{\text{final}} = 1980/1,40 \approx 1414W.$$

Es decir, el sistema propone alrededor de 1,41 kW, un compromiso razonable entre ambas fuentes. La confianza global se calcula también combinando C_s y C_c de forma que valores altos pesen más. Si la confianza resultante cae por debajo de un umbral mínimo definido en el dashboard, la celda se deja sin imputar, aunque hubiera candidatos.

Cuando solo hay un candidato (por ejemplo, solo correlación encontró un modelo aceptable), el bucle usa directamente ese valor y su confianza, siempre respetando los umbrales configurados.

12.8.4 Actualización de la tabla, criterio de parada y JSON para el Módulo 2

Una vez decidido el valor final de cada celda, el bucle:

1. Escribe las imputaciones aceptadas en la tabla de trabajo.
2. Actualiza contadores: cuántas celdas nuevas se completaron, cuántas quedan vacías.
3. Evalúa si tiene sentido seguir iterando:
 - sí en varias iteraciones seguidas se imputan muy pocas celdas nuevas, o
 - si se alcanza el máximo de iteraciones configurado, el proceso se detiene.

Al finalizar, el sistema produce dos salidas principales:

- un **Excel imputado**, que es la versión “completa” de la base de datos, lista para ser usada por el asistente de diseño del Módulo 3;

- y uno o varios **archivos JSON** que contienen:
 - las imputaciones celda por celda (valor final, confianza, método o combinación de métodos),
 - el detalle interno de lo que aportó similitud y lo que aportó correlación,
 - y todos los modelos entrenados por correlación con sus métricas.

Estos JSON son la materia prima del Módulo 2: gracias a ellos podemos reconstruir, de manera visual, las nubes de puntos, curvas y superficies que dieron lugar a las imputaciones, y revisar con lupa la calidad de los modelos antes de tomar decisiones de diseño (**Imagen 12.9**; **Imagen 12.10**; **Imagen 12.11**; **Imagen 12.12**; **Imagen 12.13**; **Imagen 12.14**).

12.9 Panel de configuración del Módulo 1 y sensibilidad del sistema

Todo este comportamiento no está “esculpido en piedra” dentro del código. Una de las decisiones que tomamos a medida que el módulo crecía fue trasladar muchos números y decisiones de diseño a un panel de configuración accesible desde el dashboard del Módulo 1. De este modo, el usuario puede ajustar la política de imputación sin editar scripts.

El panel, construido principalmente en `ux_config_panel.py` y `ux_notebook_panel.py`, se organiza en varias secciones. En la pestaña de orquestación se pueden activar o desactivar los métodos de similitud y correlación, definir el orden de ejecución y fijar criterios globales, como el mínimo de datos válidos para considerar entrenar un modelo. En las pestañas de similitud y correlación se encuentran los deslizadores y campos numéricos que controlan parámetros como el umbral máximo de diferencia entre aeronaves, el tamaño de los grupos de vecinos, los factores para detectar outliers o los umbrales mínimos de las métricas.

Hay también secciones específicas para la configuración de salidas (Excel y HTML) y para el comportamiento del sistema frente a extrapolaciones: el usuario puede elegir, por ejemplo, si quiere permitir extrapolaciones suaves dentro de cierto margen o si prefiere bloquearlas por completo, obligando al sistema a recurrir a la similitud o a dejar la celda sin imputar.

Durante el desarrollo del módulo experimentamos en carne propia la sensibilidad del sistema a estos parámetros. Umbrales demasiado estrictos dejaban muchas celdas sin imputar, sobre todo en los primeros intentos con menos aeronaves. Umbrales demasiado relajados producían modelos aceptables en términos de métricas globales, pero con comportamientos sospechosos en ciertas zonas del espacio de diseño. Esta experiencia nos convenció de que no podíamos fijar una única configuración “óptima”, sino que debíamos ofrecer un conjunto de valores por defecto razonables y, al mismo tiempo, permitir que el usuario ajuste el comportamiento según su contexto.

El panel de configuración refleja esa filosofía. Más que una colección de opciones técnicas, es una forma de hacer explícitas las decisiones que, en muchas herramientas estadísticas, quedan implícitas o escondidas en el código. El usuario no tiene por qué cambiar nada, pero, si quiere, puede adaptar la agresividad de la imputación a sus necesidades: ser muy conservador cuando está explorando un nuevo segmento de mercado, o algo más permisivo cuando trabaja con un conjunto de datos homogéneo y bien conocido.

Importante: Por defecto no se pedirán confirmaciones manuales; podés ajustar el comportamiento en la sección Confirmaciones.

Orquestación	Similitud	Correlación	Loop/Orquestación	Excel	HTML
Ruta Excel	C:/Users/delpi/OneDrive/Tesis/ADRpy-VTOL/ADRpy/analisis/Data/Datos_aeronaves.xlsx				
Carpeta de salida	Results				
Máx. iteraciones	1				
	☒ Ejecutar similitud ☒ Ejecutar correlación ☒ Permitir correlación sin filtro es...				
Mín. datos válidos	5				
	☒ Mostrar consola				
display.max_rows	200				
display.max_columns	120				
Confirmaciones Elegí cómo querés validar antes de ejecutar. En 'Automático' no se pedirá nada; en 'Resumen' verás un resumen para confirmar; en 'Consola' se intentará preguntar por teclado.					
Modo Automático (sin preguntas) ▼					
☐ Pedir confirmación de columna... ☐ Pedir confirmación de rutas (en...					
Guardar overrides		Guardar y ejecutar			

Imagen 12.9- Panel de configuración del Módulo 1: pestaña Orquestación

Pestaña donde se definen rutas de entrada/salida, comportamiento de confirmaciones y parámetros generales de ejecución del flujo.

Importante: Por defecto no se pedirán confirmaciones manuales; podés ajustar el comportamiento en la sección Confirmaciones.

Orquestación	Similitud	Correlación	Loop/Orquestación	Excel	HTML
▶ Básicos ▶ Familias ▶ Función de similitud ▶ Selección de vecinos ▶ Confianza (cv_ref & penalización k) ▶ Umbral por familia ▶ Outliers (Similitud)					
Guardar overrides		Guardar y ejecutar			

Imagen 12.10- Panel de configuración del Módulo 1: pestaña Similitud

Pestaña con secciones para configurar vecinos (Top-K), función de similitud, confianza y manejo de atípicos asociados al método por similitud.

Importante: Por defecto no se pedirán confirmaciones manuales; podés ajustar el comportamiento en la sección Confirmaciones.

Orquestación	Similitud	Correlación	Loop/Orquestación	Excel	HTML
--------------	-----------	--------------------	-------------------	-------	------

▸ Checks 2D

▸ Diversidad mínima

▸ Extrapolación

▸ Modelos

▸ Confianza

▸ Selección/LOOCV

▸ Outliers

Guardar overrides

Guardar y ejecutar

Imagen 12.11 - Panel de configuración del Módulo 1: pestaña Correlación

Pestaña con checks 2D, diversidad mínima, extrapolación, selección de modelos, confianza y criterios LOOCV/outliers para el método por correlación.

Importante: Por defecto no se pedirán confirmaciones manuales; podés ajustar el comportamiento en la sección Confirmaciones.

Orquestación	Similitud	Correlación	Loop/Orquestación	Excel	HTML
--------------	-----------	-------------	--------------------------	-------	------

▸ Orden de ejecución

▸ Combinación de métodos

▸ Criterios de parada

▸ Export/HTML

Guardar overrides

Guardar y ejecutar

Imagen 12.12- Panel de configuración del Módulo 1: pestaña Loop/Orquestación

Pestaña para definir orden de ejecución, combinación de métodos y criterios de parada del loop, además de opciones de exportación.

Importante: Por defecto no se pedirán confirmaciones manuales; podés ajustar el comportamiento en la sección Confirmaciones.

Orquestación	Similitud	Correlación	Loop/Orquestación	Excel	HTML
Color similitud	<input type="color" value="#ffff00"/>				
Color correlación	<input type="color" value="#00ff00"/>				
Color combinado	<input type="color" value="#0000ff"/>				
Color evaluado	<input type="color" value="#ffa500"/>				
Comentarios grandes	☒				
Permitir sobrescribir	☐				
Congelar paneles	☒				
Decimales exportación	2				

Imagen 12.13- Panel de configuración del Módulo 1: pestaña Excel

Opciones de exportación a Excel (colores, comentarios, sobrescritura, congelado de paneles y redondeo) para el DataFrame imputado.

Importante: Por defecto no se pedirán confirmaciones manuales; podés ajustar el comportamiento en la sección Confirmaciones.

Orquestación	Similitud	Correlación	Loop/Orquestación	Excel	HTML
HTML decimales	3				
HTML ancho (px)	1200				
HTML alto (px)	600				

Imagen 12.14- Panel de configuración del Módulo 1: pestaña HTML

Opciones de exportación de reportes HTML (decimales y dimensiones del render) para visualizaciones y resúmenes del proceso.

12.10 Registro de resultados, salidas en Excel y JSON

Imputar valores es solo la mitad del trabajo. La otra mitad es **documentar** lo que se hizo, de forma que se pueda comprender, revisar y, si hace falta, cuestionar las decisiones tomadas por el Módulo 1. Por eso dedicamos una parte importante del diseño a la estructura de salida.

Por un lado, generamos un archivo Excel que replica la tabla original de aeronaves, pero con las celdas imputadas integradas. Cada valor imputado no se añade “en silencio”: el sistema marca qué celdas fueron completadas, con qué método y, en muchos casos, incluye comentarios ampliados que describen las fuentes de la imputación (vecinos usados, modelo aplicado, advertencias sobre rango, etc.). Estos comentarios no son imprescindibles para usar la base, pero se convierten en una herramienta valiosa cuando se quiere inspeccionar un caso concreto.

Por otro lado, el Módulo 1 produce uno o varios archivos JSON que almacenan, en forma estructurada, toda la información relevante para el Módulo 2 y el Módulo 3. En esos JSON guardamos, para cada combinación de aeronave y parámetro, datos como:

- si el valor es original o imputado;
- qué método se utilizó (similitud, correlación, combinación);
- qué vecinos o modelos sustentan la imputación;
- las métricas clave asociadas (cuando aplica);
- el nivel de confianza calculado;
- y cualquier advertencia específica (por ejemplo, extrapolación moderada, datos muy escasos, presencia de outliers fuertes).

A diferencia del Excel, que está pensado para usuarios acostumbrados a trabajar con hojas de cálculo, los JSON (**Imagen 12.15**) están diseñados para ser consumidos por otros módulos de software. El Módulo 2, dedicado al análisis de modelos, lee esa información para construir tablas resumen, paneles de diagnóstico y gráficos que muestran, entre otras cosas, qué tan confiables son las imputaciones en distintos rincones del espacio de diseño. El Módulo 3, por su parte, utiliza la base imputada y la trazabilidad que encapsulan estos JSON para comparar una aeronave objetivo con las existentes, sabiendo en qué parámetros la información es sólida y en cuáles es más provisional.

Podemos pensar estos archivos como el “diario de a bordo” del Módulo 1: no solo muestran dónde hemos llegado (la base imputada), sino también qué ruta seguimos para llegar a cada valor.

Imagen 12.15 - Ejemplo de archivo JSON de resultados de imputación

Ejemplo ilustrativo de registro de resultados de imputación para una aeronave: filas por parámetro, indicando si el valor es original o imputado, método utilizado, fuentes (vecinos/modelos), métricas relevantes y nivel de confianza.

12.11 Limitaciones del Módulo 1 y puente hacia el Módulo 2

Aunque el Módulo 1 representa un avance significativo respecto a trabajar con la base de drones en bruto, es importante reconocer sus límites. La calidad de cualquier imputación está acotada por la calidad y cantidad de datos originales: cuando la base es escasa o está muy sesgada hacia ciertos tipos de

```

427379         "y_original": [
427380             0.239,
427381             0.31819504,
427382             0.31819504,
427383             0.352,
427384             0.352,
427385             0.352,
427386             0.196551724137931
427387         ],
427388         "columnas_predictores": [
427389             "Longitud del fuselaje"
427390         ]
427391     },
427392     "variable_independiente_1": 2.05,
427393     "variable_independiente_2": null
427394 },
427395 {
427396     "tipo": "log-1",
427397     "predictores": [
427398         "Longitud del fuselaje"
427399     ],
427400     "n_predictores": 1,
427401     "n_muestras_entrenamiento": 7,
427402     "coeficientes_originales": [
427403         0.285373604914421
427404     ],
427405     "intercepto_original": 0.040925614772547636,
427406     "ecuacion_string": "y = 0.04 + 0.29*log(Longitud del fuselaje)",
427407     "mape": 1.021647286522931,
427408     "r2": 0.9876652355917113,
427409     "corr": 0.9297777082450913,
427410     "Confianza": 0.0,
427411     "Confianza_LOOCV": 0.39281157006565537,
427412     "Corr_LOOCV": 0.8879801910818064,
427413     "MAPE_LOOCV": 2.9584613374742856,
427414     "R2_LOOCV": 0.9731911379952319,
427415     "Advertencia": null,
427416     "Peso de predictores": [
427417         1.0
427418     ],
427419     "transformacion": "logarítmica",
427420     "metodo_imputacion": "correlacion",
427421     "datos_entrenamiento": {
427422         "X_original": [
427423             [
427424                 2.1
427425             ],
427426             [
427427                 2.5908
427428             ],
427429             [
427430                 2.5908
427431             ],
427432             [

```

plataformas, el margen de maniobra estadístico se reduce. En esos casos, la mejor configuración posible del panel no convierte datos pobres en oro; solo nos ayuda a no engañarnos más de la cuenta.

Otro límite viene dado por la propia naturaleza de los modelos que usamos. Por muy cuidadosos que seamos con outliers y extrapolaciones, siempre existe el riesgo de que un modelo que parece funcionar bien en promedio produzca valores poco razonables en casos específicos. De hecho, durante el desarrollo nos encontramos con ejemplos de modelos que ofrecían excelentes métricas globales pero generaban predicciones negativas o con órdenes de magnitud extraños para ciertos parámetros. Justamente esas experiencias nos convencieron de que necesitábamos mirar a los modelos **de cerca**, no solo a través de resúmenes numéricos.

Finalmente, hay decisiones pragmáticas que asumimos para poder avanzar, como aceptar ciertas definiciones de peso o misión a partir de la información comercial disponible. Estas decisiones aumentan la completitud de la base y hacen posible entrenar modelos, pero introducen un grado de incertidumbre que debemos tener presente cuando interpretemos los resultados. Por eso dimos tanta importancia a la trazabilidad: cada vez que el sistema “rellena” un hueco, deja huellas de cómo y con qué nivel de confianza lo hizo.

Todo esto nos lleva de manera natural al siguiente capítulo. El Módulo 1 nos da una base imputada y un conjunto de modelos que, en principio, tienen sentido. Pero para utilizar estos modelos con tranquilidad en el diseño de una aeronave o en la comparación con el mercado, necesitamos herramientas que nos permitan ver sus curvas, sus superficies, sus zonas de mayor y menor apoyo en los datos, y sus advertencias. Ese es precisamente el papel del Módulo 2 – Análisis y visualización de modelos, que desarrollaremos en el siguiente capítulo.

13 MÓDULO 2: Panel interactivo de análisis de modelos

13.1 Del motor predictor a la visualización

En capítulo anterior vimos cómo el Módulo 1 toma la base de datos de drones, entrena modelos de correlación y produce dos salidas principales: un Excel con la tabla de aeronaves imputada y uno o varios archivos JSON con el catálogo de modelos y sus métricas.

El Módulo 2 arranca exactamente ahí. No vuelve a entrenar nada: se apoya en ese JSON para construir un panel interactivo que nos permite mirar los modelos “con lupa”. En términos simples:

el Módulo 1 calcula; el Módulo 2 nos ayuda a entender qué calculó y si tiene sentido confiar en ello.

Esta separación es deliberada. Queríamos que el entrenamiento de modelos pudiera ejecutarse como un proceso relativamente automático, pero que el análisis siguiera siendo un acto consciente, donde podamos combinar métricas numéricas con criterio de ingeniería y sentido físico.

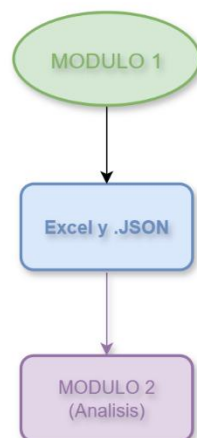


Imagen 13.1- Diagrama simple Módulo 1

Esquema de alto nivel del predictor: entradas (Excel original + configuración), entrenamiento/modelado e imputación, y salidas (Excel imputado, JSON/HTML).

13.2 De los JSON del Módulo 1 al dashboard del Módulo 2

Cuando abrimos el notebook del Módulo 2, las primeras celdas se encargan de:

- leer el JSON generado por el Módulo 1 (a través de `data_loader.py`),
- normalizar algunos campos y nombres (con `normalization_engine.py`),
- preparar tablas de trabajo que resumen, por modelo:
 - qué parámetro predice,
 - qué predictores usa,

- qué tipo de modelo es (lineal, polinómico, exponencial, etc.),
- cuántas muestras utilizó,
- qué métricas obtuvo (MAPE, R^2 , correlación, LOOCV, confianza, etc.).

Toda la lógica posterior del Módulo 2 se apoya en estas estructuras. Scripts como `plot_model_curves.py`, `plot_3d.py`, `metrics_dashboard.py` o `comparison_system.py` no vuelven a tocar los datos brutos de las aeronaves: trabajan sobre la “foto” que dejó el motor predictor.

Esto tiene dos ventajas claras, el análisis es reproducible; siempre partimos del mismo JSON y es ligero; podemos abrir y cerrar el panel, filtrar y combinar modelos, sin tener que recomputar regresiones cada vez.

13.3 Estructura general del panel: cuatro zonas que se hablan entre sí

Más que describir cada botón, nos interesa entender el tablero como un conjunto de zonas que dialogan:

1. Panel de filtros (izquierda)

Aquí elegimos el contexto de análisis: qué parámetro se quiere estudiar, qué predictor(es) interesan, qué tipo de modelos se quiere ver (lineales, polinómicos, exponenciales...) y qué elementos se quiere mostrar en las figuras (puntos de entrenamiento, puntos de imputación, curvas con o sin validación, etc.).

2. Gráfico central (2D/3D)

Es la “escena principal”. En 2D vemos el predictor en el eje horizontal y el parámetro objetivo en el vertical; aparecen los datos originales y las curvas de los modelos seleccionados. En 3D, cuando hay dos predictores, vemos superficies completas y podemos rotarlas para apreciar la geometría.

3. Tabla y ficha de modelo

La tabla “Resumen de modelos” funciona como catálogo: cada fila es un modelo y sus columnas condensan las métricas clave. Al seleccionar uno, el panel derecho muestra su “ficha completa”: tipo de modelo, ecuación, métricas, tamaño de muestra, etc.

4. Pestañas inferiores (2D / 3D / Comparación / Métricas)

Nos permiten cambiar de contexto: una vista centrada en la geometría (2D/3D), otra en la comparación agregada de varios modelos y otra más numérica, donde mandan las tablas de métricas.

En la práctica, trabajamos saltando constantemente entre estas zonas: filtramos en el panel izquierdo, vemos cómo se actualizan la tabla y las curvas, miramos la ficha del modelo que más nos convence y, si hace falta, pasamos a la pestaña de Métricas para ver cómo se comporta frente a otras opciones.

Análisis Interactivo de Modelos de Imputación

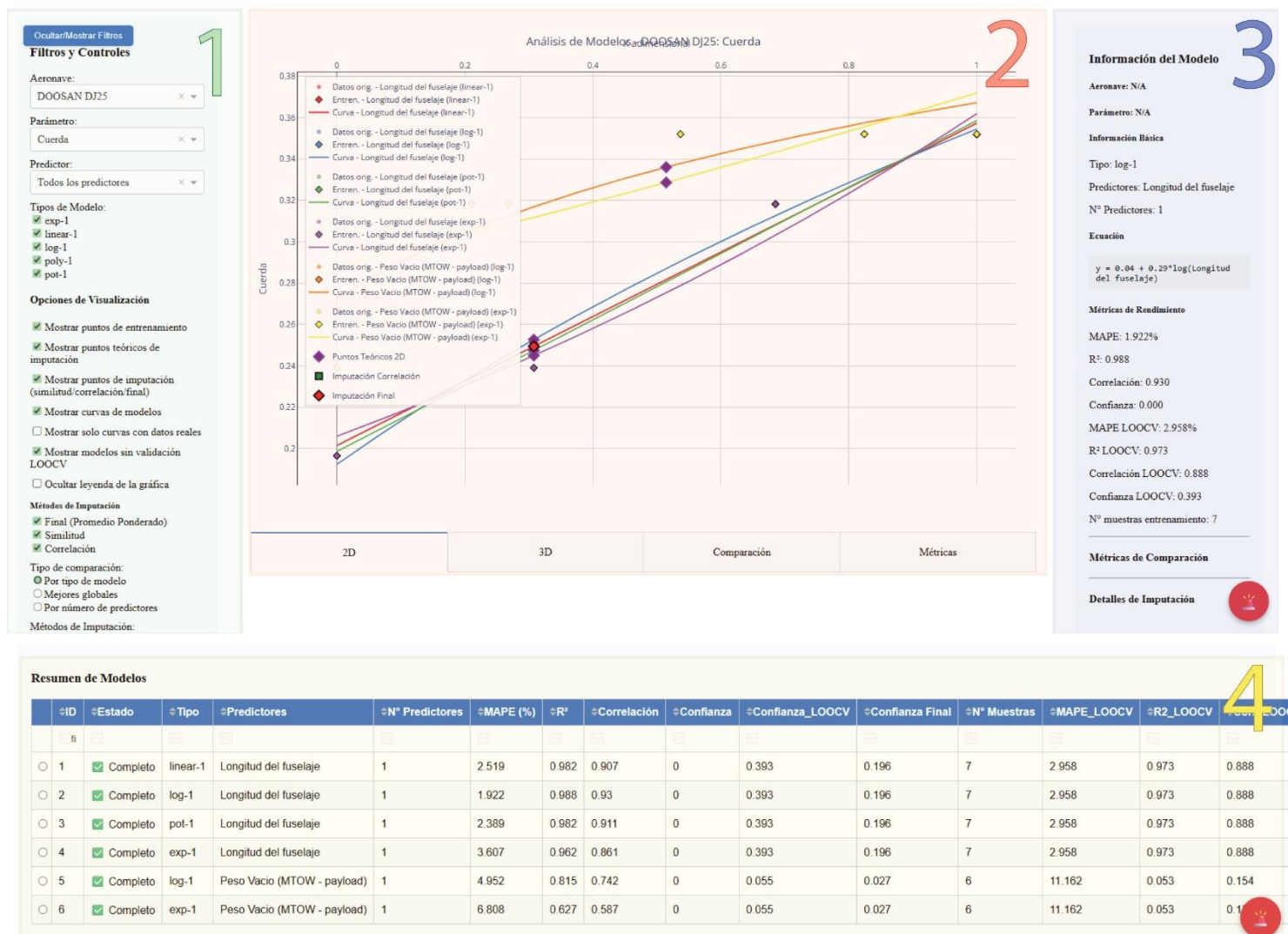


Imagen 13.2 - Estructura del dashboard del Módulo 2 (cuatro zonas)

Se identifican las zonas principales: (1) filtros/controles; (2) gráfico central; (3) resumen de modelos y métricas; (4) ficha con pestañas de detalle.

13.4 Uso del Módulo 2 en la práctica

Aunque el dashboard ofrece muchas combinaciones posibles, en la práctica seguimos algunos patrones de uso que se repiten. A continuación describimos un flujo típico de trabajo, que resume cómo concebimos este módulo.

13.4.1 Paso 1 – Elegir parámetro y predictor(es)

Partimos de una pregunta concreta, por ejemplo:

“¿Cómo está modelando el sistema la velocidad máxima en función de MTOW y payload?”.

En el panel izquierdo seleccionamos el parámetro “Velocidad máxima”, elegimos el predictor principal (o el par de predictores) y activamos los tipos de modelo que nos interesa comparar.

13.4.2 Paso 2 – Filtrar candidatos razonables

Miramos la tabla “Resumen de modelos” y hacemos un primer filtrado muy simple: descartamos modelos con muy pocas muestras o con métricas claramente malas. Aquí nos apoyamos en columnas como MAPE, R^2 , número de muestras o confianza final, pero sin absolutismos: preferimos combinaciones “razonables” antes que ganar una décima de MAPE a costa de entrenar con tres puntos.

13.4.2.1 Paso 3 – Mirar la geometría de las curvas y superficies

Con un puñado de modelos seleccionados, pasamos a la vista 2D o 3D. Allí vemos:

- cómo se distribuyen los puntos de datos originales,
- cómo se superponen las curvas (o superficies) de los modelos,
- y dónde caen los puntos de imputación para una aeronave objetivo.

En esta etapa no miramos solo números: buscamos que la forma general sea físicamente plausible. Por ejemplo, esperamos que, a igualdad de otras cosas, más MTOW no implique menos capacidad de carga de forma arbitraria, o que una superficie 3D no tenga picos imposibles en una región con muy pocos datos de entrenamiento.

13.4.3 Paso 4 – Abrir la ficha del modelo “candidato”

Cuando una curva o superficie nos resulta razonable a la vista, abrimos su ficha en el panel derecho. Allí confirmamos:

- que las métricas son coherentes con lo que vemos (por ejemplo, un R^2 moderado, un MAPE aceptable y valores LOOCV no muy degradados),
- que el número de muestras es suficiente para confiar en el modelo,
- y que la ecuación no tiene coeficientes desproporcionados que sugieran problemas de escala o colinealidad.

Esta ficha no está pensada para hacer álgebra a mano, sino para documentar y, si hace falta, poder reproducir el modelo en otro entorno.

La idea es auditar tanto la parte del pipeline como por ejemplo fallas en el código que puede ser que el flujo sea el correcto pero que falle por errores de formato de número, por ejemplo.

13.4.4 Paso 5 – Jugar con los métodos de imputación

Por último, solemos alternar entre tres vistas: imputación final, solo similitud y solo correlación. Los puntos de imputación en el gráfico se mueven ligeramente y esto nos da una idea de cuánto peso está aportando cada método en la región de diseño que nos interesa.

13.5 Casos de uso y lecciones obtenidas del análisis de modelos

El valor real del Módulo 2 se hizo evidente cuando empezamos a usarlo sobre nuestros propios modelos. Aquí resumimos algunos casos representativos que marcaron decisiones de diseño posteriores.

13.5.1 Superficies con muy poca cobertura de datos

En los primeros experimentos, algunos modelos de dos predictores mostraban métricas muy atractivas: MAPE bajo, R^2 alto y curvas suaves. Sin embargo, al visualizar la superficie 3D junto con los puntos de entrenamiento vimos algo preocupante: la superficie ocupaba una región amplia del espacio de entrenamiento, pero los puntos de datos reales se concentraban en una franja muy estrecha.

A simple vista, el ajuste parecía perfecto en la zona densa, pero en las regiones laterales la superficie estaba prácticamente “flotando” sin soporte de datos. Fue una señal clara de que, aunque las métricas globales eran buenas, la cobertura del modelo sobre el dominio de interés era pobre.

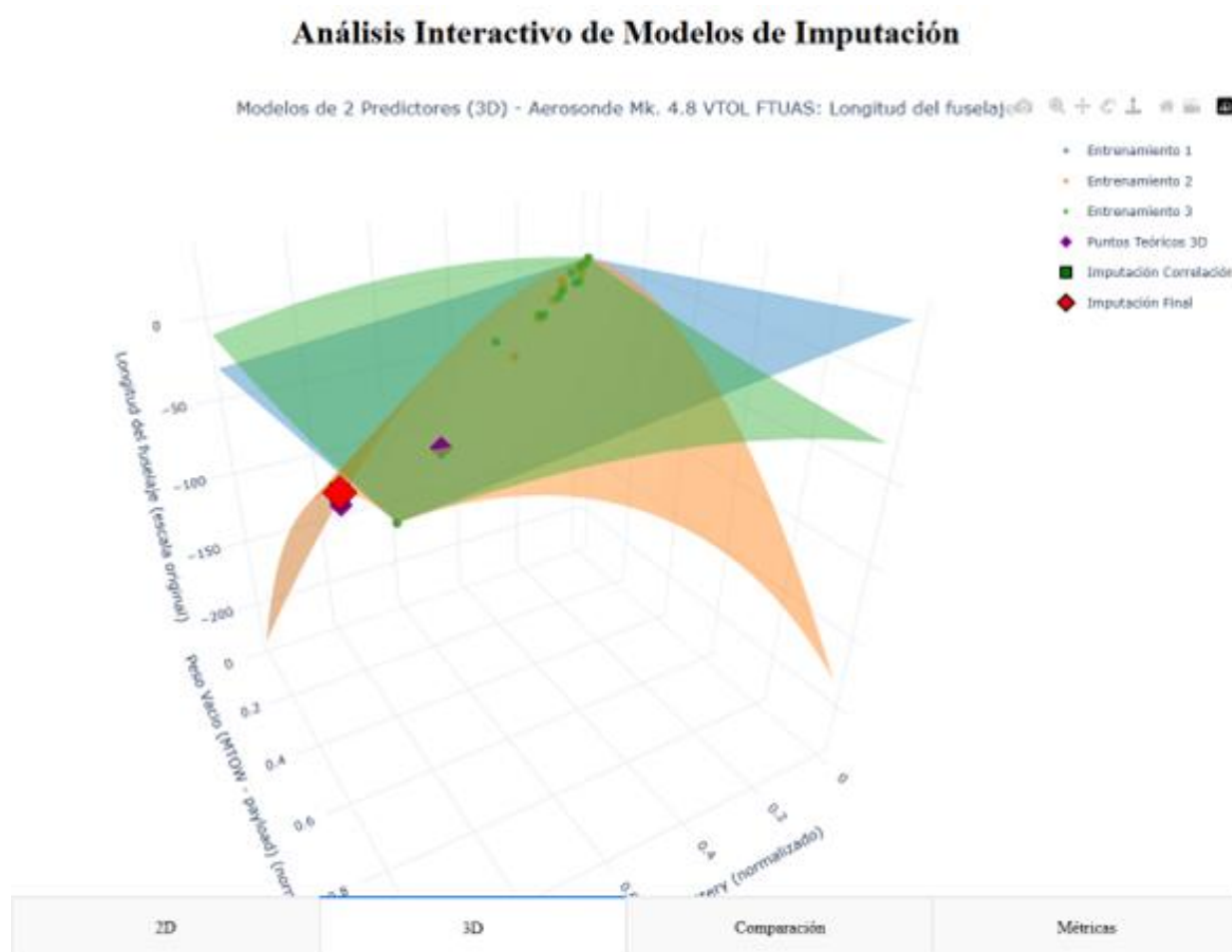


Imagen 13.3 - Ejemplo 3D: superficie con poca cobertura de datos

Captura del panel 3D mostrando una superficie ajustada con pocos puntos de entrenamiento y zonas sin soporte, que motivaron la incorporación de checks geométricos.

Esa observación visual nos llevó a introducir en el Módulo 1 varios checks geométricos, como:

- verificaciones de cobertura por pares de predictores (cobertura mínima de puntos),

- medidas basadas en **convex hull** y elipse de confianza para asegurarnos de que la aeronave objetivo no quedaba demasiado fuera del “núcleo” de datos,
- y umbrales mínimos para considerar aceptable la región donde se quería imputar.

En el Módulo 2 seguimos viendo la superficie, pero ahora sabemos que, si la aeronave objetivo queda demasiado alejada de la nube de entrenamiento, el propio motor predictor marcará ese modelo como poco confiable antes de llegar a esta instancia.

13.5.2 Modelos que daban valores físicamente imposibles

Otro patrón problemático que detectamos con el Módulo 2 fueron modelos con métricas razonables, pero predicciones físicas inaceptables. Un ejemplo típico eran curvas que, en el rango de interés, llegaban a valores negativos de velocidad, alcance o masa, algo que ningún dron real puede tener.

La combinación de la tabla de métricas con la gráfica 2D nos permitió ver rápidamente que estos modelos solían compartir dos características:

- estaban entrenados en rangos muy estrechos del predictor,
- y extrapolaban de forma violenta al alejarnos unos pocos pasos de esos rangos.

A partir de estas observaciones, ajustamos en el Módulo 1 los criterios de extrapolación aceptable y reforzamos algunos filtros basados en **PC2**, **VIF** y condición numérica. El objetivo fue claro: preferir modelos algo más “humildes” pero robustos, antes que curvas muy ajustadas en un rango minúsculo que se volvían absurdas apenas uno se alejaba de los datos.

13.5.3 Curvas polinómicas ‘perfectas’ que requirieron una revisión crítica

Uno de los episodios que más nos marcó con el Módulo 2 fue el comportamiento de algunos modelos polinómicos de segundo grado. En la tabla de “Resumen de modelos” aparecían con MAPE prácticamente nulo y un R^2 cercano a 1. A primera vista parecían modelos ideales: pasaban muy cerca de todos los puntos de entrenamiento y superaban holgadamente los umbrales de calidad que habíamos fijado.

Cuando miramos con más detalle la ecuación y la dibujamos en una herramienta externa (Geogebra), vimos el problema: la curva tenía forma de campana muy estrecha y muy pronunciada. El pico de esa campana coincidía justo con la zona donde estaban los pocos puntos de entrenamiento, pero apenas nos alejábamos un poco de ese rango, las predicciones se disparaban a valores completamente absurdos.

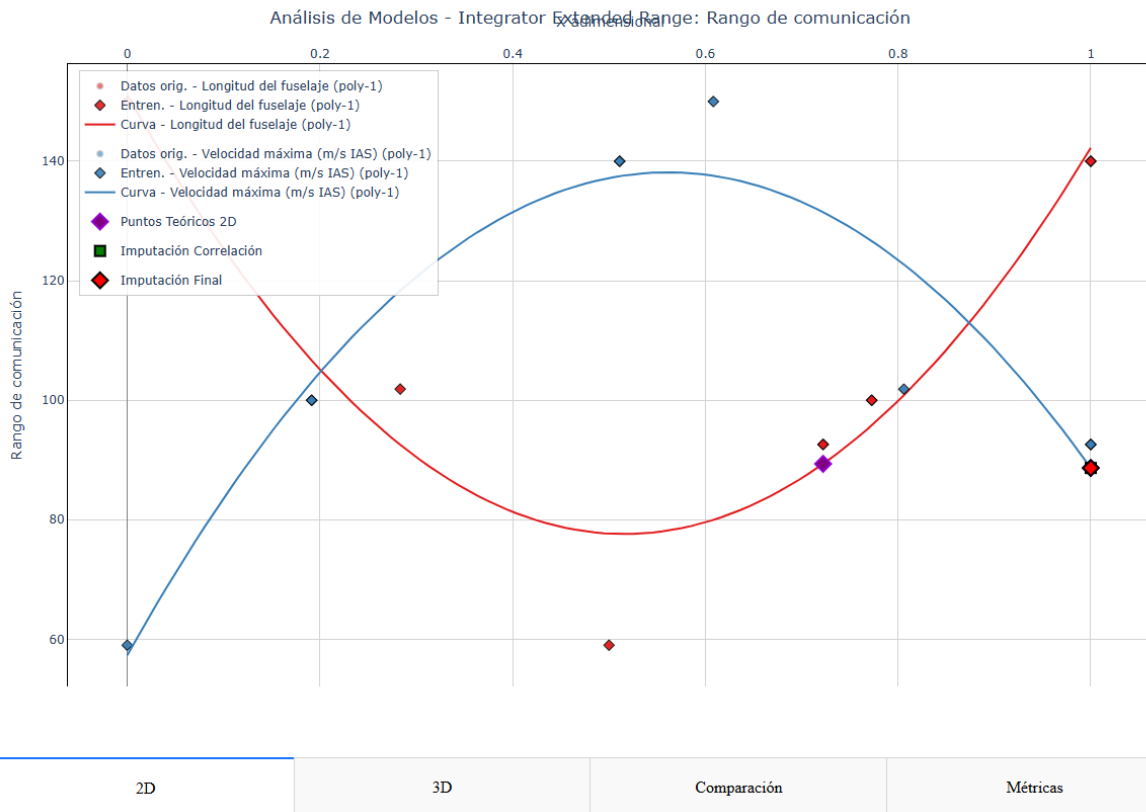


Imagen 13.4- Ejemplo 2D de modelo “engañoso” por extrapolación

Caso donde el modelo queda “clavado” en el centro de la nube y presenta comportamientos no físicos hacia los extremos, pese a métricas globales buenas.

En otras palabras, el modelo estaba “clavado” en el centro de la nube de datos y se hundía o crecía de forma brutal a ambos lados. Matemáticamente la regresión estaba muy bien ajustada; desde el punto de vista físico, era casi imposible justificar un comportamiento tan brusco para parámetros como velocidad, alcance o masa de una aeronave.

Esa experiencia nos dejó dos lecciones:

1. Necesitábamos ver todas las curvas dentro de nuestro propio sistema.

No alcanzaba con mirar métricas ni con inspeccionar una ecuación aislada. Queríamos poder abrir el panel, seleccionar un modelo polinómico y ver, en el mismo entorno, cómo se superpone su curva con los datos y con otros modelos más simples. Este fue uno de los motores directos para desarrollar el Módulo 2 y scripts como *plot_model_curves.py*, *plot_stability.py* y *equation_analyzer.py*.

2. Teníamos que ser más exigentes con los modelos polinómicos.

A partir de estos casos, ajustamos el Módulo 1 para exigir una cantidad mínima de puntos más alta cuando se trata de modelos polinómicos. La idea es simple: como son mucho más flexibles que los lineales, sólo los consideramos aceptables si están respaldados por una base de datos suficientemente amplia en el rango donde se pretende usarlos.

En contextos aeronáuticos es poco razonable esperar comportamientos en “campana” extremadamente agudos si no hay muchos datos que los sostengan. Y si ese comportamiento realmente existiera, al menos se busca que aparezca como resultado de una muestra rica, no de un ajuste con pocos puntos.

El Módulo 2 nos permitió “hacer visible” este tipo de problemas. Desde entonces, cada vez que vemos una campana muy marcada en el panel, nuestra primera reacción no es celebrar el MAPE, sino preguntar cuántos datos la están sosteniendo y si tiene sentido físico en el contexto del diseño preliminar.

13.6 Cómo el análisis del Módulo 2 alimenta al predictor y al asistente de diseño

Después de usar el Módulo 2 en múltiples sesiones, empezó a funcionar como un puente activo entre el motor predictor y el asistente de diseño.

Por un lado, muchas de las decisiones de configuración del Módulo 1 nacen de lo que vimos aquí. Los checks de cobertura, los umbrales específicos para modelos polinómicos, los límites a la extrapolación y varios filtros de estabilidad numérica no salieron de la teoría de manual, sino de mirar curvas, superficies y casos concretos hasta entender dónde se rompía la lógica física. Cada vez que detectábamos un patrón problemático en el Módulo 2, volvíamos al Módulo 1 y ajustábamos la “caja negra” del predictor para que esos modelos quedaran penalizados o directamente descartados en futuras corridas.

Por otro lado, el Módulo 2 también prepara el terreno para el asistente de diseño del Módulo 3. Cuando pensamos en definir parámetros de una aeronave objetivo, no nos interesa cualquier modelo que pase los filtros mínimos, sino aquellos que: están respaldados por un número razonable de datos, muestran una geometría coherente con el sentido común aerodinámico y mantienen un comportamiento estable en el rango donde se pretende diseñar.

El panel interactivo es el lugar donde podemos “curar” esta colección de modelos de referencia. No lo hacemos de forma automática, pero sí con más información y menos intuiciones ciegas que al principio del proyecto.

13.7 Cierre del capítulo y puente hacia el Módulo 3

En este capítulo presentamos el Módulo 2 como lo que realmente es en nuestro flujo de trabajo: una lupa visual sobre el motor estadístico del Módulo 1. Vimos cómo el panel se alimenta de los JSON del predictor, cómo organiza la información en filtros, tablas, curvas y superficies, y cómo lo usamos para combinar métricas numéricas con criterio de ingeniería.

Los casos de uso mostraron que muchas de las mejoras del sistema nacieron de aquí: superficies con poca cobertura de datos, curvas que daban valores físicamente imposibles y polinomios en forma de campana que parecían perfectos sobre el papel, pero que se comportaban de forma absurda fuera de una zona muy pequeña. Cada una de estas situaciones nos obligó a revisar umbrales, checks y criterios de aceptabilidad en el Módulo 1.

En el Capítulo siguiente nos apoyaremos en este trabajo previo para presentar el Módulo 3, el asistente de diseño. Allí veremos cómo la base imputada y los modelos filtrados se transforman en herramientas concretas para comparar una aeronave objetivo con el mercado y para escoger valores de diseño más razonables desde el inicio del proceso.

14 MÓDULO 3: Asistente de diseño

🔗 Abrir Excel original

Usá este botón para auditar los vecinos directamente en el Excel original.

α similitud

1.00

Penalizar NaN

☒

penalidad N...

1

Top-K

10

Quitar atípicos (IQR)

☒

factor IQR

1.5

Segmentar ...

(ninguno)

Modo

Global (sin segmentar)

Valor

(ninguno)

factor prefer

1.30

Recalcular

Limpiar

☐ Auto

Parámetros (dinámico)

Buscar parámetro...

☐ Solo activos

Activar visibles

Desactivar todos

Orden: Alfabético

Grupo: Todos

Vista: Detallada

Por página: 15

Página 1/3

Modo masi...

(sin cambio)

Peso masivo: 1

0

0

Aplicar a visibles

☒ Alcance de la aeronave (km)

☐ Ancho del fuselaje

☐ Autonomía de la aeronave (h)

☐ Cantidad de motores propulsión horizontal

☐ Cantidad de motores propulsión vertical

☐ Capacidad combustible

☐ Clase

☐ Consumo

☐ Cuerda

☐ Despegue

☐ Envergadura

☐ Longitud del fuselaje

☐ Maximum Crosswind (sostenido)

☐ Payload

☐ Peso frame

Fijo (= valor)

500

1.00

i

Ignorar

1.00

i

Ignorar

1.00

i

Ignorar

1.00

i

Ignorar

1.00

i

Ignorar

1.00

i

Ignorar

1.00

i

Ignorar

1.00

i

Ignorar

1.00

i

Ignorar

1.00

i

Ignorar

1.00

i

Ignorar

1.00

i

Ignorar

1.00

i

Ignorar

1.00

i

Ignorar

1.00

i

Ignorar

1.00

i

Ignorar

1.00

i

Ignorar

1.00

i

Ignorar

1.00

i

Ranking por similitud

Similitud (ranking): ordena todas las aeronaves por cercanía al objetivo (no filtra). Top-K (vecinos): trabaja sólo con los K más cercanos para sugerencias locales y análisis detallado.

Segmento: Todos

Sim mín: 0.00

Buscar:

Orden: similitud

Dirección: Desc

Top-N: 15

Limpiar

	aeronave	segmento	alerta	distancia	distancia_med	similitud	similitud_med	Alcance de la aeronave (km)	Δ Alcance de la aeronave...	Δ Alcance la aeronave
0	Objetivo (usuario)	(selección)		0.00	0.00	1.00	1.00	500.00	0.00	
1	Integrator Extended Range	Vigilancia / fotogrametría y carga		0.00	0.00	1.00	1.00	500.00	0.00	
2	DOOSAN DJ25	Vigilancia / fotogrametría y carga		0.00	0.00	1.00	1.00	500.00	0.00	
3	Vollitation VT510	Vigilancia / fotogrametría y carga		0.05	0.05	0.95	0.95	540.00	40.00	
4	JOUAV CW-25H	Vigilancia / fotogrametría y carga		0.08	0.08	0.92	0.92	435.00	-65.00	
5	Stalker VXE30 Fuel Cell	Vigilancia / fotogrametría y carga		0.09	0.09	0.92	0.92	433.00	-67.00	
6	Ascend	Vigilancia / fotogrametría y carga		0.09	0.09	0.92	0.92	432.00	-68.00	
7	V39	Vigilancia / fotogrametría y carga		0.12	0.12	0.88	0.88	405.00	-95.00	
8	Aliaca UAS ER	Vigilancia / fotogrametría y carga		0.14	0.14	0.87	0.87	388.80	-111.20	
9	Shield AI V-BAT 128	Vigilancia / fotogrametría y carga		0.15	0.15	0.86	0.86	613.08	113.08	
10	Stalker XE	Vigilancia / fotogrametría y carga		0.17	0.17	0.85	0.85	370.00	-130.00	
11	YANGDA Sky Whale Imax Electric Heavy Lift Long Endurance VTOL Drone	Vigilancia / fotogrametría y carga		0.18	0.18	0.83	0.83	360.00	-140.00	
12	Edge Autonomy Penguin C Mk 2.5 VTOL	Vigilancia / fotogrametría y carga		0.21	0.21	0.81	0.81	665.28	165.28	
13	V32	Vigilancia / fotogrametría y carga		0.23	0.23	0.80	0.80	324.00	-176.00	
14	YANGDA Sky Whale Mini Electric Long Endurance VTOL Drone	Vigilancia / fotogrametría y carga		0.23	0.23	0.79	0.79	319.10	-180.90	

Detalle del parámetro

▼ Resumen global/filtrado: Alcance de la aeronave (km)

DataFrame original

Resumen (global)

Cobertura: 64/83 (NaN 22.89%)

Min: 25.00 • Q1: 210.60 • Mediana: 365.00 • Media: 669.27 • Q3: 982.80 • Max: 3270.00

Rango IQR: [-947.70, 2141.10]

Sugerido (Top-K): 434.00

Objetivo: 500.00

DataFrame filtrado

Resumen (filtrado)

Cobertura: 64/83 (NaN 22.89%)

Min: 25.00 • Q1: 210.60 • Mediana: 365.00 • Media: 669.27 • Q3: 982.80 • Max: 3270.00

Rango IQR: [-947.70, 2141.10]

Rangos & atípicos

Distribución & accesos

Ayuda de controles

Ranking ...

Tenden...

Sugerenc...

O...

Panel ...

Ranking y similitud

Tendencias X-Y

Sugerencias (Top-K)

Outliers

14-136

Sugerencias (Top-K)

Sugerencias (Top-K)

Resumen por parámetro (Top-K)

	parametro	top_k	n_vecinos	n_usados	n_outliers	calidad_n	min	q1	mediana	media	q3	max	w_mediana	w_media	n_efectivo
0	Alcance de la aeronave (km)	10	10	10	0	buena	370.000000	411.750000	434.000000	461.688000	500.000000	613.080000	434.000000	463.009778	9.205089

Distribución por parámetro

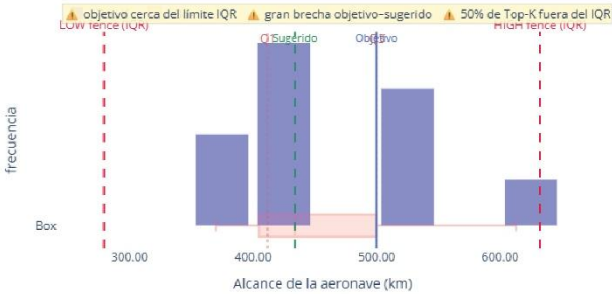
Sugerencias (Top-K) N_efectivo=9.21 / n_usados=10

Parámetro: Alcance de la aeronave (km)

Consultá el Excel original (botón "Abrir Excel original" en la parte superior) para ver la lista completa de vecinos ordenados por peso.

Distribución Top-K: Alcance de la aeronave (km)

Q1=411.75 - Q3=500.00 - n_topk=10



Para revisar los vecinos Top-K podés abrir el Excel original con el botón superior.

Tendencias (X-Y) — Global vs Filtrado

X: Velocidad a la que se realiza el crucero (m/s TAS) Y: Techo de servicio máximo Modo: Global

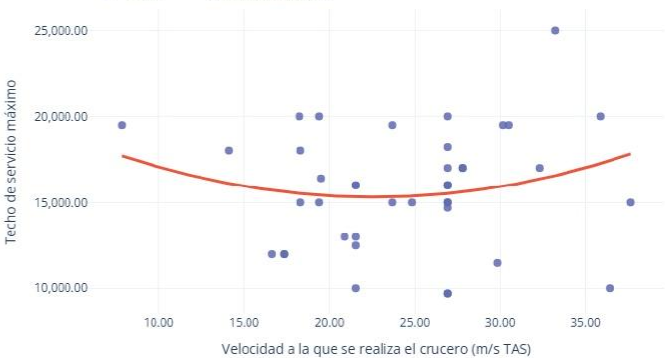
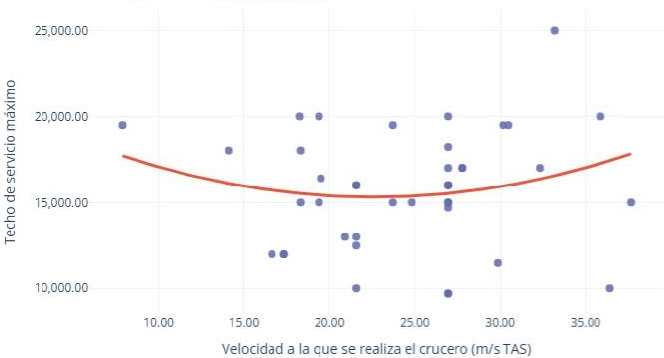
☒ Quitar atípicos (IQR) factor IQR 1.5 min_n Recalcular

DataFrame original (n=83)

DataFrame filtrado (n=83)

Tendencias (X-Y): Velocidad a la que se realiza el crucero (m/s TAS) vs Techo de servicio

Tendencias (X-Y): Velocidad a la que se realiza el crucero (m/s TAS) vs Techo de servicio



	ámbito	n	modelo	ecuación	R ² _adj	MAPE_%	calidad_n	IQR	min_n
0	Global	41	cuadrático	$y = 11.03 \cdot x^2 + -498.07 \cdot x + 20936.49$	-0.041600	18.310000	buena	ON	5

Outliers (IQR) — Global vs filtrado

DataFrame original (n=83)

DataFrame filtrado (n=83)

Outliers (IQR k=1.50)

Outliers (IQR) – DataFrame original

Resumen (IQR k=1.50)

Parámetro: Alcance de la aeronave (km)

	columna	n_validos	Q1	Q3	IQR	low	hi
20	Maximum Crosswind (sostenido)	17	13.000000	17.000000	4.000000	7.000000	23.0000
17	Peso frame	15	5.300000	11.250000	5.950000	-3.625000	20.1750
23	Consumo	5	0.600000	1.200000	0.600000	-0.300000	2.1000
12	Velocidad de pérdida (m/s CAS)	24	13.858428	17.000000	3.141572	9.146069	21.7123
4	Área del ala	45	1.000000	1.550000	0.550000	0.175000	2.3750
25	Potencia HP	7	4.000000	9.000000	5.000000	-3.500000	16.5000
3	Velocidad de pérdida limpia (m/s CAS)	30	14.000000	16.000000	2.000000	11.000000	19.0000

Outliers (IQR k=1.50)

Outliers (IQR) – DataFrame filtrado

Resumen (IQR k=1.50)

Parámetro: Alcance de la aeronave (km)

	columna	n_validos	Q1	Q3	IQR	low	hi
20	Maximum Crosswind (sostenido)	17	13.000000	17.000000	4.000000	7.000000	23.0000
17	Peso frame	15	5.300000	11.250000	5.950000	-3.625000	20.1750
23	Consumo	5	0.600000	1.200000	0.600000	-0.300000	2.1000
12	Velocidad de pérdida (m/s CAS)	24	13.858428	17.000000	3.141572	9.146069	21.7123
4	Área del ala	45	1.000000	1.550000	0.550000	0.175000	2.3750
25	Potencia HP	7	4.000000	9.000000	5.000000	-3.500000	16.5000
3	Velocidad de pérdida limpia (m/s CAS)	30	14.000000	16.000000	2.000000	11.000000	19.0000

2	Techo de servicio máximo	54	12000.000000	17750.000000	5750.000000	3375.000000	26375.0000
26	Precio	15	5899.000000	14949.500000	9050.500000	-7676.750000	28525.2500
28	Propulsión horizontal	83	1.000000	2.000000	1.000000	-0.500000	3.5000
9	Alcance de la aeronave (km)	64	210.500000	982.800000	772.200000	-947.700000	2141.1000
14	Cuerda	22	0.295992	0.345878	0.049885	0.221164	0.4207
7	Ancho del fuselaje	28	0.202135	0.277000	0.074865	0.089838	0.3892
11	Velocidad máxima (m/s IAS)	62	31.448182	38.892137	7.443955	20.282249	50.0580
1	Velocidad a la que se realiza el crucero (m/s TAS)	67	19.390000	27.790000	8.400000	6.790000	40.3900
16	Velocidad crucero (m/s IAS)	68	17.750000	25.800000	8.050000	5.675000	37.8750
6	Longitud del fuselaje	72	1.480000	2.500000	1.020000	-0.050000	4.0300
10	Autonomía de la aeronave (h)	80	2.950000	12.000000	9.050000	-10.625000	25.5750
32	Misión	83	1.000000	1.000000	0.000000	—	—
29	Propulsión vertical	83	1.000000	3.000000	2.000000	-2.000000	6.0000
27	Despegue	83	1.000000	2.000000	1.000000	-0.500000	3.5000
31	Cantidad de motores propulsión horizontal	83	1.000000	1.000000	0.000000	—	—
24	Potencia Watts	5	1280.000000	2980.000000	1700.000000	-1270.000000	5530.0000
30	Cantidad de motores propulsión vertical	83	0.000000	4.000000	4.000000	-6.000000	10.0000
0	Modelo	19	0.000000	380.000000	380.000000	-570.000000	950.0000
22	Capacidad combustible	7	12.250000	28.000000	15.750000	-11.375000	51.6250
21	Rango de comunicación	56	30.000000	132.500000	102.500000	-123.750000	286.2500
19	Peso Vacío (MTOW - payload)	71	12.500000	43.150000	30.650000	-33.475000	89.1250
18	Peso vacío - fuel y battery	10	7.000000	31.937500	24.937500	-30.406250	69.3437
15	Payload	74	2.200000	11.825000	9.625000	-12.237500	26.2625
13	Envergadura	75	2.750000	4.650000	1.900000	-0.100000	7.5000
8	Peso máximo al despegue (MTOW)	83	12.250000	53.950000	41.700000	-50.300000	116.5000
5	Relación de aspecto del ala	26	12.674539	15.265523	2.590984	8.788063	19.1519
33	Clase	83	1.000000	3.000000	2.000000	-2.000000	6.0000

Histogramas (IQR k=1.50)

Parámetro: Alcance de

factor IQR: 1.50

☒ Listar outliers



	columna	n_valido	Q1	Q3	IQR	LOW	HIGH	usable
0	Alcance de la aeronave (km)	64	210.60	982.80	772.20	-947.70	2141.10	True

Outliers (3 filas):

	Alcance de la aeronave (km)	aeronave	tipo_outlier
14	2445.120000	Integrator	HIGH
74	2663.710000	T-20	HIGH
8	3270	AAI Aerosonde	HIGH

2	Techo de servicio máximo	54	12000.000000	17750.000000	5750.000000	3375.000000	26375.0000
26	Precio	15	5899.000000	14949.500000	9050.500000	-7676.750000	28525.2500
28	Propulsión horizontal	83	1.000000	2.000000	1.000000	-0.500000	3.5000
9	Alcance de la aeronave (km)	64	210.600000	982.800000	772.200000	-947.700000	2141.1000
14	Cuerda	22	0.295992	0.345878	0.049885	0.221164	0.4207
7	Ancho del fuselaje	28	0.202135	0.277000	0.074865	0.089838	0.3892
11	Velocidad máxima (m/s IAS)	62	31.448182	38.892137	7.443955	20.282249	50.0580
1	Velocidad a la que se realiza el crucero (m/s TAS)	67	19.390000	27.790000	8.400000	6.790000	40.3900
16	Velocidad crucero (m/s IAS)	68	17.750000	25.800000	8.050000	5.675000	37.8750
6	Longitud del fuselaje	72	1.480000	2.500000	1.020000	-0.050000	4.0300
10	Autonomía de la aeronave (h)	80	2.950000	12.000000	9.050000	-10.625000	25.5750
32	Misión	83	1.000000	1.000000	0.000000	—	—
29	Propulsión vertical	83	1.000000	3.000000	2.000000	-2.000000	6.0000
27	Despegue	83	1.000000	2.000000	1.000000	-0.500000	3.5000
31	Cantidad de motores propulsión horizontal	83	1.000000	1.000000	0.000000	—	—
24	Potencia Watts	5	1280.000000	2980.000000	1700.000000	-1270.000000	5530.0000
30	Cantidad de motores propulsión vertical	83	0.000000	4.000000	4.000000	-6.000000	10.0000
0	Modelo	19	0.000000	380.000000	380.000000	-570.000000	950.0000
22	Capacidad combustible	7	12.250000	28.000000	15.750000	-11.375000	51.6250
21	Rango de comunicación	56	30.000000	132.500000	102.500000	-123.750000	286.2500
19	Peso Vacío (MTOW - payload)	71	12.500000	43.150000	30.650000	-33.475000	89.1250
18	Peso vacío - fuel y battery	10	7.000000	31.937500	24.937500	-30.406250	69.3437
15	Payload	74	2.200000	11.825000	9.625000	-12.237500	26.2625
13	Envergadura	75	2.750000	4.650000	1.900000	-0.100000	7.5000
8	Peso máximo al despegue (MTOW)	83	12.250000	53.950000	41.700000	-50.300000	116.5000
5	Relación de aspecto del ala	26	12.674539	15.265523	2.590984	8.788063	19.1519
33	Clase	83	1.000000	3.000000	2.000000	-2.000000	6.0000

Histogramas (IQR k=1.50)

Parámetro: Alcance de

factor IQR: 1.50

☒ Listar outliers



	columna	n_valido	Q1	Q3	IQR	LOW	HIGH	usable
0	Alcance de la aeronave (km)	64	210.60	982.80	772.20	-947.70	2141.10	True

Outliers (3 filas):

	Alcance de la aeronave (km)	aeronave	tipo_outlier
14	2445.120000	Integrator	HIGH
74	2663.710000	T-20	HIGH
8	3270	AAI Aerosonde	HIGH

Imagen 14.1 – Interfaz del asistente de diseño (vista detallada en tres partes)

Conjunto de tres capturas que, en conjunto, muestran la interfaz completa del asistente: controles globales, tabla de parámetros y paneles de ayuda/diagnóstico (tendencias, sugerencias Top-K, outliers), usada como referencia en secciones posteriores.

En este capítulo describimos el tercer módulo del sistema ADRpy-VTOL: el asistente de diseño. A diferencia de los módulos anteriores, que se concentran en construir y auditar modelos a partir del mercado, aquí usamos ese conocimiento para acompañar la elección de parámetros cuando se busca definir una aeronave nueva.

El módulo fue pensado principalmente como herramienta profesional (alrededor del 80 % del uso previsto): un apoyo práctico para pre-dimensionar diseños reales antes de entrar al cálculo detallado. El restante 20 % corresponde a un uso más pedagógico, útil para mostrar a estudiantes qué significa “estar dentro del mercado” o “plantear un diseño demasiado optimista”.

14.1 Rol del asistente de diseño en el flujo global

En los capítulos dónde explicamos del Módulo 1 y Módulo 2 construimos, filtramos y analizamos modelos descriptivos a partir de una base de drones existentes. Allí aprendimos, por ejemplo, cómo se comportan parámetros como alcance, MTOW o autonomía en función de la misión y del resto de variables, y qué tan confiables son esas relaciones.

El Módulo 3 se apoya directamente en ese trabajo previo. Su función es recibir una idea de misión (por ejemplo, vigilancia de larga duración con carga electro-óptica) y traducirla en una configuración numérica que sea razonable a la luz de los datos. El asistente no “diseña solo” el dron; lo que hace es:

- ordenar la base de datos por **similitud** con el diseño planteado,
- mostrar **rangos típicos** y **umbrales extremos** por parámetro,
- emitir **advertencias** cuando un valor se aleja demasiado del comportamiento observado,
- y reunir verificaciones físicas básicas en un panel único.

La decisión final sigue siendo del ingeniero. El asistente acota el espacio de búsqueda y ayuda a ver compromisos, pero no garantiza el “mejor diseño posible” ni reemplaza el análisis detallado de performance que luego se realiza en ADRpy.

14.2 Entrada de datos: definir la aeronave objetivo

El punto de partida es siempre una **aeronave objetivo**: un diseño preliminar que se pretende explorar. El panel principal del dashboard (**Imagen 14.2**) está organizado en tres niveles:

1. Controles globales, donde elegimos:

- el **factor α de similitud** (qué tan agresiva es la agregación de distancias),
- si penalizamos o no los **NaN**,
- el **Top-K** de vecinos que usaremos para sugerencias,
- el factor IQR y la política para **quitar atípicos**,

- y la **segmentación** por misión o familia (global, por segmento, solo un segmento, etc.).

2. **Panel de parámetros dinámicos**, que contiene una fila por cada parámetro relevante del dataset (Imagen 14.2):

- nombre del parámetro (por ejemplo, “*Alcance de la aeronave (km)*”, “*Velocidad de crucero (m/s TAS)*”, “*Payload*”, etc.);
- un modo por parámetro:
 - ignorar,
 - fijo (igualar a un valor),
 - objetivo (tolerancia alrededor de un valor deseado),
 - máximo / mínimo (cotas unilaterales),
 - rango (intervalo [min, max]);
- el **valor objetivo** y, si aplica, tolerancias o rangos;
- un **peso relativo**, que indica cuánto influye ese parámetro en la distancia global.

Activando la casilla de cada fila decidimos si el parámetro participa o no en el cálculo de similitud.

3. Un pequeño panel lateral con el **estado de sesión** (archivo JSON de configuración cargado o auto-guardado), que permite recuperar rápidamente configuraciones previas de parámetros. Dónde también se despliegan paneles al presionar el botón de información en cada parámetro, que nos muestran un pantallazo de métricas importantes para ayudarnos a colocar valores razonables y a su vez un panel de ayuda que funciona como guía de cada ajuste posible que podemos realizar sobre los paneles.¹⁶

En términos de interacción, lo habitual es que partamos de una intuición de misión (“quisiera un dron de vigilancia de larga duración para x kg de carga”) y especifiquemos primero los parámetros que más nos importan. En una etapa inicial solemos trabajar con modos fijo u objetivo para MTOW, carga útil, alcance y velocidad de crucero, dejando el resto en ignorar.

¹⁶ El panel de ayuda se despliega una vez que presionamos el botón de “?” que se encuentra haciendo scroll a la derecha en la parte superior derecha.

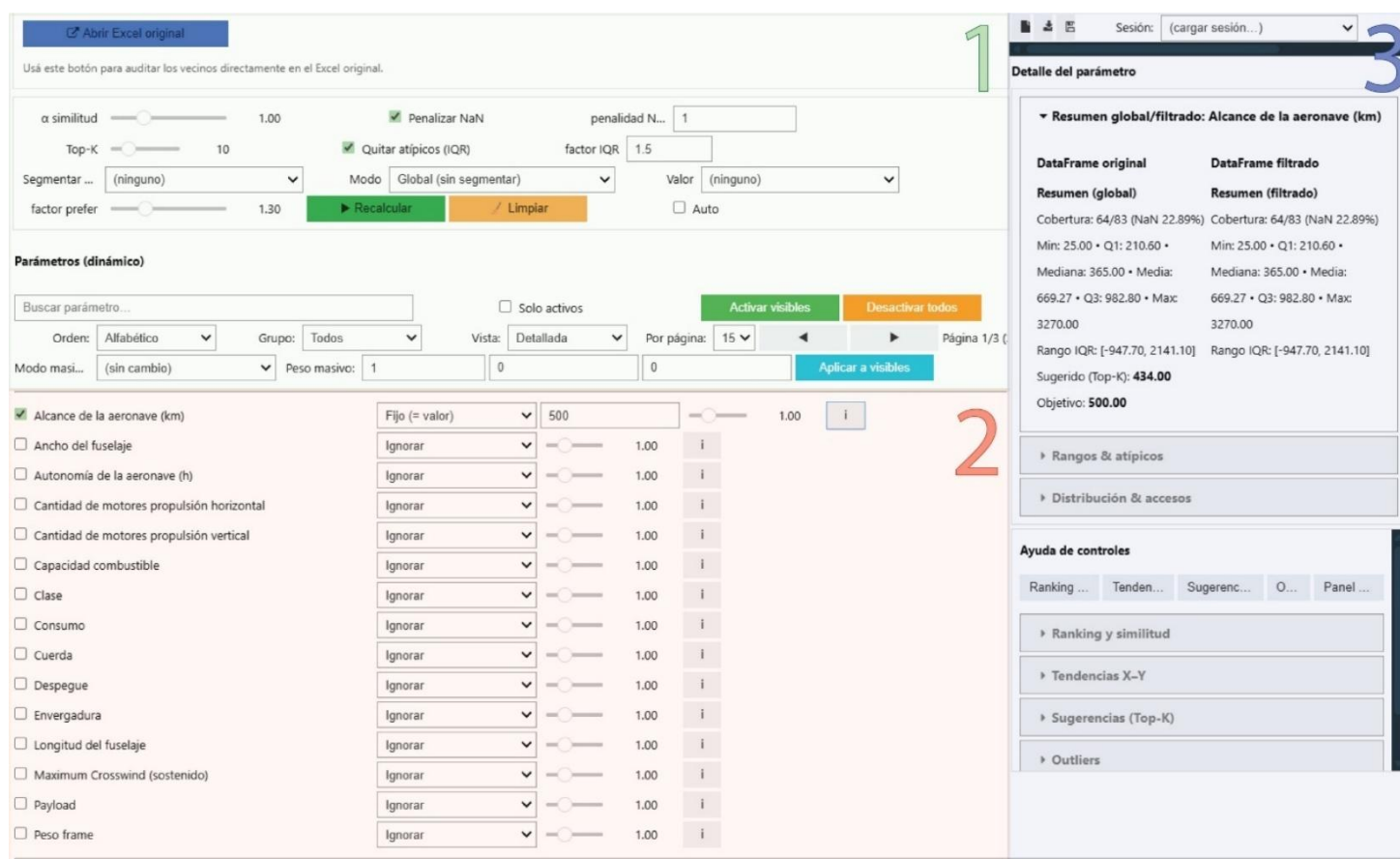


Imagen 14.2- Zonas principales del asistente de diseño (captura anotada)

Captura anotada que identifica (1) controles globales; (2) lista de parámetros con modo/valor/peso; (3) panel lateral de detalle y ayuda.

14.3 Reutilización de tendencias y modelos univariados

El asistente no vuelve a calcular toda la estadística descriptiva presentada en los capítulos anteriores. En su lugar, reutiliza las tendencias X–Y construidas en el Módulo 2 (tendencias.py y herramientas asociadas) como contexto visual.

Para cada par de variables (por ejemplo, alcance vs MTOW, velocidad de crucero vs techo de servicio) el sistema muestra:

- la **nube de puntos** de los drones de la base,
- la **curva de tendencia** seleccionada en el análisis previo (lineal, cuadrática, etc.),
- y el **punto de la aeronave objetivo**, resaltado sobre ese fondo.

El objetivo no es afinar coeficientes ni comparar modelos (eso ya se discutió en los Capítulos del Módulo 1 y Módulo 2) sino ayudar a responder preguntas prácticas del tipo:

“Si reduzco un poco la velocidad, ¿parece razonable esperar más alcance?”

“¿Mi combinación de payload y MTOW se parece a la de otros diseños del mismo segmento?”

La **Imagen 14.1** (tendencia X-Y Global vs Filtrado) ilustra este uso: las tendencias actúan como “reglas de pulgar cuantitativas” que nos permiten ver si un ajuste en un parámetro mueve el diseño hacia una zona donde existen ejemplos reales, o hacia una región poco poblada del espacio de diseño.

14.4 Ranking por similitud y exploración de vecinos (versión ampliada con base teórica)

El corazón del asistente es el motor de similitud, que implementa una versión adaptada del método de k-vecinos más cercanos (k-NN) presentado en el Marco Teórico. En lugar de entrenar un modelo paramétrico (como una regresión global), el sistema responde a la pregunta:

“¿A qué aeronaves existentes se parece más el diseño objetivo, considerando los parámetros que el usuario decidió activar?”

Representación del diseño en un espacio de parámetros

Cada aeronave (incluida la aeronave objetivo) se representa como un punto en un espacio de características donde cada dimensión corresponde a un parámetro activo del panel dinámico (alcance, payload, MTOW, velocidad, etc.).

Las decisiones del usuario en el panel se traducen en restricciones por parámetro:

- ignorar: la coordenada no participa en el cálculo de similitud;
- fijo / objetivo: se define un valor central y, en el caso de objetivo, una tolerancia;
- mínimo, máximo, rango: se establecen cotas que delimitan zonas “aceptables”.

A partir de estas decisiones, el script `similitud.py` construye una distancia parcial por parámetro: cuanto más se aleja una aeronave del valor deseado o de las cotas especificadas, mayor es su contribución a la distancia total.

14.4.1 Métrica de distancia y normalización

Como se discutió en el MARCO TEÓRICO, uno de los problemas clásicos de k-NN es que las variables pueden tener escalas muy diferentes. Para evitar que un parámetro domine a los demás, el asistente normaliza las diferencias usando medidas de dispersión robustas (típicamente el **IQR**, o alternativamente MAD/STD cuando corresponde).

De forma esquemática:

- La diferencia entre el valor de la aeronave y el objetivo se divide por una escala basada en el IQR, de modo que “una unidad de distancia” signifique aproximadamente “un salto típico” en ese parámetro.
- Cada distancia parcial se multiplica por un **peso** elegido por el usuario en el panel dinámico, lo que permite dar más importancia a, por ejemplo, alcance y payload que a otros parámetros secundarios.

- Las distancias parciales se agregan usando un exponente α de similitud, que controla si la métrica se comporta más como una norma L1 (más tolerante a grandes desvíos en un solo parámetro) o más cercana a L2 (penaliza más los desvíos concentrados). (Imagen 14.2)

El resultado es una distancia global para cada aeronave de la base. Invertiendo esa distancia se obtiene una medida de similitud, que se normaliza para facilitar la lectura en el panel.

14.4.2 k-NN como base del ranking

Una vez calculadas las distancias, el asistente ordena las aeronaves de la base de menor a mayor distancia: esto es, de mayor a menor similitud. El panel “Ranking por similitud” muestra justamente ese ordenamiento, con:

- las aeronaves más cercanas en las primeras filas,
- un gradiente de color que resume la calidad de la similitud,
- y columnas que detallan la diferencia en parámetros clave respecto al diseño objetivo.

Sobre este ranking se define el conjunto Top-K, es decir, los k vecinos más cercanos que se usan como referencia local para algunas sugerencias adicionales (por ejemplo, estadísticas por parámetro dentro de los vecinos).

La elección de un enfoque tipo k-NN para el Módulo 3 responde a varias razones prácticas:

- **No impone una forma funcional global.** El espacio de diseño de aeronaves VTOL es altamente no lineal y heterogéneo. k-NN permite trabajar con la estructura real de la base sin forzar una ecuación única que valga para todo el mercado.

- **Funciona razonablemente bien con bases moderadas.** Aunque el dataset total ronda las decenas de aeronaves, muchas combinaciones de parámetros útiles para el diseño sólo disponen de unos pocos ejemplos (6–11 casos). En ese contexto, un modelo paramétrico complejo corre riesgo de sobreajuste; en cambio, comparar directamente con los vecinos más parecidos resulta más estable.

- **Es interpretable para el usuario.** Cada sugerencia del asistente puede trazarse a aeronaves concretas: no sólo sabemos que “la similitud es 0,8”, sino que podemos abrir el Excel original y ver qué aeronave o familias de aeronaves están respaldando esa recomendación. Esta trazabilidad es coherente con el objetivo de mantener el asistente como herramienta de apoyo y no como “caja negra”.

En síntesis, el ranking por similitud puede leerse como una aplicación específica de k-NN al contexto de diseño preliminar: en lugar de predecir directamente un parámetro objetivo, el sistema nos muestra qué configuraciones reales están más cerca de nuestra propuesta, para que podamos decidir si el compromiso que buscamos es razonable o si debemos ajustar nuestras expectativas. La **Imagen 14.1** (Ranking por Similitud) ilustra esta idea al mostrar la tabla de ranking con el código de colores por distancia y las diferencias por parámetro.

14.5 Detección de outliers y verificaciones de consistencia

El análisis por similitud se complementa con dos tipos de controles:

1. Detección de outliers por parámetro, basada en *outliers.py*. Para cada columna numérica, calculamos los cuantiles Q1 y Q3, el IQR y los límites LOW/HIGH definidos como:

$$\text{LOW} = Q1 - k \cdot \text{IQR}, \text{HIGH} = Q3 + k \cdot \text{IQR}$$

donde k es el factor IQR configurable desde la interfaz. La vista de histogramas (**Imagen 14.1**) muestra:

- la distribución de un parámetro concreto (por ejemplo, alcance),
- las líneas verticales de LOW y HIGH,
- y la lista de aeronaves identificadas como outliers (tipo HIGH o LOW).

Esta herramienta no sirve solo para descartar casos extremos. En el contexto del asistente, la usamos para entender hasta dónde se ha llegado en la práctica: los outliers marcan algo así como “umbrales altos de lo posible”. Si nos interesa operar cerca de esos límites, podemos inspeccionar específicamente esas aeronaves en el Excel original.

2. Verificaciones físicas básicas, implementadas en *verificacion.py*. Aquí el objetivo no es estadístico, sino de coherencia física. Se recogen identidades como:

- $MTOW \approx W_0 + \text{Payload}$,
- $AR \approx b/c$ (relación de aspecto),
- la relación entre IAS y TAS a una densidad fija de referencia,
- o la coherencia entre alcance, autonomía y velocidad.

Cada verificación se evalúa agregando información de la base (por ejemplo, usando medianas) y se clasifica según el error relativo:

- *OK* si el desvío es menor al 2 %,
- *Ajuste sugerido* entre 2 % y 10 %,
- *No confiable* si supera el 10 %.

El resultado se resume en tarjetas con el nombre de la identidad, la ecuación legible, las entradas usadas y el dictamen final, que pueden incorporarse a la Tabla final de la **Imagen 14.1**.

Combinando histogramas y verificaciones, el asistente no solo nos dice si nuestro diseño “se parece” a otros, sino si respeta relaciones físicas básicas y si está intentando operar en rangos extremos poco poblados.

14.6 Mensajes de guía, tooltips y narrativa de apoyo

Una de las lecciones de los módulos anteriores fue que las métricas numéricas por sí solas no son suficientes para orientar a un usuario que no es especialista en estadística. Por eso el asistente incluye una capa de mensajes explicativos:

- **Tooltips** asociados a los controles globales y a los parámetros, que explican en una frase:

- qué significa “ α similitud”,
- qué implica activar “Quitar atípicos (IQR)”,
- cómo interpretar el modo objetivo frente al modo fijo, etc.

- **Mensajes de resumen** encima de algunos paneles (por ejemplo, en el Ranking por similitud o en las sugerencias Top-K), que recuerdan qué hace cada vista:

- “Similitud (ranking) ordena todas las aeronaves por cercanía al objetivo sin filtrar filas”,
- “Top-K trabaja solo con los K más cercanos para sugerencias locales y análisis detallado”.

- **Textos de ayuda contextual** en la parte superior del dashboard, que se pueden mostrar u ocultar con un botón tipo Ayuda.

En términos narrativos, estos textos permiten que alguien que se acerca por primera vez al sistema pueda interpretar los resultados sin tener que volver al capítulo teórico. La **Imagen 14.3** esquematiza cómo aparecen estos textos de guía en el dashboard.

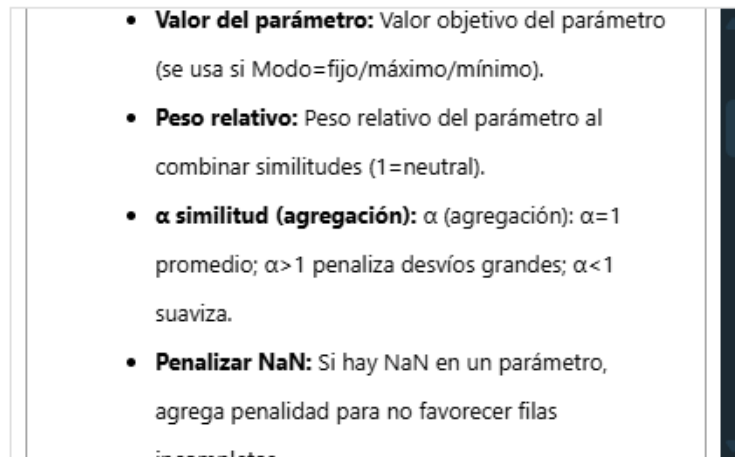


Imagen 14.3 - Ayuda contextual: definiciones de controles del asistente

Ejemplo del panel de ayuda donde se explican términos como “valor objetivo”, “peso relativo”, “ α de similitud” y penalizaciones asociadas al ranking.

15 MÓDULO 3: Ejemplos de uso para definir una aeronave objetivo

15.1 Objetivo del capítulo

En este capítulo se muestra, paso a paso, cómo usar el módulo 3 del sistema ADRpy-VTOL para elegir un conjunto inicial de parámetros de diseño a partir de la base de datos de aeronaves ya imputada.

La idea es partir de una misión clara (vigilancia de largo alcance y costo moderado) y, usando únicamente las herramientas que el sistema ya ofrece (resúmenes estadísticos, análisis de outliers, ranking por similitud y gráficas de tendencias X-Y), ir ajustando los requisitos hasta llegar a una aeronave con prestaciones coherentes con esa misión.

No se busca “probar” que haya una única respuesta correcta, sino mostrar un **proceso de toma de decisiones** reproducible:

1. mirar cómo es el conjunto de aeronaves disponibles,
2. traducir la misión a valores numéricos,
3. lanzar el módulo de similitud,
4. analizar qué devuelve,
5. y corregir los requisitos apoyándose en las gráficas de tendencias.

El resultado final de este capítulo es un **vector de parámetros iniciales** que luego puede usarse en la siguiente etapa de diseño.

15.2 Paso 1: mirar el “orden de magnitud” de la base de datos

Antes de fijar requisitos para una misión concreta conviene mirar qué valores aparecen realmente en la base de datos. La idea es muy simple: si se piden combinaciones que nunca aparecen en los datos, el módulo de similitud se verá obligado a irse a los extremos o a soluciones muy alejadas de lo que se considera razonable.

Para eso se usa directamente el panel de “Resumen global/filtrado” que ya trae el sistema.

Como ejemplo, la **Imagen 15.2** muestra el resumen para la columna “Alcance de la aeronave (km)”:

The image shows a user interface for filtering aircraft data. It has three rows of filters, each with a checkbox, a dropdown menu, a range input, a slider, a weight value, and an 'i' button. The first row is for 'Alcance de la aeronave (km)' and is checked. The second row is for 'Ancho del fuselaje' and the third for 'Autonomía de la aeronave (h)', both unchecked. The 'i' button in the first row is highlighted with a red box.

Parámetro	Modo	Rango [min..max]	Slider	Peso	Acción
☒ Alcance de la aeronave (km)	Rango [min..max]	200 500		1.00	i
☐ Ancho del fuselaje	Ignorar			1.00	i
☐ Autonomía de la aeronave (h)	Ignorar			1.00	i

Imagen 15.1- Controles por parámetro: modo, rango y peso en el asistente

Detalle de una fila mostrando selección de modo (rango/ignorar/fijo, etc.), límites min–max, peso relativo y acceso al resumen (botón “i”).

▼ Resumen global/filtrado: Alcance de la aeronave (km)	
DataFrame original	DataFrame filtrado
Resumen (global)	Resumen (filtrado)
Cobertura: 64/83 (NaN 22.89%)	Cobertura: 24/43 (NaN 44.19%)
Min: 25.00 • Q1: 210.60 • Mediana: 365.00 • Media: 669.27 • Q3: 982.80 • Max: 3270.00	Min: 216.00 • Q1: 251.24 • Mediana: 307.50 • Media: 326.88 • Q3: 392.85 • Max: 500.00
Rango IQR: [-947.70, 2141.10]	Rango IQR: [38.81, 605.27]
Sugerido (Top-K): 306.00	

Imagen 15.2- Panel “Resumen global/filtrado” para Alcance (km)

Captura del resumen estadístico del DataFrame original vs filtrado (cobertura, cuartiles, IQR, mínimos/máximos) y sugerencia Top-K para orientar orden de magnitud.

En este panel se ven dos bloques:

- A la izquierda, el resumen del DataFrame original para esa columna. El sistema muestra cuántas aeronaves tienen dato de alcance (cobertura), el porcentaje de valores vacíos, y los estadísticos básicos: mínimo, primer cuartil, mediana, media, tercer cuartil y máximo. En el ejemplo, el alcance va aproximadamente de 25 km a 3270 km, con una mediana cercana a 365 km. Esto indica que hay algunos modelos con alcances muy altos que se separan bastante del resto.

- A la derecha, el resumen del DataFrame filtrado, donde el módulo de outliers ya eliminó los valores más extremos según el criterio IQR configurado. En el ejemplo, el alcance filtrado va de unos 216 km a 500 km, con una mediana de alrededor de 308 km. El sistema también muestra un rango IQR razonable y sugiere un valor central (en este caso, unos 306 km) como referencia para esa variable.

Este mismo procedimiento se repite para otras columnas clave que van a usarse en el ejemplo:

- Peso máximo al despegue (MTOW) [kg]
- Autonomía de la aeronave (h)
- Velocidad crucero (m/s IAS)

Solo hace falta seleccionar cada columna en el panel, abrir la sección “Resumen global/filtrado” y anotar los rangos típicos (mínimo, mediana, máximo y, si hace falta, Q1 y Q3). Con eso el usuario obtiene una idea rápida del “tamaño” de la base, qué se considera “normal” para cada variable, qué valores son extremos y en qué zona del eje le conviene trabajar si quiere mantenerse cerca del grueso de las aeronaves.

A partir de esta lectura básica se puede pasar a concretar la misión en forma de números.

15.3 Paso 2: traducir la misión a requisitos numéricos iniciales (Caso A)

La misión que se quiere estudiar en este ejemplo es una misión de vigilancia de largo alcance, pensada como primer demostrador de bajo costo relativo. A grandes rasgos, el usuario quiere:

- un UAV que pueda permanecer varias horas en el aire sin necesidad de aterrizar,
- que vuele a una velocidad adecuada para vigilancia (ni tan lenta que no cubra distancia, ni tan rápida que consuma de más),
- y que no sea demasiado grande ni complejo, para que sea razonable fabricarlo y operarlo.

Después de ver los resúmenes estadísticos del paso anterior, el usuario elige con qué variables va a empezar a jugar en el módulo 3. En esta primera iteración se trabajan tres parámetros:

En primer lugar, la autonomía de la aeronave. Es el parámetro que más directamente refleja la misión de vigilancia. La base de datos muestra que la mayoría de las aeronaves tienen autonomías entre pocas horas y un valor medio del orden de 5 horas, pero también hay varios modelos con autonomías que superan las 20 horas. Para que el ejemplo sea ilustrativo, se decide arrancar con un requisito relativamente alto: se pide una autonomía mínima de 20 horas. Esto fuerza al sistema a buscar en la parte alta de la distribución y permite ver qué tipo de diseños aparecen cuando se exige tanto tiempo en vuelo.

En segundo lugar, se fija un límite sobre el peso máximo al despegue (MTOW), usado como indicador indirecto de costo y complejidad. Al mirar el resumen de MTOW, se observa que los valores van desde unos pocos kilogramos hasta cifras superiores a 100 kg. Como la intención es mantenerse en el rango de plataformas tácticas relativamente ligeras, se decide imponer una cota superior de alrededor de 20 kg de MTOW. Esto descarta desde el principio los diseños más grandes y caros, y obliga al sistema de similitud a buscar soluciones de vigilancia dentro de la familia de aeronaves ligeras.

Por último, se incluye la velocidad de crucero en IAS como tercer parámetro. La distribución de esta variable en la base muestra que la mayoría de los UAV vuelan en crucero entre aproximadamente 18 y 26 m/s, con algunos casos más lentos y otros más rápidos. Para una misión de vigilancia, tiene sentido moverse en esa zona media: velocidades demasiado bajas dificultan cubrir distancias grandes, y velocidades demasiado altas elevan el consumo y pueden no ser necesarias. Como primera aproximación, se elige una velocidad de crucero alrededor de 22–25 m/s, dejando todavía margen para ajustar este número en iteraciones posteriores.

Con estos tres requisitos (autonomía alta, MTOW limitado y velocidad de crucero moderada) el usuario está listo para hacer el primer ensayo en el módulo 3, que en este capítulo llamaremos Caso A. En el siguiente apartado se describe qué ranking devuelve el sistema con esta combinación inicial y por qué los resultados invitan a revisar las decisiones tomadas en este paso.

15.4 Paso 3: primera corrida del módulo 3 (Caso A)

Con los requisitos iniciales del Caso A:

- Autonomía mínima ≈ 20 h.

- MTOW máximo ≈ 20 kg.
- Velocidad de crucero alrededor de 22–25 m/s (aún sin ser demasiado estrictos).

se cargan estos valores en el módulo 3 y se ejecuta el cálculo de similitud sobre el DataFrame imputado.

☐ Alcance de la aeronave (km)	Ignorar	<input type="range"/>	1.00	i
☐ Ancho del fuselaje	Ignorar	<input type="range"/>	1.00	i
☒ Autonomía de la aeronave (h)	Mínimo ($\geq$ valor)	20	<input type="range"/>	1.00 i
☐ Cantidad de motores propulsión horizontal	Ignorar	<input type="range"/>	1.00	i
☐ Cantidad de motores propulsión vertical	Ignorar	<input type="range"/>	1.00	i
☐ Capacidad combustible	Ignorar	<input type="range"/>	1.00	i
☐ Clase	Ignorar	<input type="range"/>	1.00	i
☐ Consumo	Ignorar	<input type="range"/>	1.00	i
☐ Cuerda	Ignorar	<input type="range"/>	1.00	i
☐ Despegue	Ignorar	<input type="range"/>	1.00	i
☐ Envergadura	Ignorar	<input type="range"/>	1.00	i
☐ Longitud del fuselaje	Ignorar	<input type="range"/>	1.00	i
☐ Maximum Crosswind (sostenido)	Ignorar	<input type="range"/>	1.00	i
☐ Payload	Ignorar	<input type="range"/>	1.00	i
☐ Peso frame	Ignorar	<input type="range"/>	1.00	i
☒ Peso máximo al despegue (MTOW)	Máximo ($\leq$ valor)	20	<input type="range"/>	1.00 i
☐ Peso Vacío (MTOW - payload)	Ignorar	<input type="range"/>	1.00	i
☐ Peso vacío - fuel y battery	Ignorar	<input type="range"/>	1.00	i
☐ Potencia HP	Ignorar	<input type="range"/>	1.00	i
☐ Potencia Watts	Ignorar	<input type="range"/>	1.00	i
☐ Precio	Ignorar	<input type="range"/>	1.00	i
☐ Propulsión horizontal	Ignorar	<input type="range"/>	1.00	i
☐ Propulsión vertical	Ignorar	<input type="range"/>	1.00	i
☐ Rango de comunicación	Ignorar	<input type="range"/>	1.00	i
☐ Relación de aspecto del ala	Ignorar	<input type="range"/>	1.00	i
☐ Techo de servicio máximo	Ignorar	<input type="range"/>	1.00	i
☐ Velocidad a la que se realiza el crucero (m/s TAS)	Ignorar	<input type="range"/>	1.00	i
☒ Velocidad crucero (m/s IAS)	Rango [min..max]	22 25	<input type="range"/>	1.00 i
☐ Velocidad de pérdida (m/s CAS)	Ignorar	<input type="range"/>	1.00	i
☐ Velocidad de pérdida limpia (m/s CAS)	Ignorar	<input type="range"/>	1.00	i

Imagen 15.3 - Ingreso de requerimientos

Ejemplo de carga de valores objetivo/tolerancias en el asistente.

En la práctica, el módulo recorre todas las aeronaves y, para cada una, calcula cuánto se aleja de lo pedido en cada columna. Después combina esas distancias en un solo valor y ordena la tabla desde la aeronave “más parecida” hasta la que menos se ajusta a los requisitos.

Cuando se mira qué aeronaves de la base tienen autonomías tan altas, aparece una situación clara: solo cinco modelos superan o igualan las 20 horas de autonomía, mientras que el resto se queda por debajo. En la tabla siguiente se muestran esos cinco casos:

MODELO	AUTONOMÍA [H]	MTOW [KG]	VEL. CRUCERO [M/S]	IAS	ALCANCE [KM]
AAI	26,0	13,1	—		3270
ORBITER 4	24,0	55,0	24,0		—
INTEGRATOR	24,0	74,8	28,3		—
T-20	24,0	84,0	30,8		—
REACH	20,0	91,0	25,0		—

Tabla 15.1 - Aeronaves con autonomía ≥ 20 h en la base de datos

Listado de las plataformas con mayor autonomía disponibles en la base, incluyendo autonomía, MTOW y otras prestaciones cuando están declaradas, para evidenciar que los casos de endurance extremo son pocos y dominan el ranking.

	aeronave	segmento	alerta	distancia	distancia_med ia	similitud	similitud_med ia	Peso máximo al despegue..	Δ Peso máximo al...	Δ Peso máximo al...	Autonomía de la aeronave...
2	AAI Aerosonde	Vigilancia / fotogrametría y carga		2.30	0.77	0.10	0.46	13.10	0.00		26.00
5	T-20	Vigilancia / fotogrametría y carga		3.56	1.19	0.03	0.31	84.00	64.00	▲	24.00
4	Integrator	Vigilancia / fotogrametría y carga		2.65	0.88	0.07	0.41	74.80	54.80	▲	24.00
1	Orbiter 4	Vigilancia / fotogrametría y carga		2.21	0.74	0.11	0.48	55.00	35.00	▲	24.00
3	Reach	Vigilancia / fotogrametría y carga		2.45	0.82	0.09	0.44	91.00	71.00	▲	20.00
0	Objetivo (usuario)	(selección)		0.00	0.00	1.00	1.00	20.00	0.00		20.00

Imagen 15.4 - Resultado del ranking del Caso A (autonomía alta y MTOW bajo)

Captura del listado ordenado de aeronaves “más cercanas” al objetivo del Caso A, mostrando cómo el requisito extremo de autonomía concentra el ranking en pocos modelos.

Se ve algo importante:

- Cuatro de estas aeronaves pesan entre **55 y 91 kg**, muy por encima del límite de MTOW que se quería (≈ 20 kg).
- Solo una, la **AAI Aerosonde**, combina una autonomía extrema (26 h) con una masa relativamente baja (13,1 kg).¹⁷

Cuando el módulo 3 recibe la condición “autonomía ≥ 20 h” con peso alto y, al mismo tiempo, “MTOW ≤ 20 kg”, el espacio de búsqueda queda casi vacío: en la práctica solo hay una aeronave que cumple bien ambas cosas, y el resto de las opciones aparecen con penalizaciones importantes en el peso o se quedan fuera.

El ranking que devuelve el módulo en este Caso A queda por lo tanto muy concentrado:

¹⁷ “El 21 de agosto de 1998, una Aerosonda Fase 1, apodada “Laima”, en honor a la antigua deidad letona de la buena fortuna, completó un vuelo de 3270 km (2031 millas) sobre el Océano Atlántico. Este fue el primer cruce del Océano Atlántico realizado por un UAV; en ese momento, también fue la aeronave más pequeña en cruzar el Atlántico”
Fuente: (https://en.wikipedia.org/wiki/AAI_Aerosonde).

- en los primeros puestos aparece la AAI Aerosonde y, dependiendo de cómo se ponderen las tolerancias, pueden aparecer también Orbiter 4, Integrator o T-20, pero todos ellos violan claramente el límite de masa;

- el resto de aeronaves, con autonomías moderadas pero masas más cercanas a los 20 kg, quedan más abajo en la lista porque la autonomía se les penaliza mucho.

Desde el punto de vista del diseñador, el mensaje es simple: **el requisito inicial de “20 horas como mínimo” no es compatible con querer una aeronave ligera** dentro del rango de MTOW que se había fijado.

Esta primera corrida sirve, justamente, para mostrar que un conjunto de requisitos demasiado ambicioso puede empujar al sistema hacia soluciones que no encajan con la idea de “primer demostrador de bajo costo”, aunque matemáticamente el ranking esté funcionando como se espera.

15.5 Paso 4: usar las tendencias X–Y para entender el trade-off

Después del Caso A, el siguiente paso natural no es seguir probando valores al azar, sino aprovechar las tendencias X–Y que el sistema ya ofrece para entender mejor qué está pasando entre autonomía y masa.

En la interfaz del módulo de tendencias se selecciona:

- en el eje X: Peso máximo al despegue (MTOW),
- en el eje Y: Autonomía de la aeronave (h).

El sistema dibuja la nube de puntos de todas las aeronaves (o de las que no fueron marcadas como outliers) y ajusta las curvas de tendencia que ya se describieron en el capítulo del módulo 2. Para este caso, lo que interesa no es tanto la forma exacta de la curva, sino la distribución cualitativa de los puntos:

- La mayor parte de las aeronaves con autonomías entre 3 y 12 horas se concentra en un rango intermedio de masas, desde algo menos de 10 kg hasta unos 50 kg.
- Las aeronaves con autonomías superiores a 20 horas se ubican casi todas en la parte alta de MTOW, entre 55 y 90 kg, con la AAI Aerosonde destacando como un caso particular de 26 h con solo 13,1 kg.

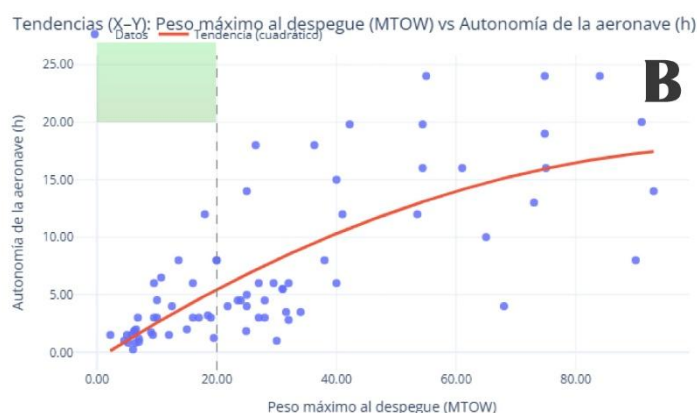
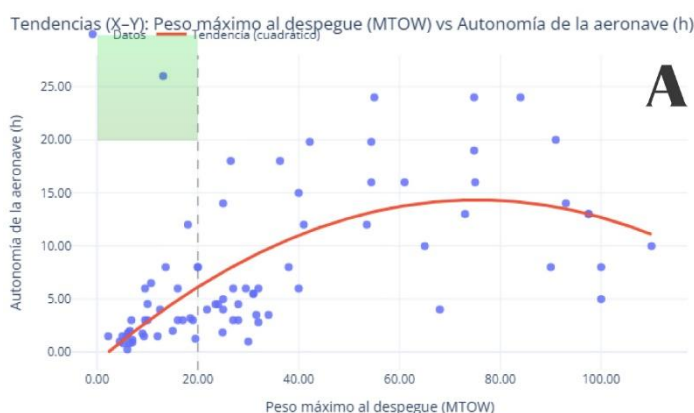


Imagen 15.5 - Caso A: conflicto autonomía vs masa (visualización)

Gráfico que evidencia la incompatibilidad práctica de pedir simultáneamente “autonomía muy alta” y “masa muy baja”, al quedar el objetivo en una región (zona verde) casi vacía de datos. (A) con outliers; (B) sin outliers.

Esta gráfica muestra muy claramente por qué el Caso A es problemático: pedir simultáneamente “autonomía muy alta” y “masa muy baja” significa buscar soluciones en una esquina de la nube donde prácticamente solo hay un punto real. El módulo 3 no “se equivoca”; simplemente está reflejando que esa combinación casi no existe en los datos.

En este mismo módulo de tendencias se pueden explorar otras combinaciones, por ejemplo:

- MTOW en X y alcance en Y, usando el rango filtrado que el sistema ya calcula para “Alcance de la aeronave (km)”.
- Autonomía en X y Velocidad de Crucero en Y, para ver cómo se distribuyen las plataformas que vuelan más rápido o más lento.

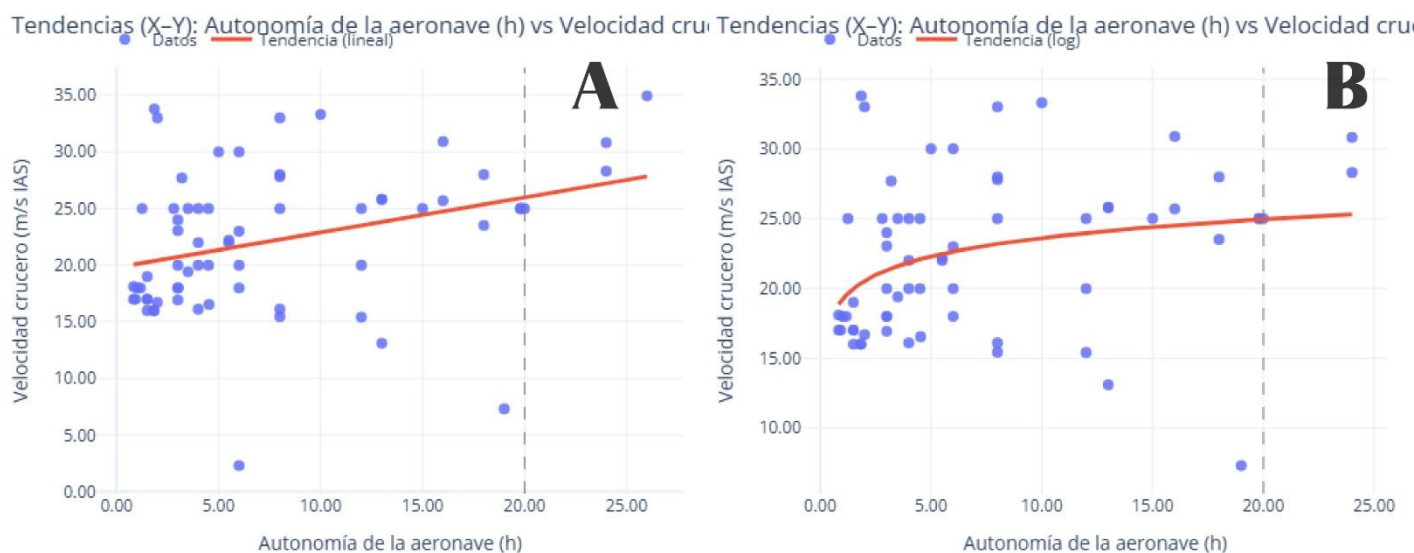


Imagen 15.6 - Distribución velocidad de crucero vs autonomía en la base

Diagrama de dispersión (autonomía en X, velocidad de crucero en Y) para observar segmentos y tendencias entre desempeño y duración de misión.

El objetivo de estas gráficas no es hacer un análisis exhaustivo de toda la base, sino apoyar una idea muy concreta para el ejemplo:

Si se quiere una aeronave ligera (alrededor de 20 kg de MTOW) no es razonable exigir autonomías de 20–24 horas. Hay que aceptar un compromiso: bajar el requisito de autonomía a un rango todavía alto, pero donde efectivamente existan varios modelos.

Con esa información, el usuario está en condiciones de reformular la misión en términos más realistas y pasar a una segunda iteración en el módulo 3, que será el Caso B. Ahí es donde se empezará a ver cómo,

al relajar y reequilibrar los requisitos, el sistema deja de proponer solo aeronaves extremas y comienza a ofrecer opciones que se parecen mucho más al tipo de plataforma que se busca para el demostrador.

15.6 Paso 5: segunda iteración (Caso B) con requisitos más realistas

Después de mirar la nube MTOW–autonomía en el módulo de tendencias, queda claro que pedir “autonomía ≥ 20 h” y al mismo tiempo limitar el MTOW a unos 20 kg deja prácticamente sin opciones. La mayoría de las aeronaves de la base con tantas horas de vuelo son bastante más pesadas, y solo la AAI Aerosonde se acerca al rango de masas que nos interesa (Convirtiéndose en un outlier par nuestro caso).

El siguiente paso es entonces ajustar la misión a un rango de autonomía todavía alto, pero alcanzable para aeronaves ligeras. A partir de la distribución que da el propio panel de “Resumen global/filtrado” para la columna *Autonomía de la aeronave (h)* (mínimo 0,25 h, mediana 5 h, tercer cuartil 12 h, máximo 26 h), se fija ahora un intervalo de trabajo entre unas 8 y 16 horas. Es un rango exigente, pero dentro de lo que se ve varias veces en la base.

Con el peso se mantiene la filosofía de “aeronave ligera”, pero se admite un poco más de margen. En lugar de fijar el límite justo en 20 kg, se trabaja con un máximo en torno a 25–30 kg. De ese modo se incluyen algunos diseños interesantes que están cerca del umbral sin entrar todavía en la liga de los UAV de 50–100 kg.

La velocidad de crucero también se ajusta. El histograma de *Velocidad crucero (m/s IAS)* muestra que la mayoría de las aeronaves vuelan entre 18 y 26 m/s, con valores mínimos de unos 13 m/s y máximos de 35 m/s. Para una misión de vigilancia de largo alcance tiene sentido centrarse en esa zona media. En esta segunda iteración se acota la velocidad de crucero deseada aproximadamente entre 18 y 25 m/s, que es donde se concentra buena parte de la base.

Con estas decisiones, el Caso B queda definido de forma aproximada así:

- autonomía entre 8 y 16 h;
- MTOW no mayor a unos 25–30 kg;
- velocidad de crucero en la franja típica de 18–25 m/s.

Al introducir estas preferencias en el módulo 3, el conjunto de aeronaves que se ajusta razonablemente a la misión crece. Ya no estamos empujando al sistema a una esquina casi vacía de la nube, sino a una zona donde aparecen varias plataformas reales con prestaciones similares.

	aeronave	segmento	alerta	distancia	distancia_med ia	similitud	similitud_med ia	Peso máximo al despegue..	Δ Peso máximo al...
0	Objetivo (usuario)	(selección)		0.00	0.00	1.00	1.00	30.00	0.00
1	Transition	Vigilancia / fotogrametría y carga		0.00	0.00	1.00	1.00	18.00	0.00
2	JOUAV CW-30E	Vigilancia / fotogrametría y carga		0.14	0.05	0.87	0.95	38.00	8.00
3	Volitation VT370	Vigilancia / fotogrametría y carga		0.18	0.06	0.84	0.94	40.00	10.00
4	Stalker VXE30 Fuel Cell	Vigilancia / fotogrametría y carga		0.22	0.07	0.81	0.93	19.95	0.00
5	Stalker XE	Vigilancia / fotogrametría y carga		0.29	0.10	0.75	0.91	13.60	0.00
6	Fulmar X	Vigilancia / fotogrametría y carga		0.32	0.11	0.73	0.90	20.00	0.00
7	Aerosonde Mk. 4.7 VTOL	Vigilancia / fotogrametría y carga		0.42	0.14	0.66	0.87	53.50	23.50
8	Edge Autonomy Penguin C Mk 2.5 VTOL	Vigilancia / fotogrametría y carga		0.49	0.16	0.61	0.85	41.00	11.00
9	L3Harris FVR-90	Vigilancia / fotogrametría y carga		0.52	0.17	0.60	0.84	54.40	24.40
10	Aerovel Flexrotor	Vigilancia / fotogrametría y carga		1.00	0.33	0.37	0.72	25.00	0.00
11	RQ Nan 21A Blackjack	Vigilancia / fotogrametría y carga		1.22	0.41	0.29	0.67	61.00	31.00
12	AeroVironment JUMP 20	Vigilancia / fotogrametría y carga		1.30	0.43	0.27	0.65	97.50	67.50

Imagen 15.7 - Ejemplo de filtrado por autonomía y MTOW (candidatos)

Listado de modelos que cumplen rangos de autonomía y MTOW dentro del DataFrame imputado/filtrado, usado para seleccionar candidatos plausibles.

En el DataFrame imputado, si se miran solo las aeronaves con autonomías entre 8 y 16 horas y MTOW hasta unos 30 kg, aparecen por ejemplo los siguientes modelos:

<i>Modelo</i>	<i>Autonomía</i> [h]	<i>MTOW</i> [kg]	<i>Vel. crucero</i> IAS [m/s]	<i>Alcance</i> [km]
<i>Stalker XE</i>	8,0	13,6	15,43	370
<i>Stalker VXE30</i> <i>Fuel Cell</i>	8,0	19,95	16,09	433
<i>Fulmar X</i>	8,0	20,0	27,8	800
<i>Transition</i>	12,0	18,0	20,0	—
<i>Aerovel</i> <i>Flexrotor</i>	14,0	25,0	—	—

Tabla 15.2 - Aeronaves ligeras con autonomía media-alta ($\approx 8\text{--}16$ h, $\text{MTOW} \leq 30$ kg)

Subconjunto de aeronaves que cumple el filtro de autonomía y masa del Caso B, usado como candidatos plausibles en el ranking por similitud del Módulo 3.

No es necesario saber el orden exacto del ranking interno para este capítulo. Lo importante es que, con los requisitos del Caso B, el módulo 3 empieza a proponer aeronaves de este estilo: masas entre 13,6 y 30 kg, autonomías de 8 a 14 h y velocidades de crucero entre 15 y 28 m/s. Es decir, diseños que encajan bastante mejor con la idea de “plataforma de vigilancia de tamaño medio y costo moderado”.

En este grupo ya aparece el Transition, con 12 h de autonomía y 18 kg de MTOW, junto a otros diseños como el Flexrotor y la familia Stalker. Esto muestra que, al relajar las condiciones del Caso A, empezamos a movernos en la región del espacio de diseño donde se encuentran las aeronaves que nos interesan.

15.7 Paso 6: tercera iteración (Caso C) añadiendo la configuración de la aeronave

Hasta aquí solo se han usado parámetros de misión “globales”: autonomía, MTOW y velocidad de crucero. El siguiente paso es aprovechar que la base imputada también guarda información sobre la configuración de cada plataforma (propulsión horizontal, propulsión vertical, número de motores, clase, etc.).

En la práctica, el usuario abre ahora las columnas de configuración para las aeronaves que salieron bien paradas en el Caso B. Por ejemplo, Stalker XE, Stalker VXE30 Fuel Cell y Fulmar X comparten una configuración más cercana a un ala fija convencional: en la columna de *Propulsión vertical* aparecen códigos que, en combinación con “Cantidad de motores propulsión vertical = 0”, indican que no disponen de un sistema VTOL dedicado.

Transition y Aerovel Flexrotor muestran, en cambio, valores distintos: tienen motores de propulsión vertical definidos y un número de motores verticales mayor que cero. Eso sugiere configuraciones híbridas o VTOL, más compatibles con la línea que seguirá la tesis en la fase de diseño detallado.

Si la misión requiere operar desde espacios reducidos o sin pista de despegue, es razonable dar prioridad a las aeronaves que ya contemplan algún tipo de propulsión vertical. El módulo 3 permite hacerlo de dos maneras, filtrando el DataFrame antes del ranking, para quedarse solo con las entradas que tengan propulsión vertical activa, o manteniendo todas las aeronaves, pero usando la columna de *Clase* o de *Propulsión vertical* como criterio de “preferencia” dentro del cálculo de distancia.

En este caso de uso se adopta la primera opción para hacer el ejemplo más claro: se trabaja solo con aeronaves que tienen propulsión vertical y se mantiene el rango de autonomía y masa de la iteración anterior (alrededor de 8–16 horas y MTOW no muy por encima de 40 kg).

Si se filtra el DataFrame imputado con estas condiciones (autonomía entre 8 y 16 h, MTOW hasta 40 kg, propulsión vertical activa y Clase igual a 3, que agrupa plataformas de tipo VTOL/híbrido), el grupo de aeronaves candidatas se reduce a muy pocas opciones. Entre ellas aparecen, por ejemplo:

<i>Modelo</i>	<i>Autonomía [h]</i>	<i>MTOW [kg]</i>	<i>Vel. crucero IAS [m/s]</i>
<i>Volitation VT370</i>	15,0	40,0	25,0
<i>Transition</i>	12,0	18,0	20,0
<i>JOUAV CW-30E</i>	8,0	38,0	25,0

Tabla 15.3 - Aeronaves VTOL/híbridas con autonomía media-alta y MTOW moderado

Listado de candidatas VTOL/híbridas que cumplen el filtro de autonomía, MTOW y propulsión vertical activa (Clase 3), utilizado para la iteración orientada a operación sin pista en el Módulo 3.

En un ranking por similitud con estos criterios, las aeronaves que cumplen al mismo tiempo:

- masa moderada,
- autonomía claramente superior a la media,
- y presencia de propulsión vertical,

son las que quedan arriba. Dentro de este pequeño grupo, el Transition se encuentra en una posición muy atractiva: su MTOW es mucho menor que el del VT370 o el CW-30E, mantiene 12 horas de autonomía (en la parte alta del rango) y su velocidad de crucero de 20 m/s es muy razonable para misiones de vigilancia.

En este punto, el módulo 3 ya ha hecho su papel: a partir de un proceso iterativo y de varios ajustes guiados por los datos, el usuario ha llegado a una combinación de requisitos que considera razonable y, en nuestro caso, la aeronave elegida es la ALTI Transition. Los valores seleccionados en esta etapa constituyen el conjunto de parámetros globales de partida que se utilizarán a continuación para construir el vector de entrada de ADRpy.



Imagen 15.8 - Plataforma VTOL de referencia: Alti Transition

Imagen del dron Alti Transition como ejemplo representativo usado para contextualizar el caso de estudio.
[10]

15.8 Paso 7: vector final de parámetros para la aeronave objetivo

Los requisitos y parámetros globales definidos mediante el módulo 3 no se utilizan de forma aislada: en este paso se traducen a la estructura de entrada que requiere ADRpy para poder realizar cálculos de diseño conceptual.

En el módulo 3 se definieron los requisitos iniciales de la aeronave objetivo “en lenguaje de usuario”: autonomía, alcance, masa máxima al despegue, velocidad de crucero, etc. Para poder utilizar estos requisitos en ADRpy es necesario traducirlos al vector de entrada que espera la clase AircraftConcept, organizado en tres bloques principales:

- **brief:** requerimientos de misión y entorno de operación,
- **design:** parámetros geométricos y de configuración,
- **performance:** datos de prestaciones y fracciones de peso.

En esta sección se completa el **bloque brief** usando tres fuentes:

1. La ficha técnica de la ALTI Transition, versión 2020.2.¹⁸
2. Los valores ya seleccionados en el Capítulo 13 a partir del análisis de la base de datos.
3. Supuestos razonables tomados de ejemplos de Keane, cuando la información no está disponible.

15.8.1 Tabla 1 – Requerimientos de misión (brief)

<i>Parámetro ADRpy</i>	<i>Descripción resumida</i>	<i>Valor usado / estado para Transition</i>	<i>Unidades</i>	<i>Origen / comentario auditable</i>
<i>cruisealt_m</i>	Altitud a la que se define la velocidad de crucero	5000	m	Columna “Altitud a la que se realiza el crucero” del Excel.
<i>cruisespeed_ktas</i>	Velocidad de crucero (TAS) a <i>cruisealt_m</i>	≈ 41.9	kt	Excel: “Velocidad a la que se realiza el crucero (m/s TAS)” = 21.55 m/s → conversión a kt.
<i>cruisethrustfact</i>	Fracción de empuje máximo usada en crucero	1.0	–	Se adopta el mismo valor del ejemplo de ADRpy (Keane / User’s Guide, brief de referencia). No se dispone de dato específico del fabricante.
<i>servceil_m</i>	Techo de servicio requerido	13000	m	Excel: “Techo de servicio máximo”.

¹⁸ Ver apéndice 18.2 - Ficha técnica “ALTI TRANSITION”

<i>secclimbspd_kias</i>	IAS a la que se alcanza el techo de servicio	—	kt	No figura en la ficha técnica ni en la base. No se define la restricción explícita de techo en esta tesis; se deja para la etapa 2.
<i>climbalt_m</i>	Altitud donde se pide la tasa de ascenso	0	m	Se mantiene el valor por defecto de ADRpy (nivel del mar). En esta tesis no se explota la restricción de ascenso.
<i>climbspeed_kias</i>	IAS a la que se debe cumplir la tasa de ascenso	—	kt	No hay dato ni estimación validada. No se dimensiona con esta restricción en esta etapa.
<i>climbrate_fpm</i>	Tasa de ascenso requerida	—	ft/min	No se dispone de valor medido ni imputado fiable para el Transition en la base actual. Se deja sin fijar.
<i>vstallclean_kcas</i>	Velocidad de pérdida en configuración limpia	≈ 25.3	kt	Excel: “Velocidad de pérdida limpia (m/s CAS)” = 13 m/s → conversión a kt.
<i>groundrun_m</i>	Longitud de carrera de despegue	—	m	No aplica a la aeronave objetivo (Transition es VTOL, sin carrera de despegue con ruedas). La restricción de despegue convencional no se usa.
<i>rwyelevation_m</i>	Elevación de la pista usada para groundrun_m	0	m	Se mantiene el default 0 m (nivel del mar) de ADRpy; sin efecto porque no se aplica restricción de despegue.
<i>to_headwind_kts</i>	Componente de viento en pista para el despegue	0	kt	Se asume sin viento, igual que en el ejemplo de ADRpy (condición estándar).
<i>to_slope_perc</i>	Pendiente de pista	0	%	Se asume pista horizontal (default ADRpy). No hay requisito específico.

<i>stloadfactor</i>	Factor de carga sostenido en viraje nivelado	—	—	No se dimensiona el Transition con la restricción de viraje en este trabajo. Se deja para la etapa 2.
<i>turnalt_m</i>	Altitud a la que se define el viraje	0	m	Default ADRpy (nivel del mar); sin efecto porque no se define la restricción.
<i>turnspeed_ktas</i>	TAS a la que debe cumplirse el viraje	—	kt	No se especifica; la restricción de viraje no se usa en esta tesis.

Tabla 15.4 - Vector de entrada ADRpy (ALTI Transition): requerimientos de misión (brief)

Parámetros de misión y entorno usados para construir el “brief” en ADRpy (altitud/velocidad de crucero, techo de servicio, autonomías, velocidades características y factores operativos), incluyendo unidades y el origen o justificación de cada valor.

15.8.2 Tabla 2 – Variables de diseño (design)

Parámetro ADRpy	Descripción resumida	Valor usado / estado para Transition	Unidades	Origen / comentario auditable
<i>aspectratio</i>	Relación de aspecto del ala	≈ 16.4	—	Excel: “Relación de aspecto” (coincide con b^2/S calculado).
<i>wingarea_m2</i>	Superficie alar de referencia	0.9289	m ²	Excel: “Área del ala”.
<i>sweep_le_deg</i>	Flecha del borde de ataque	5	°	Inspección del plano de planta (hoja de datos): ala con leve ángulo, se estima a ojo.
<i>sweep_mt_deg</i>	Flecha en la línea de máximo espesor	—	°	Se deja al default ADRpy (= <i>sweep_le_deg</i>). No se dispone de dato específico.
<i>sweep_25_deg</i>	Flecha al 25 % de la cuerda	—	°	ADRpy calcula un valor consistente a partir de <i>sweep_le_deg</i> cuando no se especifica.

<i>roottaperratio</i>	Relación cuerda punta / cuerda raíz	—	—	No se modela la geometría detallada del ala; se deja el valor óptimo que ADRpy calcula por defecto.
<i>wingheightratio</i>	h/b para efecto de suelo	100	—	Se mantiene el default ADRpy (ala “lejos del suelo”), ya que no se dimensiona despegue rodado.
<i>bpr</i>	Tipo de planta propulsora	-1	—	Saito FG-21 es un motor de pistón → bpr = -1 según la documentación de ADRpy.
<i>spooluptime_s</i>	Tiempo hasta empuje de despegue	5	s	Default ADRpy (Keane); no hay dato experimental para el FG-21 en esta aeronave concreta.
<i>totalstaticthrust_n</i>	Empuje estático máximo a nivel del mar	≈ 46.1	N	Hoja técnica del motor Saito FG-21: empuje estático ≈ 4.7 kgf con hélice 16×8 → 4.7·9.81 ≈ 46.1 N. Se toma como empuje del motor de crucero; la contribución de los motores VTOL no se modela en esta versión de ADRpy.
<i>tr</i>	Throttle ratio (relevante para turbinas)	—	—	No aplica a motor de pistón; ADRpy no lo usa en este caso. Se deja sin fijar.
<i>weight_n</i>	Peso máximo al despegue	≈ 176.6	N	Excel: MTOW = 18 kg → 18·9.81 ≈ 176.6 N.
<i>weightfractions</i>	Fracciones de peso para cada fase (TO, climb, cruise, turn, ceil)	no especificadas (todas = 1.0)	—	Se usan las fracciones por defecto de ADRpy (cada fase a MTOW completo), según

				documentación oficial.
<i>runwayalpha_deg</i>	Ángulo de ataque durante la carrera de despegue	—	°	No se utiliza la restricción de despegue rodado para Transition; se dejan los valores por defecto de ADRpy.
<i>runwayalpha_max_deg</i>	Ángulo de ataque máximo antes del lift-off	—	°	Ídem anterior; no se fija explícitamente.

Tabla 15.5 - Vector de entrada ADRpy (ALTI Transition): variables de diseño geométrico (design)

Parámetros geométricos y de configuración (p. ej., superficie alar, relación de aspecto, flechas, taper y fracciones de peso), con unidades, valores adoptados y trazabilidad (base de datos, ficha técnica o supuestos).

15.8.3 Tabla 3 – Parámetros de performance y aerodinámica (performance + propulsion)

15.8.3.1 Coeficientes aerodinámicos / de rodadura

Parámetro ADRpy	Descripción resumida	Valor usado / estado para Transition	Unidades	Origen / comentario auditable
<i>CDOTO</i>	Coeficiente de arrastre a sustentación nula en despegue	—	—	No se modela despegue rodado en esta tesis; no se usa la restricción correspondiente.
<i>CDTO</i>	Coeficiente total de arrastre en configuración de despegue	—	—	Ídem anterior; ADRpy usaría el default 0.09 si se activara esta restricción (según User's Guide).
<i>mu_R</i>	Coeficiente de rozamiento rueda-pista	—	—	No aplica a Transition VTOL en este modelo; la carrera de despegue no se usa como restricción.
<i>CLOTO</i>	Coef. de sustentación a $\alpha = 0^\circ$ en despegue	—	—	No se define porque no se evalúa la fase de despegue con flaps/ruedas.

<i>CLTO</i>	Coefficiente de sustentación en despegue	—	—	Ídem. ADRpy tiene default 0.95 para casos con tren y flaps; no se usa aquí.
<i>CLmaxTO</i>	CL máximo en configuración de despegue	—	—	No se aplica (Transition no dispone de flaps convencionales para carrera de despegue).

Tabla 15.6 - Vector de entrada ADRpy (ALTI Transition): coeficientes de despegue y rodadura (no aplicables en esta etapa)

Conjunto de coeficientes asociados a despegue convencional y rodadura (p. ej., CD0TO, CLTO, μR), documentados para completitud, pero dejados sin definir porque el caso de estudio asume operación VTOL y no activa esa familia de restricciones.

15.8.3.2 Coeficientes en configuración limpia

Parámetro ADRpy	Descripción resumida	Valor usado / estado para Transition	Unidades	Origen / comentario auditable
<i>CDminclean</i>	Coefficiente de arrastre a CL = 0 en configuración limpia	default ADRpy (0.03)	—	Se adopta el valor por defecto indicado en la User's Guide (CDminclean = 0.03 para alas subsónicas de baja carga). No hay polar específica publicada del Transition.
<i>CLmaxclean</i>	CL máximo en configuración limpia	≈ 1.3	—	Para el cálculo de la restricción de giro se adoptó $C_{Lmax, clean} = 1,3$, consistente con perfiles convencionales de ala recta utilizados en UAV de este tamaño (NACA 4412 / 23012).
<i>CLminclean</i>	CL mínimo (negativo) en configuración limpia	—	—	No hay información específica y la restricción de vuelo con carga negativa no se usa en esta tesis. Se mantiene el valor por defecto interno de ADRpy; no se fija numéricamente aquí.
<i>CLSlope</i>	Pendiente de la curva de	—	—	Para alas sub-sónicas suele aproximarse a 2π , pero no se fuerza un valor específico. Se deja al default

sustentación ($dCL/d\alpha$, en rad^{-1})			de ADR_{py} (2π) indicado en la documentación.
--	--	--	--

Tabla 15.7 - Vector de entrada ADR_{py} (ALTI Transition): coeficientes aerodinámicos en configuración limpia
Valores adoptados para coeficientes básicos en configuración limpia (p. ej., $CD_{minclean}$, $CL_{maxclean}$, CL_{slope}), necesarios para restricciones aerodinámicas; cuando no hay datos publicados se usan valores por defecto o supuestos conservadores explicitados.

15.8.3.3 Eficiencias de hélice y tipo de propulsión

Parámetro ADR_{py}	Descripción resumida	Valor usado / estado para Transition	Unidades	Origen / comentario auditable
<i>etaprop</i>	Eficiencia de propulsor en TO, climb, cruise, turn...	no especificada (defaults ADR_{py})	—	Se usan los valores por defecto de ADR_{py} : 0.45 (TO), 0.75 (climb), 0.85 (cruise/turn), 0.65 (servceil), según User's Guide. No se dispone de polares detalladas para las hélices del Transition.
<i>propulsion</i>	Tipo genérico de planta propulsora	"piston"	texto	Configuración equivalente a motor Saito FG-21 con hélice; se usa el tipo genérico "piston" documentado en ADR_{py} .

Tabla 15.8 - Vector de entrada ADR_{py} (ALTI Transition): eficiencias de hélice y tipo de propulsión
Definición del tipo genérico de planta motriz y de la eficiencia propulsiva (*etaprop*) por fase de vuelo; en ausencia de datos específicos se adoptan los valores por defecto recomendados por ADR_{py} y una equivalencia razonable para el sistema propulsivo.

Una vez definidos los valores de partida mediante el módulo 3, el siguiente paso consiste en completar un vector de entrada compatible con ADR_{py} . Las **Tabla 15.4**, **Tabla 15.5**, **Tabla 15.6**, **Tabla 15.7**, **Tabla 15.8**, muestran un ejemplo de este vector para el caso de la aeronave objetivo basada en la ALTI Transition, organizado en tres grupos: requerimientos de misión (*brief*), parámetros de diseño geométrico (*design*) y parámetros aerodinámicos y de propulsión (*performance*).

En el bloque de requerimientos de misión (**Tabla 15.4**) se reúnen las magnitudes que describen la forma de volar que se quiere estudiar con el diagrama de restricciones: altitud y velocidad de crucero, techo de servicio, rango y autonomía, velocidades características y condiciones de despegue, ascenso y maniobra. Algunos de estos valores provienen de manera directa de la base de datos de UAV, por ejemplo la altitud y la velocidad de crucero o el techo de servicio. Otros se obtienen por conversiones sencillas de unidades, como la velocidad de pérdida limpia, que se expresa en nudos a partir del valor en m/s.

Varios parámetros aparecen indicados explícitamente como "no definidos" o "no utilizados" en esta etapa. Esto ocurre, por ejemplo, con la tasa de ascenso requerida o las velocidades específicas de ascenso

y viraje. El motivo no es una falta de información, sino que en esta tesis no se construye todavía el conjunto completo de restricciones que se usarán en la segunda fase del proyecto. En particular, no se activan las ecuaciones asociadas a ascenso y maniobrabilidad que ya están disponibles en ADRpy y que serán responsabilidad de la segunda fase del proceso de diseño, cuando se definan en detalle los perfiles de misión y las configuraciones de propulsión.

En la misma línea, la variable `groundrun_m`, que en el ejemplo de Keane representa la carrera de despegue de un avión de tren convencional, se deja sin valor en nuestro caso. La aeronave elegida como referencia opera con despegue vertical y no dispone de tren de aterrizaje, por lo que no existe una carrera de despegue comparable a la de un avión de ala fija clásico. Esta diferencia de configuración hace que toda la familia de restricciones ligadas al despegue horizontal quede, de momento, fuera del alcance de este trabajo y se traslade a la fase 2, donde será necesario decidir si la configuración final incorporará también la posibilidad de despegue horizontal (por ejemplo, con un módulo desmontable de VTOL como el Stalker UAS de lockheedmartin¹⁹ y sistema de catapulta en vez de tren de aterrizaje, (**Imagen 15.9**) o si se mantendrá una combinación de catapulta y recuperación mediante paracaídas.



Imagen 15.9 - Catapulta neumática PL-80 de Eli Airborne Solutions.

Este sistema está diseñado para lanzar vehículos aéreos no tripulados (UAV) y presenta las siguientes características clave:

Está diseñado para acelerar UAV con un peso máximo de despegue de hasta 100 kg.

Puede lanzar aeronaves a velocidades de hasta 25 m/s.

Se puede transportar fácilmente en estuches resistentes especiales.

Se pueden crear carros personalizados para diferentes tipos de carrocería de UAV.

El sistema es ligero, funciona con batería y se monta rápidamente sin necesidad de herramientas. [11]

La **Tabla 15.5** resume los parámetros de diseño relacionados con geometría y masa. Aquí el punto de partida es la información de la base de datos (MTOW, superficie alar, relación de aspecto), complementada con datos de la ficha técnica (envergadura) y con la reconstrucción geométrica básica del ala. A partir de esos valores se calculan el peso en newton y la relación de aspecto, y se fija una flecha del borde de ataque igual a cinco grados (suposición respecto a la **Imagen 15.8**), coherente con el plano de planta, donde se

¹⁹ <https://www.lockheedmartin.com/en-us/products/stalker.html>

aprecia que no es totalmente perpendicular. El empuje estático máximo se estima a partir de la ficha técnica del motor Saito FG-21 recomendado por el fabricante, donde se declara un empuje del orden de 4,7 kgf para la hélice estándar; al multiplicar por $9,81 \text{ m/s}^2$ se obtiene un empuje equivalente aproximado de 46 N, que se utiliza como *totalstaticthrust_n* para el motor de crucero.

En cambio, no se personaliza todavía el vector completo de fracciones de peso (*weightfractions*). ADRpy permite representar la pérdida de peso debida al consumo de combustible a lo largo de cada fase de la misión, siguiendo el enfoque clásico de Raymer. Esa flexibilidad es fundamental cuando se diseña una aeronave con propulsión puramente térmica y depósitos dimensionados desde cero. En esta tesis, el objetivo principal es mostrar cómo el sistema de imputación y el módulo 3 permiten construir un vector coherente a partir de datos de UAV comerciales, de modo que se adopta el esquema simple donde todas las fases se consideran a MTOW y se pospone el ajuste fino de fracciones de peso para la segunda etapa del proyecto.

Las **Tabla 15.6**, **Tabla 15.7**, **Tabla 15.8**, recoge los parámetros aerodinámicos y de propulsión. Aquí se combinan tres tipos de información: datos medidos o declarados (como la velocidad de pérdida limpia), cálculos directos a partir de magnitudes conocidas (por ejemplo, el $C_{L\max}$ limpio obtenido a partir de la condición de pérdida) y parámetros que se mantienen en los valores por defecto de ADRpy cuando no se dispone de una polar aerodinámica completa para esta aeronave. Este es el caso de coeficientes de arrastre mínimos, pendientes de la curva de sustentación o eficiencias propulsivas por fase, que se toman de los valores recomendados en la documentación del código, pensados para aeronaves subsónicas de ala recta.

Algo similar ocurre con los parámetros pensados para despegue y aterrizaje en avión convencional. Coeficientes como $CL_{\max TO}$, CD_{TO} , el coeficiente de rozamiento rueda–pista o el ángulo de ataque en carrera de despegue (*runwayalpha*) no se fijan para el Transition, ya que la aeronave no dispone de flaps ni de un tren de aterrizaje utilizado de forma rutinaria. En esta tesis se asume un único $C_{L\max}$ correspondiente a la configuración limpia, derivado de la velocidad de pérdida de ala fija, y se deja claro que el análisis de despegue horizontal y de operaciones mixtas VTOL/STOL será abordado en la fase 2.

En conjunto, este vector no pretende ser todavía el diseño definitivo de la aeronave que se busca, sino un ejemplo completo y trazable de cómo el sistema ADRpy-VTOL permite pasar desde un conjunto de datos de UAV comerciales y un vector de diseño conceptual, hasta un conjunto de parámetros compatible con ADRpy. El resultado respeta las prestaciones y órdenes de magnitud de la ALTI Transition allí donde se dispone de datos (masa, superficie alar, velocidades, techo de servicio) y utiliza parámetros estándar o por defecto en aquellos aspectos que dependen de decisiones de configuración que aún no están cerradas: tipo de despegue horizontal, nivel de detalle en la modelización de la polar aerodinámica y estrategia de fracciones de peso.

Esta forma de presentar el caso deja explícito qué partes del vector están plenamente respaldadas por datos de mercado, cuáles se apoyan en cálculos básicos o en bibliografía, y qué componentes del modelo quedan abiertos para la etapa siguiente, en particular todo lo relacionado con el despegue vertical y la transición entre vuelo estacionario y vuelo de ala fija.

A partir del vector de entrada construido en este capítulo se genera, a modo de ejemplo, un diagrama clásico de restricciones que relaciona la carga alar W/S con la relación empuje–peso T/W . Esta gráfica se incluye únicamente con carácter ilustrativo, con el objetivo de materializar el recorrido completo desde los datos de mercado y las imputaciones hasta una salida típica del diseño conceptual. En esta fase del trabajo no se realiza un análisis detallado ni una interpretación profunda de las curvas obtenidas: el diagrama no pretende dimensionar de forma definitiva la aeronave, sino mostrar en qué se concreta el proceso de selección y estimación de parámetros desarrollado a lo largo de la tesis.

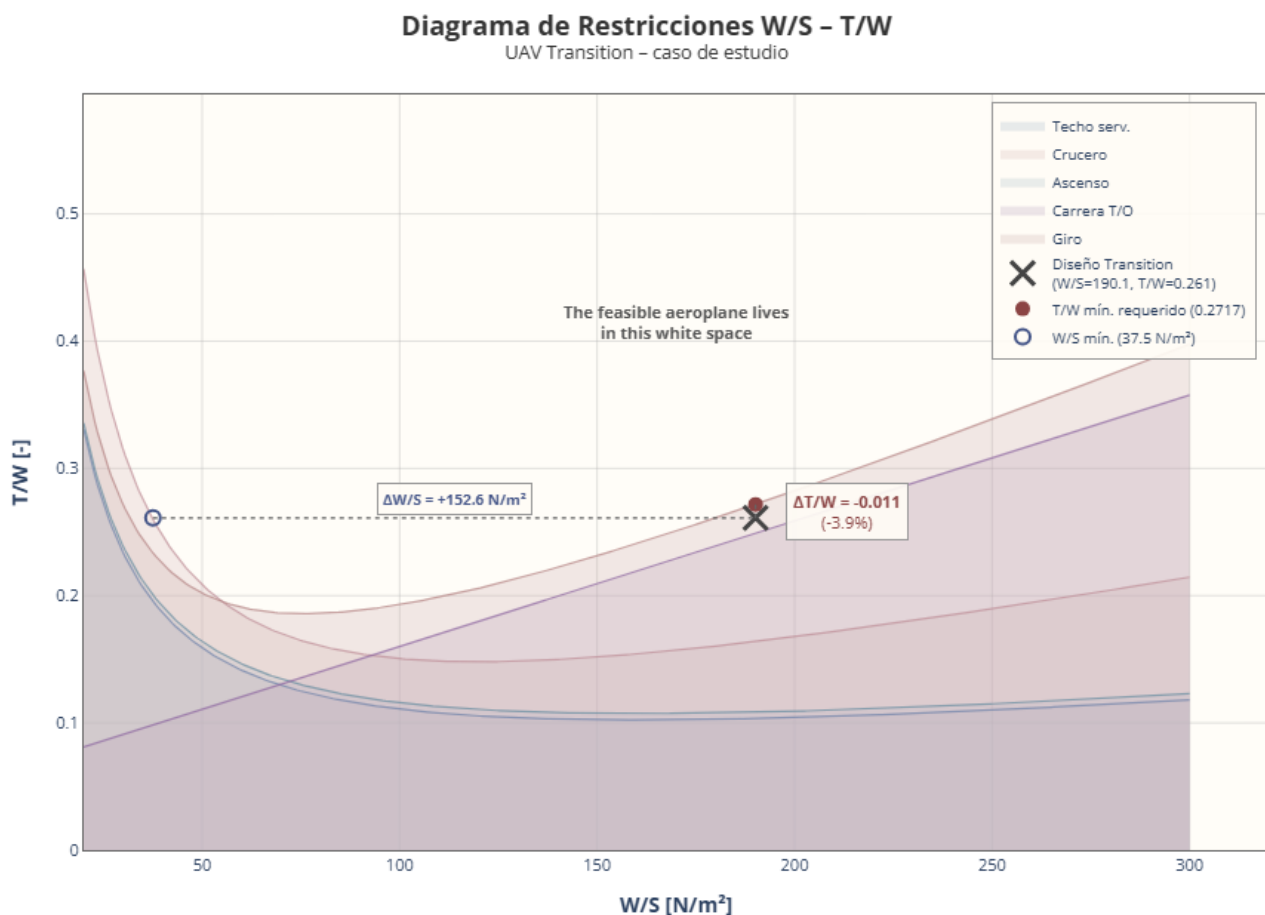


Imagen 15.10 - Diagrama de restricciones WS–T/W para caso de estudio

Diagrama de restricciones W/S – T/W generado a partir del vector de entrada construido en el asistente de diseño. La imagen se presenta únicamente con fines ilustrativos, para mostrar una salida típica del diseño conceptual derivada del flujo propuesto.

En la gráfica actual se ve claramente el “clásico” diagrama de restricciones W/S – T/W en versión estilizada tipo Keane dónde:

- Las curvas de techo, crucero, ascenso, giro y carrera de T/O marcan, para cada carga alar W/S , el T/W mínimo que exigiría cada restricción. Todo lo que está coloreado es “no viable” para esa condición; el avión factible vive en la franja blanca donde ninguna curva lo supera.
- Aunque en nuestro caso el Transition no despegue con carreteo, se mantuvo la curva de carrera de T/O como referencia, para conservar la comparación con el ejemplo de Keane y mostrar cómo se vería una versión “ala fija equivalente”. En la tesis podemos aclarar explícitamente que, para un $VTOL$ real, la restricción de T/O horizontal no sería la limitante principal.
- Para los cálculos se adoptó CL_{max} limpio ≈ 1.3 , en lugar del valor más alto (~ 1.84) obtenido de la V_{stall} . Ese 1.3 es más representativo de un perfil NACA razonable sin dispositivos hipersustentadores; hace que la curva de giro sostenido sea más exigente y, en la práctica, es la que termina levantando la envolvente roja en la zona de W/S altos.

- El punto de diseño del Transition aparece como la X negra alrededor de $W/S \approx 190 \text{ N/m}^2$ y $T/W \approx 0.261$.

El marcador rojo indica el T/W mínimo requerido por la envolvente en esa misma carga alar, $T/W \approx 0.272$, de modo que el diseño real queda unos $\sim 4\%$ por debajo del margen teórico. Es decir: está muy cerca del borde de la zona factible, pero ligeramente “por dentro” de la región coloreada (ligeramente submotorizado bajo nuestros supuestos conservadores).

- El círculo azul en la parte izquierda marca el W/S mínimo asociado a la estructura/ala que estamos usando como aproximación para el Transition, y sirve para visualizar cuánto margen hay, en el eje horizontal, entre la carga alar de diseño y el límite inferior aceptable.

En resumen: la figura muestra que, con los parámetros y supuestos actuales, el Transition queda muy cerca del borde de la región factible; la curva de giro (afectada por CL_{max} limpio) es la que más condiciona el T/W requerido en la zona de W/S donde cae nuestra aeronave, y la inclusión de la carrera de T/O se mantiene solo como referencia “ala fija”, no como restricción real de un *VTOL*.

16 DISCUSIÓN, ALCANCES Y CONTINUACIÓN

16.1 Visión general: qué aporta realmente el sistema

En esta tesis intenté responder una pregunta muy simple de formular y muy difícil de ejecutar:

¿Hasta qué punto puedo usar estadística y herramientas computacionales para apoyar el diseño conceptual de un dron de ala fija con capacidad VTOL a partir de lo que ya existe en el mercado?

El resultado fue un sistema de tres módulos que permite pasar de una base de datos incompleta y heterogénea a un entorno interactivo donde se puede imputar datos, entrenar modelos, visualizar sus comportamientos y finalmente comparar una aeronave objetivo con sus “vecinas” en el mercado.

En términos prácticos, el sistema permite transformar una tabla dispersa de fichas técnicas en: (i) una base imputada con trazabilidad, (ii) un laboratorio visual para modelos de correlación y (iii) un asistente de diseño que ayuda a situar nuevas configuraciones dentro del mapa actual de drones. Lo que se considera más original no es cada pieza por separado, sino la combinación: imputación configurable + análisis visual profundo + asistente de diseño, todo enfocado en un dominio muy específico y con énfasis en la transparencia de las decisiones.

Lo innovador de este enfoque es que propone una mirada distinta sobre la etapa preliminar y la elección de parámetros de una aeronave, más allá de las reglas convencionales. Al trabajar con drones se advirtió que existen muchas más variables, formas, tamaños, materiales y tecnologías que en la aviación tradicional, y que no era evidente si las reglas clásicas de Raymer y otros autores se extendían sin más a este universo. En lugar de asumir que esas reglas siguen siendo válidas, se eligió dejar abierta la posibilidad de que existan otras relaciones empíricas por descubrir. En este contexto, cada modelo que se entrenó se puede leer como una “regla nueva” que aplica al conjunto de aeronaves que la sostienen: es un punto de apoyo para entender cómo se comporta ese segmento del mercado, no un límite rígido para la creatividad.

El camino que llevó hasta aquí no fue lineal. La misión inicial de la tesis era mucho más ambiciosa desde el punto de vista aeronáutico: se quiso avanzar a lo largo de buena parte del proceso de diseño de un dron, incluyendo el uso de herramientas de fluidos computacionales (CFD). El encontrarse con el problema de datos incompletos fue un giro inesperado que reorientó casi todo el proyecto. En la práctica, se recorrió quizás un diez por ciento del “camino clásico” de diseño, pero con una complejidad técnica en programación y estadística mucho mayor de la que se imaginó al inicio. Se aceptó ese giro porque se entendió que, si no se resolvía mínimamente el problema de los datos, cualquier análisis aerodinámico posterior iba a apoyarse sobre cimientos demasiado frágiles.

16.2 Lecciones sobre datos: calidad, cantidad y segmentación

Uno de los aprendizajes más claros de este trabajo es que la calidad de los datos en el mundo real está muy lejos del ideal de los libros de texto. La base de drones que se contruyó es relativamente modesta en tamaño (del orden de unas decenas de aeronaves), pero el problema principal no fue tanto la cantidad como la heterogeneidad: fichas de fabricantes con criterios distintos para definir “peso vacío”, autonomías calculadas con hipótesis no siempre especificadas, datos faltantes en los parámetros más importantes y, en algunos casos, incoherencias directas.

En la práctica, esto se tradujo en que muchos parámetros clave presentaban porcentajes altos de celdas vacías, o bien mezclaban definiciones que no eran estrictamente comparables. Para poder usarlos en modelos se tuvo que tomar decisiones de limpieza, descartar algunos registros y aceptar que ciertos parámetros solo serían utilizables dentro de subgrupos muy concretos. La famosa frase “garbage in, garbage out” dejó de ser un eslogan y se convirtió en un límite muy tangible: por sofisticado que sea el algoritmo, si la base está mal definida, el resultado será frágil.

También se aprendió que, en un mercado tan diverso como el de los drones, la segmentación es casi obligatoria. Mezclar en un mismo análisis plataformas VTOL pesadas de vigilancia, pequeños drones de fotogrametría, sistemas experimentales y conceptos emergentes diluye cualquier señal estadística. La tesis solo pudo dar pasos iniciales en esa dirección, pero el propio sistema deja preparada la estructura para trabajar por segmentos, familias o tecnologías, siempre que futuras extensiones de la base de datos aporten más casos y mejor documentación.

16.3 Lecciones sobre modelos: no confiar solo en los números

El Módulo 2 fue una escuela acelerada sobre por qué no se puede confiar ciegamente en métricas numéricas. Descubrí que es relativamente fácil obtener modelos con MAPE muy bajos y R^2 altos que, sin embargo, son físicamente absurdos o dependen de muy pocos datos. Un caso típico eran los modelos polinómicos de segundo grado que, ajustados sobre un rango estrecho, producían curvas con forma de campana muy pronunciada. El MAPE podía ser prácticamente cero porque la curva pasaba rozando todos los puntos de entrenamiento, pero el máximo local quedaba en un rango donde no había datos, y el valor que el sistema intentaba imputar para la aeronave objetivo era justamente ese pico artificial.

Antes de tener el panel de análisis, el instinto natural fue validar estas ecuaciones en una herramienta externa (por ejemplo, Geogebra) para ver qué estaba ocurriendo. Esa comprobación confirmó la intuición: el problema no era el algoritmo de mínimos cuadrados en sí mismo, sino la combinación de pocos datos, alta flexibilidad del modelo y falta de controles visuales. A partir de ahí se comprendió que se necesitaban dos cosas: un espacio visual para ver las nubes de puntos y las curvas, y reglas más estrictas para aceptar modelos polinómicos. Una de las decisiones concretas fue aumentar el número mínimo de muestras exigidas para entrenar polinomios, precisamente porque su capacidad de “dibujar cualquier cosa” es una ventaja peligrosa cuando los datos son escasos.

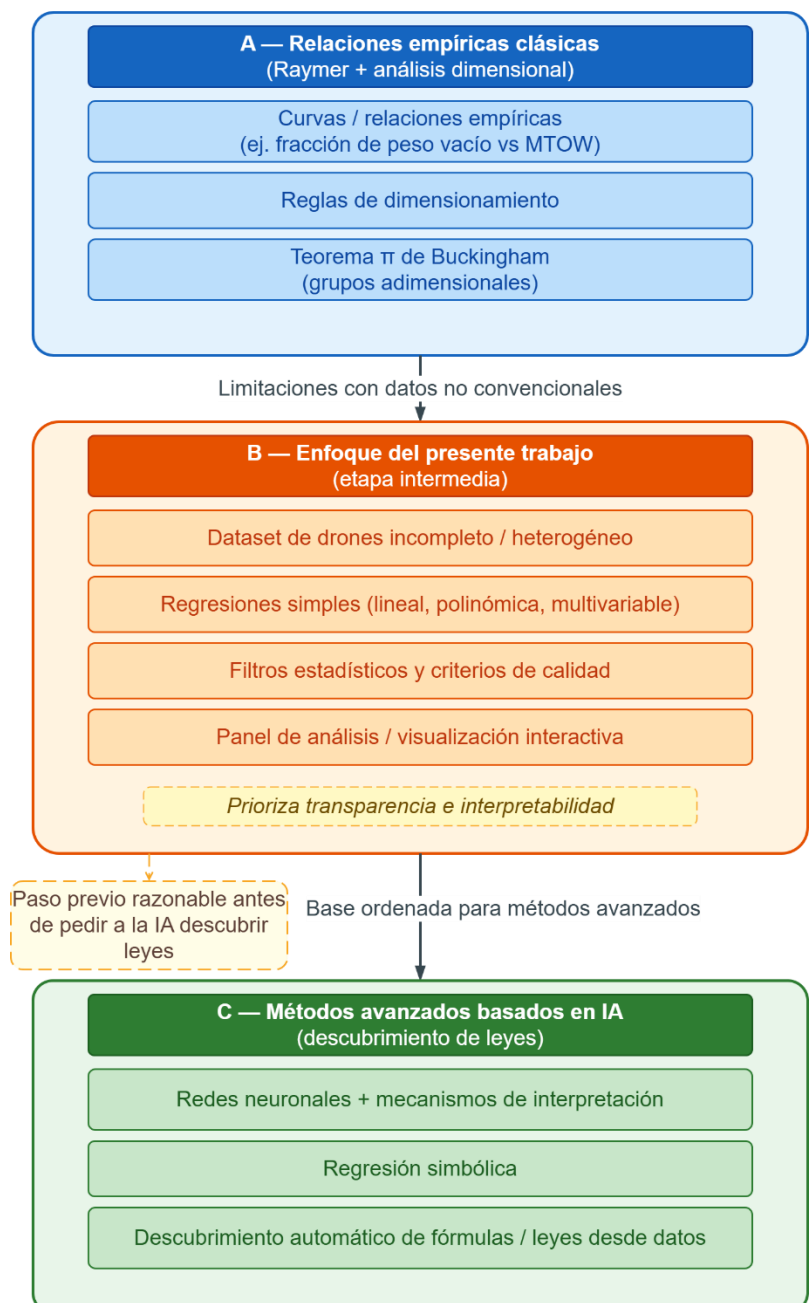
El panel también ayudó a valorar modelos que, sobre el papel, parecían menos brillantes. En muchos casos un modelo lineal con métricas moderadas, entrenado sobre una muestra más grande y con una forma coherente con la física del problema, resultaba preferible a un polinomio espectacular pero sostenido por media docena de puntos. Esta tensión entre “modelo óptimo según la métrica” y “modelo razonable según la ingeniería” se volvió un tema central de la tesis, y reforzó ideas ya discutidas en el MARCO TEÓRICO sobre underfitting y overfitting.

Algo similar ocurrió en el Módulo 3 con la elección del método de similitud. El plan original era llegar a esa etapa con un dataframe prácticamente completo y usar sobre todo relaciones de correlación para guiar el diseño: ecuaciones, cocientes y fórmulas que describieran cómo se conectan los parámetros entre sí. La realidad fue otra: no se logró un dataset suficientemente completo y homogéneo como para sostener esa estrategia. La solución fue apoyarse en un método de vecinos más cercanos (k-NN) también en el asistente de diseño. Paradójicamente, la limitación de los datos abrió una puerta nueva: k-NN no solo permite rellenar huecos, sino también ordenar las aeronaves según su cercanía al objetivo, lo que establece un contacto mucho más directo con el “mapa” del mercado que el que se hubiera tenido con fórmulas aisladas.

16.4 Análisis dimensional y descubrimiento de fórmulas: un primer paso

Una de las motivaciones de fondo de este proyecto fue un acercamiento, aunque fuera de forma rudimentaria, al tipo de relaciones empíricas que aparecen en textos clásicos de diseño conceptual, como las curvas de fracción de peso vacío en función de la MTOW o las reglas de dimensionamiento basadas en análisis dimensional. El teorema π de Buckingham ofrece un marco elegante para construir grupos adimensionales y buscar estructuras invariantes en los datos [6] [5]. Sin embargo, la diversidad de configuraciones y tecnologías en el universo de los drones hizo que, en esta primera etapa, yo eligiera partir de un enfoque puramente numérico. Si suponía desde el inicio que las reglas de los aviones convencionales seguían valiendo, corría el riesgo de forzar a los drones a encajar en un molde que quizá ya no les corresponde. Entre motores eléctricos, arquitecturas VTOL poco habituales y futuras opciones como la propulsión por hidrógeno, parecía más honesto asumir que “no sé sabía” cuáles son las reglas correctas y dejar que los datos hablen primero. A partir de ahí, las relaciones empíricas resultantes pueden dialogar con la teoría clásica, pero no están condicionadas desde el principio por ella.

La literatura reciente en física e inteligencia artificial va mucho más lejos, proponiendo métodos automáticos para descubrir conceptos físicos o fórmulas a partir de datos, combinando redes neuronales profundas con mecanismos de interpretación o regresión simbólica. [3] [2] [4]. En comparación con estas propuestas, el sistema es deliberadamente modesto: trabaja con regresiones relativamente simples, filtros estadísticos y visualización interactiva. Pero puede verse como un paso previo razonable en la misma dirección: antes de pedirle a una red neuronal que “descubra” leyes del diseño de drones, hay que entender qué tipo de relaciones básicas son plausibles, qué tan ruidosos son los datos y hasta dónde se puede llegar con herramientas más transparentes.



16.5 Posibles extensiones estadísticas y de IA

Mirando hacia adelante, el sistema abre varias líneas claras de mejora técnica. La primera es ampliar el repertorio de modelos dentro del propio marco estadístico clásico: introducir regresiones robustas menos sensibles a outliers, explorar modelos multivariados con más predictores allí donde haya datos suficientes y estudiar variantes de regularización que reduzcan el riesgo de sobreajuste sin necesidad de imponer umbrales manuales tan conservadores.

Una segunda línea es incorporar métodos de machine learning cuando la base de datos lo permita. Árboles de decisión, random forests o modelos de tipo gradient boosting podrían capturar patrones no lineales sin necesidad de elegir a priori una forma funcional específica. Redes neuronales poco profundas, combinadas con capas de interpretación o con técnicas de regresión simbólica, podrían servir para generar hipótesis de fórmulas a partir de los datos, que luego se contrastarían con criterios de ingeniería.

Todo esto tiene una condición clara: sin mejores datos, cualquier sofisticación algorítmica corre el riesgo de convertirse en una caja negra poco útil. Un aspecto que sería deseable preservar en futuras extensiones es precisamente la interpretabilidad: que un ingeniero con formación estándar pueda entender qué está haciendo el sistema, qué datos sostienen cada modelo y cómo se llegó a una recomendación concreta.

16.6 Escalabilidad, continuidad y potencial institucional

Aunque el sistema nació para este caso concreto de drones de ala fija VTOL, gran parte de la arquitectura es genérica. El núcleo de imputación, la lógica de entrenamiento y filtrado de modelos y la estructura de dashboards podrían adaptarse, con cambios razonables, a otros tipos de aeronaves o incluso a productos industriales distintos que compartan el mismo problema de base: catálogos incompletos y necesidad de apoyar decisiones de diseño. Lo que sí es muy específico son las etiquetas, los nombres de columnas y ciertos criterios de diseño propios del dominio de los drones; esas capas podrían modularizarse mejor en versiones futuras.

Más allá de los drones del mercado, se entiende este enfoque como un laboratorio para explorar familias de diseños propios. Se podría cargar en la base varios prototipos de una misma aeronave, ir variando parámetros de forma controlada y observar cómo responde el resto del sistema: qué pasa con la autonomía si se ajusta el área alar, cómo se mueve el peso estructural cuando se cambia la configuración de propulsión, etc. También se puede aislar subsistemas, por ejemplo distintos tipos de alas, y estudiar cómo una dimensión particular afecta a otra. En esta lectura, el valor no está solo en encontrar correlaciones fuertes, sino también en confirmar que ciertos parámetros *no* muestran relación alguna. Eso puede significar que faltan datos válidos para estudiar el fenómeno, o que en esa región del espacio de diseño hay mucha más libertad de la que se imaginaba. Todo esto ayuda a evitar el error clásico de hacer cada vez más eficiente un sistema que quizá no debería existir, en lugar de cuestionar el concepto y buscar alternativas radicalmente distintas.

Desde el punto de vista de la institución, el proyecto fue concebido desde el inicio como algo continuable. Desarrollar el diseño preliminar completo de una aeronave según las metodologías de Keane o Raymer es un esfuerzo costoso, incluso si se trabaja solo con modelos conceptuales; se suele estimar que un proyecto de este tipo, con todas sus fases y herramientas, implica recursos del orden de decenas de miles de dólares. [8]. El objetivo no era “resolver todo”, sino plantar una semilla en la etapa de datos y

modelos, de manera que futuras generaciones puedan avanzar sobre una base ya funcional, detectando errores, ampliando la base, integrando nuevas etapas o migrando a entornos web más maduros.

El sistema no está “terminado” en el sentido industrial del término. Es una herramienta en fase avanzada de desarrollo, útil para muchas cosas, pero que seguramente contiene bugs y casos no contemplados. También se sabe que existen soluciones que probablemente serían mejores, pero cuyo nivel técnico requería, por sí solo, una tesis completa (por ejemplo, ciertos enfoques sofisticados de análisis dimensional o de aprendizaje profundo). Parte de la madurez del proyecto fue aceptar ese límite y encontrar un equilibrio entre ambición y realismo: construir algo que funcione, sea entendible y abra caminos, aunque no agote todas las posibilidades.

16.7 Del análisis estadístico a la formula física

Un análisis estadístico de correlaciones no es un fin en sí mismo, sino una herramienta para descubrir estructuras subyacentes en los datos. Cuando se lleva a cabo correctamente, ese análisis debe permitirnos pasar de la simple observación a la modelización formal. Es decir, construir expresiones matemáticas que representen, con cierto grado de precisión, cómo se comportan las variables entre sí.

Esto implica que, a partir de un conjunto de relaciones detectadas (lineales, polinómicas, o multivariantes), el investigador debe estar en condiciones de derivar fórmulas predictivas funcionales, como:

$$L = \frac{1}{2} \rho \cdot V^2 \cdot S \cdot C_L$$

Donde:

- L = fuerza de sustentación (N)
- ρ = densidad del aire $\left(\frac{kg}{m^3}\right)$
- V = velocidad del flujo de aire sobre las alas $\left(\frac{m}{s}\right)$
- S = Superficie alar (m^2)
- C_L = coeficiente de sustentación (adimensional)

Esta ecuación ilustra cómo diferentes variables interactúan de forma multiplicativa para generar un resultado físico observable. Es un excelente ejemplo de cómo relaciones empíricas bien entendidas pueden traducirse en expresiones matemáticas precisas que luego son utilizadas para diseño, simulación o validación, además este tipo de expresiones no solo permite imputar valores faltantes, sino que también, favorece la **automatización** en sistemas de diseño o control, aporta **transparencia** al proceso (se entiende cómo y por qué se llega a un resultado), establece una **base objetiva** para posteriores validaciones experimentales.

¿Por qué es importante llegar a fórmulas matemáticas? ²⁰

²⁰ Las ecuaciones finales dependen de los predictores disponibles. En el ejemplo anterior se muestran variables que, para poder incluirse en la relación, deben contar con datos medidos o estimados. A modo ilustrativo, la ecuación clásica de sustentación presenta una complejidad relativamente alta, ya que integra varias variables físicas. En general, a medida que aumenta el grado de una ecuación o la cantidad de términos, crece su complejidad; sin embargo, esto no implica que siempre sea preferible una formulación más compleja. En muchos casos, relaciones más

1. Porque simplifican la complejidad.

Un modelo basado en una fórmula explícita permite captar relaciones no obvias de manera compacta, lo que es fundamental en campos como la ingeniería, donde muchas veces se deben tomar decisiones rápidas sobre múltiples variables.

2. Porque son replicables.

A diferencia de una intuición o una visualización, una fórmula es concreta: puede ser validada, probada, mejorada o descartada.

3. Porque permiten generalización.

Una vez obtenida una relación funcional entre parámetros, esta puede aplicarse a otros vehículos, sistemas o entornos, evaluando su validez fuera del conjunto original de datos.

Cabe aclarar que, si bien el desarrollo de fórmulas matemáticas explícitas representa un objetivo ideal en todo análisis predictivo, no es el alcance central del presente trabajo llevar a cabo una formalización algebraica exhaustiva de los modelos generados. Nuestro enfoque se centra en la imputación práctica de valores faltantes y en la validación estadística de las combinaciones más prometedoras. Sin embargo, destacamos que realizar este tipo de modelización funcional es altamente recomendable en estudios posteriores, ya que permite consolidar el conocimiento extraído, facilitar su transferencia a sistemas automáticos de diseño, y fomentar la replicabilidad y comprensión científica del fenómeno observado (entendiendo que es un campo aún en exploración).

Una vez contruidos y validados los modelos basados en correlaciones y regresiones, surgen nuevas oportunidades como, automatizar procesos de diseño o verificación técnica (por ejemplo, prever si una configuración de propulsión es viable sin tener que simularla) o explorar métodos más sofisticados si las relaciones no pueden ser capturadas por expresiones simples.

Es aquí donde se abren las puertas al uso de modelos de tipo **machine learning**, como **Random Forest** o **Redes Neuronales**, que permiten descubrir patrones no lineales y de alta dimensionalidad sin necesidad de presuponer una forma funcional.

16.8 Métodos complementarios

16.8.1 Random Forest

Random Forest es un modelo basado en conjuntos de árboles de decisión entrenados de forma aleatoria. Cada árbol aprende sobre una muestra diferente del conjunto de datos, y la predicción final se obtiene promediando los resultados individuales. Se caracteriza por su robustez ante el ruido, su capacidad para capturar relaciones no lineales y su versatilidad para trabajar con variables numéricas y categóricas. Es especialmente útil cuando hay muchas variables y no se conoce a priori la estructura del problema.

Segmentación tipo reglas: " $si\ x > 10, y = 30$ "

simples son perfectamente viables y, además, deseables por su facilidad de aplicación en etapas preliminares. Por ejemplo, el peso vacío puede aproximarse mediante una regla lineal del tipo: **Peso vacío** $\approx 0,8 \cdot \text{MTOW}$.

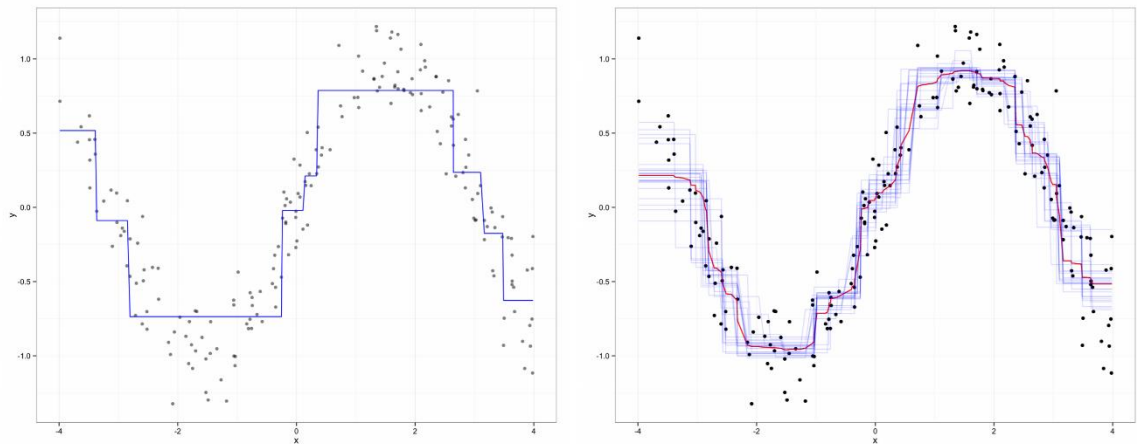


Imagen 16.2 - Árbol de decisión vs Random Forest en regresión

Comparación conceptual entre un único árbol (izquierda) y un ensamble tipo Random Forest (derecha), destacando cómo el promedio de múltiples árboles suaviza y generaliza. [12]

16.8.2 XGBoost (Extreme Gradient Boosting)

XGBoost es una técnica basada en el principio de boosting por gradiente, en la que los modelos se construyen de forma secuencial, corrigiendo en cada etapa los errores cometidos por el modelo anterior. Se trata de uno de los métodos más precisos y eficientes para tareas de regresión y clasificación, y ha sido ampliamente validado en competencias de ciencia de datos. Su principal ventaja radica en su capacidad para optimizar directamente métricas de error y adaptarse a relaciones complejas entre variables con gran capacidad de generalización.

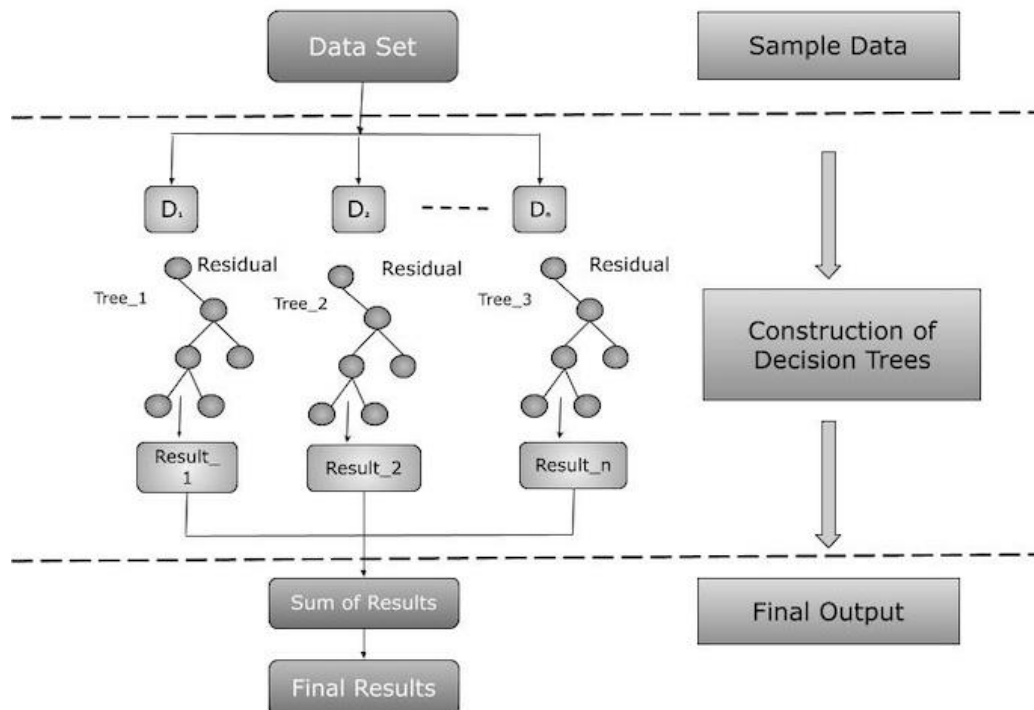


Imagen 16.3 - Arquitectura general de XGBoost (gradient boosting)

Diagrama del flujo de boosting: construcción secuencial de árboles donde cada nuevo árbol corrige los residuales del anterior, y la suma produce la salida final. [13]

16.8.3 Redes Neuronales Artificiales (ANN)

Las redes neuronales son modelos inspirados en el funcionamiento del cerebro humano. Están compuestas por capas de nodos conectados, que transforman y combinan los datos de entrada mediante funciones matemáticas. Son capaces de aprender patrones complejos y no lineales, incluso en entornos de alta dimensionalidad. Requieren más datos y recursos para su entrenamiento, pero ofrecen el máximo poder predictivo disponible actualmente. Su uso es recomendable cuando se dispone de un volumen suficiente de datos y cuando otras técnicas no logran capturar la estructura del problema.

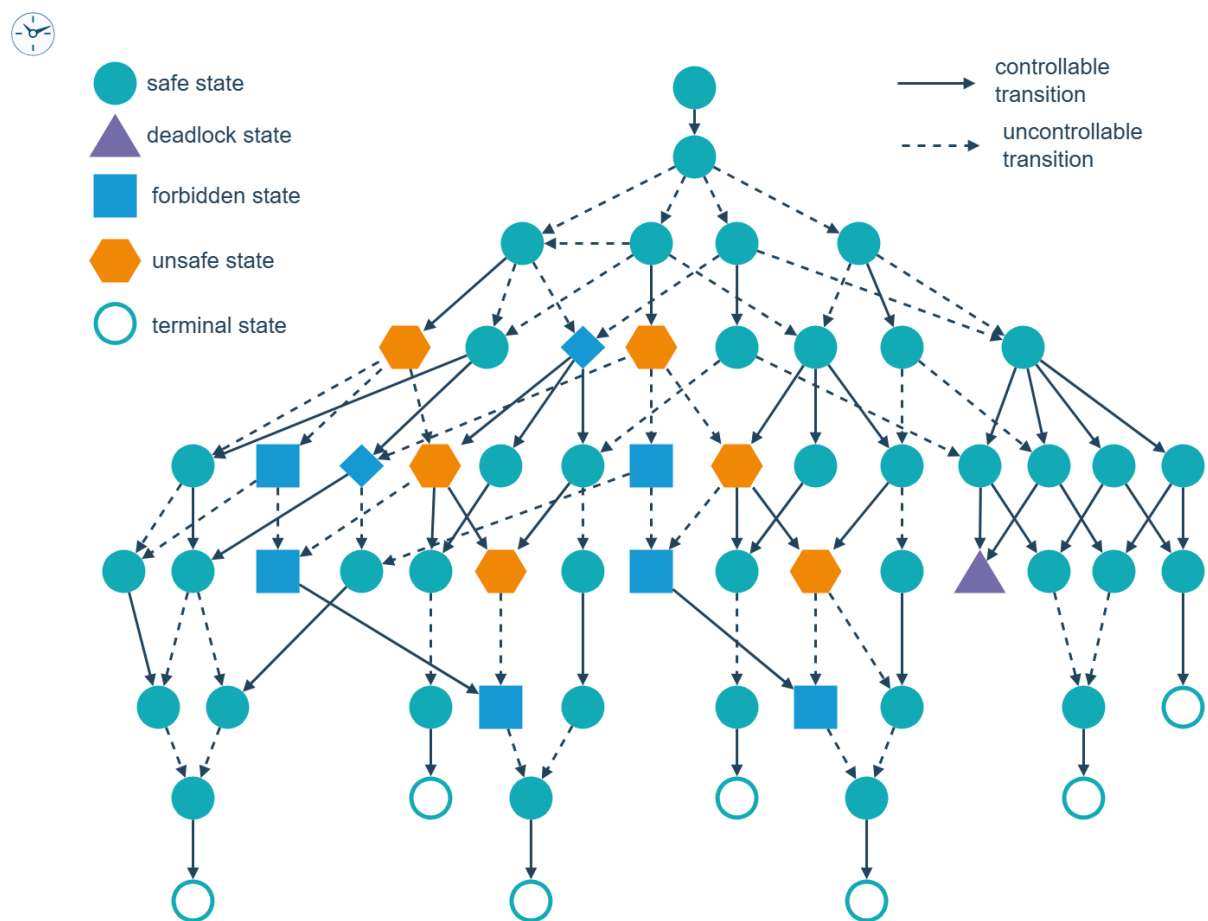


Imagen 16.4 - Ejemplo de diagrama de estados y transiciones

Diagrama ilustrativo de estados (seguros, prohibidos, terminales, etc.) y transiciones controlables/no controlables, usado como analogía de estructuras discretas. [14]

A modo de cierre de esta revisión metodológica, se presentan de forma resumida las principales características de los métodos evaluados, en relación a su aplicabilidad para tareas de imputación de datos técnicos como las abordadas en este trabajo:

<i>Método</i>	<i>Forma funcional conocida</i>	<i>Captura de no linealidades</i>	<i>Soporte a múltiples variables</i>	<i>Interpretabilidad</i>	<i>Robustez / Precisión</i>
<i>Regresión lineal</i>	✓ Sí	X No	✓ Sí	✓ Alta	! Limitada
<i>Regresión polinómica</i>	✓ Sí (más compleja)	✓ Limitada	✓ Sí	! Moderada	✓ Mejora puntual
<i>Regresión logarítmica</i>	✓ Sí	✓ Sí (específica)	✓ Sí	✓ Alta	! Limitada
<i>Regresión exponencial</i>	✓ Sí	✓ Sí (específica)	✓ Sí	✓ Alta	! Limitada
<i>Regresión potencial (potencia)</i>	✓ Sí	✓ Sí (específica)	✓ Sí	✓ Alta	! Limitada
<i>Random Forest</i>	X No explícita	✓ Sí	✓ Sí	! Intermedia	✓ Alta
<i>XGBoost</i>	X No explícita	✓ Sí (gradual)	✓ Sí	! Baja/Moderada	✓ Muy alta
<i>Redes Neuronales (ANN)</i>	X No explícita	✓ Sí (alta complejidad)	✓ Sí	X Baja	✓✓ Máxima (con datos)

Tabla 16.1 - Comparación de métodos de modelado para imputación de datos técnicos

Síntesis comparativa de enfoques de modelado (regresiones, árboles, boosting, redes neuronales, etc.) según forma funcional, capacidad de no linealidad, soporte multivariable, interpretabilidad y robustez/precisión, para justificar la selección metodológica del trabajo.

16.9 Lecciones del desarrollo: programación, estadística e inteligencia artificial

Este proyecto fue sobre todo una escuela en tres frentes: programación, estadística e inteligencia artificial. Se inició con una base relativamente limitada en programación y se concluyó construyendo un sistema que, por tamaño y complejidad, está a la altura del trabajo de un ingeniero de software con varios años de experiencia. Eso obligó a aprender buenas prácticas, modularizar código, diseñar interfaces y mantener la trazabilidad de decisiones técnicas a lo largo del tiempo.

En estadística, el aprendizaje fue todavía más profundo. Más allá de las fórmulas, lo que se cambió fue la forma de pensar problemas de ingeniería: entender qué significa realmente imputar datos, cómo se comportan las métricas cuando la variable objetivo tiene una forma simple, por qué un modelo “perfecto” puede ser engañoso y qué papel juega el juicio de ingeniería para aceptar o no una relación empírica. Sin

este contexto práctico, muchas nociones de regresión, validación cruzada o análisis de outliers habrían quedado como conceptos abstractos.

Finalmente, el contacto con la literatura de inteligencia artificial aplicada a la física abrió una ventana a un campo que en algún momento se consideró convertir en tema central de la tesis. La idea de descubrir conceptos físicos o fórmulas directamente desde los datos resultó ser fascinante, y hubo un planteo serio sobre un cambio de rumbo hacia ese enfoque. Sin embargo, el compromiso con la estructura original del proyecto, con el trabajo complementario de la fase 2 en etapas posteriores del diseño y con los plazos reales de una tesis individual fueron causas suficientes para que se mantenga el foco. Entendemos el resultado como un buen compromiso: no se llegó al estado del arte en deep learning físico, pero se contruyó una plataforma concreta donde esos métodos podrían integrarse en el futuro, y en el proceso se entendió mejor tanto su potencial como sus requisitos.

En paralelo, todo este recorrido reforzó la comprensión de los límites: los del tiempo disponible, los de la calidad de los datos y los de nuestras capacidades en el momento actual. Aun así, el proyecto demostró que, con una base sólida y mucha perseverancia, es posible construir herramientas de nivel profesional incluso en un contexto de recursos limitados. También se tomó consciencia del estado de desarrollo de nuestra sociedad, apreciar la inteligencia humana y mucha intriga de como se desarrollarán los siguientes años, con una tecnología como la IA brindando un fuerte impulso y velocidad en muchos sectores. En nuestro proyecto dicha tecnología fue base para poder desarrollar el programa, ya que el nivel alcanzado de desarrollo de código fue propio de un ingeniero, pero nuestra formación previa muy básica, gracias se pudo salir de lo teórico y abstracto para materializar las ideas y aplicación, haciendo todo el proyecto mucho mas interesante y medible.

16.10 Síntesis y puente a las conclusiones

En este capítulo se tomó distancia del código y de las figuras para responder, que se logró realmente con este proyecto y cual fue el aprendizaje. El sistema que se desarrolló transforma una base de datos problemática en una plataforma interactiva de imputación, análisis de modelos y apoyo al diseño; revela con claridad las limitaciones de los datos y de los modelos; y deja abierta una ruta plausible hacia métodos estadísticos e inteligentes más avanzados, siempre que se respeten ciertas condiciones de calidad e interpretabilidad.

También se dejó explícitos los principales límites: la base de datos es pequeña y heterogénea, el sistema no está libre de bugs, y muchas de las extensiones más ambiciosas solo podrán abordarse con proyectos futuros. Aun así, la tesis cumple su papel de “semilla”: muestra que vale la pena invertir en herramientas de este tipo, enmarca el problema dentro de una conversación más amplia sobre descubrimiento de relaciones físicas y abre un espacio concreto para que otros continúen el desarrollo.

En el Capítulo siguiente se retoman los objetivos planteados al inicio de la tesis, se confrontan con los resultados obtenidos y se formulan conclusiones más directas sobre el grado de cumplimiento, las principales contribuciones y las líneas prioritarias para trabajos futuros.

Aunque el esfuerzo de programación y experimentación estadística se concentró en buena medida en el desarrollo del sistema predictor y de sus herramientas de auditoría, el producto más tangible para un proyecto de diseño es el asistente del Módulo 3. Es este módulo el que transforma el trabajo previo en decisiones concretas sobre una aeronave objetivo y el que proporciona los valores de partida que, en el caso de estudio, se convierten en un vector de entrada completo para ADRpy.

17 CONCLUSIONES

En este capítulo retomamos los objetivos planteados al inicio de la tesis, se confrontan con los resultados obtenidos y se sintetizan las principales contribuciones, limitaciones y líneas de trabajo futuro del sistema ADRpy-VTOL. Mientras que el capítulo anterior se reflexionó sobre el proceso recorrido, aquí el foco está en lo que el proyecto logró efectivamente como herramienta de ingeniería y en cómo se sitúa dentro del flujo de diseño de un dron de ala fija con capacidad VTOL.

17.1 Grado de cumplimiento de los objetivos

El objetivo general de la tesis era transformar una base de datos incompleta de UAV del mercado en una herramienta utilizable para apoyar el diseño conceptual de un dron VTOL de ala fija, mediante un sistema modular de imputación, análisis de modelos y asistencia al diseño. En ese sentido, el proyecto alcanzó un **mínimo producto viable** (MVP) que cumple con los requisitos básicos planteados: existe un flujo reproducible desde un Excel desordenado hasta un conjunto de parámetros globales coherentes para una aeronave objetivo, acompañado por herramientas para auditar el camino seguido por los datos y por los modelos.

En relación con los objetivos específicos, el balance puede resumirse así:

- El **módulo predictor** (Módulo 1) se desarrolló de forma completa para el alcance definido: filtra la base, prepara subconjuntos de datos, entrena modelos de regresión con hasta dos predictores, calcula métricas como MAPE, MAE y R^2 y genera tablas estructuradas de resultados. En la práctica, este módulo concentra la mayor parte del esfuerzo de programación y experimentación; incluso fue necesario simplificar ideas iniciales (como la generación sistemática de modelos con cuatro predictores) para mantener el proyecto dentro de los plazos razonables de una tesis individual.
- El **panel de análisis y auditoría de modelos** (Módulo 2) se implementó hasta un nivel que permite revisar y filtrar ecuaciones, inspeccionar gráficamente su comportamiento y detectar outliers. Desde el punto de vista técnico supuso un desafío importante a nivel de interfaz y manejo de datos, y aunque su potencial podría explotarse más en términos de resultados, deja resuelto el problema central de hacer visibles y discutibles las decisiones tomadas por el predictor.
- El **asistente de diseño** (Módulo 3) alcanza el objetivo de traducir la base depurada y los modelos validados en decisiones concretas sobre una aeronave objetivo. A partir de unos pocos requerimientos iniciales, propone combinaciones de parámetros globales razonables y permite iterar sobre ellas hasta llegar a una solución coherente. En el caso de estudio, estos valores sirven de base para construir un vector de entrada completo en ADRpy, que a su vez se utiliza para generar un diagrama ilustrativo de restricciones $W/S-T/W$.
- En cuanto a la **documentación y enfoque pedagógico**, el texto de este trabajo final y el repositorio de código se complementan: el documento explica la lógica del sistema y sus decisiones de diseño, y el código permite reproducir los resultados y experimentar con nuevos escenarios. En este aspecto, el sistema cumple su papel de “plataforma formativa” más que de producto maduro listo para uso industrial.

En síntesis, puede afirmarse que los objetivos planteados se cumplen en su versión mínima viable: el sistema funciona, demuestra el flujo completo desde los datos de mercado hasta una primera solución numérica razonable y deja un espacio claro para mejoras y extensiones futuras.

17.2 Aportes principales del sistema ADRpy-VTOL

El primer aporte del sistema es **metodológico**: mostrar que el diseño conceptual de una aeronave puede abordarse no sólo como un conjunto de fórmulas aisladas, sino como un flujo de datos, modelos y herramientas que debe ser trazable y auditable. El Módulo 1 estructura y enriquece una base de datos problemática; el Módulo 2 la hace interpretable; y el Módulo 3 la traduce en decisiones de diseño, integrando todo en un pipeline coherente.

El segundo aporte es **práctico**. El sistema reduce de forma drástica el trabajo manual necesario para explorar combinaciones de parámetros, evaluar modelos y justificar decisiones numéricas. La cantidad de comprobaciones lógicas, cálculos y cruces de información que realiza el programa sería, en la práctica, imposible de sostener de forma manual con un esfuerzo razonable. Nadie dedicaría ese tiempo a mano para justificar únicamente valores iniciales aproximados; sin embargo, al automatizar esas tareas, la tesis demuestra que es factible elevar el nivel de rigor de la etapa conceptual sin paralizar el avance del proyecto.

El tercer aporte es **instrumental**: el desarrollo se concreta en una herramienta ejecutable y disponible públicamente. El repositorio ADRpy-VTOL reúne el código, los notebooks y los archivos de configuración necesarios para reproducir los ejemplos de la tesis y experimentar con nuevos datasets:

<https://github.com/Delpoo/ADRpy-VTOL>

La tesis funciona, así como guía de uso y de lectura crítica de la herramienta, y el repositorio como base para su reutilización y extensión.

Por último, a nivel de **experiencia de diseño**, el módulo que ofrece un beneficio más tangible para un proyecto real es el Módulo 3. Aunque el predictor y el panel de modelos representan la mayor inversión de tiempo y complejidad, es el asistente de diseño el que condensa todo ese trabajo previo en decisiones concretas sobre una aeronave objetivo y el que proporciona los valores de partida que, en el caso de estudio, se convierten en un vector de entrada completo para ADRpy.

17.3 Hipótesis, descubrimiento de relaciones y alcance real del sistema

Respecto de la **primera hipótesis**, los resultados confirman que es posible imputar valores faltantes de forma coherente combinando similitud entre aeronaves, modelos de regresión y criterios físicos simples. Para varios parámetros se obtuvieron modelos razonablemente estables y útiles para completar la base de datos. Sin embargo, la calidad y cantidad del dataset se revelaron como la limitación dominante: en muchos casos el problema no era la técnica de imputación, sino la escasez y heterogeneidad de los datos disponibles.

En relación con la **segunda hipótesis**, el sistema permitió identificar patrones y tendencias que pueden interpretarse como “indicios” de reglas de diseño, más que como leyes universales del mercado de UAV. La expectativa inicial de descubrir nuevas relaciones físicas generales se vio moderada por las limitaciones de la muestra: con una base de datos pequeña, cargada manualmente y con definiciones no siempre homogéneas, resultó imposible cerrar la tesis con fórmulas nuevas robustas a todo el espectro de drones. Quedó, en cierto sentido, un “gusto raro” por no haber llegado más lejos en ese frente; pero al

mismo tiempo se hizo evidente que, con una campaña de relevamiento más sistemática y rigurosa, el sistema podría ser una plataforma útil para explorar ese tipo de relaciones en el futuro.

En cuanto a la **tercera hipótesis**, la automatización del flujo de trabajo se valida de forma clara. El sistema reduce en órdenes de magnitud el esfuerzo necesario para generar, evaluar y depurar modelos, y para iterar sobre parámetros globales de diseño. Al mismo tiempo, el proyecto deja explícito que el sistema no produce “números mágicos”: las salidas deben interpretarse como aproximaciones macro iniciales, que requieren todavía el juicio del ingeniero y análisis adicionales antes de convertirse en decisiones definitivas. Comprender qué ocurre dentro del sistema (qué suposiciones se hacen, cómo está estructurada la hoja Excel, qué unidades se usan) sigue siendo condición necesaria para interpretar correctamente los resultados.

Una lección transversal que emerge de las tres hipótesis es el riesgo de perder el foco macro cuando se profundiza demasiado en subproblemas técnicamente interesantes. La curiosidad intelectual y el perfeccionismo empujaron el desarrollo del sistema predictor y de las herramientas de auditoría mucho más allá de lo mínimo necesario para un MVP. No hay arrepentimiento por esa profundización (el aprendizaje técnico fue enorme), pero sí una toma de conciencia: en un proyecto de diseño de un dron completo, dedicar tanto esfuerzo a un único tramo del flujo puede penalizar fuertemente el avance hacia el objetivo global.

17.4 Limitaciones y estado actual de la herramienta

Desde el punto de vista técnico, la limitación más importante del trabajo es la **calidad y cantidad de datos** disponibles. La base se construyó a partir de fichas comerciales y documentos públicos que no siempre comparten criterios de definición, unidades o nivel de detalle. Esto obliga a asumir que muchas de las tendencias encontradas describen el comportamiento de un subconjunto concreto del mercado y no el de todos los UAV posibles.

Una segunda limitación clave es la **falta de validación externa sistemática**. Aunque se aplican criterios de coherencia física y se revisan las salidas del predictor y del asistente, el sistema no ha sido todavía probado en un proyecto real de diseño que culmine en la construcción de una aeronave. En ese sentido, y siendo honestos con el contexto en el que se desarrolló, hoy el programa es sobre todo un sistema sofisticado para argumentar selecciones numéricas de parámetros iniciales y para explorar el espacio de diseño, más que una herramienta lista para dimensionar un dron que vaya a fabricarse.

En tercer lugar, la herramienta presenta **limitaciones de uso práctico**: la interfaz está pensada para ejecutarse en local, con una curva de aprendizaje considerable y con fuertes dependencias respecto al formato de la hoja Excel de entrada. La presencia de filas o columnas con valores aleatorios, encabezados invertidos, unidades inconsistentes o cambios en el orden de los parámetros puede alterar por completo el comportamiento del sistema. Antes de utilizar el programa como soporte de decisiones en un entorno profesional habría que invertir en una interfaz más robusta, en validaciones de entrada más estrictas y en flujos de trabajo que reduzcan el riesgo de errores por formato.

Por todo ello, la conclusión honesta es que ADRpy-VTOL, en su estado actual, **no está todavía listo** para ser usado sin reservas en un proyecto de diseño de aeronaves con impacto económico real. Requiere auditorías adicionales, pruebas de validación con casos reales y mejoras importantes en la experiencia de usuario. Como herramienta de exploración, aprendizaje y argumentación numérica de primeras decisiones, en cambio, ya cumple un papel valioso.

17.5 Líneas de trabajo futuro

De cara al futuro, se abren dos líneas de trabajo complementarias.

La primera consiste en **perfeccionar la plataforma ADRpy-VTOL**. Esto incluye ampliar y depurar la base de datos con un relevamiento más sistemático y homogéneo, incorporar métodos más avanzados de modelado (por ejemplo, técnicas de inteligencia artificial interpretables) sobre la infraestructura ya construida, mejorar el módulo de auditoría y, muy especialmente, rediseñar la interfaz para que el sistema pueda usarse con mayor seguridad y menor curva de aprendizaje, idealmente a través de un entorno web. También sería deseable mostrar más casos de uso en la herramienta y explotar con calma las relaciones encontradas, dedicando tiempo específicamente al análisis de resultados y no sólo al desarrollo de código.

La segunda línea, igualmente o aún más importante, es **continuar el diseño de la aeronave VTOL** que motivó el proyecto. En el esquema global, esta tesis representa la fase 1: construir y explotar una base de datos que permita definir una “primera solución razonable” para un dron VTOL. La futura fase 2, adapta ADRpy para incorporar la configuración VTOL y avanzar en el diseño preliminar propiamente dicho. A partir de ahí, las fases siguientes —análisis estructurales y aerodinámicos detallados, simulaciones CFD, estudios por elementos finitos, diseño de detalle y fabricación de prototipos— prometen ser progresivamente más cercanas al “corazón” del diseño de la aeronave y, en muchos sentidos, más gratificantes para el estudiante que las aborde.

En ese contexto, el sistema desarrollado en esta tesis ya es suficiente como base de datos y generador de parámetros iniciales para continuar el diseño de la aeronave. No es imprescindible seguir ampliándolo si el foco está en el vehículo en sí; las mejoras del programa cobran sentido sobre todo si se aspira a convertirlo en una plataforma de uso más general o incluso comercial.

17.6 Consideraciones finales

Mirado en perspectiva, el proyecto no llegó a diseñar por completo el dron VTOL soñado ni a descubrir nuevas fórmulas universales de diseño a partir de los datos. En cambio, dejó algo distinto, pero igual de valioso: una plataforma concreta que conecta datos desordenados del mercado con una primera solución numérica defendible para alimentar ADRpy, un conjunto de herramientas que automatizan tareas que serían inviables a mano y un ejemplo completo que culmina en un vector de entrada y un diagrama de restricciones ilustrativo, que materializan el flujo de trabajo propuesto.

La tesis muestra que, incluso en un contexto de recursos limitados, es posible construir herramientas de nivel casi profesional si se acepta que el resultado será un MVP a perfeccionar y no un producto final acabado. El proyecto deja así una “semilla” para futuros desarrollos: una base sobre la que otros podrán seguir construyendo, ya sea para refinar la plataforma ADRpy-VTOL o para llevar adelante el diseño y la construcción de la aeronave VTOL que le dio sentido desde el principio.

18 APÉNDICES

18.1 Descripción de variables del vector de la aeronave objetivo

Grupo	Variable	Descripción
Parámetros de misión (brief)	cruisealt_m	Altitud de crucero de la misión, en metros. Define la densidad del aire y las condiciones en las que se evalúa el tramo principal de vuelo.
Parámetros de misión (brief)	cruisespeed_ktas	Velocidad verdadera de crucero, en nudos (TAS). Se usa para calcular empuje/potencia requerida y alcance.
Parámetros de misión (brief)	range_km	Alcance horizontal requerido de la misión, en kilómetros. Representa la distancia total que debe poder recorrer la aeronave.
Parámetros de misión (brief)	endurance_h	Autonomía total de vuelo, en horas. Tiempo que la aeronave puede permanecer en el aire desde el despegue hasta el aterrizaje.
Parámetros de misión (brief)	loiter_time_h	Tiempo destinado a patrulla o merodeo sobre la zona de interés, en horas. Es una fracción de la autonomía total.
Parámetros de misión (brief)	servceil_m	Techo de servicio, en metros. Altitud máxima a la que la aeronave mantiene una tasa de ascenso mínima especificada.
Parámetros de misión (brief)	climbrate_fpm	Tasa de ascenso requerida en el tramo principal, en pies por minuto. Define una restricción de potencia/empuje en ascenso.
Parámetros de misión (brief)	groundrun_m	Longitud de carrera de despegue, en metros. Se usa para verificar restricciones de operación desde pista o rodadura.
Parámetros de misión (brief)	turnloadfactor	Factor de carga en giro sostenido, adimensional. Indica cuántos "g" debe soportar la aeronave en un viraje nivelado.
Parámetros de misión (brief)	n_turn	Factor de carga en giro sostenido, adimensional. Indica cuántos "g" debe soportar la aeronave en un viraje nivelado.
Parámetros de diseño (design)	wingarea_m2	Superficie alar de referencia, en metros cuadrados. Es la base para calcular la carga alar W/S.
Parámetros de diseño (design)	aspectratio	Relación de aspecto del ala $AR=b^2/S$, adimensional. Mide cuán alargada es el ala y condiciona la eficiencia aerodinámica.
Parámetros de diseño (design)	roottaperratio	Relación de estrechamiento del ala $\lambda=c_{tip}/c_{root}$, adimensional. Describe cómo se afina el ala desde la raíz hacia la punta.
Parámetros de diseño (design)	sweep_le_deg	Ángulo de flecha del borde de ataque del ala, en grados. Influye en el comportamiento a alta velocidad.
Parámetros de diseño (design)	sweep_25_deg	Ángulo de flecha medido al 25 % de la cuerda, en grados. Es el valor habitual para correlaciones de estabilidad y rendimiento.
Parámetros de diseño (design)	sweep_mt_deg	Ángulo de flecha en la línea de máximo espesor del perfil, en grados. Se utiliza en algunas correcciones aerodinámicas.

Parámetros de diseño (design)	wingheightratio	Relación entre altura del ala sobre el suelo y envergadura h/b, adimensional. Indica el grado de efecto suelo en despegue y aterrizaje.
Parámetros de diseño (design)	bpr	Parámetro de tipo de propulsor, adimensional. En el código se utiliza para seleccionar el modelo de motor (pistón, turboprop, eléctrico, etc.).
Parámetros de diseño (design)	spooluptime_s	Tiempo de respuesta del motor desde ralentí hasta potencia máxima, en segundos. Afecta maniobras críticas y márgenes de seguridad.
Parámetros de diseño (design)	weight_n	Peso máximo al despegue (MTOW), en Newtons. Es el peso de referencia para todas las restricciones de diseño.
Parámetros de diseño (design)	totalstaticthrust_n	Empuje estático máximo total al nivel del mar, en Newtons. Vincula el cociente T/W con motores reales.
Parámetros de diseño (design)	tr	Parámetro de “throttle ratio”, adimensional. Controla cómo varía el empuje disponible con la altura y la posición del acelerador en el modelo.
Parámetros de diseño (design)	runwayalpha_deg	Ángulo de ataque del ala durante la carrera de despegue, en grados. Relevante en configuraciones de despegue convencional.
Parámetros de diseño (design)	runwayalpha_max_deg	Ángulo de ataque máximo permitido durante la carrera de despegue, en grados. Limita la maniobra de rotación y la carga sobre el tren.
Parámetros de performance y globales	CDOTO	Coefficiente de arrastre parásito a sustentación cero en configuración de despegue, adimensional.
Parámetros de performance y globales	CDTO	Coefficiente de arrastre total representativo del punto de despegue, adimensional.
Parámetros de performance y globales	CDminclean	Coefficiente de arrastre mínimo en configuración limpia (sin flaps ni tren extendido), adimensional.
Parámetros de performance y globales	mu_R	Coefficiente de rozamiento de rodadura del tren de aterrizaje sobre la pista, adimensional.
Parámetros de performance y globales	CLOTO	Coefficiente de sustentación a ángulo de ataque cero en configuración de despegue, adimensional.
Parámetros de performance y globales	CLTO	Coefficiente de sustentación típico durante la rotación y el despegue, adimensional.
Parámetros de performance y globales	CLmaxTO	Coefficiente de sustentación máximo en configuración de despegue (con flaps), adimensional.
Parámetros de performance y globales	CLmaxclean	Coefficiente de sustentación máximo en configuración limpia, adimensional.
Parámetros de performance y globales	CLminclean	Coefficiente de sustentación mínimo en configuración limpia (ángulos de ataque negativos), adimensional.
Parámetros de performance y globales	CLslope	Pendiente de la curva de sustentación $dCL/d\alpha$, en rad^{-1} . Relaciona variaciones de ángulo de ataque con variaciones de sustentación.
Parámetros de performance y globales	etaprop['take-off']	Eficiencia del sistema propulsivo en despegue, adimensional.

Parámetros de performance y globales	etaprop['climb']	Eficiencia del sistema propulsivo en ascenso, adimensional.
Parámetros de performance y globales	etaprop['cruise']	Eficiencia del sistema propulsivo en crucero, adimensional.
Parámetros de performance y globales	etaprop['turn']	Eficiencia del sistema propulsivo en giro sostenido, adimensional.
Parámetros de performance y globales	etaprop['servceil']	Eficiencia del sistema propulsivo al alcanzar el techo de servicio, adimensional.
Parámetros de performance y globales	weightfractions['take-off']	Fracción de peso utilizada en la fase de despegue, adimensional. En este trabajo se asume igual a 1,0.
Parámetros de performance y globales	weightfractions['climb']	Fracción de peso en la fase de ascenso, adimensional. En este trabajo se asume igual a 1,0.
Parámetros de performance y globales	weightfractions['cruise']	Fracción de peso en crucero, adimensional. En este trabajo se asume igual a 1,0.
Parámetros de performance y globales	weightfractions['turn']	Fracción de peso en giro sostenido, adimensional. En este trabajo se asume igual a 1,0.
Parámetros de performance y globales	weightfractions['servceil']	Fracción de peso en la fase de techo de servicio, adimensional. En este trabajo se asume igual a 1,0.
Atmósfera	Atmósfera de referencia (ISA)	El modelo utiliza una atmósfera estándar (ISA) implementada en los scripts auxiliares de ADRpy. Incluye leyes de variación de temperatura, presión y densidad con la altura, y se considera fija para todos los casos analizados.

18.2 Ficha técnica “ALTI TRANSITION”

Technical Datasheet

TRANSITION



Aircraft

Dimensions	
Wingspan	3000 mm
Length	2300 mm
Height (Empennage)	525 mm
Ground Clearance (Fuselage)	170 mm
Ground Clearance (Pusher Propeller)	65 mm
Weight	
MTOW (Maximum Takeoff Weight)	18 kg
Empty Airframe	5.8 kg
RTF (Dry Weight)	11.8 kg
RTF (Including Fuel & Batteries)	16.5 kg
Max Payload	1.5 kg
Flight Performance	
Stall Speed	13-15 m/s
Cruise Speed	20 m/s
VNE (Velocity Never Exceed)	30 m/s
Maximum Crosswind	10-15 m/s
Maximum Service Ceiling	13,000 ft AMSL
Flight Time (VTOL)	Up to 3 minutes
Flight Time (Fixed Wing)	Up to 12 hours
Temperature Range	-10° C to +50° C
VTOL Propulsion	
Motors	T-Motor ALTI U7
Motor KV	420 KV
ESC	T-Motor ALTI 80A Flame
Propeller	T-Motor 18x6.1 Carbon Fiber
Battery Power Supply	2 x 9000mAh 4S Lipo
Fixed Wing Propulsion	
Internal Combustion Engine	Saito FG21



Command & Control | C²

Dimensions	
Length	630 mm
Width	394 mm
Height	208 mm
Weight	
Ground Control Station	12 kg
Volumetric Weight	14 kg (Avg) (Shipping Purposes)
Case	Pelican 1555 AIR
Hardware	
Intel NUC8	
Storage (SSD)	120 GB
Display	2x ASUS 15.6" HD Monitors
HDMI Out	1x External UHD Compatible
USB	3x External USB 3.0 Ports
Bluetooth	Yes
LAN	1x External LAN port
Cooling	2x Active Cooling Fans
Pilot Controller	Futaba
Keyboard & Mouse	Yes
Main AC Power	120-220v - 19V
Battery Power	2 x 9000mAh 4S Lipo
Intelligent Power Management	Yes
Links	
Groundside	
Data Telemetry / Video Link: TaiSync	
- AES128 or 256	
- Omni Antenna, 4 Section Collinear, 6 dBi Gain	
- Cylindrical Sector Antenna, 12 dBi Gain	
Control Link: 868 / 915 MHz	
- TBS Crossfire	
- TBS Diamond Antenna Gain: 2.88dB	
Airside	
Data Telemetry / Video Link: TaiSync	
- AES128 or 256	
- 2 x Omni Antenna, Half Wave Dipole 2.15dBi Gain	
Control Link: 868 / 915 MHz	
- TBS Crossfire	
- Custom Build 1/4 Wave dipole antennas 0dBi Gain	

v2020.2 | All information subject to change without notice

GET IN TOUCH

info@altiuas.com | www.altiuas.com | +27 44 382 7051

Imagen 18.1 - Ficha técnica de UAV Altitude Transition (1/2)

Características técnicas del dron seleccionado como aeronave más similar para completar el vector de entrada al sistema ADRpy

Technical Datasheet

TRANSITION



Advanced Tracking Antenna Systems

Groundside

Video Link: 2.4GHz

Silvus Radio (SC4200E)

Antenna:

Tracking antenna

OSPT10 (100km) | OSPT25 (150km)

Control Link: 868/915 MHz

TBS Crossfire

TBS Diamond Antenna Gain: 2.88dB

Airside

Video Link: 2.4GHz

Silvus Radio (SC4200E) OEM

2x Omni Antenna, Half Wave Dipole 2.15 dBi

Control Link: 868/915 MHz

TBS Crossfire

Custom Build 1/4 Wave dipole antennas 0dBi Gain

Transportation Case

Dimensions

Length	1740 mm
Width	1040 mm
Height	550 mm

Weight

Aircraft, GCS & Accessories	110 kg
Volumetric Weight	316 kg (Average) (Shipping Purposes)

Sound Levels

Ground @ 5m Distance	
Front Idle	65 dB
Left-side Idle	60 dB
Rear Idle	64 dB
Right-side Idle	61 dB
Passing Overhead @ 70m AGL	57 dB
Passing Overhead @ 100m AGL	52 dB
In VTOL Hover @ 20m AGL	70 dB
In VTOL Hover @ 1m AGL	90 dB

Please note that the maximum range from point of operation indicated above is with clear Line of Sight (LOS), in other words, there are no major obstructions between the GCS and Aircraft.

Autonomous Flight Modes

> Auto Take-off and Landing

Allowing for fully autonomous VTOL take-off and landing

> VTOL Transition

Seamless autonomous transition between multirotor and fixed-wing modes

> Payload Triggering

Auto trigger payload once or multiple times using Geo Reference, Time or Distance Interval. An override for manual triggering is also available

> Guided Mode

Point and click autonomous mission flight mode. Aircraft will fly to and loiter (circle) at the selected location and altitude

> Loiter

Circle around the point where you started the loiter, holding altitude for efficient flight

> Return to Launch (RTL)

Aircraft will return to the home (take-off) position in Cruise mode until it is within the RTL radius, after which it will transition to Q-Loiter (VTOL) mode and land. Radius and RTL altitude is configurable in Mission Planner

> Various other advanced mission planning and flight features available

v2020.2 | All information subject to change without notice

ALTI

GET IN TOUCH

info@altiuas.com

www.altiuas.com

+27 44 382 7051

Imagen 18.2 - Ficha técnica de UAV Alti Transition (2/2)

Características técnicas del dron seleccionado cómo aeronave más similar para completar el vector de entrada al sistema ADRpy

18.3 Proyecto final formato Revista

Sistema modular de imputación y análisis estadístico de parámetros de UAV para apoyo al diseño conceptual de un dron VTOL de ala fija

Modular System for UAV Parameter Imputation and Statistical Analysis to Support the Conceptual Design of a Fixed-Wing VTOL Drone

Ezequiel Delpino

*Universidad del Trabajo del Uruguay
Instituto Tecnológico Superior
Instituto de Adiestramiento Aeronáutico*

Recibido: 18 Feb 2026

Aceptado: 26 Feb 2026

Disponible en línea: 5 Mar
2026

Resumen

El diseño conceptual de aeronaves cuenta con metodologías y referencias ampliamente consolidadas; sin embargo, en el caso de los UAV (en particular los de ala fija con configuraciones VTOL) la diversidad de arquitecturas y la fragmentación de información pública dificultan disponer de criterios generalistas y de conjuntos de valores iniciales comparables. Esta situación se vuelve especialmente relevante cuando se busca iniciar de forma sistemática y automatizada un proceso de dimensionamiento conceptual mediante herramientas programables como ADRpy, que requieren vectores de entrada coherentes y verificables. En este trabajo se presenta ADRpy-VTOL, un sistema modular en Python desarrollado para apoyar ese objetivo: transformar una planilla heterogénea de UAV de ala fija (con y sin capacidad VTOL) en una base de datos organizada, completada y auditada, y facilitar la generación de vectores de parámetros listos para análisis conceptual.

El sistema aplica una estrategia de imputación local por celda faltante, combinando (a) imputación por similitud mediante transferencia desde vecinos cercanos y (b) imputación por correlación a partir de regresiones supervisadas con formas funcionales simples y hasta dos predictores, bajo criterios explícitos de consistencia y evidencia. Los modelos se validan con LOOCV y se evalúan mediante métricas complementarias (MAE, MAPE y R^2), integradas en indicadores operativos y en un score de confianza que penaliza complejidad y tamaño muestral. Un módulo de auditoría en modo lectura organiza la trazabilidad de cada imputación (método, métricas, advertencias y confianza) y clasifica resultados según su aceptabilidad y prioridad de revisión. Finalmente, un asistente de diseño construye un vector de parámetros compatible con ADRpy, preservando el origen y la calidad asociada a cada valor.

Los resultados muestran que el enfoque permite obtener una base enriquecida y trazable que puede facilitar etapas tempranas del diseño conceptual y el análisis de tendencias de primer orden. No obstante, su desempeño y alcance quedan condicionados por las limitaciones inherentes a la base de datos disponible, particularmente en términos de completitud, homogeneidad y confiabilidad de las especificaciones de origen.

Palabras clave: UAV, VTOL, diseño conceptual, imputación de datos, regresión, LOOCV, MAPE, R^2 , confianza, ADRpy.

Abstract

Conceptual aircraft design benefits from well-established methodologies and widely used references; however, in the UAV domain (particularly for fixed-wing platforms with VTOL configurations) the diversity of architectures and the fragmentation of publicly available information make it difficult to rely on general-purpose criteria and comparable sets of initial values. This becomes especially relevant when aiming to start a systematic, automated conceptual-sizing workflow using programmable tools such as ADRpy, which require coherent and verifiable input vectors. This paper presents ADRpy-VTOL, a modular Python system developed to support this goal by transforming a heterogeneous spreadsheet of fixed-wing UAVs (with and without VTOL capability) into an organized, completed, and audited database, and by facilitating the generation of ADRpy-ready parameter vectors for conceptual analyses.

The system applies a local, per-missing-cell imputation strategy that combines (a) similarity-based imputation via transfer from nearest neighbors and (b) correlation-based imputation through supervised regressions with simple functional forms and up to two predictors, under explicit consistency and evidence criteria. Models are validated using LOOCV and evaluated with complementary metrics (MAE, MAPE, and R^2), which are integrated into operational indicators and a confidence score that penalizes model complexity and sample size. A read-only audit module provides full traceability for each imputation (method, metrics, warnings, and confidence) and classifies results by acceptability and review priority. Finally, a design assistant builds an ADRpy-compatible parameter vector while preserving the origin and quality associated with each value.

Results show that the approach can produce an enriched and traceable dataset that may facilitate early-stage conceptual design and first-order trend analyses. Nevertheless, its performance and scope are constrained by limitations inherent to the available database, particularly regarding completeness, consistency, and the reliability of the source specifications.

Keywords: UAV, VTOL, conceptual design, data imputation, regression, LOOCV, MAPE, R^2 , confidence, ADRpy.

1 Introducción

El diseño conceptual de aeronaves tiene como propósito delimitar configuraciones viables y orientar decisiones tempranas con un costo de desarrollo reducido. En esta etapa se evalúan órdenes de magnitud de masa, potencia, geometría y desempeño mediante relaciones simplificadas y modelos de primer orden. A pesar de su carácter preliminar, el diseño conceptual condiciona decisiones posteriores de alto impacto, ya que define restricciones físicas, compromisos entre variables y rangos plausibles para las etapas de diseño preliminar y de detalle.

En plataformas no convencionales (particularmente en UAVs y en aeronaves con capacidad VTOL) el diseño conceptual enfrenta una dificultad adicional: el acceso a información homogénea de aeronaves de referencia es limitado. Los datos publicados suelen provenir de fichas técnicas, reportes parciales o material comercial; por lo tanto, no siempre se explicitan las condiciones de medición ni se mantiene consistencia en definiciones. La autonomía puede reportarse con distintas cargas útiles; la velocidad puede corresponder a regímenes diferentes (crucero, máxima, económica); y las masas pueden no referirse a la misma configuración operativa. A este problema de comparabilidad se suma la presencia de valores faltantes, de modo que el dataset disponible típicamente es incompleto y heterogéneo al mismo tiempo. Estas condiciones afectan no solo la calidad de un análisis exploratorio, sino también la posibilidad de automatizar un flujo de diseño conceptual con trazabilidad.

Las herramientas computacionales orientadas al diseño conceptual permiten formalizar el análisis temprano y reducir la dependencia de cálculos manuales. En particular, entornos en Python como ADRpy facilitan la evaluación de desempeño bajo hipótesis estándar y la representación de restricciones mediante diagramas de carga alar y relación empuje/peso u otros enfoques equivalentes. Sin embargo, para que el análisis sea reproducible y útil en un proceso sistemático, es necesario disponer de un conjunto de parámetros de entrada completo y consistente. En la práctica, cuando el dataset posee valores faltantes en variables clave, el análisis se interrumpe, se simplifica en exceso o se apoya en supuestos no documentados, lo cual reduce la confiabilidad de las conclusiones.

Este trabajo presenta ADRpy-VTOL, un sistema modular implementado en Python cuyo objetivo es robustecer el uso de datos de referencia en las etapas iniciales del diseño conceptual de un dron de ala fija con capacidad VTOL. El sistema aborda el problema desde una perspectiva orientada a ingeniería: no se limita a imputar valores faltantes, sino que incorpora mecanismos de depuración, evaluación y trazabilidad que permiten interpretar cada valor completado como una estimación respaldada por evidencia cuantitativa. En términos generales, el flujo propuesto transforma un dataset inicial incompleto en una base procesada que incluye, junto a cada valor imputado, información sobre el método empleado, la calidad del ajuste y un indicador de confianza.

La contribución del trabajo se articula en tres componentes. En primer lugar, se propone una etapa de

depuración inicial que normaliza tipos de datos, trata registros inválidos y resuelve duplicados, reduciendo sesgos que podrían propagarse hacia métodos estadísticos. En segundo lugar, se implementan dos estrategias complementarias de imputación: un enfoque basado en similitud (vecinos cercanos) y un enfoque basado en regresión supervisada²¹ que explora distintas familias funcionales con combinaciones acotadas de predictores. En tercer lugar, se define un mecanismo de evaluación que integra métricas de precisión y capacidad explicativa con penalizaciones por complejidad y tamaño efectivo de muestra, resultando en un indicador de confianza que puede utilizarse para filtrar imputaciones o priorizar revisiones. Esta lógica busca mantener un equilibrio entre completitud del dataset y robustez de las estimaciones, evitando que el completado de datos se convierta en una fuente silenciosa de error.

En términos de arquitectura, ADRpy-VTOL se organiza en tres módulos. El Módulo 1 (Predictor) prepara e imputa el dataset; el Módulo 2 (Auditor) permite inspeccionar coherencia y comportamiento de los modelos y valores completados; y el Módulo 3 (Asistente de diseño) traduce el dataset procesado a un conjunto de entradas compatibles con el análisis conceptual (Imagen 18.4) y genera salidas típicas de dimensionado preliminar (por ejemplo, diagramas de restricciones) (Imagen 18.4). En consecuencia, la contribución del trabajo no se evalúa únicamente por el nivel de completitud alcanzado, sino por su capacidad de convertir datos incompletos en insumos accionables para decisiones tempranas de diseño.

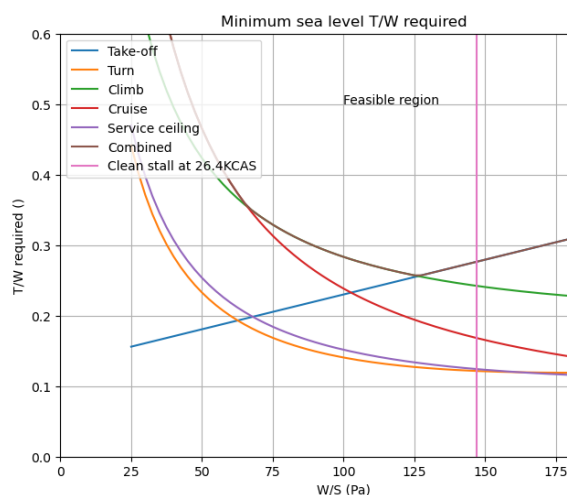


Imagen 18.3 - Diagramas de restricciones ADRpy para una aeronave objetivo.

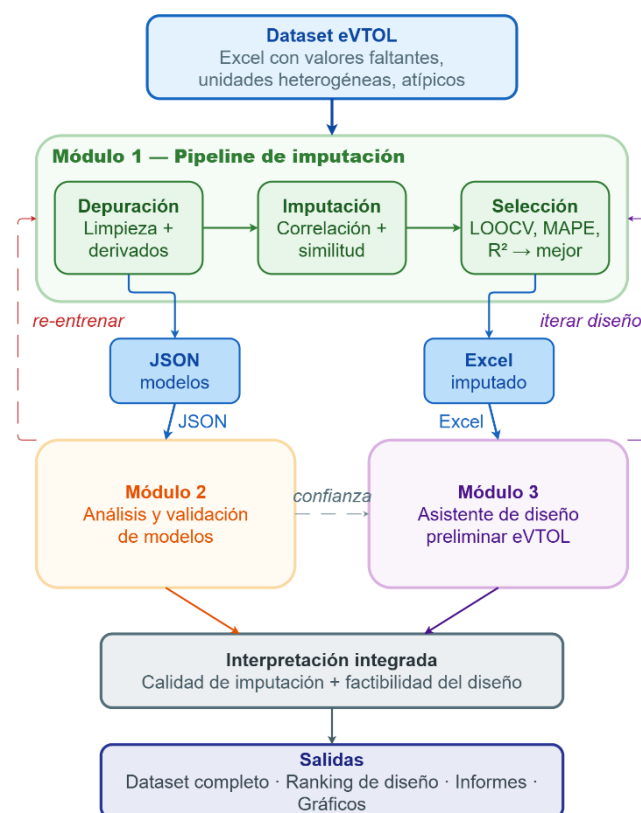


Imagen 18.4 – Arquitectura del sistema de imputación y selección inicial de parámetros para diseño preliminar de eVTOL

2 Contexto y trabajos relacionados

En el diseño conceptual, el uso de aeronaves de referencia cumple un rol doble. Por un lado, permite validar órdenes de magnitud y acotar decisiones de configuración. Por otro, habilita la construcción de correlaciones empíricas o reglas de escalamiento que conectan parámetros geométricos, de masa y desempeño. En consecuencia, la confiabilidad del proceso depende de la calidad del conjunto de referencia: valores inconsistentes o no comparables pueden inducir relaciones espurias y afectar decisiones en etapas tempranas, donde aún no existen modelos detallados que actúen como “control” independiente.

El dominio de UAVs y eVTOL presenta particularidades que complican la creación de bases homogéneas. La rápida evolución tecnológica, la diversidad de configuraciones y la falta de estandarización en la divulgación de especificaciones hacen frecuente la coexistencia de definiciones distintas para un mismo parámetro. Aun cuando un valor esté presente, su comparabilidad puede depender de condiciones no explicitadas. Por ello, un dataset útil para diseño conceptual debe tratar tanto faltantes como coherencia interna, y debería preservar metadatos o criterios de selección que permitan entender qué tan representativa es cada aeronave respecto del conjunto.

²¹ Regresión Supervisada = imputación por correlación

En este escenario, las herramientas computacionales de análisis temprano resultan valiosas porque permiten explorar rápidamente regiones de diseño factible. ADRpy, por ejemplo, ofrece un marco de cálculo que facilita la evaluación conceptual de desempeño y restricciones bajo hipótesis físicas estándar. Este tipo de herramienta tiende a amplificar el efecto de los datos de

entrada: un conjunto coherente habilita análisis reproducibles, mientras que un dataset incompleto fuerza supuestos ad hoc o reduce el alcance de los resultados. Por ello, la integración efectiva de herramientas de diseño conceptual con datasets de referencia requiere un proceso explícito de depuración e imputación que sea verificable.

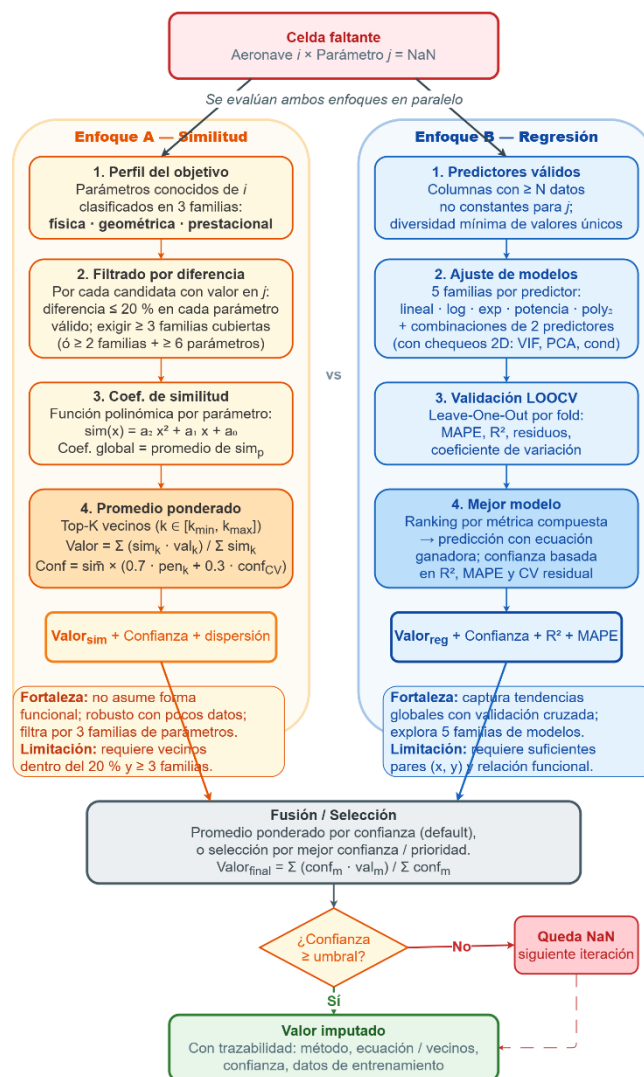
La imputación de datos faltantes puede abordarse mediante enfoques locales o globales. Los métodos basados en similitud (típicamente formalizados como vecinos cercanos) estiman un valor faltante a partir de aeronaves “próximas” en un espacio de características. Su ventaja es que pueden capturar comportamientos diferenciados entre subfamilias de aeronaves sin imponer una relación global única. Sin embargo, su calidad depende de la existencia de vecinos realmente comparables y de una selección adecuada de variables de similitud. Por otra parte, los métodos basados en regresión supervisada buscan aprender una relación predictiva entre parámetros disponibles y el parámetro objetivo. Este enfoque puede producir ecuaciones interpretables y permite evaluar desempeño con métricas estándar; no obstante, es sensible al tamaño efectivo de muestra y al riesgo de sobreajuste, especialmente si se incrementa la complejidad del modelo sin respaldo de datos. (Imagen 18.5)

En aplicaciones de ingeniería, el problema no se agota en producir un número imputado; es necesario además evaluar si la estimación es suficientemente confiable para ser utilizada como entrada de cálculos posteriores. Por esta razón, resulta pertinente combinar métricas de precisión con métricas de capacidad explicativa y penalizaciones por condiciones de ajuste. El uso de un error relativo como MAPE es atractivo porque permite comparar desempeño en variables de distinta escala, aunque debe interpretarse con cuidado cuando los valores reales son pequeños. Complementariamente, R^2 aporta información sobre la variabilidad explicada por el modelo, mitigando escenarios en los que un ajuste reduce error aparente sin capturar estructura. Finalmente, penalizaciones explícitas por tamaño de muestra y complejidad permiten reflejar el hecho de que un modelo con muchos parámetros efectivos y pocos datos suele generalizar peor, incluso si ajusta bien el conjunto de entrenamiento.

En síntesis, el contexto sugiere que una solución orientada al diseño conceptual debe: (i) tratar faltantes con métodos complementarios, (ii) preservar trazabilidad del valor imputado (modelo o vecinos utilizados), y (iii) incorporar un criterio cuantitativo que traduzca desempeño y condiciones del ajuste en un indicador utilizable para filtrar o priorizar revisiones. ADRpy-VTOL se apoya en estas ideas para construir un flujo que transforme un dataset

heterogéneo en una base apta para análisis conceptual reproducible.

En el marco del proceso de diseño, el alcance de ADRpy-VTOL se ubica en la transición entre la recopilación/normalización de datos de referencia y la generación de insumos para dimensionado conceptual. El sistema no pretende reemplazar análisis aerodinámicos o estructurales de etapas preliminares; su aporte se concentra en habilitar que la etapa conceptual opere con entradas



coherentes y trazables, de modo que los resultados posteriores puedan interpretarse con conocimiento explícito sobre la procedencia y confiabilidad de los parámetros utilizados.

Imagen 18.5 – Enfoques de imputación y su rol en el flujo, Similitud vs Regresión

3 Metodología

3.1 Arquitectura del sistema y flujo de información

ADRPy-VTOL se implementa como un flujo de transformación de datos orientado a habilitar análisis de diseño conceptual a partir de especificaciones públicas heterogéneas. El sistema recibe un dataset tabular incompleto, lo depura, genera estimaciones trazables para celdas faltantes y produce un conjunto de entradas utilizable para el análisis conceptual. Para ordenar responsabilidades y reducir ambigüedades, la solución se divide en tres módulos como se mencionó anteriormente en la introducción.

En términos de reproducibilidad, el desarrollo se realizó sobre un entorno Python con control de versiones. La asistencia de herramientas de autocompletado (por ejemplo, VS Code con Copilot) se utilizó como apoyo de implementación, pero el comportamiento del sistema se define por reglas explícitas, validaciones y chequeos internos; es decir, la consistencia metodológica depende de la lógica implementada y no de la herramienta de edición utilizada.

3.2 Construcción y normalización de la base de datos de UAV

El punto de partida es una base tabular donde cada fila representa una aeronave (UAV/eVTOL) y cada columna un parámetro técnico potencialmente relevante para diseño conceptual (por ejemplo, masa máxima al despegue, carga útil, alcance, autonomía, velocidades, geometría alar, potencia instalada). La información proviene de fichas técnicas y fuentes públicas no uniformes, lo que introduce dos dificultades estructurales: (i) valores faltantes y (ii) heterogeneidad (unidades distintas, definiciones no equivalentes y diferente nivel de verificación).

Antes de entrenar modelos o estimar valores se aplica una etapa de preparación orientada a consistencia interna. Primero, se normalizan tipos de datos y unidades a un sistema coherente, y se depuran campos no numéricos o ambiguos cuando no pueden traducirse a magnitudes comparables. Segundo, se habilita el cálculo de campos derivados únicamente cuando existe una identidad física directa y auditable y la celda correspondiente está vacía; por ejemplo, la relación de aspecto a partir de envergadura y cuerda, o coherencias simples entre alcance, autonomía y velocidad cuando están definidas bajo el mismo criterio. La regla de diseño es evitar “inventar” información: si la derivación depende de supuestos no controlados (condiciones de ensayo, perfil de misión, márgenes), el valor se conserva como faltante. Tercero, se preservan atributos categóricos (misión, tipo de despegue, configuración de propulsión) para habilitar filtros por “familia” que reduzcan comparaciones injustas entre plataformas de naturaleza distinta.

3.3 Estrategia general de imputación: modelos locales y enfoque modular

El sistema adopta un principio metodológico central: evitar un único modelo global por parámetro y , en su lugar, generar estimaciones locales adaptadas a cada celda faltante. La motivación de este enfoque es práctica: en un dataset con alta incompletitud, una correlación “válida” en términos estadísticos puede resultar inutilizable para una aeronave particular si el predictor requerido no está disponible en esa fila; por lo tanto, el proceso debe seleccionar, caso a caso, relaciones predictivas que sean simultáneamente aceptables y aplicables con la información efectivamente presente en la aeronave objetivo.

Imputar un valor equivale a formular un problema específico: para una aeronave objetivo A^* y un parámetro faltante y , se construye un conjunto de evidencia con aeronaves donde y está disponible y con variables predictoras X que también están presentes en A^* . Este enfoque reduce dos riesgos frecuentes: entrenar modelos con predictores inexistentes para el caso real y extrapolar sin control sobre regiones del espacio de datos poco representadas.

La imputación se resuelve mediante dos mecanismos complementarios. La imputación por similitud opera por transferencia desde vecinos cercanos en el espacio de parámetros y tiende a ser robusta cuando existe una vecindad comparable. La imputación por correlación (regresión supervisada) busca ecuaciones explícitas a partir de los datos disponibles, priorizando modelos que combinen buen desempeño con evidencia suficiente. En ambos casos, además del valor estimado se conserva una traza que incluye método, soporte del cálculo, métricas y advertencias. Esta traza permite auditar el resultado y evita que el completado se convierta en un proceso opaco.

Para evitar ambigüedades de notación, se utiliza el siguiente convenio mínimo a lo largo del capítulo: A^* es la aeronave objetivo; y el parámetro a imputar; $X = \{x_i\}$ el conjunto de variables disponibles utilizadas para comparación o predicción; m el número efectivo de variables comparadas; K el número de vecinos utilizados; n el tamaño efectivo de muestra; y k la complejidad efectiva del modelo seleccionado.

3.4 Imputación por similitud: vecinos, umbral y confianza

La imputación por similitud se basa en la hipótesis de vecindad: si una aeronave objetivo A^* es suficientemente comparable a otras aeronaves del dataset según variables disponibles, entonces un parámetro faltante y puede estimarse a partir de esas aeronaves vecinas. Para cuantificar comparabilidad se define un conjunto de variables de comparación $\{x_i\}_{i=1}^m$, restringido a aquellas que están presentes tanto en A^* como en la aeronave candidata.

Para cada variable comparable se calcula una diferencia relativa:

$$\Delta_i(\%) = \frac{|x_i - x_i^*|}{\max(|x_i^*|, \varepsilon)} \cdot 100$$

donde x_i es el valor en la aeronave candidata, x_i^* el valor en A^* , y ε es un término pequeño para evitar indeterminaciones cuando x_i^* es cercano a cero. La diferencia Δ_i se transforma en una similitud parcial $s_i \in [0,1]$ mediante una función monótona decreciente $g(\cdot)$:

$$s_i = g(\Delta_i)$$

En términos operativos, $g(\cdot)$ se define para penalizar diferencias grandes y permitir un control explícito del criterio de vecindad mediante un umbral. Las similitudes parciales se agregan en una similitud global:

$$s_{\text{global}} = \frac{1}{m} \sum_{i=1}^m s_i$$

donde m es la cantidad efectiva de variables comparadas para ese par de aeronaves. Se aceptan como candidatos únicamente los vecinos con $s_{\text{global}} \geq s_{\text{min}}$, siendo s_{min} un umbral configurable. Con los vecinos aceptados que además poseen y disponible, se seleccionan los K mejores ($Top - K$).

El valor imputado se calcula mediante promedio ponderado:

$$\hat{y}_{\text{sim}} = \frac{\sum_{j=1}^K w_j y_j}{\sum_{j=1}^K w_j}$$

donde y_j es el valor observado del parámetro objetivo en el vecino j , y w_j es un peso asociado a su similitud global. Se define:

$$w_j = f(s_{\text{global},j})$$

con $f(\cdot)$ una función monótona creciente que mapea $s_{\text{global},j} \in [0,1]$ a un peso no negativo. Esta formulación garantiza que vecinos más similares influyan más en la estimación y permite ajustar la sensibilidad del método mediante la forma de $f(\cdot)$ (por ejemplo, leyes potenciales o funciones exponenciales suavizadas), sin alterar el principio de monotonía.

Además del valor imputado, el método reporta una confianza por similitud $C_{\text{sim}} \in [0,1]$ que resume (i) la calidad del vecindario, reflejada por similitud promedio y/o dispersión de similitudes, y (ii) la evidencia disponible, representada por K . En consecuencia, si el vecindario es escaso o heterogéneo, la confianza disminuye, permitiendo

tratar el valor estimado como una imputación de mayor incertidumbre.

3.5 Imputación por correlación: generación de candidatos y filtros de validez

En una segunda etapa, el sistema intenta imputar mediante correlación, interpretada aquí como regresión supervisada sobre el dataset disponible. Este flujo se ejecuta por celda faltante y se apoya en cuatro ideas: entrenar con un conjunto local de evidencia, evitar extrapolación extrema, generar candidatos de distinta complejidad y filtrar inestabilidades geométricas cuando existen dos predictores.

3.5.1 Construcción del set de entrenamiento local.

Para una celda objetivo (A^*, y) , se construye un set de entrenamiento con aeronaves donde y (parámetro a imputar) está disponible y con predictores X (conjunto de variables disponibles) presentes también en A^* . Adicionalmente, se aplican filtros por “familia” de forma progresiva: primero se intenta con subconjuntos más estrictos (plataformas comparables por misión y configuración) y, solo si la evidencia es insuficiente, se flexibiliza el criterio para ampliar el conjunto, emitiendo una alerta en la salida para informar al usuario. Con ello se busca maximizar comparabilidad sin bloquear la imputación en regiones de baja densidad de datos.

3.5.2 Evitar extrapolación extrema.

Antes de entrenar, se verifica que el valor de cada predictor en A^* no quede fuera del rango observado en el entrenamiento, admitiendo una tolerancia configurable. Si una combinación de predictores fuerza extrapolación excesiva, se descarta para esa imputación concreta. Este control es importante porque, en datasets heterogéneos y pequeños, una extrapolación moderada puede dominar el resultado aun cuando el modelo muestre buen ajuste dentro del rango observado.

3.5.3 Generación de modelos candidatos.

Con el set de entrenamiento definido, se generan modelos candidatos variando la cantidad de predictores (uno o dos) y la familia funcional. Para un predictor se consideran, entre otras, familias lineal, cuadrática, logarítmica, potencia y exponencial; el objetivo es capturar tanto tendencias aproximadamente lineales como curvaturas simples sin introducir complejidad innecesaria. Para dos predictores se consideran un plano lineal y un polinomio de segundo grado, incluyendo términos cuadráticos e interacción, de forma general:

$$y = a_0 + a_1 x_1 + a_2 x_2 ;$$

$$y = a_0 + a_1 x_1 + a_2 x_2 + a_3 x_1^2 + a_4 x_2^2 + a_5 x_1 x_2$$

La exploración se restringe a combinaciones donde exista un número mínimo de muestras efectivas, de modo que el ajuste sea estadísticamente defendible y no dependa de pocos puntos.

3.5.4 Filtros de diversidad y estabilidad en 2D (cuando hay dos predictores).

Los modelos con dos predictores pueden fallar aun cuando muestren un error bajo en entrenamiento debido a razones geométricas: nube de puntos casi alineada, multicolinealidad fuerte o cobertura pobre del espacio de entrenamiento. Por ello, antes de comparar candidatos por métricas se calculan indicadores de estabilidad y se descartan combinaciones inestables. En términos prácticos, se controlan señales de colinealidad (por ejemplo, correlaciones elevadas o VIF alto), degeneración dimensional (por ejemplo, dominancia de un componente principal) y condición numérica. Complementariamente, se evalúa diversidad/coertura mediante medidas simples (dispersión, unicidad de pares y aproximaciones de cobertura del dominio). El objetivo de estos filtros no es maximizar la complejidad, sino evitar superficies que “parecen buenas” por memorizar pocos puntos y producen comportamientos físicamente inverosímiles fuera de la nube observada.

3.6 Métricas y selección: MAPE, R^2 , robustez LOOCV y confianza final

Para comparar modelos candidatos se calculan métricas de desempeño y se construye un indicador de confianza. Se utiliza el MAPE como medida de error relativo medio y el coeficiente de determinación R^2 como medida de capacidad explicativa. Para combinarlas en una escala común se definen términos normalizados en $[0,1]$:

$$\begin{aligned} mape_{\text{term}} &= \max\left(0, 1 - \frac{MAPE}{MAPE_{\text{ref}}}\right), r2_{\text{term}} \\ &= \max(0, \min(1, R^2)) \end{aligned}$$

donde $MAPE_{\text{ref}}$ es un umbral de referencia configurable que fija el punto a partir del cual el desempeño se considera insuficiente.

Con estos términos se define una base de calidad:

$$base = w_{R^2} r2_{\text{term}} + w_{MAPE} mape_{\text{term}}$$

con w_{R^2} y w_{MAPE} pesos configurables (típicamente $w_{R^2} + w_{MAPE} = 1$). Sin embargo, en datasets pequeños un buen ajuste puede ser engañoso si se sostiene con poca evidencia o con un modelo demasiado flexible. Por ello, la base se penaliza por complejidad y tamaño de muestra:

$$conf_{\text{intermedia}} = base \cdot pen_k(k) \cdot pen_n(n)$$

donde k representa la complejidad efectiva del modelo (número de parámetros efectivos), n es el tamaño efectivo de

muestra disponible para entrenar/validar en esa imputación, y $pen_k(\cdot)$, $pen_n(\cdot) \in [0,1]$ son funciones monótonas que penalizan modelos complejos con pocos datos.

Para modelos con dos predictores se incorpora además una penalización geométrica agregada:

$$conf_{\text{intermedia}} \leftarrow conf_{\text{intermedia}} \cdot pen_{2D}$$

donde $pen_{2D} \in [0,1]$ resume los chequeos de estabilidad y cobertura del conjunto de entrenamiento en 2D. Finalmente, la consistencia del modelo se ajusta con validación LOOCV (leave-one-out cross-validation). En lugar de usar LOOCV solo como métrica adicional, se clasifica el candidato en una categoría de robustez y se aplica un factor $f_{\text{LOOCV}} \in [0,1]$ dependiente de esa clase:

$$\begin{aligned} Confianza_{\text{final}} &= conf_{\text{intermedia}} \\ &\cdot ((1 - w_{\text{LOOCV}}) + w_{\text{LOOCV}} \cdot f_{\text{LOOCV}}) \end{aligned}$$

donde $w_{\text{LOOCV}} \in [0,1]$ controla cuánto influye la robustez LOOCV en la confianza final. La confianza resultante se utiliza como criterio de aceptación mínima y también como peso cuando se combinan estimaciones provenientes de métodos distintos.

3.7 Bucle de imputación y combinación de métodos

El sistema no imputa una sola vez, sino que ejecuta un bucle iterativo sobre una copia de la base. En cada iteración, los métodos de similitud y correlación proponen valores para distintas celdas; el bucle decide qué valores se escriben finalmente, actualiza el dataset y recalcula disponibilidad de evidencia para la siguiente iteración. Este comportamiento es importante porque imputaciones tempranas pueden habilitar predictores o variables comparables que no estaban disponibles al inicio.

Cuando ambos métodos proponen un valor para la misma celda, se aplica una política de fusión basada en confianza. Una opción típica es el promedio ponderado:

$$\hat{y} = \frac{C_{\text{sim}} \hat{y}_{\text{sim}} + C_{\text{corr}} \hat{y}_{\text{corr}}}{C_{\text{sim}} + C_{\text{corr}}}$$

donde C_{sim} y C_{corr} son las confianzas asociadas a cada método. Si solo uno de los métodos produce una estimación aceptable, se adopta ese valor y su traza correspondiente. Si ninguno supera los criterios mínimos (por falta de evidencia, extrapolación o inestabilidad), la celda se conserva como faltante.

El bucle incorpora criterios de parada (máximo de iteraciones y “iteraciones sin mejora”) para evitar completar artificialmente con imputaciones cada vez menos sustentadas. Al finalizar, el módulo exporta un dataset imputado con marcado de celdas estimadas y genera trazas estructuradas (por ejemplo, en JSON) que almacenan, por

celda, el valor adoptado, el método, la confianza, la ecuación o los vecinos utilizados y las advertencias detectadas. El flujo completo se considera satisfactorio cuando el dataset resultante permite construir el vector de entrada del caso de estudio y ejecutar el análisis conceptual sin recurrir a completados manuales no documentados, manteniendo trazabilidad sobre el origen de cada parámetro.

3.8 Auditoría y control de coherencia (Módulo 2)

Una vez generado el dataset imputado por el Módulo 1, se ejecuta una etapa de auditoría destinada a verificar coherencia y a reducir el riesgo de adoptar valores completados que, aun siendo numéricamente plausibles, estén sustentados por evidencia débil o impliquen tendencias no razonables. Esta auditoría utiliza como insumo tanto la tabla final como las trazas por celda (método aplicado, vecinos o ecuación, métricas y confianza), de manera que cada imputación pueda interpretarse en función de su soporte.

El proceso de auditoría se organiza en dos niveles. En el nivel global, se inspeccionan distribuciones, rangos típicos y outliers por parámetro, identificando valores que requieren revisión por exceder límites físicos o por situarse en colas extremas coherentes con subpoblaciones específicas. En el nivel por imputación, se evalúa la consistencia del valor estimado con el comportamiento de los datos que lo sustentan. Para imputaciones por similitud, se analiza la calidad del vecindario (similitud agregada y dispersión entre vecinos) y se detectan casos donde el valor imputado queda dominado por uno o pocos vecinos. Para imputaciones por correlación, se inspecciona el ajuste en relación con la nube de puntos, la cobertura efectiva del conjunto de entrenamiento y la estabilidad geométrica cuando existen dos predictores. En particular, la auditoría permite identificar modelos con métricas aceptables, pero con superficies inestables o dependientes de regiones estrechas del dominio.

Como salida, el Módulo 2 produce un estado de control por imputación que permite clasificar valores en tres categorías operativas: imputaciones aceptables, imputaciones que deben priorizarse para revisión (por ejemplo, confianza baja o advertencias de extrapolación) e imputaciones que deben rechazarse o mantenerse como faltantes por falta de evidencia suficiente. Esta etapa no modifica la naturaleza del método de imputación, pero sí establece un mecanismo explícito para separar completitud de confiabilidad, requisito necesario cuando se busca utilizar el dataset como soporte de decisiones de diseño.

Imagen 18.6 – Dashboard del Módulo 2: filtros de selección, gráfico principal, resumen de modelos/métricas y panel de detalle

3.9 Asistente de diseño conceptual



(Módulo 3)

El flujo metodológico culmina con la traducción del dataset procesado a un conjunto de entradas directamente utilizable en diseño conceptual. El Módulo 3 (Asistente de diseño) recibe el dataset auditado y construye un vector de parámetros compatible con el análisis conceptual, actuando como interfaz entre datos de referencia y la ejecución del dimensionado preliminar. Esta etapa es clave porque define el criterio práctico de finalización del proceso: el pipeline se considera satisfactorio cuando permite ejecutar el análisis conceptual sin recurrir a completados manuales no documentados, manteniendo trazabilidad sobre el origen de los parámetros utilizados.

La construcción del vector se realiza bajo reglas de prioridad que preservan interpretabilidad. Cuando un parámetro requerido está disponible de forma directa en el dataset, se utiliza ese valor. Si el valor no está disponible, pero existe una relación física directa y auditable dentro del marco de hipótesis asumido, se utiliza un valor derivado. Si el parámetro no puede derivarse, se recurre a imputaciones; en ese caso, el valor se incorpora junto con su confianza y con el registro del método que lo sustenta (similitud o correlación). Si la confianza es insuficiente o la auditoría marcó la imputación como frágil, el sistema puede optar por mantener el parámetro como faltante o adoptar un supuesto conservador documentado, evitando que el análisis conceptual se apoye implícitamente en estimaciones de baja evidencia sin que el lector o el diseñador lo adviertan.

Una vez construido el vector, el asistente ejecuta el análisis conceptual y genera una salida verificable que sintetiza el resultado del flujo. En configuraciones típicas, esta salida corresponde a un diagrama de restricciones u otra representación equivalente del espacio de diseño, que permite identificar regiones factibles y comprender la sensibilidad del diseño a los parámetros de entrada. La contribución metodológica de esta etapa no es únicamente producir la gráfica final, sino hacerlo con un vector cuya procedencia es trazable: cada parámetro queda etiquetado según su origen (directo, derivado, imputado o supuesto) y, cuando corresponde, acompañado por su confianza. Esto habilita una lectura crítica del resultado: si una conclusión de

diseño depende de parámetros imputados con baja confianza, el flujo lo expone y orienta iteraciones posteriores hacia la mejora dirigida del dataset o la revisión de supuestos.

4 Resultados

4.1 Diagnóstico inicial del dataset: completitud y parámetros críticos

El primer resultado del trabajo es el diagnóstico cuantitativo del estado inicial del dataset. Esta evaluación es necesaria porque condiciona el alcance real de cualquier imputación: no todos los parámetros pueden completarse con el mismo nivel de evidencia, y existen variables cuya baja cobertura obliga a adoptar un comportamiento conservador (rechazo de imputación o confianza baja). En consecuencia, antes de aplicar métodos predictivos se calcula la completitud por columna, identificando parámetros “ancla” (de alta disponibilidad) y parámetros de cobertura limitada.

En la base utilizada se registran 83 aeronaves y un conjunto de parámetros técnicos que combinan variables de misión, masas, geometría, desempeño y potencia instalada. El análisis de completitud muestra un patrón típico en este dominio: algunos parámetros estructurales y de clasificación presentan alta disponibilidad, mientras que otros (frecuentemente asociados a desempeño detallado o a condiciones específicas de operación) exhiben vacíos significativos (Tabla 18.1). Esta distribución asimétrica justifica la estrategia metodológica del sistema: imputación contextual por celda, filtros por familia y mecanismos explícitos de confianza, en lugar de un completado indiscriminado.

PARÁMETRO	% COMPLETADO
BATTERY WEIGHT	1,20%
MAXIMUM CROSSWIND (RACHAS)	2,41%
RADIO DE GIRO	3,61%
TIEMPO DE EMERGENCIA EN VUELO	4,82%
POTENCIA HP	7,23%
CUERDA	8,43%
RELACIÓN DE ASPECTO DEL ALA	9,64%
ANCHO DEL FUSELAJE	12,05%
PRECIO	18,07%
PESO VACIO (MTOW- PAYLOAD)	24,10%
VELOCIDAD MÁXIMA (M/S IAS)	57,83%
TECHO DE SERVICIO MÁXIMO	63,86%
PAYLOAD	86,75%
ENVERGADURA	90,36%
AUTONOMÍA DE LA AERONAVE (H)	95,18%
PESO MÁXIMO AL DESPEGUE	100,00%
DESPEGUE	100,00%

Tabla 18.1 – Tabla parcial de completitud de celdas.

Para documentar el punto de partida y facilitar reproducibilidad, se reportan: (i) número de aeronaves N , (ii) número total de parámetros P , (iii) completitud global inicial C_0 , y (iv) completitud por parámetro. Este diagnóstico también permite priorizar los parámetros que aportan mayor valor en análisis conceptual, en particular aquellos que habilitan el vector de entrada para dimensionado preliminar.

4.2 Heterogeneidad y outliers: evidencia de la necesidad de filtrado y auditoría

El segundo resultado es la caracterización de la heterogeneidad del dataset, particularmente la presencia de valores extremos asociados a plataformas muy disímiles. En mercados de UAV/eVTOL, conviven aeronaves de misión y escala diferentes, por lo que ciertas variables presentan colas largas o multimodalidad. Si no se aplican filtros o segmentación, estos extremos pueden dominar regresiones y generar tendencias numéricamente ajustadas, pero conceptualmente inadecuadas para una familia específica.

Como ejemplo representativo, la variable alcance muestra un rango amplio en el dataset. En el resumen global se observan valores aproximadamente entre 25 km y 3270 km, con una mediana cercana a 365 km. Al aplicar filtrado por IQR, el rango central queda aproximadamente entre 216 km y 500 km, con mediana cercana a 308 km, y un valor de referencia del orden de 306 km como guía de orden de magnitud. Este comportamiento evidencia que el dataset contiene subpoblaciones distintas y que, en ausencia de filtros, un modelo global podría mezclar fenómenos no equivalentes.

Este resultado refuerza la decisión de incorporar el Módulo 2 (Auditor) como mecanismo de control: aun con métricas favorables, un modelo puede ser engañoso si se entrenó sobre datos dominados por valores extremos o por cobertura parcial. La auditoría permite inspeccionar distribuciones, outliers y consistencia de tendencias antes de utilizar relaciones como soporte de diseño.

4.3 Resultados del Módulo 1 (Predictor): imputación trazable y controlada por evidencia

El Módulo 1 produce como salida un dataset completado junto con trazas por imputación. El resultado relevante no es únicamente el aumento de completitud, sino la posibilidad de auditar cada valor imputado mediante: (i) método utilizado (similitud, correlación o combinación), (ii) soporte del cálculo (vecinos o ecuación), (iii) métricas asociadas (por ejemplo, MAPE y R^2 en regresión), (iv) advertencias (extrapolación, evidencia limitada, inestabilidad 2D), y (v) un indicador de confianza final.

En la imputación por similitud, la evidencia se basa en la existencia de vecinos suficientemente comparables para la

aeronave objetivo y el parámetro faltante. El sistema registra el conjunto de vecinos seleccionados ($Top - K$), la similitud agregada y la coherencia local de la vecindad. En la imputación por correlación, el sistema registra la familia funcional seleccionada, la ecuación final, el tamaño efectivo de muestra y el desempeño del ajuste. En ambos casos, la confianza permite reflejar la calidad esperada de la imputación y diferenciar completados robustos de completados frágiles.

Un hallazgo operativo observado durante la generación de modelos candidatos es la aparición de ajustes con métricas aparentemente buenas, pero con comportamiento problemático fuera del rango central de datos, particularmente en modelos polinómicos o en superficies 2D entrenadas con cobertura geométrica pobre. Este punto es crítico: sin controles adicionales, el completado podría introducir tendencias no plausibles y afectar decisiones de diseño conceptual. Por ello, el Predictor se concibe como un generador de estimaciones auditables (Imagen 18.7), no como un mecanismo de completado ciego.

0,212	6.5	75.0	2.0	25.6	Mean	N
0,258	=== IMPUTED VALUE ===					
Nan	Aeronave: ScanEagle					
Nan	Parámetro: Ancho del fuselaje					
Nan	Valor imputado: 0.26					
0,222	Confianza: 0.00					
0,265	Método predictivo: Correlacion					
0,271	Iteración imputación: 1.00					
Nan	variable_independiente_1: 18.00					
Nan	=== CORRELATION DETAILS ===					
Nan	Aeronave: ScanEagle					
Nan	Parámetro: Ancho del fuselaje					
0,287	Valor imputado: 0.26					
0,287	Confianza: 0.00					
0,287	Tipo Modelo: log-1					
0,287	Predictores: Autonomía de la aeronave (h)					
0,287	Coeficientes: [0.05]					
0,287	Peso de predictores: [1.00]					
0,287	Aeronaves entrenamiento: [Stalker XE, Stalker VXE30 Fuel Cell, Stalker Battery, Aerosonde Mk. 4.7 Fixed Wing, Aerosonde Mk. 4.8 Fixed wing]					
0,195	k: 5.00					
0,208	Penalización_k: 0.27					
0,243	Corr: 0.72					
0,242	MAPE: 6.12					
0,375	R2: 0.84					
0,375	Confianza_LOOCV: 0.22					
0,375	k_LOOCV: 5.00					
0,375	Corr_LOOCV: 0.81					
Nan	MAPE_LOOCV: 4.08					
0,26	R2_LOOCV: 0.90					
0,26	Método predictivo: Correlacion					
0,306	Advertencia: Modelo robusto					
Nan	Outliers_duros_removidos: 0.00					
Nan	Outliers_suaves_pesados: 4.00					
Nan	Familia: F0					
Nan	Iteración imputación: 1.00					
Nan	variable_independiente_1: 18.00					

Imagen 18.7 – Comentario generado en celdas imputadas para auditar la lógica y veracidad del valor.

4.4 Resultados del Módulo 2 (Audit): verificación de coherencia y detección de modelos engañosos

Durante la ejecución del pipeline, la auditoría permitió identificar casos en los que una imputación o un modelo candidato presentaba métricas aceptables, pero mostraba señales de fragilidad al inspeccionar la evidencia que lo

sustentaba. Se observaron escenarios donde el ajuste dependía de pocos puntos efectivos, donde la región de entrenamiento era estrecha respecto del dominio de interés, o donde la cobertura geométrica en modelos con dos predictores resultaba insuficiente, aumentando el riesgo de obtener superficies “buenas” en el rango central pero poco confiables fuera de él.

Estas situaciones se registraron como advertencias y/o degradación de confianza, de modo que su impacto sobre el análisis conceptual posterior pudiera interpretarse explícitamente. En términos de resultado, el aporte del módulo se refleja en su capacidad de separar completados aceptables de completados que requieren revisión adicional antes de ser utilizados como soporte de decisiones de diseño.

4.5 Resultados del Módulo 3 (Design Assistant): integración con diseño conceptual y salida final utilizable

El resultado de mayor utilidad práctica del sistema se obtiene en el Módulo 3, donde el dataset procesado se transforma en un insumo directamente utilizable para diseño conceptual. Este módulo implementa el puente entre datos de mercado y el proceso de dimensionado preliminar: a partir de una aeronave objetivo o un conjunto de requisitos de misión, construye un vector de parámetros compatible con el esquema de entrada requerido por la herramienta de análisis conceptual (por ejemplo, ADRpy) y permite ejecutar el análisis sin depender de completados manuales no reproducibles.

4.5.1 Construcción del vector de entrada y trazabilidad de origen

El vector de entrada se organiza en bloques coherentes con el flujo conceptual: requerimientos (*brief*), variables de configuración (*design*) y parámetros de desempeño/aerodinámica (*performance/aero*). Para cada parámetro del vector, se documenta el origen, distinguiendo entre: (i) valor directo del dataset, (ii) valor derivado por relación física, (iii) valor imputado por similitud o por correlación (con su confianza), y (iv) supuesto conservador documentado cuando no existe evidencia suficiente. Esta clasificación es esencial para evitar que el análisis conceptual oculte incertidumbre: el objetivo no es eliminar supuestos, sino hacerlos explícitos, trazables y revisables en función de su impacto sobre la salida del análisis.

4.5.2 Ejecución del análisis conceptual y generación de la salida final

Una vez construido el vector, el módulo ejecuta el análisis conceptual y produce una salida verificable en formato gráfico, típicamente un diagrama de restricciones (por ejemplo, T/W vs W/S , o un equivalente adecuado a la configuración) (Imagen 18.3). Esta figura constituye el cierre natural del trabajo, ya que demuestra que el pipeline no solo completa datos, sino que habilita la etapa donde esos datos

se vuelven útiles: delimitar regiones factibles, comparar configuraciones y apoyar decisiones tempranas.

El valor de esta salida no se limita a ejecutar el análisis conceptual (ADRpy). Su importancia está en que el análisis se ejecuta con un conjunto de entradas construido de forma reproducible, con trazabilidad sobre el origen de cada parámetro y con un indicador de confianza asociado a los valores imputados. En consecuencia, el resultado final puede interpretarse junto con su nivel de robustez: si el diagrama depende fuertemente de parámetros imputados con baja confianza, el sistema lo hace visible y habilita iteraciones dirigidas (mejorar datos específicos o revisar supuestos).

5 Discusión

Los resultados obtenidos confirman que, en dominios donde la información técnica proviene de fuentes no uniformes, la principal dificultad no es únicamente la ausencia de valores, sino la combinación de incompletitud con heterogeneidad conceptual. En este marco, la elección de un flujo que integra depuración, imputación y auditoría resulta determinante para evitar que el completado de celdas se convierta en un mecanismo de propagación de errores. En particular, el comportamiento observado durante la generación de modelos evidencia que, si no se imponen restricciones explícitas sobre evidencia disponible y estabilidad geométrica, es posible obtener ajustes con métricas aparentemente favorables que, sin embargo, no generalizan y producen relaciones físicamente poco plausibles.

La utilización de dos estrategias complementarias (similitud y regresión) permite balancear robustez e interpretabilidad. La imputación por similitud tiende a ser adecuada cuando el dataset contiene subfamilias comparables y cuando la aeronave objetivo se ubica en una región densamente poblada del espacio de parámetros. En esos casos, el método opera como una transferencia de conocimiento local que reduce la dependencia de hipótesis funcionales globales. No obstante, su desempeño se degrada cuando la vecindad disponible es pobre o heterogénea, situación en la cual la confianza asociada debe disminuir para reflejar la debilidad de la evidencia. Por su parte, la imputación por regresión aporta ecuaciones explícitas y escalables, pero es más sensible a la cantidad efectiva de datos y a la colinealidad entre predictores, especialmente en modelos de dos variables. El uso de filtros previos (diversidad, cobertura y estabilidad) constituye, en consecuencia, un componente crítico del enfoque propuesto, ya que permite descartar candidatos que pueden ajustarse por casualidad o por degeneración geométrica del entrenamiento.

La estrategia de evaluación basada en MAPE y R^2 responde a la necesidad de considerar simultáneamente precisión e interpretabilidad. En conjuntos pequeños, el MAPE puede inducir una percepción optimista del desempeño cuando la variable objetivo presenta variación limitada o cuando los valores reales son cercanos a cero; por ello, el uso de R^2 como medida complementaria contribuye a

detectar casos en los que el modelo explica poco aun si el error relativo promedio es bajo. Aun así, ninguna métrica aislada garantiza validez en ingeniería. La decisión de sintetizar el desempeño mediante un indicador de confianza penalizado por complejidad y tamaño de muestra se justifica por un principio simple: un modelo complejo con evidencia limitada debe considerarse menos confiable que un modelo más simple respaldado por mayor cobertura, incluso si ambos producen un valor puntual.

Desde el punto de vista de la trazabilidad, una ventaja práctica de ADRpy-VTOL es que preserva el “rastreo” de cada imputación: método aplicado, ecuación o vecinos, y métricas. Este diseño favorece la auditabilidad y habilita una interacción más segura con el dataset, en contraste con enfoques donde el completado ocurre de forma silenciosa. No obstante, deben reconocerse limitaciones inherentes. En primer lugar, la imputación no resuelve por sí misma la falta de estandarización en definiciones: un valor completado puede ser numéricamente coherente y, aun así, no ser estrictamente comparable si la fuente original reporta bajo condiciones distintas. En segundo lugar, la representatividad del dataset condiciona la validez de las inferencias; si un segmento tecnológico está subrepresentado, el sistema tenderá a extrapolar o a rechazar imputaciones (según configuración del usuario), y esa conducta es preferible a forzar completitud. Finalmente, el indicador de confianza propuesto es una síntesis operativa; su calibración depende de umbrales y funciones de penalización configurables, por lo que su interpretación debe contextualizarse como un mecanismo de control relativo dentro del flujo, más que como una probabilidad absoluta.

En síntesis, el enfoque es especialmente útil cuando el objetivo es construir una base utilizable para diseño conceptual, donde la prioridad es disponer de parámetros consistentes y trazables para alimentar análisis preliminares. En este contexto, el sistema contribuye a formalizar una práctica frecuente (“completar y corregir” datos de mercado) con criterios reproducibles, cuantitativos y auditables.

6 Conclusiones y trabajo futuro

En este trabajo se presentó ADRpy-VTOL, un sistema modular en Python orientado a apoyar etapas tempranas del diseño conceptual de un UAV con capacidad VTOL, mediante un flujo reproducible que conecta datos de mercado con análisis conceptual. El problema abordado no se limita a la presencia de valores faltantes, sino a la coexistencia de incompletitud, heterogeneidad y comparabilidad imperfecta en las especificaciones públicas disponibles. En este contexto, el principal aporte no es “completar un dataset”, sino proponer un mecanismo para convertir información dispersa en un insumo de diseño que conserve trazabilidad y permita interpretar resultados con criterio de ingeniería.

Los resultados sostienen que el enfoque modular es adecuado para este dominio por dos razones complementarias. En primer lugar, la imputación deja de ser

un procedimiento aislado y se transforma en una etapa subordinada a una finalidad de diseño: alimentar un vector de entrada y habilitar un análisis conceptual verificable. En segundo lugar, la metodología evita que la “completitud” se convierta en un objetivo en sí mismo. En datasets pequeños y heterogéneos, completar por completar aumenta el riesgo de introducir relaciones espurias; por ello, la incorporación de criterios de aceptación y penalizaciones por evidencia disponible permite un comportamiento más conservador, donde el sistema puede degradar confianza o abstenerse de imputar cuando la evidencia es insuficiente.

Se identifican tres contribuciones técnicas principales. La primera es la formalización de un proceso de imputación contextual por celda, donde el conjunto de entrenamiento y los predictores se definen en función de la disponibilidad real para la aeronave objetivo. Esta decisión reduce incompatibilidades frecuentes de enfoques globales (modelos entrenados con variables ausentes para el caso objetivo) y acota el riesgo de extrapolación. La segunda contribución es el diseño de un indicador de confianza que integra desempeño del ajuste con penalizaciones por complejidad y tamaño efectivo de muestra, y que puede incorporar factores adicionales cuando se ajustan superficies con dos predictores. Esta confianza no se interpreta como una probabilidad absoluta, sino como una medida operativa que permite jerarquizar imputaciones, filtrar estimaciones frágiles y transparentar la calidad de soporte disponible. La tercera contribución, y la de mayor impacto práctico, es la materialización de la utilidad del pipeline en el Módulo 3, donde el dataset procesado se transforma en un vector de entrada compatible con el análisis conceptual y se obtiene una salida final (por ejemplo, un diagrama de restricciones) que actúa como evidencia verificable de que el flujo habilita decisiones tempranas sin recurrir a completados manuales no trazables.

Desde una perspectiva de ingeniería, un hallazgo relevante es que métricas agregadas pueden ser insuficientes para garantizar validez conceptual si el conjunto de datos que sostiene un modelo es pequeño, sesgado o geoméricamente degenerado. En particular, en este tipo de datasets es posible obtener ajustes aparentemente “buenos” que dependen de pocos puntos o de una cobertura pobre del espacio de entrenamiento, lo que puede inducir tendencias físicamente inverosímiles cuando se interpretan como reglas generales. Esta observación justifica la existencia del Módulo 2 como mecanismo de control y verificación, aunque no constituya el resultado final del sistema: su función es asegurar que las relaciones aprendidas o transferidas no se adopten de forma acrítica. En otras palabras, el flujo propuesto promueve una separación explícita entre “estimación” y “uso para diseño”, y esa separación es especialmente valiosa cuando los datos de entrada provienen de fuentes no estandarizadas.

Asimismo, el trabajo permite concluir que la utilidad del sistema no depende únicamente del porcentaje de completitud alcanzado, sino de la capacidad de documentar el origen de cada parámetro utilizado en el análisis conceptual (directo, derivado, imputado o supuesto). Esta trazabilidad aporta una ventaja interpretativa: el resultado

del dimensionado conceptual puede discutirse junto con el nivel de robustez de sus entradas. Si una región de diseño factible depende de manera dominante de parámetros imputados con confianza baja, el flujo lo hace visible y orienta la siguiente iteración hacia una mejora dirigida del dataset o una revisión del supuesto asociado. Esto convierte al sistema en una herramienta no solo de automatización, sino de gestión explícita de incertidumbre en etapas tempranas.

No obstante, también se identifican limitaciones inherentes al problema. Primero, la imputación no puede resolver por sí sola la falta de estandarización en definiciones y condiciones de medición: un valor completo puede seguir siendo conceptualmente no comparable si proviene de supuestos distintos (por ejemplo, autonomía con cargas útiles diferentes). Segundo, la representatividad del dataset condiciona la generalidad de los modelos: si ciertas configuraciones están subrepresentadas, el sistema tenderá a operar con mayor conservadurismo, ya sea degradando confianza o rechazando imputaciones. Este comportamiento se considera deseable en términos de seguridad metodológica, pero implica que la mejora del dataset es una condición necesaria para fortalecer el alcance predictivo del enfoque. Tercero, el indicador de confianza, aunque útil como criterio operativo, requiere calibración y validación adicional si se desea interpretarlo de manera más “absoluta” o transferible a otros datasets.

Como líneas de trabajo futuro se propone: (a) ampliar y estandarizar el dataset incorporando metadatos sobre condiciones de medición y criterios explícitos de comparabilidad; (b) complementar la confianza con cuantificación explícita de incertidumbre (por ejemplo, intervalos de predicción) para que el análisis conceptual pueda incorporar bandas y no solo valores puntuales; (c) incorporar segmentación automática de familias de aeronaves para mejorar la pertinencia de vecinos y reducir sesgos en regresión; y (d) propagar incertidumbres hacia análisis de sensibilidad en dimensionado conceptual, de modo que el resultado final no sea solo una región factible, sino también una caracterización de qué parámetros dominan la incertidumbre del diseño. Estas extensiones reforzarían el rol de ADRpy-VTOL como puente entre datos de mercado y decisiones tempranas de diseño, manteniendo un equilibrio entre automatización, trazabilidad y criterio de ingeniería.

19 BIBLIOGRAFÍA & RECURSOS

- 1] D. P. Raymer, *A Conceptual Approach* (Seventh Edition), American Institute of Aeronautics and Astronautics (AIAA), 2024.
- 2] R. Iten, T. Metger, H. Wilming, L. d. Rio y R. Renner, «Discovering Physical Concepts with Neural Networks,» *Physical Review Letters*, p. 010508, 2020.
- 3] M. Cranmer, A. Sanchez-Gonzalez, P. Battaglia, R. Xu, K. Cranmer, D. Spergel y S. Ho, «arxiv.org,» 18 noviembre 2020. [En línea]. Available: <https://arxiv.org/abs/2006.11287>.
- 4] S. J. Wetzel, S. Ha, R. Iten, M. Klopotek y Z. Liu, «arxiv.org,» 1 abril 2025. [En línea]. Available: <https://arxiv.org/abs/2503.23616>.
- 5] E. Buckingham, «On physically similar systems; illustrations of the use of dimensional equations,» *Physical Review*, p. 345–376, 1914.
- 6] A. A. Sonin, «The Physical Basis of Dimensional Analysis (Second Edition),» Department of Mechanical Engineering, MIT, Cambridge, MA, 2001.
- 7] A. Sobester, «ADRP: Aircraft Design Recipes in Python,» GitHub, 26 01 2024. [En línea]. Available: <https://github.com/sobester/ADRP>. [Último acceso: 15 Febrero 2026].
- 8] A. J. Keane, A. Sóbester y J. P. Scanlan, *Small Unmanned Fixed-wing Aircraft Design: A Practical Approach*, John Wiley & Sons, 2017.
- 9] M. Hu, «graphs of overfitted and underfitted model (Figure 3),» ResearchGate, 15 Marzo 2024. [En línea]. Available: https://www.researchgate.net/figure/graphs-of-overfitted-and-underfitted-model_fig2_379846879. [Último acceso: 15 Febrero 2026].
- 10] Alti, «Instagram oficial profile,» Instagram, [En línea]. Available: <https://www.instagram.com/p/DEzpFk5pnIA/>. [Último acceso: 15 Febrero 2026].
- 11] «Eli Airborne Solutions,» [En línea]. Available: <https://www.uav.ee/products/pneumatic-catapult-pl-80/>. [Último acceso: 1 Febrero 2026].
- 12] J. Rosa, «Jhoan Rosa,» [En línea]. Available: <https://www.johan-rosa.com/post/nota-metodologica-random-forest-i/>. [Último acceso: 04 Febrero 2026].
- 13] S. Ahmed, B. Raza, L. Hussain, T. Sadiq y A. K. Dutta, «Enhancing multiclass COVID-19 prediction with ESN-MDFS: Extreme smart network using mean dropout feature selection technique,» *PLOS ONE*, vol. 19, nº 11, 2024.
- 14] ioDraw, «ioDraw,» [En línea]. Available: <https://www.iodraw.com/es/template/nerve>. [Último acceso: 06 Febrero 2026].
- 15] J. L. Devore, *Probabilidad y estadística para ingeniería y ciencias*, Cengage Learning, 2016.

- D. C. Montgomery y G. C. Runger, Probabilidad y estadística aplicadas a la ingeniería,
16] Wiley, 2003.
- T. S. Corporation, «Aerosonde® Mk. 4.7 VTOL UAS (Maritime),» 2024. [En línea]. Available:
17] https://www.textronsystems.com/sites/default/files/_documents/001_AIR%20SYSTEMS_Aerosonde%20Mk.%204.7%20VTOL%20UAS_Maritime_2024.pdf.
- Textron Systems Corporation, «Aerosonde® UAS (Aerosonde HQ – Maritime Brochure),»
18] 2023. [En línea]. Available:
https://www.textronsystems.com/sites/default/files/_documents/Aerosonde%20HQ%20Maritime%20Brochure.pdf.
- Textron Systems Corporation, «Aerosonde® VTOL FTUAS,» 2023. [En línea]. Available:
19] https://www.textronsystems.com/sites/default/files/_documents/001_AIR%20SYSTEMS_Aerosonde_VTOL%20FTUAS_2024_SINGLE%20DIGITAL_V1.pdf.
- Textron Systems Corporation, «Improved Shadow Tactical Unmanned Aircraft System
20] (TUAS),» 2022. [En línea]. Available:
https://www.textronsystems.com/sites/default/files/_documents/001_AIR%20SYSTEMS_Improved%20Shadow_%202022_SINGLE_DIGITAL.pdf.
- Atmos, «Marlyn Cobalt Specifications (V2.4, December 2023),» 2023. [En línea]. Available:
21] https://assets-global.website-files.com/602fd111547334bf67fc7ead/65bada4e39ee52376f7b254f_Marlyn_Cobalt_Specs_2.4_EN.pdf.
- AeroVironment, Inc., «Puma™ 3AE (Datasheet),» 2025. [En línea]. Available:
22] https://www.avinc.com/images/uploads/product_docs/2025_Puma3AE_Datasheet_250707_v08.pdf.
- Aeronautics Ltd., «Orbiter® 3 (Small UAS),» 2023. [En línea]. Available:
23] https://aeronautics-sys.com/wp-content/uploads/2023/06/58319_All_brochures_A4_09_Final_orbiter3.pdf.
- Aeronautics Ltd., «Orbiter® 4 (Tactical UAS),» 2023. [En línea]. Available:
24] https://aeronautics-sys.com/wp-content/uploads/2023/06/58319_All_brochures_A4_09_Final_orbiter4.pdf.
- AeroVironment, Inc., «Product Catalog (2022),» 2022. [En línea]. Available:
25] https://www.avinc.com/images/uploads/product_docs/2022_Product_Catalog.pdf.
- Airbus Defence and Space, «Survey Copter Aliaca: Maritime at heart, global in mind,»
26] 2025. [En línea]. Available: https://mediaassets.airbus.com/pm_38_689_689175-9phtg2pjx.pdf.
- Python Software Foundation, «Python 3 Documentation,» 2026. [En línea]. Available:
27] <https://docs.python.org/3/>.
- Project Jupyter, «Project Jupyter Documentation,» [En línea]. Available:
28] <https://docs.jupyter.org/>. [Último acceso: 2026].

- 29] E. Delpino, «ADRPY-VTOL,» [En línea]. Available: <https://github.com/Delpoo/ADRPY-VTOL>. [Último acceso: 19 Feb 2026].
- 30] A. Sobester, «ADRPY (repositorio en GitHub),» [En línea]. Available: <https://github.com/sobester/ADRPY>. [Último acceso: 19 Feb 2026].
- 31] I. Anaconda, «Anaconda Documentation,» [En línea]. Available: <https://docs.anaconda.com/>. [Último acceso: 2026].
- 32] Microsoft, «Code Documentation,» [En línea]. Available: <https://code.visualstudio.com/docs>. [Último acceso: 2026].
- 33] pandas development team, «pandas Documentation,» [En línea]. Available: <https://pandas.pydata.org/docs/>. [Último acceso: 2026].
- 34] OpenAI, «ChatGPT,» [En línea]. Available: <https://chat.openai.com/>. [Último acceso: 2026].